

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ ĐÔNG Á**  
**KHOA CÔNG NGHỆ THÔNG TIN**



## **BÀI TẬP LỚN**

**HỌC PHẦN: CÔNG NGHỆ JAVA**

**CHỦ ĐỀ 7: Các bài toán quản lý khác**

**MÃ ĐỀ CNJ70: Xây dựng ứng dụng desktop quản lý hàng  
trang sức sử dụng Java Spring, JDBC, PostgreSQL và JPA**

**LỚP TÍN CHỈ : Công nghệ Java-1-1-26(N03)**

**Giảng viên giảng dạy: Trần Bá Hùng**

**Danh sách sinh viên thực hiện: Nhóm 2**

| STT | Mã sinh viên | Sinh viên thực hiện | Lớp hành chính |
|-----|--------------|---------------------|----------------|
| 1   | 20232139     | Nguyễn Thanh Hùng   | DCCNTT.14.6    |
| 2   |              |                     |                |
| 3   |              |                     |                |

**Bắc Ninh, năm 2026**

# MỤC LỤC

|                                                            |    |
|------------------------------------------------------------|----|
| DANH MỤC BẢNG BIỂU .....                                   | 7  |
| DANH MỤC HÌNH ẢNH .....                                    | 8  |
| LỜI CẢM ƠN .....                                           | 11 |
| CHƯƠNG 1. CƠ SỞ LÝ THUYẾT .....                            | 14 |
| 1.1.    Giới thiệu về đề tài .....                         | 14 |
| 1.1.1.    Đặt vấn đề .....                                 | 14 |
| 1.1.2.    Lý do chọn đề tài .....                          | 15 |
| 1.1.3.    Mục tiêu của đề tài .....                        | 16 |
| 1.1.4.    Phạm vi của đề tài .....                         | 18 |
| 1.1.5.    Đối tượng sử dụng hệ thống .....                 | 19 |
| 1.2.    Phương pháp, mô hình và công cụ .....              | 22 |
| 1.2.1.    Phương pháp phân tích và xây dựng hệ thống ..... | 22 |
| 1.2.2.    Mô hình Client - Server .....                    | 25 |
| 1.2.3.    Kiến trúc RESTful API .....                      | 27 |
| 1.2.4.    Mô hình kiến trúc nhiều tầng .....               | 32 |
| 1.2.5.    Công cụ phát triển Backend .....                 | 37 |
| 1.2.6.    Công cụ phát triển Desktop .....                 | 41 |
| 1.3.    Các công nghệ sử dụng .....                        | 45 |
| 1.3.1.    Ngôn ngữ lập trình Java .....                    | 45 |
| 1.3.2.    Spring Framework .....                           | 46 |
| 1.3.3.    Spring Boot .....                                | 46 |
| 1.3.4.    Spring Data JPA .....                            | 47 |
| 1.3.5.    Hibernate .....                                  | 48 |
| 1.3.6.    JDBC .....                                       | 49 |
| 1.3.7.    PostgreSQL .....                                 | 50 |

|                                                          |                                                          |    |
|----------------------------------------------------------|----------------------------------------------------------|----|
| 1.3.8.                                                   | <i>Spring Security</i> .....                             | 50 |
| 1.3.9.                                                   | <i>JSON Web Token</i> .....                              | 51 |
| 1.3.10.                                                  | <i>RESTful API</i> .....                                 | 52 |
| 1.3.11.                                                  | <i>React</i> .....                                       | 53 |
| 1.3.12.                                                  | <i>Electron</i> .....                                    | 54 |
| 1.3.13.                                                  | <i>TypeScript</i> .....                                  | 55 |
| <b>CHƯƠNG 2. THIẾT KẾ VÀ XÂY DỰNG CHƯƠNG TRÌNH</b> ..... |                                                          | 57 |
| 2.1.                                                     | <b>Phân tích yêu cầu bài toán</b> .....                  | 57 |
| 2.1.1.                                                   | <i>Mô tả bài toán quản lý kinh doanh trang sức</i> ..... | 57 |
| 2.1.2.                                                   | <i>Các đối tượng tham gia hệ thống</i> .....             | 61 |
| 2.1.3.                                                   | <i>Yêu cầu chức năng</i> .....                           | 65 |
| a.                                                       | Chức năng đăng nhập và xác thực người dùng.....          | 65 |
| b.                                                       | Chức năng quản lý tài khoản.....                         | 66 |
| c.                                                       | Chức năng quản lý vai trò và phân quyền.....             | 66 |
| d.                                                       | Chức năng quản lý nhân viên.....                         | 67 |
| e.                                                       | Chức năng quản lý khách hàng .....                       | 67 |
| f.                                                       | Chức năng quản lý loại vàng và loại trang sức .....      | 68 |
| g.                                                       | Chức năng quản lý sản phẩm trang sức .....               | 68 |
| h.                                                       | Chức năng quản lý kho và tồn kho .....                   | 69 |
| i.                                                       | Chức năng quản lý bảng giá.....                          | 69 |
| j.                                                       | Chức năng quản lý giao dịch bán hàng.....                | 70 |
| k.                                                       | Chức năng tìm kiếm và tra cứu .....                      | 70 |
| l.                                                       | Chức năng báo cáo và thống kê.....                       | 70 |
| m.                                                       | Chức năng kiểm soát và xử lý dữ liệu .....               | 71 |
| 2.1.4.                                                   | <i>Yêu cầu phi chức năng</i> .....                       | 71 |
| 2.2.                                                     | <b>Phân tích chức năng hệ thống</b> .....                | 76 |
| 2.2.1.                                                   | <i>Chức năng đăng nhập</i> .....                         | 76 |
| 2.2.2.                                                   | <i>Chức năng quản lý tài khoản</i> .....                 | 78 |
| 2.2.3.                                                   | <i>Chức năng quản lý nhân viên</i> .....                 | 82 |
| 2.2.4.                                                   | <i>Chức năng quản lý sản phẩm trang sức</i> .....        | 85 |

|                                            |                                                                 |     |
|--------------------------------------------|-----------------------------------------------------------------|-----|
| 2.2.5.                                     | <i>Chức năng quản lý loại vàng</i> .....                        | 89  |
| 2.2.6.                                     | <i>Chức năng quản lý kho</i> .....                              | 92  |
| 2.2.7.                                     | <i>Chức năng quản lý tồn kho</i> .....                          | 96  |
| 2.2.8.                                     | <i>Chức năng quản lý khách hàng</i> .....                       | 99  |
| 2.2.9.                                     | <i>Chức năng quản lý bảng giá</i> .....                         | 102 |
| 2.2.10.                                    | <i>Chức năng quản lý giao dịch mua bán</i> .....                | 105 |
| 2.2.11.                                    | <i>Chức năng báo cáo thống kê</i> .....                         | 110 |
| 2.2.12.                                    | <i>Chức năng phân quyền</i> .....                               | 114 |
| 2.3.                                       | <b>Kiến trúc hệ thống</b> .....                                 | 118 |
| 2.3.1.                                     | <i>Kiến trúc tổng thể hệ thống</i> .....                        | 118 |
| 2.3.2.                                     | <i>Kiến trúc Desktop Client</i> .....                           | 120 |
| 2.3.3.                                     | <i>Kiến trúc Backend Spring Boot</i> .....                      | 122 |
| 2.3.4.                                     | <i>Kiến trúc cơ sở dữ liệu</i> .....                            | 124 |
| 2.3.5.                                     | <i>Luồng giao tiếp giữa React Electron và Spring Boot</i> ..... | 125 |
| 2.3.6.                                     | <i>Luồng xử lý Request - Response</i> .....                     | 127 |
| 2.4.                                       | <b>Thiết kế giao diện</b> .....                                 | 129 |
| 2.4.1.                                     | <i>Thiết kế giao diện đăng nhập</i> .....                       | 129 |
| 2.5.                                       | <b>Thiết kế cơ sở dữ liệu</b> .....                             | 134 |
| 2.5.1.                                     | <i>Sơ đồ thực thể liên kết ERD</i> .....                        | 134 |
| 2.5.2.                                     | <i>Thiết kế bảng tài khoản</i> .....                            | 136 |
| 2.6.                                       | <b>Tổ chức dự án và mô tả mã nguồn</b> .....                    | 166 |
| 2.6.1.                                     | <i>Cấu trúc Backend Spring Boot</i> .....                       | 166 |
| 2.6.2.                                     | <i>Thư mục desktop Electron</i> .....                           | 168 |
| <b>CHƯƠNG 3. PHÁT TRIỂN HỆ THỐNG</b> ..... |                                                                 | 171 |
| 3.1.                                       | <b>Phát triển tầng dữ liệu</b> .....                            | 171 |
| 3.1.1.                                     | <i>Kết nối PostgreSQL</i> .....                                 | 171 |
| 3.1.2.                                     | <i>Xây dựng Entity</i> .....                                    | 172 |

|        |                                                |     |
|--------|------------------------------------------------|-----|
| 3.1.3. | <i>Xây dựng Repository với Spring Data JPA</i> | 173 |
| 3.1.4. | <i>Xử lý Transaction</i>                       | 175 |
| 3.2.   | <b>Phát triển tầng nghiệp vụ</b>               | 176 |
| 3.2.1. | <i>Xây dựng Service Layer</i>                  | 176 |
| 3.2.2. | <i>Xử lý nghiệp vụ tài khoản</i>               | 177 |
| 3.2.3. | <i>Xử lý nghiệp vụ sản phẩm</i>                | 178 |
| 3.2.4. | <i>Xử lý nghiệp vụ kho</i>                     | 178 |
| 3.2.5. | <i>Xử lý nghiệp vụ tồn kho</i>                 | 179 |
| 3.2.6. | <i>Xử lý nghiệp vụ băng giá</i>                | 179 |
| 3.2.7. | <i>Validation dữ liệu</i>                      | 179 |
| 3.2.8. | <i>Xử lý Exception</i>                         | 180 |
| 3.3.   | <b>Phát triển tầng API và bảo mật</b>          | 181 |
| 3.3.1. | <i>Xây dựng Controller</i>                     | 181 |
| 3.3.2. | <i>Chuẩn hóa Response API</i>                  | 181 |
| 3.3.3. | <i>Xây dựng chức năng đăng nhập</i>            | 182 |
| 3.3.4. | <i>Mã hóa mật khẩu bằng Bcrypt</i>             | 182 |
| 3.3.5. | <i>Sinh JWT</i>                                | 182 |
| 3.3.6. | <i>Xác thực JWT</i>                            | 182 |
| 3.3.7. | <i>Cấu hình Spring Security</i>                | 183 |
| 3.4.   | <b>Phát triển ứng dụng Desktop</b>             | 183 |
| 3.4.1. | <i>Khởi tạo React Electron</i>                 | 183 |
| 3.4.2. | <i>Xây dựng Electron Main Process</i>          | 183 |
| 3.4.3. | <i>Xây dựng Preload Script</i>                 | 183 |
| 3.4.4. | <i>Xây dựng Renderer Process</i>               | 184 |
| 3.4.5. | <i>Xây dựng giao diện React</i>                | 184 |
| 3.4.6. | <i>Xây dựng hệ thống Router</i>                | 184 |

|                                      |     |
|--------------------------------------|-----|
| 3.4.7. Xây dựng Service gọi API..... | 184 |
| <b>KẾT LUẬN</b> .....                | 186 |
| 1. Kết quả thu được.....             | 186 |
| 2. Hạn chế của đề tài .....          | 187 |
| 3. Hướng phát triển.....             | 187 |
| <b>PHỤ LỤC</b> .....                 | 191 |
| <b>TÀI LIỆU THAM KHẢO</b> .....      | 192 |

## DANH MỤC BẢNG BIỂU

|                                                       |     |
|-------------------------------------------------------|-----|
| Bảng 2.1: Bảng dữ liệu tài khoản .....                | 136 |
| Bảng 2.2: Cấu trúc bảng hồ sơ người dùng.....         | 138 |
| Bảng 2.3: Cấu trúc bảng doanh nghiệp.....             | 139 |
| Bảng 2.4: Cấu trúc bảng vai trò.....                  | 141 |
| Bảng 2.5: Cấu trúc bảng quyền .....                   | 142 |
| Bảng 2.6: Cấu trúc bảng liên kết vai trò - quyền..... | 144 |
| Bảng 2.7: <i>Cấu trúc bảng nhân viên</i> .....        | 145 |
| Bảng 2.8: Cấu trúc bảng khách hàng .....              | 147 |
| Bảng 2.9: Cấu trúc bảng loại vàng .....               | 149 |
| Bảng 2.10: Cấu trúc bảng sản phẩm trang sức.....      | 151 |
| Bảng 2.11: Cấu trúc bảng kho .....                    | 153 |
| Bảng 2.12: Cấu trúc bảng tồn kho .....                | 155 |
| Bảng 2.13: Cấu trúc bảng bảng giá.....                | 157 |
| Bảng 2.14: Cấu trúc bảng chi tiết bảng giá.....       | 159 |
| Bảng 2.15: Cấu trúc bảng giao dịch.....               | 161 |
| Bảng 2.16: Bảng chứa dữ liệu mua bán.....             | 164 |
| Bảng 3.17: Bảng nghiệp vụ Exception .....             | 180 |

## DANH MỤC HÌNH ẢNH

|                                                                  |    |
|------------------------------------------------------------------|----|
| Hình 1.1: Quy trình phát triển phần mềm.....                     | 25 |
| Hình 1.2: Sơ đồ luồng hoạt động tổng quát.....                   | 27 |
| Hình 1.3: Sơ đồ luồng dữ liệu tạo mới sản phẩm.....              | 29 |
| Hình 1.4: Sơ đồ luồng dữ liệu.....                               | 30 |
| Hình 1.5: Sơ đồ phản hồi từ server.....                          | 31 |
| Hình 1.6: Sơ đồ kiến trúc phân tầng hệ thống.....                | 33 |
| Hình 1.7: Sơ đồ luồng yêu cầu và phản hồi.....                   | 36 |
| Hình 1.8: IntelliJ IDEA công cụ hỗ trợ phát triển phần mềm.....  | 37 |
| Hình 1.9: Postman công cụ hỗ trợ kiểm thử API.....               | 39 |
| Hình 1.10: Hệ quản trị cơ sở dữ liệu mã nguồn mở PostgreSQL..... | 39 |
| Hình 1.11: GitHub công cụ lưu trữ mã nguồn.....                  | 40 |
| Hình 1.12: Visual Studio Code công cụ lập trình.....             | 41 |
| Hình 1.13: NodeJs nền tảng xây dựng ứng dụng.....                | 42 |
| Hình 1.14: React Native xây dựng ứng dụng đa nền tảng.....       | 42 |
| Hình 1.15: Lập trình ứng dụng desktop đa nền tảng.....           | 43 |
| Hình 2.16: Quy trình nghiệp vụ tổng quát của hệ thống.....       | 60 |
| Hình 2.17: Các đối tượng tham gia vào hệ thống.....              | 65 |
| Hình 2.18: Sơ đồ tuần tự đăng nhập.....                          | 78 |
| Hình 2.19: Sơ đồ hoạt động quản lý tài khoản.....                | 81 |
| Hình 2.20: Biểu đồ hoạt động quản lý nhân viên.....              | 85 |
| Hình 21: Sơ đồ hoạt động quản lý sản phẩm trang sức.....         | 89 |
| Hình 2.22: Biểu đồ hoạt động quản lý loại vàng.....              | 92 |
| Hình 2.23: Biểu đồ hoạt động quản lý kho.....                    | 95 |
| Hình 2.24: Biểu đồ tuần tự quản lý tồn kho.....                  | 99 |

|                                                                 |     |
|-----------------------------------------------------------------|-----|
| Hình 2.25: Biểu đồ hoạt động chức năng quản lý khách hàng ..... | 102 |
| Hình 2.26: Biểu đồ hoạt động chức năng bảng giá .....           | 105 |
| Hình 2.27: Biểu đồ tuần tự giao dịch mua bán.....               | 110 |
| Hình 2.28: Biểu đồ tuần tự báo cáo thống kê.....                | 114 |
| Hình 2.29: Biểu đồ tuần tự chức năng phân quyền.....            | 118 |
| Hình 2.30: Kiến trúc tổng thể hệ thống .....                    | 120 |
| Hình 2.31: Kiến trúc Desktop Client .....                       | 122 |
| Hình 2.32: Sơ đồ kiến trúc spring boot.....                     | 124 |
| Hình 2.33: Sơ đồ luồng cách client nói chuyện với server.....   | 127 |
| Hình 2.34: Thiết kế giao diện đăng nhập.....                    | 129 |
| Hình 2.35: Thiết kế giao diện dashboard.....                    | 129 |
| Hình 2.36: Thiết kế giao diện quản lý sản phẩm.....             | 130 |
| Hình 2.37: Thiết kế giao diện quản lý kho.....                  | 130 |
| Hình 2.38: Thiết kế giao diện quản lý nhân viên .....           | 131 |
| Hình 2.39: Thiết kế giao diện quản lý khách hàng.....           | 131 |
| Hình 2.40: Thiết kế giao diện bảng giá.....                     | 132 |
| Hình 2.41: Thiết kế giao diện bán hàng.....                     | 132 |
| Hình 42: Thiết kế giao diện giao dịch.....                      | 133 |
| Hình 2.43: Thiết kế giao diện báo cáo .....                     | 133 |
| Hình 2.44: Sơ đồ thực thể ERD .....                             | 134 |
| Hình 2.45: Cấu trúc thư mục bachken .....                       | 166 |
| Hình 2.46: Cấu trúc thư mục desktop.....                        | 169 |
| Hình 3.47: Kết nối sql.....                                     | 171 |
| Hình 3.48: Entities được tạo bởi JPA.....                       | 173 |
| Hình 3.49: Tìm kiếm tài khoản bằng JPA.....                     | 174 |
| Hình 3.50: Sử dụng transaction vào sql.....                     | 175 |

|                                                 |     |
|-------------------------------------------------|-----|
| Hình 3.51: Service Layer trong spring boot..... | 177 |
| Hình 3.52: Đăng nhập tại tầng service.....      | 178 |

## LỜI CẢM ƠN

Trong quá trình thực hiện bài tập lớn học phần Công nghệ Java với đề tài “Xây dựng ứng dụng desktop quản lý hàng trang sức sử dụng Java Spring, JDBC, PostgreSQL và JPA”, em đã nhận được nhiều sự hướng dẫn và hỗ trợ cần thiết để từng bước hoàn thiện nội dung nghiên cứu cũng như sản phẩm chương trình.

Trước hết, em xin trân trọng cảm ơn Trường Đại học Công nghệ Đông Á và Khoa Công nghệ Thông tin đã tạo điều kiện học tập, cung cấp nền tảng kiến thức chuyên môn và môi trường thuận lợi để sinh viên có cơ hội vận dụng kiến thức lý thuyết vào việc giải quyết các bài toán thực tế trong lĩnh vực công nghệ phần mềm.

Em xin gửi lời cảm ơn chân thành tới Thầy Trần Bá Hùng, giảng viên phụ trách học phần Công nghệ Java, đã trực tiếp giảng dạy, định hướng phương pháp tiếp cận và cung cấp những kiến thức quan trọng về lập trình Java, tổ chức ứng dụng, làm việc với cơ sở dữ liệu cũng như các công nghệ liên quan. Những kiến thức được tiếp thu trong quá trình học tập là cơ sở quan trọng để em phân tích yêu cầu, lựa chọn công nghệ và triển khai các chức năng của hệ thống.

Trong quá trình xây dựng chương trình, em đã có cơ hội củng cố kiến thức về Java, Spring Boot, JDBC, JPA, PostgreSQL, đồng thời tìm hiểu thêm về mô hình Client–Server, RESTful API và phương pháp xây dựng ứng dụng desktop. Việc thực hiện đề tài không chỉ giúp em nâng cao kỹ năng lập trình mà còn rèn luyện khả năng phân tích bài toán, thiết kế cơ sở dữ liệu, tổ chức mã nguồn và xử lý các vấn đề phát sinh trong quá trình phát triển phần mềm.

Do thời gian thực hiện và kinh nghiệm thực tế còn giới hạn, báo cáo và chương trình khó tránh khỏi những thiếu sót. Em kính mong nhận được những nhận xét và góp ý từ thầy để có thể tiếp tục hoàn thiện kiến thức, kỹ năng cũng như phương pháp làm việc trong các dự án sau này.

Em xin trân trọng cảm ơn!

## LỜI MỞ ĐẦU

Sự phát triển mạnh mẽ của công nghệ thông tin đang tạo ra những thay đổi rõ rệt trong phương thức tổ chức và quản lý hoạt động kinh doanh. Thay vì phụ thuộc hoàn toàn vào sổ sách hoặc các bảng dữ liệu rời rạc, nhiều đơn vị đã chuyển sang sử dụng các hệ thống phần mềm nhằm quản lý thông tin tập trung, giảm sai sót trong quá trình xử lý dữ liệu và nâng cao khả năng kiểm soát hoạt động kinh doanh.

Trong lĩnh vực kinh doanh trang sức, yêu cầu quản lý dữ liệu có nhiều đặc điểm riêng. Sản phẩm thường có giá trị cao và cần được quản lý theo nhiều thuộc tính như mã sản phẩm, loại trang sức, loại vàng, trọng lượng, đơn giá và tình trạng tồn kho. Bên cạnh đó, quá trình nhập hàng, bán hàng, cập nhật bảng giá, quản lý nhân viên và theo dõi giao dịch tạo ra lượng dữ liệu lớn, có mối liên hệ chặt chẽ với nhau. Nếu các thông tin này được quản lý riêng lẻ hoặc xử lý thủ công, việc tra cứu, đối chiếu và tổng hợp số liệu có thể mất nhiều thời gian và dễ xảy ra sai sót.

Xuất phát từ thực tế đó, đề tài “Xây dựng ứng dụng desktop quản lý hàng trang sức sử dụng Java Spring, JDBC, PostgreSQL và JPA” được lựa chọn nhằm nghiên cứu và xây dựng một hệ thống hỗ trợ quản lý các nghiệp vụ cơ bản trong hoạt động kinh doanh trang sức. Hệ thống hướng tới việc tổ chức dữ liệu một cách có hệ thống, hỗ trợ người dùng quản lý sản phẩm, kho hàng, nhân viên, khách hàng, bảng giá, giao dịch và các thông tin liên quan thông qua một ứng dụng desktop.

Về mặt công nghệ, phần xử lý nghiệp vụ của hệ thống được xây dựng trên nền tảng Java và Spring Boot. Dữ liệu được lưu trữ bằng PostgreSQL, trong khi JPA, Hibernate và JDBC được sử dụng để hỗ trợ quá trình truy cập và thao tác với cơ sở dữ liệu. Hệ thống được tổ chức theo mô hình Client–Server, trong đó Backend cung cấp các RESTful API để trao đổi dữ liệu với ứng dụng desktop. Phần giao diện người dùng được định hướng xây dựng bằng React kết hợp Electron, qua đó tạo ra một ứng dụng có thể hoạt động trên môi trường desktop nhưng vẫn duy trì sự tách biệt giữa tầng giao diện và tầng xử lý nghiệp vụ.

Thông qua quá trình thực hiện đề tài, mục tiêu không chỉ dừng lại ở việc xây dựng một chương trình có khả năng đáp ứng các chức năng quản lý cơ bản, mà còn hướng tới việc vận

dụng những kiến thức đã học trong học phần Công nghệ Java vào một bài toán cụ thể. Quá trình này giúp làm rõ cách tổ chức một ứng dụng Java theo nhiều tầng, cách thiết kế và xây dựng RESTful API, phương pháp ánh xạ dữ liệu với JPA, sử dụng JDBC trong quá trình truy xuất dữ liệu và kết nối ứng dụng với hệ quản trị cơ sở dữ liệu PostgreSQL.

Nội dung báo cáo được trình bày theo ba chương chính. Chương 1 trình bày cơ sở lý thuyết, mô hình và các công nghệ được sử dụng trong đề tài. Chương 2 tập trung phân tích yêu cầu, thiết kế kiến trúc, giao diện, cơ sở dữ liệu và tổ chức mã nguồn của hệ thống. Chương 3 trình bày quá trình phát triển các tầng của ứng dụng, kết quả thực hiện và quá trình kiểm thử hệ thống. Phần cuối của báo cáo tổng kết những kết quả đạt được, đồng thời chỉ ra các hạn chế và định hướng phát triển trong tương lai.

Qua đề tài này, các kiến thức về Java và các công nghệ liên quan được vận dụng trong một quy trình phát triển phần mềm tương đối hoàn chỉnh, từ phân tích yêu cầu, thiết kế hệ thống, xây dựng cơ sở dữ liệu cho tới triển khai, tích hợp và kiểm thử chương trình.

# CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

## 1.1. Giới thiệu về đề tài

### 1.1.1. Đặt vấn đề

Trong những năm gần đây, việc ứng dụng công nghệ thông tin vào hoạt động quản lý và kinh doanh ngày càng trở nên phổ biến. Các hệ thống phần mềm không chỉ giúp doanh nghiệp lưu trữ thông tin một cách tập trung mà còn hỗ trợ kiểm soát quy trình nghiệp vụ, giảm sai sót trong quá trình xử lý dữ liệu và nâng cao hiệu quả vận hành. Đối với lĩnh vực kinh doanh trang sức, nhu cầu này càng trở nên cần thiết do đặc thù hàng hóa có giá trị cao, nhiều thuộc tính cần quản lý và thường xuyên phát sinh các nghiệp vụ liên quan đến nhập hàng, bán hàng, tồn kho, cập nhật giá và theo dõi giao dịch.

Trong thực tế, một sản phẩm trang sức thường được quản lý theo nhiều thông tin như mã sản phẩm, tên sản phẩm, loại trang sức, loại vàng, trọng lượng, hàm lượng, đơn giá, số lượng và trạng thái tồn kho. Bên cạnh đó, doanh nghiệp còn phải quản lý thông tin nhân viên, khách hàng, kho hàng, bảng giá và lịch sử giao dịch. Các dữ liệu này có mối quan hệ chặt chẽ với nhau và thường xuyên thay đổi trong quá trình kinh doanh. Nếu được quản lý bằng sổ sách, bảng tính hoặc nhiều công cụ rời rạc, việc cập nhật và đối chiếu thông tin có thể mất nhiều thời gian, đồng thời làm tăng nguy cơ sai lệch dữ liệu.

Một vấn đề khác là giá trị của sản phẩm trang sức có thể thay đổi theo giá nguyên liệu, trọng lượng, loại vàng và các yếu tố liên quan đến gia công. Vì vậy, hệ thống quản lý cần đảm bảo khả năng cập nhật bảng giá, liên kết giá với sản phẩm và hỗ trợ tính toán chính xác trong quá trình giao dịch. Đồng thời, sự biến động của số lượng hàng hóa sau mỗi lần nhập hoặc bán cũng đặt ra yêu cầu phải kiểm soát tồn kho một cách nhất quán để hạn chế tình trạng chênh lệch giữa dữ liệu trên hệ thống và hàng hóa thực tế.

Bên cạnh yêu cầu về quản lý dữ liệu, một hệ thống phục vụ hoạt động kinh doanh còn cần chú trọng đến vấn đề phân quyền và bảo mật. Mỗi nhóm người dùng có nhiệm vụ khác nhau nên phạm vi truy cập và thao tác trên hệ thống cũng cần được kiểm

soát phù hợp. Việc xác định rõ quyền hạn của từng tài khoản giúp hạn chế các thao tác không được phép, đồng thời tăng khả năng kiểm soát và truy vết các hoạt động phát sinh trong hệ thống.

Từ những yêu cầu trên, việc xây dựng một ứng dụng desktop quản lý hàng trang sức có khả năng kết nối với hệ thống xử lý nghiệp vụ và cơ sở dữ liệu tập trung là cần thiết. Giải pháp được đề xuất trong đề tài sử dụng Java Spring Boot để xây dựng phần xử lý nghiệp vụ và cung cấp RESTful API, kết hợp JPA, Hibernate và JDBC trong quá trình truy xuất dữ liệu, sử dụng PostgreSQL làm hệ quản trị cơ sở dữ liệu. Phần giao diện được xây dựng theo hướng ứng dụng desktop bằng React và Electron, qua đó tạo sự tách biệt giữa tầng giao diện, tầng nghiệp vụ và tầng dữ liệu.

Thông qua việc xây dựng hệ thống, đề tài hướng tới giải quyết các nghiệp vụ quản lý cơ bản của hoạt động kinh doanh trang sức như quản lý sản phẩm, kho hàng, nhân viên, khách hàng, bảng giá và giao dịch, đồng thời vận dụng các kiến thức về công nghệ Java vào quá trình phân tích, thiết kế và phát triển một ứng dụng có cấu trúc tương đối hoàn chỉnh.

### ***1.1.2. Lý do chọn đề tài***

Trong hoạt động kinh doanh trang sức, việc quản lý hàng hóa đòi hỏi độ chính xác cao do sản phẩm có giá trị lớn, nhiều đặc tính riêng và thường xuyên phát sinh các nghiệp vụ nhập, xuất, mua, bán và điều chỉnh giá. Mỗi sản phẩm không chỉ được quản lý theo tên hoặc mã hàng mà còn liên quan đến các thông tin như loại trang sức, loại vàng, trọng lượng, đơn giá, số lượng tồn kho và trạng thái giao dịch. Khi số lượng sản phẩm và dữ liệu kinh doanh tăng lên, phương pháp quản lý thủ công bằng sổ sách hoặc bảng tính sẽ bộc lộ nhiều hạn chế, đặc biệt trong việc cập nhật, tra cứu, đối chiếu và tổng hợp thông tin.

Bên cạnh đó, hoạt động kinh doanh trang sức còn liên quan đến nhiều nhóm dữ liệu khác nhau như nhân viên, khách hàng, kho hàng, bảng giá và lịch sử giao dịch. Các dữ liệu này có mối quan hệ chặt chẽ, vì vậy nếu được quản lý bằng nhiều công cụ riêng biệt sẽ dễ xảy ra tình trạng thiếu đồng bộ, trùng lặp hoặc sai lệch thông tin. Điều này

có thể ảnh hưởng trực tiếp đến quá trình kiểm soát tồn kho, tính toán giá trị giao dịch và đánh giá hiệu quả kinh doanh.

Xuất phát từ những yêu cầu thực tế trên, việc xây dựng một hệ thống quản lý tập trung là cần thiết nhằm hỗ trợ người dùng xử lý các nghiệp vụ một cách khoa học, chính xác và thuận tiện hơn. Một ứng dụng desktop có giao diện trực quan, kết hợp với hệ thống backend xử lý nghiệp vụ và cơ sở dữ liệu tập trung, có thể giúp giảm bớt thao tác thủ công, nâng cao khả năng kiểm soát dữ liệu và tạo điều kiện thuận lợi cho việc mở rộng các chức năng quản lý trong tương lai.

Ngoài ý nghĩa thực tiễn, đề tài còn phù hợp với nội dung của học phần Công nghệ Java. Việc xây dựng hệ thống cho phép vận dụng nhiều kiến thức quan trọng như lập trình Java, Spring Boot, JDBC, JPA, Hibernate, kết nối cơ sở dữ liệu PostgreSQL, xây dựng RESTful API và tổ chức ứng dụng theo mô hình nhiều tầng. Đồng thời, việc kết hợp React và Electron để xây dựng giao diện desktop giúp đề tài có tính hoàn chỉnh cao hơn, thể hiện rõ sự phối hợp giữa tầng giao diện, tầng xử lý nghiệp vụ và tầng dữ liệu.

Vì những lý do trên, đề tài được lựa chọn với mục tiêu vừa giải quyết một bài toán quản lý có tính thực tế, vừa tạo điều kiện để củng cố và vận dụng các kiến thức đã học vào quá trình xây dựng một hệ thống phần mềm tương đối hoàn chỉnh.

### ***1.1.3. Mục tiêu của đề tài***

Đề tài được thực hiện với mục tiêu xây dựng một hệ thống phần mềm hỗ trợ quản lý các nghiệp vụ cơ bản trong hoạt động kinh doanh trang sức theo hướng tập trung, chính xác và dễ sử dụng. Hệ thống hướng đến việc thay thế các thao tác quản lý rời rạc bằng một quy trình xử lý thống nhất, trong đó dữ liệu về sản phẩm, kho hàng, nhân viên, khách hàng, bảng giá và giao dịch được tổ chức có hệ thống và có khả năng liên kết với nhau.

Về mặt nghiệp vụ, hệ thống cần đáp ứng khả năng quản lý thông tin hàng trang sức từ thời điểm được khai báo, nhập kho, cập nhật thông tin cho đến khi phát sinh giao dịch bán hàng. Các dữ liệu quan trọng như mã sản phẩm, loại trang sức, loại vàng, trọng

lượng, giá bán, số lượng tồn kho và trạng thái của sản phẩm cần được lưu trữ và cập nhật chính xác. Bên cạnh đó, chương trình phải hỗ trợ quản lý nhân viên, khách hàng, bảng giá, giao dịch và các thông tin có liên quan nhằm giúp người sử dụng theo dõi hoạt động kinh doanh một cách thuận tiện hơn.

Một mục tiêu quan trọng khác của đề tài là xây dựng cơ chế quản lý tài khoản và phân quyền phù hợp với từng nhóm người dùng. Thông qua việc xác thực và kiểm soát quyền truy cập, hệ thống có thể giới hạn các chức năng mà từng tài khoản được phép sử dụng, từ đó góp phần bảo vệ dữ liệu và hạn chế các thao tác không phù hợp với nhiệm vụ của người dùng.

Về mặt kỹ thuật, đề tài hướng tới việc vận dụng các kiến thức và công nghệ Java đã được học để xây dựng một hệ thống có cấu trúc rõ ràng. Java Spring Boot được sử dụng để phát triển backend và xử lý các nghiệp vụ chính; Spring Data JPA, Hibernate và JDBC hỗ trợ quá trình truy xuất, ánh xạ và thao tác dữ liệu; PostgreSQL đảm nhiệm việc lưu trữ dữ liệu của hệ thống. Backend cung cấp các RESTful API làm phương thức trao đổi dữ liệu với ứng dụng phía người dùng, qua đó đảm bảo sự tách biệt giữa phần giao diện, xử lý nghiệp vụ và cơ sở dữ liệu.

Đối với phía người dùng, đề tài hướng đến xây dựng một ứng dụng desktop có giao diện trực quan và thuận tiện cho quá trình thao tác. React kết hợp Electron được sử dụng để phát triển giao diện desktop, trong khi toàn bộ nghiệp vụ và dữ liệu được xử lý thông qua các API do Spring Boot cung cấp. Cách tổ chức này giúp chương trình có tính mô-đun, thuận lợi hơn cho việc bảo trì, kiểm thử và mở rộng chức năng.

Thông qua quá trình triển khai, đề tài đồng thời đặt mục tiêu củng cố khả năng phân tích yêu cầu, thiết kế cơ sở dữ liệu, tổ chức mã nguồn theo các tầng, xây dựng API, xử lý bảo mật và tích hợp giữa các thành phần của một hệ thống phần mềm. Kết quả cuối cùng hướng tới một ứng dụng có khả năng vận hành ổn định với các nghiệp vụ quản lý trang sức cơ bản, đồng thời tạo nền tảng để tiếp tục phát triển thêm các chức năng nâng cao trong tương lai.

#### **1.1.4. Phạm vi của đề tài**

Phạm vi của đề tài tập trung vào việc xây dựng một ứng dụng desktop phục vụ công tác quản lý hoạt động kinh doanh trang sức trong phạm vi cửa hàng hoặc đơn vị kinh doanh có quy mô vừa và nhỏ. Hệ thống được định hướng giải quyết các nghiệp vụ quản lý cơ bản, đồng thời bảo đảm dữ liệu được tổ chức tập trung, dễ tra cứu và thuận tiện trong quá trình sử dụng.

Về nghiệp vụ, chương trình hỗ trợ quản lý thông tin sản phẩm trang sức, loại vàng, kho hàng, số lượng tồn, nhân viên, khách hàng, bảng giá và các giao dịch phát sinh trong quá trình mua bán. Người dùng có thể thực hiện các thao tác thêm mới, cập nhật, tìm kiếm, tra cứu và theo dõi trạng thái của từng nhóm dữ liệu. Đối với nghiệp vụ bán hàng, hệ thống hỗ trợ ghi nhận thông tin giao dịch, xác định sản phẩm, khách hàng, đơn giá và tổng giá trị thanh toán, đồng thời cập nhật dữ liệu tồn kho sau khi giao dịch được hoàn tất.

Bên cạnh các chức năng nghiệp vụ, hệ thống còn hỗ trợ quản lý tài khoản và phân quyền người dùng. Mỗi tài khoản được cấp vai trò và phạm vi thao tác phù hợp với nhiệm vụ được giao, qua đó hạn chế việc truy cập hoặc thay đổi dữ liệu không đúng thẩm quyền. Chức năng đăng nhập và xác thực được xây dựng nhằm bảo đảm chỉ những người dùng hợp lệ mới có thể sử dụng các chức năng của chương trình.

Về phạm vi kỹ thuật, backend được phát triển bằng Java Spring Boot và đảm nhiệm việc xử lý nghiệp vụ, kiểm tra dữ liệu và cung cấp các RESTful API cho ứng dụng desktop. Dữ liệu được lưu trữ trên PostgreSQL, trong khi JPA, Hibernate và JDBC được sử dụng để hỗ trợ quá trình truy xuất và thao tác với cơ sở dữ liệu. Phần giao diện được xây dựng bằng React kết hợp Electron, giúp triển khai chương trình dưới dạng ứng dụng desktop và giao tiếp với backend thông qua giao thức HTTP.

Đề tài chủ yếu hướng tới các nhóm người dùng như quản trị viên, người quản lý và nhân viên. Mỗi nhóm có phạm vi chức năng khác nhau tùy theo vai trò, từ quản lý tài khoản, hàng hóa và kho đến thực hiện giao dịch và theo dõi thông tin báo cáo.

Trong giới hạn của bài tập lớn, hệ thống chưa đi sâu vào các chức năng mở rộng như tích hợp thanh toán trực tuyến, hóa đơn điện tử, máy quét mã vạch chuyên dụng, cân điện tử, đồng bộ dữ liệu theo thời gian thực giữa nhiều hệ thống hoặc các chức năng dự báo kinh doanh nâng cao. Những nội dung này có thể được xem xét phát triển thêm khi hệ thống được mở rộng trong tương lai.

Nhìn chung, phạm vi của đề tài được xác định ở mức vừa đủ để thể hiện đầy đủ quy trình phân tích, thiết kế, xây dựng và kiểm thử một ứng dụng quản lý trang sức, đồng thời phù hợp với mục tiêu và thời lượng của học phần Công nghệ Java.

#### **1.1.5. Đối tượng sử dụng hệ thống**

Hệ thống được xây dựng nhằm phục vụ các cá nhân trực tiếp tham gia vào quá trình quản lý, vận hành và thực hiện các nghiệp vụ kinh doanh trang sức. Do đặc thù của lĩnh vực này liên quan đến hàng hóa có giá trị cao, nhiều thông tin cần kiểm soát và thường xuyên phát sinh các giao dịch, việc xác định rõ đối tượng sử dụng là cơ sở quan trọng để tổ chức chức năng, phân chia quyền truy cập và bảo đảm dữ liệu được xử lý đúng phạm vi trách nhiệm. Mỗi nhóm người dùng trong hệ thống có vai trò, nhiệm vụ và mức độ truy cập khác nhau, từ đó tạo nên một quy trình vận hành có kiểm soát và hạn chế các thao tác không phù hợp.

Đối tượng có phạm vi quản lý cao nhất là **quản trị viên hệ thống**. Nhóm người dùng này chịu trách nhiệm quản lý các thành phần liên quan đến tài khoản, vai trò và quyền truy cập. Quản trị viên có thể tạo mới tài khoản, cập nhật thông tin người dùng, thay đổi trạng thái hoạt động, gán vai trò hoặc điều chỉnh quyền sử dụng chức năng. Trong trường hợp nhân viên thay đổi vị trí công việc hoặc không còn tham gia vận hành hệ thống, quản trị viên có thể thực hiện việc thay đổi quyền hoặc vô hiệu hóa tài khoản để bảo đảm an toàn dữ liệu. Ngoài ra, nhóm này còn có trách nhiệm duy trì cấu trúc phân quyền và kiểm soát những chức năng mang tính hệ thống mà người dùng thông thường không được phép tác động.

Việc tồn tại nhóm quản trị viên giúp phân tách rõ giữa nhiệm vụ quản lý kỹ thuật và nhiệm vụ xử lý nghiệp vụ kinh doanh. Không phải mọi người dùng đều cần có quyền thay đổi tài khoản, phân quyền hoặc cấu hình hệ thống. Bằng cách giới hạn các chức năng này cho một nhóm người dùng cụ thể, chương trình có thể giảm nguy cơ thao tác nhầm, thay đổi dữ liệu không đúng thẩm quyền hoặc làm ảnh hưởng đến hoạt động của các tài khoản khác. Đây cũng là cơ sở để thực hiện cơ chế xác thực và phân quyền một cách chặt chẽ hơn trong quá trình sử dụng.

Nhóm người dùng thứ hai là **người quản lý**. Đây là đối tượng chịu trách nhiệm theo dõi và kiểm soát hoạt động kinh doanh trong phạm vi được giao. Người quản lý cần có khả năng truy cập các thông tin tổng hợp như danh sách sản phẩm, số lượng tồn kho, tình trạng hàng hóa, bảng giá, giao dịch phát sinh và các báo cáo liên quan. Những

thông tin này giúp người quản lý nắm được tình hình hoạt động tại từng thời điểm, kiểm tra sự biến động của hàng hóa và đánh giá kết quả kinh doanh dựa trên dữ liệu đã được ghi nhận trong hệ thống.

Đối với hoạt động quản lý hàng trang sức, người quản lý còn có thể thực hiện hoặc phê duyệt các thao tác liên quan đến cập nhật thông tin sản phẩm, điều chỉnh bảng giá, kiểm tra số lượng hàng trong kho và theo dõi các giao dịch đã hoàn thành. Do giá trị của sản phẩm có thể thay đổi theo loại vàng, trọng lượng và bảng giá tại từng thời điểm, việc cung cấp dữ liệu đầy đủ và có tổ chức giúp người quản lý đưa ra quyết định phù hợp hơn. Thay vì tổng hợp thông tin từ nhiều nguồn riêng biệt, dữ liệu được tập trung trong cùng một hệ thống, nhờ đó quá trình tra cứu và đối chiếu trở nên thuận tiện hơn.

Nhóm **nhân viên** là đối tượng thường xuyên thao tác trực tiếp với chương trình trong quá trình làm việc hằng ngày. Đây có thể là nhân viên bán hàng, nhân viên quản lý kho hoặc các vị trí khác có liên quan đến việc nhập, kiểm tra và cập nhật dữ liệu. Tùy theo phạm vi công việc, nhân viên có thể được cấp quyền tra cứu sản phẩm, kiểm tra thông tin hàng hóa, xem bảng giá, quản lý khách hàng, tạo giao dịch hoặc cập nhật một số dữ liệu cần thiết. Việc giới hạn chức năng theo từng vai trò giúp nhân viên chỉ tiếp cận những nội dung phục vụ trực tiếp cho công việc của mình.

Đối với nhân viên bán hàng, hệ thống đóng vai trò hỗ trợ tra cứu thông tin sản phẩm và thực hiện các giao dịch với khách hàng. Trong quá trình tạo giao dịch, nhân viên có thể lựa chọn sản phẩm, kiểm tra thông tin về loại hàng, trọng lượng, đơn giá, số lượng hoặc trạng thái tồn kho trước khi xác nhận giao dịch. Các dữ liệu liên quan được gửi tới backend để xử lý và lưu trữ, từ đó bảo đảm thông tin giao dịch được ghi nhận thống nhất. Sau khi giao dịch hoàn thành, dữ liệu tồn kho cũng được cập nhật tương ứng, giúp hạn chế sự chênh lệch giữa số liệu trên hệ thống và tình trạng hàng hóa thực tế.

Đối với nhân viên phụ trách kho, phạm vi sử dụng tập trung nhiều hơn vào hoạt động kiểm soát hàng hóa. Người dùng có thể theo dõi số lượng tồn, kiểm tra trạng thái sản phẩm, ghi nhận các nghiệp vụ nhập hoặc xuất và tra cứu lịch sử biến động khi cần

thiết. Các thao tác này có ảnh hưởng trực tiếp đến dữ liệu hàng hóa nên cần được kiểm soát thông qua quyền truy cập phù hợp. Nhân viên không được phân quyền cho chức năng quản lý kho sẽ không thể thực hiện các thao tác làm thay đổi số liệu tồn, qua đó giảm nguy cơ phát sinh những thay đổi không được kiểm soát.

Bên cạnh các nhóm người dùng trực tiếp đăng nhập, **khách hàng** cũng là một đối tượng quan trọng trong phạm vi quản lý của chương trình. Tuy nhiên, khách hàng không trực tiếp sử dụng ứng dụng desktop mà thông tin của họ được lưu trữ để phục vụ các nghiệp vụ kinh doanh. Dữ liệu khách hàng có thể bao gồm các thông tin nhận dạng và liên hệ cần thiết, đồng thời được liên kết với những giao dịch đã phát sinh. Việc quản lý thông tin này giúp nhân viên có thể tra cứu khách hàng khi lập giao dịch, đồng thời hỗ trợ theo dõi lịch sử mua bán khi cần thiết.

Một nguyên tắc quan trọng trong việc xác định đối tượng sử dụng là mỗi người dùng chỉ nên được cấp quyền tương ứng với nhiệm vụ thực tế. Hệ thống không áp dụng một tập quyền giống nhau cho toàn bộ tài khoản mà tổ chức quyền theo vai trò và chức năng. Chẳng hạn, nhân viên bán hàng có thể được phép lập giao dịch nhưng không được thay đổi quyền của tài khoản khác; người quản lý có thể xem báo cáo và kiểm tra tồn kho nhưng không nhất thiết được phép chỉnh sửa các cấu hình hệ thống; quản trị viên có thể quản lý tài khoản và quyền truy cập nhưng các nghiệp vụ kinh doanh vẫn được tổ chức độc lập theo phạm vi được thiết kế. Cách phân chia này giúp giảm sự phụ thuộc giữa các nhóm chức năng và hỗ trợ kiểm soát hoạt động của người dùng tốt hơn.

Ngoài việc giới hạn chức năng, việc xác định đúng đối tượng sử dụng còn có ý nghĩa đối với thiết kế giao diện. Mỗi nhóm người dùng có nhu cầu thao tác khác nhau nên giao diện cần ưu tiên hiển thị những chức năng phù hợp với nhiệm vụ của họ. Những chức năng không có quyền sử dụng có thể được ẩn hoặc vô hiệu hóa, giúp giao diện trở nên rõ ràng hơn và giảm khả năng thao tác nhầm. Đồng thời, backend vẫn thực hiện kiểm tra quyền đối với từng yêu cầu gửi tới hệ thống nhằm bảo đảm rằng việc kiểm soát truy cập không chỉ phụ thuộc vào phía giao diện.

Hệ thống được xây dựng cho ba nhóm người dùng chính gồm quản trị viên, người quản lý và nhân viên, trong đó mỗi nhóm được phân quyền phù hợp với nhiệm

vụ và trách nhiệm cụ thể. Bên cạnh đó, thông tin khách hàng được quản lý như một thành phần dữ liệu phục vụ quá trình giao dịch và tra cứu. Việc xác định rõ phạm vi sử dụng của từng nhóm người dùng là cơ sở để thiết kế chức năng, tổ chức giao diện, xây dựng cơ chế xác thực và kiểm soát quyền truy cập, góp phần bảo đảm hệ thống vận hành an toàn và thống nhất.

## **1.2. Phương pháp, mô hình và công cụ**

### ***1.2.1. Phương pháp phân tích và xây dựng hệ thống***

Quá trình phân tích và xây dựng hệ thống được thực hiện theo hướng tiếp cận từ yêu cầu thực tế đến thiết kế, triển khai và kiểm thử. Phương pháp này giúp bảo đảm các chức năng của chương trình được hình thành dựa trên nhu cầu nghiệp vụ cụ thể, đồng thời tạo sự liên kết giữa phần giao diện, xử lý nghiệp vụ và dữ liệu. Thay vì xây dựng từng chức năng một cách độc lập, hệ thống được phân chia thành các thành phần có trách nhiệm rõ ràng để thuận lợi cho việc phát triển, kiểm tra và mở rộng.

Giai đoạn đầu tiên tập trung vào khảo sát và xác định yêu cầu. Các nghiệp vụ liên quan đến quản lý sản phẩm trang sức, loại vàng, kho hàng, nhân viên, khách hàng, bảng giá, giao dịch và tài khoản được xem xét để xác định dữ liệu cần quản lý cũng như những thao tác mà người dùng cần thực hiện. Từ đó, các yêu cầu được phân chia thành yêu cầu chức năng và yêu cầu phi chức năng. Yêu cầu chức năng mô tả những nghiệp vụ chương trình phải đáp ứng, trong khi yêu cầu phi chức năng đề cập đến các yếu tố như khả năng sử dụng, bảo mật, tính ổn định và khả năng mở rộng.

Sau khi xác định phạm vi nghiệp vụ, hệ thống được phân tích theo các nhóm người dùng và chức năng tương ứng. Các đối tượng như quản trị viên, người quản lý và nhân viên được xác định dựa trên trách nhiệm thực tế trong quá trình vận hành. Mỗi nhóm người dùng được gán với một phạm vi chức năng nhất định, qua đó hình thành cơ sở cho việc thiết kế cơ chế xác thực và phân quyền. Các chức năng chính cũng được mô hình hóa để làm rõ mối quan hệ giữa người dùng với chương trình, hạn chế tình trạng bỏ sót yêu cầu hoặc xây dựng chức năng không phù hợp với bài toán ban đầu.

Trong quá trình phân tích, các mô hình trực quan như Use Case, Activity Diagram và Sequence Diagram được sử dụng để biểu diễn chức năng và luồng xử lý. Use Case giúp xác định người dùng nào có thể thực hiện chức năng nào; Activity Diagram thể hiện trình tự các bước của một nghiệp vụ; trong khi Sequence Diagram mô tả quá trình trao đổi dữ liệu giữa giao diện, API, tầng nghiệp vụ và cơ sở dữ liệu. Việc sử dụng các mô hình này giúp quá trình thiết kế trở nên rõ ràng hơn trước khi chuyển sang giai đoạn lập trình.

Tiếp theo là giai đoạn thiết kế kiến trúc hệ thống. Chương trình được tổ chức theo mô hình Client–Server, trong đó ứng dụng desktop đóng vai trò phía client và Java Spring Boot đảm nhiệm phần backend. Client không truy cập trực tiếp cơ sở dữ liệu mà gửi yêu cầu tới backend thông qua RESTful API. Backend tiếp nhận yêu cầu, kiểm tra dữ liệu, thực hiện nghiệp vụ và trao đổi với PostgreSQL trước khi trả kết quả cho ứng dụng. Cách tổ chức này tạo sự tách biệt giữa phần trình bày và phần xử lý, đồng thời giảm sự phụ thuộc trực tiếp của giao diện vào cấu trúc dữ liệu.

Ở phía backend, hệ thống tiếp tục được phân chia thành các tầng có trách nhiệm riêng. Controller đảm nhiệm việc tiếp nhận và phản hồi các yêu cầu HTTP; Service thực hiện các quy tắc và nghiệp vụ của chương trình; Repository phụ trách truy xuất dữ liệu; các Entity biểu diễn dữ liệu dưới dạng đối tượng Java và được ánh xạ tới các bảng trong PostgreSQL. DTO được sử dụng trong quá trình nhận và trả dữ liệu nhằm hạn chế việc sử dụng trực tiếp Entity tại tầng giao tiếp. Việc phân chia này giúp mã nguồn có cấu trúc rõ ràng, giảm sự phụ thuộc giữa các thành phần và thuận lợi hơn khi cần sửa đổi một chức năng.

Song song với thiết kế kiến trúc là thiết kế cơ sở dữ liệu. Trước tiên, các thực thể cần quản lý được xác định dựa trên yêu cầu nghiệp vụ, sau đó phân tích thuộc tính, khóa chính, khóa ngoại và mối quan hệ giữa các thực thể. Sơ đồ ERD được sử dụng để biểu diễn cấu trúc dữ liệu ở mức tổng thể trước khi chuyển thành các bảng trong PostgreSQL. Các ràng buộc dữ liệu được xây dựng nhằm hạn chế dữ liệu không hợp lệ và bảo đảm tính liên kết giữa các bảng. JPA và Hibernate được sử dụng để ánh xạ giữa các đối tượng Java và dữ liệu quan hệ, trong khi JDBC có thể được áp dụng đối với

những trường hợp cần thao tác dữ liệu trực tiếp hoặc thực hiện các truy vấn phù hợp với yêu cầu xử lý.

Đối với thiết kế giao diện, các màn hình được xây dựng dựa trên nhóm chức năng đã xác định trong quá trình phân tích. Các chức năng thường xuyên sử dụng được bố trí theo hướng dễ tìm kiếm và thao tác, trong khi những chức năng phụ thuộc vào quyền người dùng được kiểm soát theo tài khoản đăng nhập. React được sử dụng để tổ chức giao diện theo các component có khả năng tái sử dụng, còn Electron cung cấp môi trường để triển khai giao diện này dưới dạng ứng dụng desktop. Phần giao diện chịu trách nhiệm hiển thị và tiếp nhận thao tác của người dùng, còn các nghiệp vụ quan trọng được xử lý tập trung tại backend.

Quá trình xây dựng chương trình được thực hiện theo từng nhóm chức năng thay vì triển khai toàn bộ hệ thống cùng lúc. Mỗi chức năng được phát triển từ tầng dữ liệu, tầng nghiệp vụ, API đến giao diện, sau đó tiến hành tích hợp để kiểm tra luồng xử lý hoàn chỉnh. Chẳng hạn, đối với chức năng quản lý sản phẩm, trước tiên cần xây dựng cấu trúc dữ liệu và Entity, tiếp đến là Repository, Service và Controller, sau đó mới kết nối API với giao diện desktop. Cách thực hiện này giúp dễ dàng phát hiện lỗi ở từng thành phần và hạn chế việc lỗi của một chức năng ảnh hưởng đến toàn bộ chương trình. Bên cạnh việc phát triển chức năng, kiểm thử được thực hiện xuyên suốt quá trình xây dựng hệ thống. Các API được kiểm tra về dữ liệu đầu vào, kết quả trả về và các trường hợp lỗi; giao diện được kiểm tra về khả năng hiển thị và tương tác; những luồng nghiệp vụ quan trọng như đăng nhập, phân quyền, cập nhật hàng hóa, giao dịch và thay đổi tồn kho được kiểm tra từ đầu đến cuối. Khi phát hiện vấn đề, chức năng được điều chỉnh và kiểm tra lại trước khi tích hợp với các phần còn lại. Phương pháp này giúp giảm số lượng lỗi tích lũy và nâng cao mức độ ổn định của chương trình.

Phương pháp phân tích và xây dựng được lựa chọn giúp quá trình phát triển diễn ra theo trình tự rõ ràng như hình dưới. Mỗi giai đoạn đều tạo ra cơ sở cho giai đoạn tiếp theo, qua đó bảo đảm các thành phần của hệ thống được xây dựng thống nhất và đáp ứng đúng phạm vi nghiệp vụ đã xác định.



Hình 1.1: Quy trình phát triển phần mềm

### 1.2.2. Mô hình Client - Server

Mô hình Client – Server là một trong những mô hình kiến trúc được sử dụng phổ biến trong quá trình xây dựng các hệ thống phần mềm hiện đại. Trong mô hình này, ứng dụng được chia thành hai thành phần chính gồm Client và Server. Hai thành phần đảm nhiệm những nhiệm vụ khác nhau nhưng phối hợp với nhau thông qua cơ chế gửi yêu cầu và nhận phản hồi. Việc phân tách này giúp hệ thống có cấu trúc rõ ràng hơn, thuận lợi cho việc tổ chức mã nguồn, quản lý dữ liệu và mở rộng chức năng.

Client là thành phần trực tiếp tương tác với người dùng. Client chịu trách nhiệm hiển thị giao diện, tiếp nhận các thao tác như đăng nhập, nhập dữ liệu, tìm kiếm, lựa chọn chức năng và hiển thị kết quả trả về từ hệ thống. Trong phạm vi đề tài, ứng dụng desktop được xây dựng bằng React kết hợp Electron và đóng vai trò là Client. Phần này không trực tiếp truy cập cơ sở dữ liệu mà thực hiện trao đổi dữ liệu với Server thông qua các API đã được cung cấp.

Server là thành phần đảm nhiệm việc xử lý nghiệp vụ, kiểm tra dữ liệu, xác thực người dùng, phân quyền và truy xuất cơ sở dữ liệu. Backend của hệ thống được xây dựng bằng Java Spring Boot và đóng vai trò là Server. Khi nhận được yêu cầu từ Client, Server sẽ phân tích nội dung yêu cầu, thực hiện xử lý tương ứng tại các tầng nghiệp vụ, truy vấn hoặc cập nhật dữ liệu trong PostgreSQL, sau đó trả kết quả về cho Client theo một cấu trúc thống nhất.

Quá trình giao tiếp giữa Client và Server được thực hiện thông qua giao thức HTTP. Client gửi các yêu cầu đến các RESTful API bằng những phương thức phổ biến như GET, POST, PUT, PATCH hoặc DELETE tùy theo loại nghiệp vụ. Dữ liệu trao đổi chủ yếu sử dụng định dạng JSON, giúp việc truyền nhận dữ liệu giữa ứng dụng React Electron và Java Spring Boot trở nên thuận tiện, độc lập với ngôn ngữ lập trình được sử dụng ở mỗi phía.

Ví dụ, khi người dùng thực hiện chức năng đăng nhập, Client sẽ tiếp nhận tên đăng nhập và mật khẩu từ giao diện, sau đó gửi một yêu cầu đến API xác thực của Server. Backend tiếp nhận dữ liệu, kiểm tra thông tin tài khoản trong cơ sở dữ liệu, xác thực mật khẩu và xử lý kết quả đăng nhập. Nếu thông tin hợp lệ, Server tạo token xác thực và trả về cho Client. Ứng dụng desktop sử dụng token này trong các yêu cầu tiếp theo nhằm chứng minh danh tính người dùng và hỗ trợ kiểm soát quyền truy cập.

Đối với các nghiệp vụ quản lý dữ liệu, quy trình xử lý cũng tuân theo nguyên tắc tương tự. Khi người dùng yêu cầu xem danh sách sản phẩm, Client gửi yêu cầu đến API sản phẩm. Server tiếp nhận yêu cầu, gọi tầng Service để xử lý nghiệp vụ, sau đó sử dụng Repository để truy vấn dữ liệu từ PostgreSQL. Kết quả được chuyển đổi thành dữ liệu phù hợp và trả về cho Client để hiển thị trên giao diện. Khi người dùng thêm hoặc cập nhật sản phẩm, dữ liệu sẽ đi theo chiều ngược lại từ Client đến Server trước khi được kiểm tra và lưu vào cơ sở dữ liệu.

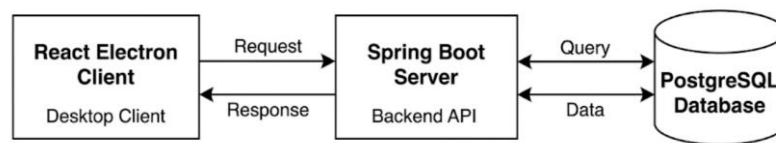
Một đặc điểm quan trọng của mô hình này là Client không cần biết chi tiết cách Server tổ chức dữ liệu và xử lý nghiệp vụ. Client chỉ cần biết địa chỉ API, phương thức gửi yêu cầu, dữ liệu cần truyền và cấu trúc phản hồi. Tương tự, Server không phụ thuộc trực tiếp vào cách giao diện được thiết kế. Sự tách biệt này giúp giảm mức độ liên kết giữa các thành phần và tạo điều kiện thuận lợi cho việc thay đổi hoặc nâng cấp từng phần mà không phải sửa đổi toàn bộ hệ thống.

Mô hình Client – Server cũng hỗ trợ tốt cho vấn đề bảo mật. Các thao tác liên quan đến dữ liệu quan trọng không được xử lý hoàn toàn tại phía Client mà được kiểm tra lại tại Server. Backend có thể thực hiện xác thực token, kiểm tra vai trò và quyền của tài khoản trước khi cho phép truy cập vào một chức năng cụ thể. Cơ chế này giúp

hạn chế trường hợp người dùng can thiệp trực tiếp vào giao diện để thực hiện các thao tác vượt quá quyền hạn được cấp.

Bên cạnh đó, việc tập trung xử lý dữ liệu tại Server giúp bảo đảm tính nhất quán của hệ thống. Các quy tắc nghiệp vụ như kiểm tra dữ liệu hợp lệ, cập nhật tồn kho, xác định trạng thái giao dịch hoặc kiểm tra quyền truy cập được thực hiện tại một nơi thống nhất. Nhờ đó, dữ liệu gửi từ các màn hình khác nhau vẫn tuân theo cùng một bộ quy tắc xử lý, giảm nguy cơ phát sinh sai lệch giữa các chức năng.

Trong hệ thống quản lý trang sức, mô hình Client – Server có thể được mô tả theo luồng tổng quát sau:



Hình 1.2: Sơ đồ luồng hoạt động tổng quát

Khi có dữ liệu trả về, quá trình được thực hiện theo chiều ngược lại từ PostgreSQL qua các tầng của Backend, sau đó Server tạo Response và gửi tới ứng dụng desktop để hiển thị cho người dùng. Cách tổ chức này giúp mỗi thành phần chỉ tập trung vào nhiệm vụ chính của mình: giao diện đảm nhiệm tương tác, Backend xử lý nghiệp vụ và cơ sở dữ liệu thực hiện lưu trữ.

Việc lựa chọn mô hình Client – Server phù hợp với yêu cầu của hệ thống vì cho phép tách biệt rõ ràng giữa giao diện desktop và phần xử lý bằng Java. Đồng thời, mô hình này tạo nền tảng thuận lợi cho việc bảo trì và mở rộng. Trong trường hợp cần phát triển thêm ứng dụng web hoặc mobile trong tương lai, các ứng dụng mới có thể tiếp tục sử dụng hệ thống RESTful API hiện có mà không cần xây dựng lại toàn bộ phần xử lý nghiệp vụ.

### 1.2.3. Kiến trúc RESTful API

RESTful API là một phương thức tổ chức giao tiếp giữa các thành phần phần mềm dựa trên các nguyên tắc của kiến trúc REST (Representational State Transfer). Trong mô hình này, các tài nguyên của hệ thống được xác định thông qua các địa chỉ

URI và được thao tác bằng những phương thức chuẩn của giao thức HTTP. RESTful API thường được sử dụng để kết nối giữa ứng dụng phía người dùng và hệ thống backend do có cấu trúc rõ ràng, dễ mở rộng và không phụ thuộc vào công nghệ được sử dụng ở mỗi phía.

Trong hệ thống, RESTful API đóng vai trò là lớp giao tiếp giữa ứng dụng desktop được xây dựng bằng React kết hợp Electron và backend Java Spring Boot. Ứng dụng desktop không truy cập trực tiếp vào PostgreSQL mà gửi các yêu cầu HTTP tới những API do backend cung cấp. Backend tiếp nhận yêu cầu, kiểm tra dữ liệu, thực hiện nghiệp vụ cần thiết và truy xuất cơ sở dữ liệu trước khi trả kết quả về cho Client. Cách tổ chức này giúp tách biệt phần giao diện, xử lý nghiệp vụ và lưu trữ dữ liệu.

Một trong những đặc điểm quan trọng của REST là việc xem dữ liệu trong hệ thống dưới dạng các tài nguyên (Resource). Mỗi tài nguyên được biểu diễn bằng một URI phù hợp với ý nghĩa của dữ liệu. Chẳng hạn, tài nguyên liên quan đến sản phẩm có thể được tổ chức dưới đường dẫn `/api/products`, tài nguyên khách hàng sử dụng `/api/customers`, trong khi giao dịch có thể được truy cập thông qua `/api/transactions`. Việc tổ chức URI theo tài nguyên giúp cấu trúc API có tính thống nhất và hỗ trợ lập trình viên dễ dàng xác định chức năng của từng endpoint.

Các thao tác trên tài nguyên được thực hiện thông qua phương thức HTTP. Phương thức GET thường được sử dụng để lấy dữ liệu; POST dùng để tạo mới dữ liệu; PUT hoặc PATCH hỗ trợ cập nhật thông tin; trong khi DELETE được sử dụng để thực hiện thao tác xóa tài nguyên khi nghiệp vụ cho phép. Việc sử dụng đúng ý nghĩa của các phương thức HTTP giúp API dễ đọc và phản ánh rõ mục đích của từng yêu cầu.

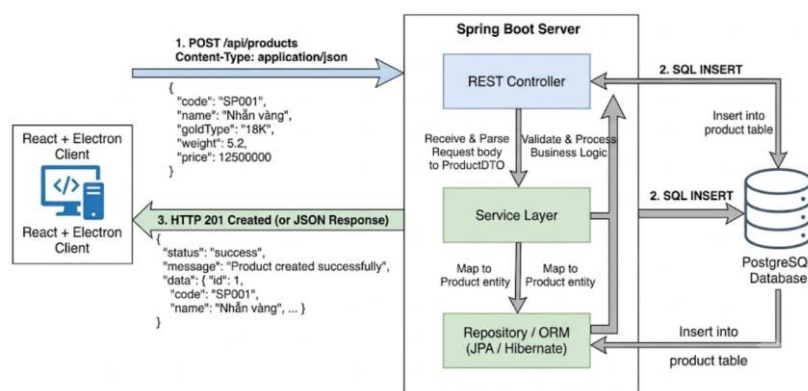
Ví dụ, đối với chức năng quản lý sản phẩm, ứng dụng có thể gửi yêu cầu:

**GET /api/products**

để lấy danh sách sản phẩm. Khi cần lấy thông tin của một sản phẩm cụ thể, Client có thể gửi:

**GET /api/products/1001**

Khi tạo mới sản phẩm, dữ liệu được gửi đến Server bằng:



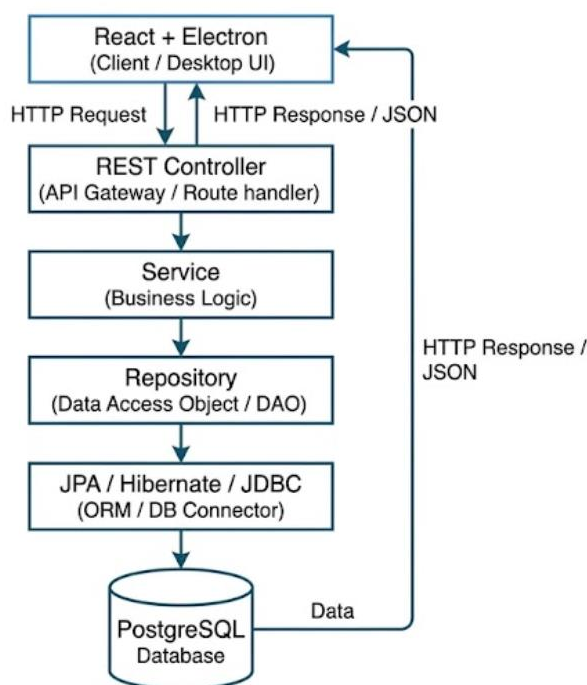
Hình 1.3: Sơ đồ luồng dữ liệu tạo mới sản phẩm

Backend tiếp nhận dữ liệu, thực hiện kiểm tra tính hợp lệ, xử lý các quy tắc nghiệp vụ và lưu dữ liệu vào PostgreSQL thông qua tầng Repository. Kết quả sau đó được đóng gói thành Response và gửi trở lại ứng dụng desktop.

Dữ liệu trao đổi giữa Client và Server chủ yếu sử dụng định dạng JSON (JavaScript Object Notation). Đây là định dạng dữ liệu dạng văn bản có cấu trúc đơn giản, hỗ trợ tốt trên nhiều ngôn ngữ lập trình và phù hợp với việc truyền dữ liệu qua HTTP. React có thể dễ dàng chuyển đổi dữ liệu JSON thành các đối tượng để hiển thị trên giao diện, trong khi Spring Boot hỗ trợ tự động chuyển đổi giữa JSON và các đối tượng Java thông qua cơ chế serialization và deserialization.

Ở phía backend, các yêu cầu RESTful API được tiếp nhận tại tầng Controller. Controller chịu trách nhiệm xác định endpoint, HTTP Method, tham số và dữ liệu đầu vào của yêu cầu. Sau đó, dữ liệu được chuyển tới tầng Service để thực hiện nghiệp vụ. Service tiếp tục sử dụng Repository để truy xuất hoặc cập nhật dữ liệu trong PostgreSQL. Kết quả được trả theo chiều ngược lại từ Repository đến Service, Controller và cuối cùng là Client.

Luồng xử lý một yêu cầu API có thể biểu diễn như sau:



Hình 1.4: Sơ đồ luồng dữ liệu

Bên cạnh dữ liệu phản hồi, RESTful API sử dụng HTTP Status Code để thể hiện trạng thái xử lý của yêu cầu. Chẳng hạn, mã 200 OK cho biết yêu cầu được thực hiện thành công, 201 Created được sử dụng khi tài nguyên mới được tạo, 400 Bad Request thể hiện dữ liệu yêu cầu không hợp lệ, 401 Unauthorized liên quan đến trường hợp chưa được xác thực, 403 Forbidden biểu thị người dùng không có quyền thực hiện chức năng và 404 Not Found được sử dụng khi tài nguyên yêu cầu không tồn tại. Đối với những lỗi phát sinh bên trong Server, hệ thống có thể trả về 500 Internal Server Error.

Việc sử dụng đúng HTTP Status Code giúp Client xác định được kết quả của từng yêu cầu mà không cần phụ thuộc hoàn toàn vào nội dung thông báo. Ứng dụng desktop có thể dựa vào trạng thái nhận được để quyết định cách phản hồi tới người dùng, chẳng hạn hiển thị dữ liệu khi yêu cầu thành công, yêu cầu đăng nhập lại khi thông tin xác thực không còn hợp lệ hoặc hiển thị thông báo khi người dùng không có quyền thao tác.

Một nguyên tắc quan trọng khác của REST là Stateless. Theo nguyên tắc này, Server không phụ thuộc vào trạng thái của các yêu cầu trước đó để xử lý yêu cầu hiện tại. Mỗi request gửi tới Server cần chứa đủ thông tin cần thiết để backend xác định

người gửi yêu cầu và thực hiện nghiệp vụ. Trong hệ thống có xác thực bằng JWT, Client gửi token kèm theo các request cần bảo vệ thông qua HTTP Header:

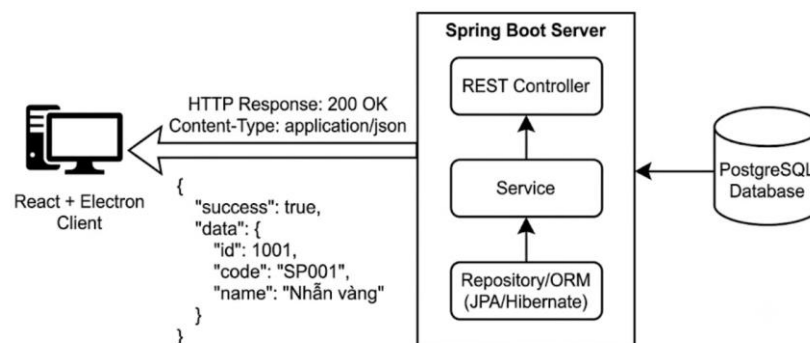
**Authorization: Bearer <token>**

Khi nhận được yêu cầu, backend tiến hành kiểm tra token, xác định tài khoản và quyền truy cập trước khi chuyển yêu cầu tới chức năng nghiệp vụ tương ứng. Cách tổ chức này giúp việc xác thực phù hợp với đặc điểm Stateless của REST và giảm sự phụ thuộc vào trạng thái phiên được lưu trên Server.

RESTful API cũng hỗ trợ tốt cơ chế phân quyền truy cập. Mặc dù giao diện có thể ẩn các chức năng mà người dùng không được phép sử dụng, Server vẫn phải kiểm tra quyền đối với từng API. Điều này ngăn trường hợp người dùng cố tình gửi yêu cầu trực tiếp đến một endpoint mà tài khoản không có quyền truy cập. Việc kiểm tra tại backend giúp cơ chế bảo mật không phụ thuộc vào giao diện phía Client.

Ngoài ra, dữ liệu gửi tới API cần được kiểm tra trước khi tiến hành xử lý. Spring Boot cung cấp cơ chế Validation để kiểm tra các điều kiện như trường bắt buộc, độ dài chuỗi, định dạng dữ liệu hoặc phạm vi giá trị. Nếu dữ liệu không đáp ứng yêu cầu, Server trả về phản hồi lỗi phù hợp thay vì thực hiện nghiệp vụ với dữ liệu không hợp lệ. Cơ chế này góp phần bảo đảm tính chính xác và nhất quán của dữ liệu được lưu trữ trong cơ sở dữ liệu.

Để thuận tiện cho Client trong quá trình xử lý, cấu trúc Response nên được chuẩn hóa giữa các API. Ví dụ, đối với một yêu cầu thành công, Server có thể trả về:



Hình 1.5: Sơ đồ phản hồi từ server

Đối với trường hợp xảy ra lỗi, thông tin trả về cần thể hiện rõ loại lỗi, trạng thái và nội dung liên quan để Client có thể xử lý thống nhất. Việc sử dụng chung một quy ước phản hồi giúp giảm sự khác biệt giữa các module và hỗ trợ quá trình phát triển giao diện thuận lợi hơn.

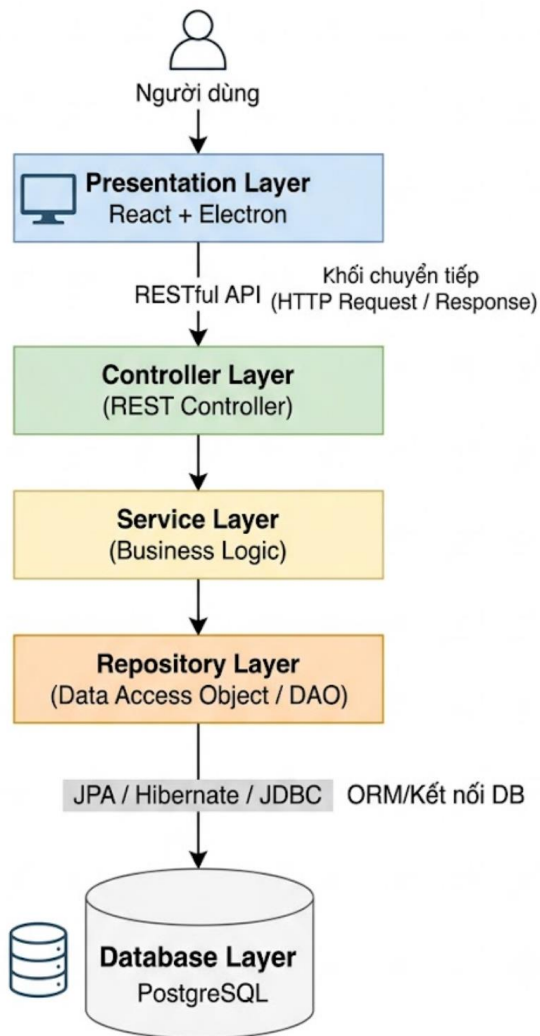
Trong phạm vi hệ thống quản lý trang sức, kiến trúc RESTful API tạo ra ranh giới rõ ràng giữa ứng dụng desktop và backend. React Electron tập trung vào việc hiển thị dữ liệu và tiếp nhận thao tác của người dùng, trong khi Spring Boot đảm nhiệm xử lý nghiệp vụ, xác thực, phân quyền và làm việc với cơ sở dữ liệu. Việc giao tiếp thông qua các endpoint thống nhất giúp các thành phần có thể được phát triển tương đối độc lập, đồng thời tạo điều kiện thuận lợi cho quá trình bảo trì và mở rộng hệ thống sau này.

#### ***1.2.4. Mô hình kiến trúc nhiều tầng***

Mô hình kiến trúc nhiều tầng là phương pháp tổ chức hệ thống phần mềm bằng cách phân chia chương trình thành các tầng có nhiệm vụ và phạm vi xử lý riêng biệt. Mỗi tầng đảm nhận một nhóm chức năng cụ thể và chỉ tương tác với các tầng có liên quan thông qua những quy tắc đã được xác định. Cách tổ chức này giúp hạn chế tình trạng toàn bộ nghiệp vụ, giao diện và thao tác dữ liệu được xử lý chung trong một thành phần, từ đó làm cho hệ thống dễ phát triển, kiểm thử, bảo trì và mở rộng hơn.

Trong quá trình xây dựng ứng dụng, kiến trúc nhiều tầng được áp dụng chủ yếu ở phía backend Java Spring Boot. Các chức năng xử lý không được thực hiện trực tiếp tại một lớp duy nhất mà được phân chia thành các tầng như Controller, Service, Repository và tầng dữ liệu. Bên cạnh đó, ứng dụng desktop React kết hợp Electron đóng vai trò là tầng giao diện, chịu trách nhiệm tiếp nhận thao tác từ người dùng và trao đổi dữ liệu với backend thông qua RESTful API.

Có thể biểu diễn cấu trúc tổng quát của hệ thống theo mô hình sau:



Hình 1.6: Sơ đồ kiến trúc phân tầng hệ thống

Tầng Presentation là tầng trực tiếp tương tác với người dùng. Trong hệ thống, tầng này được xây dựng bằng React và Electron. Nhiệm vụ chính của tầng giao diện là hiển thị thông tin, tiếp nhận dữ liệu nhập vào và gửi các yêu cầu đến backend. Các nghiệp vụ quan trọng không được xử lý trực tiếp tại tầng này mà được chuyển đến server thông qua các API. Điều này giúp giao diện tập trung vào nhiệm vụ trình bày dữ liệu và xử lý tương tác thay vì chứa các quy tắc nghiệp vụ phức tạp.

Ví dụ, khi người dùng thực hiện chức năng thêm mới một sản phẩm trang sức, giao diện tiếp nhận các thông tin như mã sản phẩm, tên sản phẩm, loại vàng, trọng lượng và giá bán. Sau khi người dùng xác nhận, ứng dụng gửi dữ liệu đến API tương

ứng. Việc kiểm tra dữ liệu, xác định sản phẩm có hợp lệ hay không và thực hiện thao tác lưu trữ được xử lý phía backend thay vì thực hiện hoàn toàn tại giao diện.

Tầng Controller là điểm tiếp nhận các yêu cầu HTTP từ ứng dụng phía Client. Trong Spring Boot, Controller thường được xây dựng bằng các lớp sử dụng các annotation như `@RestController`, `@RequestMapping`, `@GetMapping` hoặc `@PostMapping`. Tầng này chịu trách nhiệm xác định endpoint, phương thức HTTP, tiếp nhận dữ liệu đầu vào và chuyển yêu cầu đến tầng xử lý nghiệp vụ.

Controller không nên chứa quá nhiều logic nghiệp vụ. Vai trò chính của tầng này là làm trung gian giữa Client và Service. Sau khi nhận được kết quả xử lý từ Service, Controller tạo phản hồi thích hợp và gửi trở lại ứng dụng desktop. Việc giữ Controller ở mức đơn giản giúp mã nguồn dễ theo dõi và tránh tình trạng các quy tắc nghiệp vụ bị phân tán tại nhiều endpoint khác nhau.

Tầng Service là nơi tập trung phần lớn các quy tắc và logic nghiệp vụ của hệ thống. Các yêu cầu sau khi được Controller tiếp nhận sẽ được chuyển đến Service tương ứng để xử lý. Tại đây, chương trình có thể thực hiện các công việc như kiểm tra điều kiện nghiệp vụ, xác định trạng thái dữ liệu, kiểm tra quyền thực hiện chức năng, tính toán giá trị hoặc phối hợp nhiều Repository trong cùng một thao tác.

Chẳng hạn, trong nghiệp vụ bán hàng, Service có thể thực hiện nhiều bước liên tiếp như kiểm tra sản phẩm có tồn tại hay không, kiểm tra trạng thái hoặc số lượng tồn, lấy bảng giá đang áp dụng, tính giá trị giao dịch, lưu thông tin giao dịch và cập nhật lại dữ liệu tồn kho. Nếu toàn bộ các thao tác này được đặt trực tiếp tại Controller thì mã nguồn sẽ trở nên khó quản lý. Việc đưa chúng vào Service giúp nghiệp vụ được tổ chức tập trung và có thể tái sử dụng trong nhiều chức năng khác nhau.

Tầng Repository đảm nhận nhiệm vụ truy xuất và thao tác dữ liệu. Trong hệ thống sử dụng Spring Data JPA, các Repository đóng vai trò kết nối giữa tầng nghiệp vụ và cơ sở dữ liệu. Thông qua Repository, Service có thể thực hiện các thao tác như tìm kiếm, thêm mới, cập nhật hoặc xóa dữ liệu mà không cần trực tiếp viết toàn bộ quá trình kết nối với PostgreSQL.

Đối với các truy vấn phổ biến, Spring Data JPA có thể cung cấp sẵn nhiều phương thức thao tác dữ liệu. Với các trường hợp cần truy vấn phức tạp hơn, hệ thống có thể sử dụng JPQL, Native Query hoặc JDBC tùy theo yêu cầu. Việc tập trung các thao tác dữ liệu tại Repository giúp tách biệt phần truy xuất dữ liệu khỏi logic nghiệp vụ.

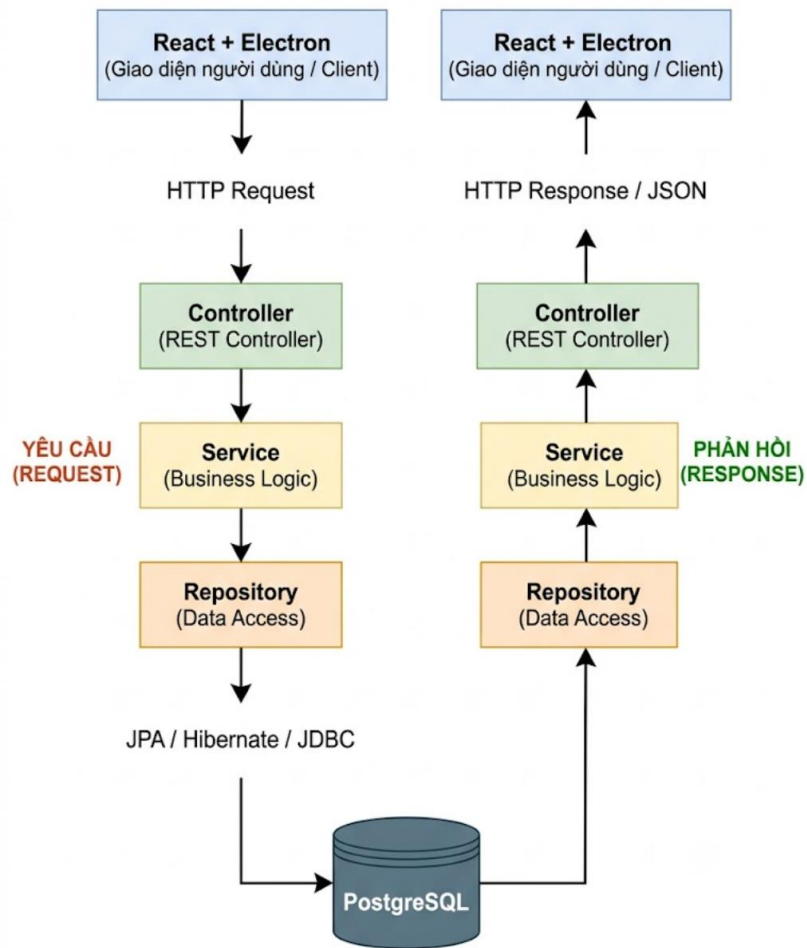
Tầng Database là nơi lưu trữ dữ liệu lâu dài của hệ thống. PostgreSQL được sử dụng để quản lý các dữ liệu liên quan đến tài khoản, vai trò, quyền truy cập, nhân viên, khách hàng, sản phẩm, kho, bảng giá và giao dịch. Các bảng được liên kết với nhau bằng khóa chính, khóa ngoại và các ràng buộc nhằm bảo đảm tính toàn vẹn của dữ liệu.

JPA và Hibernate đóng vai trò hỗ trợ ánh xạ dữ liệu giữa các bảng trong PostgreSQL và các đối tượng Java. Một bảng trong cơ sở dữ liệu có thể được biểu diễn bằng một Entity tương ứng trong chương trình. Các trường dữ liệu được ánh xạ với thuộc tính của Entity, trong khi các mối quan hệ giữa các bảng có thể được mô hình hóa thông qua các annotation của JPA.

Ngoài các tầng chính, hệ thống còn sử dụng một số thành phần hỗ trợ để tránh làm các tầng nghiệp vụ phụ thuộc trực tiếp vào nhau. DTO được sử dụng để truyền dữ liệu giữa Client và backend, hạn chế việc trả trực tiếp Entity ra ngoài API. Các lớp xử lý Exception được tổ chức riêng nhằm thống nhất cách phản hồi lỗi. Các thành phần Security đảm nhiệm xác thực và kiểm tra quyền truy cập trước khi yêu cầu được chuyển đến chức năng nghiệp vụ phù hợp.

Luồng xử lý một yêu cầu trong kiến trúc nhiều tầng có thể mô tả thông qua chức năng xem danh sách sản phẩm. Khi người dùng mở màn hình quản lý sản phẩm, React Electron gửi yêu cầu GET đến API sản phẩm. Controller tiếp nhận yêu cầu và gọi Service tương ứng. Service thực hiện các xử lý cần thiết rồi gọi Repository để truy vấn dữ liệu. Repository sử dụng JPA hoặc JDBC để làm việc với PostgreSQL. Dữ liệu sau khi được lấy từ cơ sở dữ liệu sẽ đi theo chiều ngược lại từ Repository đến Service, Controller và cuối cùng được trả về Client dưới dạng JSON.

Quá trình này có thể biểu diễn như sau:



Hình 1.7: Sơ đồ luồng yêu cầu và phản hồi

Một lợi ích quan trọng của mô hình kiến trúc nhiều tầng là khả năng phân tách trách nhiệm. Khi cần thay đổi giao diện, các thay đổi chủ yếu được thực hiện tại phía Client mà không nhất thiết phải thay đổi toàn bộ nghiệp vụ. Khi cần điều chỉnh quy tắc xử lý, lập trình viên có thể tập trung tại Service. Tương tự, khi thay đổi cách truy vấn dữ liệu, các chỉnh sửa chủ yếu được thực hiện tại Repository mà không ảnh hưởng trực tiếp đến giao diện.

Kiến trúc này cũng hỗ trợ tốt cho quá trình kiểm thử. Các tầng có thể được kiểm tra độc lập hoặc kết hợp tùy theo mục tiêu. Service có thể được kiểm thử dựa trên các tình huống nghiệp vụ, Controller có thể được kiểm tra thông qua API, trong khi Repository có thể được kiểm tra khả năng truy vấn dữ liệu. Việc phân chia trách nhiệm rõ ràng giúp xác định nguyên nhân lỗi dễ dàng hơn khi một chức năng hoạt động không đúng yêu cầu.

Ngoài khả năng bảo trì, mô hình nhiều tầng còn tạo điều kiện cho hệ thống mở rộng về sau. Khi cần bổ sung một chức năng mới, lập trình viên có thể xây dựng thêm các thành phần tương ứng tại từng tầng mà không phải thay đổi toàn bộ cấu trúc hiện có. Điều này phù hợp với các hệ thống quản lý có số lượng chức năng tăng dần trong quá trình phát triển.

Việc áp dụng kiến trúc nhiều tầng giúp hệ thống được tổ chức theo một luồng xử lý thống nhất từ giao diện đến cơ sở dữ liệu. Mỗi tầng tập trung vào một nhiệm vụ riêng và phối hợp với các tầng khác thông qua các giao diện đã được xác định. Cách tiếp cận này góp phần làm cho mã nguồn rõ ràng hơn, giảm sự phụ thuộc giữa các thành phần và tạo nền tảng thuận lợi cho quá trình phát triển, kiểm thử và bảo trì ứng dụng.

#### **1.2.5. Công cụ phát triển Backend**

Trong quá trình xây dựng Backend, việc lựa chọn công cụ phát triển phù hợp có vai trò quan trọng đối với khả năng tổ chức mã nguồn, quản lý thư viện, kiểm thử và triển khai hệ thống. Các công cụ được sử dụng cần đáp ứng tốt môi trường Java, hỗ trợ Spring Boot và tạo điều kiện thuận lợi cho quá trình làm việc với cơ sở dữ liệu PostgreSQL.



Hình 1.8: IntelliJ IDEA công cụ hỗ trợ phát triển phần mềm

IntelliJ IDEA được sử dụng làm môi trường phát triển chính cho phần Backend. Đây là một IDE hỗ trợ mạnh cho Java và hệ sinh thái Spring, cung cấp nhiều chức năng phục vụ quá trình lập trình như gợi ý mã nguồn, kiểm tra cú pháp, tìm kiếm lớp và phương thức, tái cấu trúc mã nguồn, quản lý cấu trúc dự án, chạy chương trình và gỡ lỗi. Khả năng hỗ trợ trực tiếp Spring Boot giúp việc phát triển các thành phần như Controller, Service, Repository, Entity và các lớp cấu hình trở nên thuận tiện hơn.

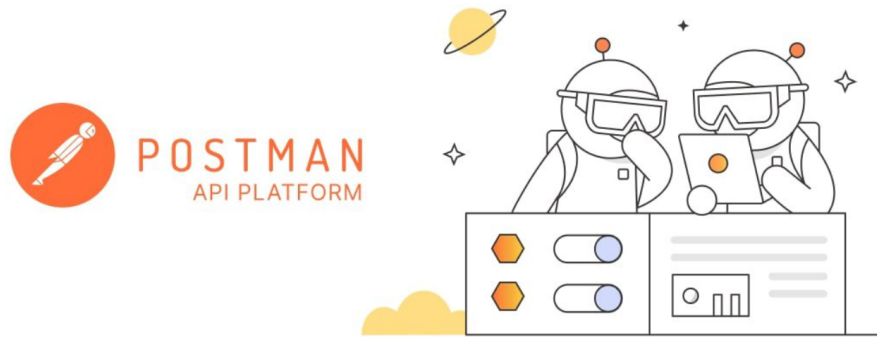
Trong quá trình phát triển, chức năng Debug của IntelliJ IDEA được sử dụng để theo dõi luồng xử lý của chương trình và xác định nguyên nhân khi xảy ra lỗi. Lập trình viên có thể kiểm tra giá trị biến, trạng thái đối tượng và quá trình xử lý giữa các tầng của hệ thống. Điều này đặc biệt hữu ích đối với những nghiệp vụ có nhiều bước xử lý hoặc liên quan đến việc truy xuất dữ liệu từ cơ sở dữ liệu.

Java Development Kit là nền tảng cần thiết để biên dịch và thực thi mã nguồn Java. JDK cung cấp trình biên dịch, môi trường chạy và các thư viện nền tảng phục vụ quá trình xây dựng ứng dụng. Phiên bản JDK được lựa chọn cần tương thích với phiên bản Spring Boot được sử dụng để bảo đảm tính ổn định trong quá trình phát triển và triển khai.

Gradle được sử dụng làm công cụ quản lý dependency và tự động hóa quá trình build của Backend. Thông qua tệp cấu hình của Gradle, các thư viện cần thiết cho dự án được khai báo và quản lý tập trung. Các thành phần như Spring Web, Spring Data JPA, Spring Security, Validation và PostgreSQL Driver có thể được cấu hình trong cùng một hệ thống quản lý dependency, giúp hạn chế việc bổ sung thư viện thủ công và giảm nguy cơ xảy ra xung đột phiên bản.

Bên cạnh việc quản lý thư viện, Gradle còn đảm nhiệm các tác vụ như biên dịch mã nguồn, chạy kiểm thử và đóng gói ứng dụng. Việc sử dụng Gradle Wrapper giúp dự án duy trì phiên bản Gradle thống nhất giữa các môi trường phát triển, từ đó hạn chế sự khác biệt khi chạy chương trình trên nhiều máy khác nhau. Đây là yếu tố cần thiết đối với quá trình phát triển và bảo trì dự án có nhiều thành phần phụ thuộc.

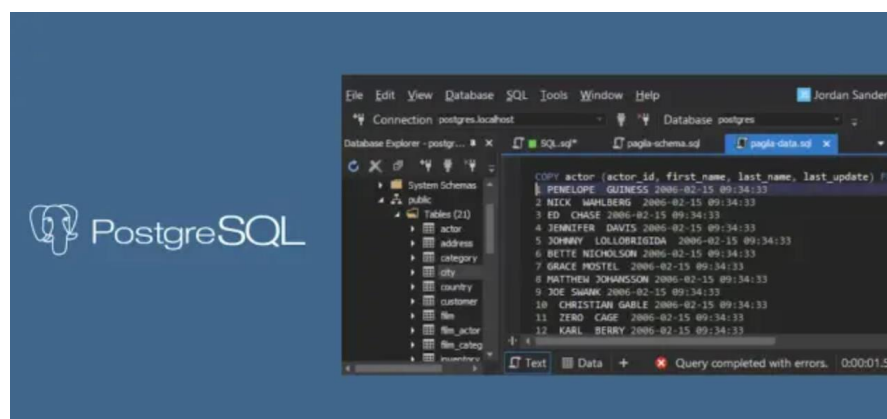
Spring Initializr được sử dụng trong giai đoạn khởi tạo dự án Spring Boot. Công cụ này cho phép lựa chọn hệ thống build, phiên bản Java, phiên bản Spring Boot và các dependency cần thiết trước khi tạo cấu trúc dự án ban đầu. Việc khởi tạo dự án thông qua Spring Initializr giúp giảm thời gian cấu hình và tạo ra cấu trúc phù hợp với cách tổ chức tiêu chuẩn của Spring Boot.



Hình 1.9: Postman công cụ hỗ trợ kiểm thử API

Postman được sử dụng để kiểm tra các RESTful API được xây dựng tại Backend. Công cụ này cho phép gửi các yêu cầu HTTP đến Server và kiểm tra kết quả trả về mà không cần phụ thuộc vào ứng dụng desktop. Qua đó, các chức năng như đăng nhập, quản lý sản phẩm, quản lý kho, quản lý nhân viên, giao dịch và phân quyền có thể được kiểm tra độc lập trước khi tích hợp với giao diện.

Trong quá trình kiểm thử API, Postman hỗ trợ theo dõi HTTP Status Code, nội dung dữ liệu phản hồi và các thông tin trong Header. Đối với những API có yêu cầu xác thực, token có thể được gửi kèm theo Request để kiểm tra khả năng truy cập theo quyền của người dùng. Việc kiểm tra độc lập Backend giúp phát hiện lỗi sớm và giảm số lượng vấn đề phát sinh trong giai đoạn tích hợp với ứng dụng desktop.



Hình 1.10: Hệ quản trị cơ sở dữ liệu mã nguồn mở PostgreSQL

Đối với cơ sở dữ liệu, PostgreSQL được sử dụng làm hệ quản trị cơ sở dữ liệu chính, trong khi pgAdmin hỗ trợ quá trình quản trị và kiểm tra dữ liệu. Thông qua

pgAdmin, có thể theo dõi cấu trúc database, schema, bảng, khóa chính, khóa ngoại, constraint và dữ liệu được lưu trữ. Công cụ này cũng hỗ trợ thực thi các câu lệnh SQL khi cần kiểm tra trực tiếp kết quả của các thao tác dữ liệu.

Việc sử dụng pgAdmin đặc biệt hữu ích khi cần đối chiếu dữ liệu giữa kết quả do API trả về và dữ liệu thực tế trong cơ sở dữ liệu. Trong quá trình phát triển các chức năng liên quan đến JPA, Hibernate hoặc JDBC, dữ liệu có thể được kiểm tra trực tiếp để xác định quá trình ánh xạ và lưu trữ có diễn ra đúng với thiết kế hay không.



Hình 1.11: GitHub công cụ lưu trữ mã nguồn

Git được sử dụng để quản lý phiên bản mã nguồn trong suốt quá trình phát triển. Các thay đổi được lưu lại theo từng phiên bản, giúp theo dõi lịch sử chỉnh sửa và hỗ trợ khôi phục mã nguồn khi cần thiết. Đối với một dự án có nhiều module và nhiều giai đoạn phát triển, việc quản lý phiên bản giúp giảm nguy cơ mất mã nguồn và tạo điều kiện thuận lợi cho việc kiểm soát các thay đổi.

Các công cụ trên được kết hợp để hình thành một môi trường phát triển Backend tương đối đầy đủ. IntelliJ IDEA đảm nhiệm quá trình viết và gỡ lỗi mã nguồn, JDK cung cấp môi trường Java, Gradle quản lý thư viện và quá trình build, Postman hỗ trợ kiểm thử API, pgAdmin hỗ trợ kiểm tra cơ sở dữ liệu và Git đảm nhiệm quản lý phiên bản. Việc sử dụng đồng bộ các công cụ này giúp quá trình phát triển Backend được tổ chức rõ ràng và thuận lợi hơn trong các giai đoạn xây dựng, kiểm thử và hoàn thiện hệ thống.

### 1.2.6. Công cụ phát triển Desktop

Phần ứng dụng Desktop được xây dựng theo hướng kết hợp React với Electron nhằm tạo ra giao diện có khả năng chạy trực tiếp trên hệ điều hành máy tính. Quá trình phát triển yêu cầu các công cụ hỗ trợ viết mã nguồn giao diện, quản lý thư viện JavaScript, chạy thử ứng dụng và kiểm tra quá trình trao đổi dữ liệu giữa Desktop Client với Backend. Việc lựa chọn công cụ phù hợp giúp quá trình phát triển giao diện được tổ chức thuận tiện hơn, đồng thời hỗ trợ phát hiện và xử lý lỗi trong giai đoạn tích hợp hệ thống.



Hình 1.12: Visual Studio Code công cụ lập trình

Visual Studio Code được sử dụng làm môi trường phát triển chính cho phần Desktop. Đây là trình soạn thảo mã nguồn hỗ trợ tốt JavaScript, TypeScript, React và các công nghệ phát triển giao diện hiện đại. Công cụ cung cấp khả năng gợi ý mã nguồn, kiểm tra cú pháp, quản lý thư mục dự án, tìm kiếm và thay thế nội dung, tích hợp Terminal cũng như hỗ trợ cài đặt các extension phục vụ quá trình lập trình.

Trong quá trình xây dựng giao diện, Visual Studio Code giúp tổ chức các thành phần của ứng dụng thành những thư mục riêng như components, pages, layouts, services và các thành phần dùng chung. Cách tổ chức này tạo điều kiện thuận lợi cho việc phân chia trách nhiệm giữa các phần của giao diện, hạn chế việc tập trung toàn bộ mã nguồn vào một khu vực và hỗ trợ tái sử dụng các component trong nhiều màn hình khác nhau.



Hình 1.13: NodeJs nền tảng xây dựng ứng dụng

Node.js được sử dụng làm môi trường thực thi cần thiết cho quá trình phát triển React và Electron. Node.js cho phép sử dụng hệ sinh thái các thư viện JavaScript phía ngoài trình duyệt, đồng thời cung cấp môi trường để thực hiện các tác vụ liên quan đến cài đặt dependency, chạy dự án, build và đóng gói ứng dụng. Đây là thành phần quan trọng trong quá trình thiết lập môi trường phát triển Desktop.

Cùng với Node.js, npm được sử dụng để quản lý các thư viện và dependency của ứng dụng. Các thành phần cần thiết như React, React DOM, Electron, thư viện định tuyến, thư viện giao tiếp HTTP và các gói hỗ trợ giao diện được quản lý thông qua tệp cấu hình của dự án. Việc quản lý dependency tập trung giúp xác định rõ những thư viện mà ứng dụng cần sử dụng và tạo điều kiện thuận lợi khi cài đặt lại môi trường trên máy tính khác.



Hình 1.14: React Native xây dựng ứng dụng đa nền tảng

React được sử dụng để xây dựng phần giao diện người dùng theo mô hình component. Các thành phần như thanh điều hướng, bảng dữ liệu, biểu mẫu nhập liệu, hộp thoại, khu vực tìm kiếm và các màn hình nghiệp vụ có thể được chia thành những component độc lập. Việc tổ chức giao diện theo component giúp giảm sự lặp lại của

mã nguồn, tăng khả năng tái sử dụng và thuận tiện hơn khi cần điều chỉnh giao diện trong quá trình phát triển.



Hình 1.15: Lập trình ứng dụng desktop đa nền tảng

Electron cung cấp môi trường để đưa giao diện React vào ứng dụng Desktop. Electron kết hợp môi trường trình duyệt với khả năng hoạt động của Node.js, cho phép ứng dụng web được đóng gói và chạy dưới dạng chương trình độc lập trên hệ điều hành. Trong cấu trúc Electron, Main Process chịu trách nhiệm quản lý vòng đời ứng dụng và cửa sổ chương trình, trong khi Renderer Process đảm nhiệm hiển thị giao diện React và xử lý tương tác của người dùng.

Preload Script được sử dụng làm thành phần trung gian giữa Main Process và Renderer Process trong những trường hợp cần truy cập các chức năng của Electron hoặc hệ điều hành. Việc tách biệt thành phần này giúp hạn chế việc để giao diện truy cập trực tiếp vào các API nhạy cảm của Node.js, từ đó nâng cao mức độ an toàn cho ứng dụng Desktop.

Ứng dụng Desktop giao tiếp với Backend thông qua các RESTful API. Các yêu cầu từ giao diện được gửi đến Java Spring Boot qua giao thức HTTP, sau đó dữ liệu phản hồi được nhận và hiển thị trên các màn hình tương ứng. Các thư viện HTTP phía Client có thể được sử dụng để tổ chức quá trình gửi Request, nhận Response, xử lý lỗi và bổ sung thông tin xác thực vào Header của những yêu cầu cần bảo vệ.

Trong quá trình chạy thử ứng dụng, công cụ Developer Tools được sử dụng để kiểm tra giao diện và theo dõi hoạt động của Renderer Process. Công cụ này hỗ trợ quan sát cấu trúc giao diện, kiểm tra CSS, theo dõi thông tin được ghi ra Console và kiểm tra các yêu cầu mạng được gửi tới Backend. Khi xảy ra lỗi trong quá trình gọi API, các thông tin về URL, HTTP Method, Status Code và dữ liệu phản hồi có thể được kiểm tra trực tiếp để xác định nguyên nhân.

React Developer Tools cũng hỗ trợ việc theo dõi cấu trúc component và trạng thái của giao diện trong quá trình phát triển. Đối với những màn hình có nhiều component hoặc dữ liệu thay đổi theo thao tác của người dùng, công cụ này giúp kiểm tra quá trình cập nhật state và xác định các thành phần đang được render. Qua đó, việc xử lý các lỗi liên quan đến hiển thị hoặc quản lý trạng thái trở nên thuận tiện hơn.

Quá trình phát triển Desktop không chỉ tập trung vào việc xây dựng giao diện mà còn bao gồm việc kiểm tra khả năng tích hợp với Backend. Trong giai đoạn đầu, các màn hình có thể được xây dựng và kiểm tra độc lập với dữ liệu mẫu. Sau khi các RESTful API được hoàn thiện, phần giao diện được kết nối với Server để nhận dữ liệu thực tế. Các trường hợp như mất kết nối, dữ liệu đầu vào không hợp lệ, phiên đăng nhập hết hiệu lực hoặc người dùng không có quyền truy cập cũng cần được xử lý tại giao diện để bảo đảm trải nghiệm sử dụng ổn định.

Khi hoàn thiện các chức năng, Electron hỗ trợ quá trình đóng gói ứng dụng thành chương trình Desktop có thể triển khai trên hệ điều hành Windows. Quá trình đóng gói bao gồm mã nguồn giao diện đã được build, các thành phần cần thiết của Electron và thông tin cấu hình của ứng dụng. Sau khi được đóng gói, người dùng có thể khởi động chương trình như một ứng dụng Desktop thông thường mà không cần trực tiếp chạy môi trường phát triển React.

Sự kết hợp giữa Visual Studio Code, Node.js, npm, React và Electron tạo thành môi trường phát triển phù hợp cho phần Desktop của hệ thống. Trong đó, React đảm nhiệm việc tổ chức giao diện, Electron cung cấp môi trường Desktop, Node.js và npm hỗ trợ vận hành cũng như quản lý dependency, còn các công cụ kiểm tra tích hợp giúp theo dõi và xử lý lỗi trong quá trình xây dựng chương trình. Cách tổ chức này hỗ trợ

tách biệt phần giao diện khỏi Backend Java, đồng thời vẫn duy trì khả năng trao đổi dữ liệu thống nhất thông qua RESTful API.

### 1.3. Các công nghệ sử dụng

#### 1.3.1. *Ngôn ngữ lập trình Java*

Java là ngôn ngữ lập trình hướng đối tượng được phát triển với mục tiêu tạo ra các chương trình có khả năng hoạt động trên nhiều nền tảng khác nhau. Mã nguồn Java sau khi được biên dịch không chuyển trực tiếp thành mã máy của một hệ điều hành cụ thể mà được chuyển thành bytecode. Bytecode được thực thi thông qua Java Virtual Machine, nhờ đó chương trình có thể hoạt động trên nhiều môi trường khác nhau miễn là môi trường đó có JVM phù hợp.

Java hỗ trợ đầy đủ các đặc trưng quan trọng của lập trình hướng đối tượng như đóng gói, kế thừa, đa hình và trừu tượng. Những đặc điểm này giúp lập trình viên tổ chức chương trình thành các lớp và đối tượng có trách nhiệm riêng, từ đó nâng cao khả năng tái sử dụng và bảo trì mã nguồn. Đối với những hệ thống có nhiều chức năng nghiệp vụ, việc tổ chức chương trình theo mô hình hướng đối tượng giúp giảm sự phụ thuộc giữa các thành phần và tạo điều kiện thuận lợi cho quá trình mở rộng.

Một ưu điểm khác của Java là hệ sinh thái thư viện và framework phong phú. Các nền tảng như Spring Framework, Spring Boot, Hibernate và Spring Data JPA được xây dựng dựa trên Java, cung cấp nhiều cơ chế hỗ trợ phát triển ứng dụng doanh nghiệp. Java cũng có khả năng làm việc tốt với các hệ quản trị cơ sở dữ liệu quan hệ thông qua JDBC và các công nghệ ORM.

Trong hệ thống quản lý hàng trang sức, Java được sử dụng làm ngôn ngữ chính để xây dựng Backend. Các lớp xử lý nghiệp vụ, mô hình dữ liệu, xác thực người dùng, phân quyền, truy xuất cơ sở dữ liệu và cung cấp RESTful API đều được triển khai trên nền tảng Java. Việc sử dụng Java giúp phần xử lý nghiệp vụ của hệ thống có cấu trúc rõ ràng, phù hợp với yêu cầu của học phần Công nghệ Java và tạo cơ sở để tích hợp các công nghệ thuộc hệ sinh thái Spring.

### **1.3.2. *Spring Framework***

Spring Framework là một framework mã nguồn mở dành cho nền tảng Java, được phát triển nhằm hỗ trợ xây dựng các ứng dụng có cấu trúc rõ ràng và giảm mức độ phụ thuộc giữa các thành phần. Spring cung cấp một hệ sinh thái gồm nhiều module phục vụ các nhu cầu khác nhau như quản lý đối tượng, truy cập dữ liệu, xử lý giao dịch, phát triển ứng dụng web và bảo mật.

Hai khái niệm quan trọng trong Spring là Inversion of Control và Dependency Injection. Inversion of Control cho phép việc khởi tạo và quản lý vòng đời của các đối tượng được chuyển cho Spring Container thay vì được thực hiện trực tiếp trong mã nguồn của ứng dụng. Dependency Injection cho phép các đối tượng cần thiết được cung cấp cho một lớp từ bên ngoài, qua đó giảm sự phụ thuộc chặt chẽ giữa các lớp.

Cách tổ chức này giúp các thành phần trong chương trình có thể thay đổi hoặc kiểm thử độc lập dễ dàng hơn. Chẳng hạn, một lớp Service không cần tự tạo đối tượng Repository mà Repository có thể được Spring cung cấp thông qua cơ chế Dependency Injection. Nhờ đó, mối quan hệ giữa các thành phần trở nên rõ ràng hơn và mã nguồn hạn chế việc khởi tạo phụ thuộc thủ công.

Spring Framework còn cung cấp nền tảng cho nhiều dự án khác trong hệ sinh thái Spring như Spring Boot, Spring Data và Spring Security. Trong hệ thống, Spring đóng vai trò là nền tảng tổ chức Backend, quản lý các Bean và liên kết giữa Controller, Service, Repository cùng các thành phần bảo mật. Việc sử dụng Spring giúp ứng dụng tuân theo kiến trúc phân tầng và thuận lợi hơn trong quá trình phát triển, kiểm thử và bảo trì.

### **1.3.3. *Spring Boot***

Spring Boot là nền tảng được xây dựng trên Spring Framework nhằm đơn giản hóa quá trình cấu hình và phát triển ứng dụng Java. Thay vì yêu cầu lập trình viên thực hiện nhiều bước cấu hình thủ công, Spring Boot cung cấp cơ chế tự động cấu hình dựa trên các dependency có trong dự án và các thuộc tính đã được khai báo.

Một đặc điểm quan trọng của Spring Boot là cơ chế Auto Configuration. Khi chương trình được khởi động, Spring Boot kiểm tra các thư viện có trong project và tự động thiết lập nhiều cấu hình cần thiết. Điều này giúp giảm lượng mã cấu hình và giúp lập trình viên tập trung nhiều hơn vào việc phát triển chức năng nghiệp vụ.

Spring Boot cũng hỗ trợ các Starter Dependency. Mỗi starter tập hợp các thư viện thường được sử dụng cho một mục đích cụ thể. Chẳng hạn, một ứng dụng cung cấp RESTful API có thể sử dụng Spring Web, còn ứng dụng làm việc với cơ sở dữ liệu có thể sử dụng Spring Data JPA. Cách quản lý này giúp quá trình thiết lập dự án trở nên đơn giản và hạn chế việc phải lựa chọn từng thư viện riêng lẻ.

Ngoài ra, Spring Boot tích hợp Web Server vào ứng dụng, giúp chương trình có thể được chạy trực tiếp mà không cần triển khai thủ công lên một Application Server bên ngoài. Điều này phù hợp với quá trình phát triển Backend độc lập, trong đó Server khởi động và cung cấp các API cho Desktop Client.

Trong hệ thống, Spring Boot được sử dụng làm nền tảng chính của Backend. Các API, lớp nghiệp vụ, cơ chế truy cập dữ liệu, xác thực và phân quyền đều được tổ chức trong ứng dụng Spring Boot. Backend sau khi khởi động tiếp nhận yêu cầu từ ứng dụng React Electron, xử lý nghiệp vụ và làm việc với PostgreSQL trước khi trả kết quả về Client.

#### **1.3.4. *Spring Data JPA***

Spring Data JPA là một module thuộc hệ sinh thái Spring Data, được xây dựng nhằm đơn giản hóa quá trình truy cập cơ sở dữ liệu khi sử dụng Java Persistence API. Công nghệ này giúp giảm lượng mã nguồn cần viết cho những thao tác dữ liệu phổ biến và tạo ra một cơ chế thống nhất để làm việc với các Entity trong ứng dụng.

Trong cách tiếp cận truyền thống, lập trình viên thường phải tự xây dựng các lớp truy xuất dữ liệu, tạo câu lệnh SQL và thực hiện các thao tác kết nối. Với Spring Data JPA, Repository có thể được định nghĩa thông qua các interface và kế thừa những interface có sẵn như JpaRepository. Nhiều thao tác cơ bản như thêm, cập nhật, xóa, lấy dữ liệu theo khóa chính hoặc phân trang đã được cung cấp sẵn.

Spring Data JPA cũng hỗ trợ cơ chế xây dựng truy vấn dựa trên tên phương thức. Đối với các yêu cầu phức tạp hơn, lập trình viên có thể sử dụng JPQL hoặc Native Query. Nhờ đó, những truy vấn đơn giản có thể được triển khai nhanh chóng trong khi vẫn duy trì khả năng xử lý các trường hợp đặc thù.

Trong hệ thống quản lý trang sức, Spring Data JPA được sử dụng tại tầng Repository để thao tác với dữ liệu liên quan đến tài khoản, nhân viên, sản phẩm, kho, bảng giá, giao dịch và các thực thể khác. Việc sử dụng Spring Data JPA giúp tầng nghiệp vụ không phải trực tiếp xử lý chi tiết quá trình kết nối cơ sở dữ liệu, từ đó làm cho mã nguồn rõ ràng và dễ bảo trì hơn.

Spring Data JPA không phải là một hệ quản trị cơ sở dữ liệu và cũng không phải công nghệ thay thế hoàn toàn JDBC. Nó hoạt động ở mức trừu tượng cao hơn, sử dụng JPA làm tiêu chuẩn ánh xạ và thường sử dụng Hibernate làm implementation để thực hiện các thao tác dữ liệu.

#### **1.3.5. *Hibernate***

Hibernate là một framework ORM dành cho Java, được sử dụng phổ biến để ánh xạ giữa mô hình đối tượng trong chương trình và mô hình dữ liệu quan hệ trong cơ sở dữ liệu. Trong các ứng dụng sử dụng Spring Data JPA, Hibernate thường đóng vai trò là JPA Provider thực hiện các thao tác ánh xạ và tương tác với cơ sở dữ liệu.

Trong mô hình hướng đối tượng, dữ liệu được biểu diễn thông qua các lớp và đối tượng. Trong cơ sở dữ liệu quan hệ, dữ liệu lại được tổ chức dưới dạng bảng, cột và các mối quan hệ khóa ngoại. Sự khác biệt giữa hai cách biểu diễn này tạo ra nhu cầu về một cơ chế chuyển đổi. Hibernate giải quyết vấn đề đó bằng cách ánh xạ các lớp Java thành các bảng và các thuộc tính của đối tượng thành các cột tương ứng.

Thông qua các annotation như Entity, Table, Id, Column, OneToMany, ManyToOne và OneToOne, lập trình viên có thể khai báo cách một đối tượng Java được ánh xạ tới cơ sở dữ liệu. Hibernate sau đó thực hiện các thao tác cần thiết để đọc, thêm, sửa hoặc xóa dữ liệu.

Hibernate còn hỗ trợ quản lý quan hệ giữa các thực thể, quản lý transaction, cơ chế lazy loading và nhiều kỹ thuật tối ưu truy xuất dữ liệu. Tuy nhiên, việc sử dụng Hibernate cũng yêu cầu thiết kế quan hệ Entity phù hợp để tránh các vấn đề như truy vấn dư thừa hoặc tải quá nhiều dữ liệu không cần thiết.

Trong hệ thống, Hibernate là thành phần trung gian giúp các Entity Java tương tác với PostgreSQL thông qua JPA. Nhờ đó, phần lớn thao tác dữ liệu có thể được thực hiện thông qua đối tượng Java thay vì phải viết câu lệnh SQL trực tiếp cho mọi nghiệp vụ.

### **1.3.6. JDBC**

JDBC, viết tắt của Java Database Connectivity, là API tiêu chuẩn của Java được sử dụng để kết nối và giao tiếp với các hệ quản trị cơ sở dữ liệu quan hệ. JDBC cung cấp các interface và lớp cần thiết để thiết lập kết nối, gửi câu lệnh SQL, nhận kết quả truy vấn và quản lý transaction.

Để Java có thể làm việc với một hệ quản trị cơ sở dữ liệu cụ thể, hệ thống cần sử dụng JDBC Driver tương ứng. Trong trường hợp PostgreSQL, PostgreSQL JDBC Driver đảm nhiệm việc chuyển đổi các lời gọi JDBC thành giao thức mà PostgreSQL có thể xử lý.

JDBC cung cấp khả năng kiểm soát trực tiếp câu lệnh SQL nên phù hợp với những trường hợp cần chủ động tối ưu truy vấn hoặc thực hiện các thao tác mà việc ánh xạ ORM không mang lại hiệu quả mong muốn. Tuy nhiên, sử dụng JDBC trực tiếp thường yêu cầu lập trình viên xử lý nhiều chi tiết hơn so với JPA, bao gồm câu lệnh SQL, tham số, kết quả truy vấn và một số vấn đề liên quan đến kết nối.

Trong hệ thống, JDBC được sử dụng song song với JPA tùy theo yêu cầu xử lý dữ liệu. Các nghiệp vụ thông thường được triển khai thông qua Spring Data JPA và Hibernate, trong khi JDBC có thể được sử dụng đối với những truy vấn cần mức độ kiểm soát cao hơn hoặc cần xử lý trực tiếp với SQL.

Về mặt kiến trúc, JPA và Hibernate cuối cùng vẫn phải giao tiếp với hệ quản trị cơ sở dữ liệu thông qua JDBC Driver. Có thể hiểu mối quan hệ tổng quát theo hướng

JPA cung cấp tiêu chuẩn ánh xạ, Hibernate triển khai tiêu chuẩn đó và JDBC đảm nhiệm lớp giao tiếp thấp hơn với cơ sở dữ liệu.

### **1.3.7. PostgreSQL**

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở có khả năng quản lý dữ liệu theo mô hình bảng và hỗ trợ đầy đủ nhiều tính năng của SQL. PostgreSQL được sử dụng trong nhiều loại hệ thống do có khả năng quản lý dữ liệu có cấu trúc, hỗ trợ transaction và cung cấp các cơ chế bảo đảm tính toàn vẹn dữ liệu.

Dữ liệu trong PostgreSQL được tổ chức thông qua database, schema, table, column và các đối tượng cơ sở dữ liệu khác. Quan hệ giữa các bảng có thể được thiết lập thông qua khóa chính và khóa ngoại. Các constraint như NOT NULL, UNIQUE, CHECK và FOREIGN KEY giúp kiểm soát dữ liệu ngay tại tầng cơ sở dữ liệu, hạn chế những bản ghi không đáp ứng điều kiện đã được thiết kế.

PostgreSQL hỗ trợ transaction theo các nguyên tắc ACID, phù hợp với những hệ thống có nhiều nghiệp vụ cần bảo đảm tính nhất quán. Điều này đặc biệt quan trọng đối với các nghiệp vụ liên quan đến giao dịch và tồn kho. Một thao tác có nhiều bước cần được xử lý thống nhất để tránh trường hợp chỉ một phần dữ liệu được cập nhật khi xảy ra lỗi.

Hệ quản trị này cũng hỗ trợ index nhằm cải thiện tốc độ truy vấn đối với những trường dữ liệu thường xuyên được tìm kiếm. Khi dữ liệu của hệ thống tăng lên, việc thiết kế index phù hợp có thể làm giảm thời gian tra cứu và tăng hiệu quả xử lý.

Trong hệ thống, PostgreSQL đảm nhiệm việc lưu trữ tập trung các dữ liệu như tài khoản, quyền, vai trò, nhân viên, khách hàng, sản phẩm trang sức, kho hàng, tồn kho, bảng giá và giao dịch. Backend Spring Boot kết nối tới PostgreSQL thông qua PostgreSQL JDBC Driver, JPA và Hibernate để thực hiện các thao tác dữ liệu.

### **1.3.8. Spring Security**

Spring Security là framework thuộc hệ sinh thái Spring được sử dụng để xây dựng các cơ chế bảo mật cho ứng dụng Java. Framework cung cấp nhiều thành phần

hỗ trợ xác thực người dùng, kiểm soát quyền truy cập, mã hóa mật khẩu và bảo vệ các tài nguyên của hệ thống.

Hai khái niệm cơ bản trong Spring Security là Authentication và Authorization. Authentication là quá trình xác định người dùng là ai, thường được thực hiện thông qua thông tin đăng nhập. Authorization là quá trình xác định người dùng sau khi được xác thực có được phép sử dụng một tài nguyên hoặc thực hiện một hành động cụ thể hay không.

Trong hệ thống, mật khẩu của tài khoản không được lưu trực tiếp ở dạng văn bản thông thường. Cơ chế mã hóa một chiều như BCrypt được sử dụng trước khi mật khẩu được lưu vào cơ sở dữ liệu. Khi người dùng đăng nhập, mật khẩu nhập vào được kiểm tra với giá trị đã mã hóa thay vì giải mã mật khẩu đã lưu.

Spring Security cũng cho phép xây dựng các quy tắc kiểm soát truy cập theo Role và Permission. Điều này phù hợp với hệ thống có nhiều nhóm người dùng như quản trị viên, người quản lý và nhân viên. Một tài khoản có thể được giới hạn chỉ sử dụng những API và chức năng phù hợp với nhiệm vụ được giao.

Trong kiến trúc của hệ thống, Spring Security hoạt động ở phía Backend. Mọi yêu cầu cần bảo vệ đều được kiểm tra trước khi đến phần xử lý nghiệp vụ. Vì vậy, ngay cả khi người dùng cố gắng gửi yêu cầu trực tiếp tới API mà không thông qua giao diện, Server vẫn có khả năng từ chối nếu tài khoản không được xác thực hoặc không có quyền phù hợp.

### **1.3.9. *JSON Web Token***

JSON Web Token, thường được gọi là JWT, là một tiêu chuẩn dùng để biểu diễn thông tin dưới dạng token có cấu trúc. JWT thường được sử dụng trong các hệ thống RESTful API để hỗ trợ cơ chế xác thực không phụ thuộc vào Session lưu tại Server.

Một JWT thông thường gồm ba thành phần là Header, Payload và Signature. Header chứa thông tin về loại token và thuật toán ký. Payload chứa các thông tin cần thiết phục vụ quá trình xác thực, chẳng hạn định danh người dùng và một số claim liên

quan. Signature được sử dụng để kiểm tra tính toàn vẹn của token, giúp Server phát hiện trường hợp nội dung token bị thay đổi.

Sau khi người dùng đăng nhập thành công, Backend tạo JWT và trả token cho ứng dụng Desktop. Client lưu token và gửi kèm token trong các yêu cầu tiếp theo cần xác thực. Khi nhận yêu cầu, Server kiểm tra chữ ký, thời hạn và các thông tin có liên quan trước khi quyết định cho phép yêu cầu tiếp tục được xử lý.

JWT phù hợp với kiến trúc RESTful vì Server không nhất thiết phải duy trì trạng thái phiên của từng Client. Mỗi yêu cầu mang theo thông tin cần thiết để xác định danh tính người dùng. Điều này làm giảm sự phụ thuộc vào Session phía Server và tạo thuận lợi khi hệ thống được mở rộng.

Trong hệ thống, JWT được kết hợp với Spring Security để xây dựng quá trình đăng nhập và kiểm soát truy cập. Token chỉ đóng vai trò xác thực và truyền một số thông tin cần thiết; quyền thực hiện nghiệp vụ vẫn phải được Backend kiểm tra theo quy tắc phân quyền của hệ thống.

### **1.3.10. *RESTful API***

RESTful API là phương thức xây dựng giao diện giao tiếp giữa các hệ thống dựa trên các nguyên tắc của kiến trúc REST và giao thức HTTP. Trong REST, dữ liệu được biểu diễn dưới dạng các tài nguyên và mỗi tài nguyên được xác định thông qua URI.

Các thao tác trên tài nguyên thường được thực hiện bằng các HTTP Method như GET để lấy dữ liệu, POST để tạo dữ liệu, PUT hoặc PATCH để cập nhật và DELETE để xóa. Việc sử dụng các phương thức HTTP theo đúng mục đích giúp API có cấu trúc nhất quán và dễ hiểu.

RESTful API cũng sử dụng HTTP Status Code để thể hiện kết quả xử lý yêu cầu. Các trạng thái phản hồi giúp Client xác định yêu cầu đã thành công, dữ liệu đầu vào không hợp lệ, tài nguyên không tồn tại, người dùng chưa được xác thực hoặc không có quyền truy cập.

Dữ liệu trao đổi giữa Client và Server trong hệ thống chủ yếu sử dụng JSON. Ứng dụng React Electron gửi Request tới Backend Spring Boot, Server thực hiện xử lý và trả Response dưới dạng JSON. Cách giao tiếp này giúp hai phía không phụ thuộc trực tiếp vào công nghệ của nhau.

Một đặc điểm quan trọng của REST là tính Stateless. Mỗi yêu cầu cần mang đủ thông tin để Server có thể xử lý mà không phụ thuộc hoàn toàn vào trạng thái của yêu cầu trước đó. Khi kết hợp RESTful API với JWT, Client có thể gửi token cùng từng yêu cầu cần xác thực.

Trong hệ thống quản lý trang sức, RESTful API là cầu nối giữa phần Desktop và Backend. Nhờ đó, giao diện React Electron có thể được phát triển tương đối độc lập với phần xử lý Java, trong khi toàn bộ quy tắc nghiệp vụ và truy cập dữ liệu vẫn được tập trung tại Server.

### **1.3.11. React**

React là thư viện JavaScript được sử dụng để xây dựng giao diện người dùng theo hướng component. Thay vì xây dựng toàn bộ giao diện thành một khối lớn, React cho phép phân chia giao diện thành nhiều thành phần nhỏ có chức năng riêng và có khả năng tái sử dụng.

Component có thể đại diện cho một phần giao diện như thanh điều hướng, bảng dữ liệu, biểu mẫu nhập liệu, hộp thoại hoặc một màn hình hoàn chỉnh. Việc tổ chức theo component giúp cấu trúc mã nguồn rõ ràng hơn và hạn chế việc lặp lại những phần giao diện có chức năng tương tự.

React quản lý sự thay đổi của giao diện dựa trên state. Khi dữ liệu thay đổi, các component có liên quan được cập nhật để phản ánh trạng thái mới. Các cơ chế như Hooks hỗ trợ quản lý state, thực hiện xử lý khi component thay đổi và tổ chức các logic dùng chung giữa các thành phần.

Trong ứng dụng quản lý trang sức, React được sử dụng để xây dựng các màn hình như đăng nhập, Dashboard, quản lý sản phẩm, kho hàng, nhân viên, khách hàng,

bảng giá, giao dịch và báo cáo. Dữ liệu hiển thị trên các màn hình được lấy từ RESTful API do Spring Boot cung cấp.

Việc sử dụng React giúp tách phần giao diện khỏi Backend. React tập trung vào hiển thị dữ liệu và xử lý tương tác của người dùng, trong khi các quy tắc nghiệp vụ quan trọng được chuyển đến Server. Cách phân chia này phù hợp với kiến trúc Client – Server được áp dụng trong hệ thống.

React vốn được sử dụng phổ biến cho ứng dụng web, tuy nhiên khi kết hợp với Electron, giao diện React có thể được đóng gói và vận hành dưới dạng ứng dụng Desktop mà vẫn tận dụng được mô hình component và hệ sinh thái thư viện của nền tảng web.

### **1.3.12. *Electron***

Electron là framework cho phép phát triển ứng dụng Desktop bằng các công nghệ thường được sử dụng trong phát triển web như HTML, CSS và JavaScript hoặc TypeScript. Electron cung cấp môi trường để giao diện web có thể chạy như một chương trình độc lập trên hệ điều hành.

Kiến trúc Electron chủ yếu bao gồm Main Process và Renderer Process. Main Process chịu trách nhiệm quản lý vòng đời của ứng dụng, tạo cửa sổ và tương tác với các chức năng của hệ điều hành. Renderer Process chịu trách nhiệm hiển thị giao diện của từng cửa sổ và trong hệ thống được sử dụng để chạy giao diện React.

Ngoài hai thành phần trên, Preload Script được sử dụng để xây dựng cầu nối an toàn giữa Renderer Process và những API cần thiết của Electron hoặc Node.js. Việc giới hạn những chức năng được đưa ra giao diện giúp giảm mức độ truy cập trực tiếp của Renderer tới các tài nguyên nhạy cảm.

Electron còn cung cấp cơ chế IPC để các Process trao đổi thông tin khi ứng dụng cần sử dụng chức năng phía Main Process. Việc phân tách này giúp tổ chức rõ ràng hơn các chức năng liên quan đến giao diện và các chức năng liên quan trực tiếp đến môi trường Desktop.

Trong hệ thống, Electron đóng vai trò biến phần giao diện được xây dựng bằng React thành ứng dụng Desktop. Người dùng có thể khởi động chương trình như một phần mềm thông thường trên máy tính, trong khi ứng dụng vẫn giao tiếp với Java Spring Boot thông qua RESTful API.

Việc kết hợp React với Electron giúp tận dụng khả năng xây dựng giao diện linh hoạt của React đồng thời đáp ứng yêu cầu triển khai dưới dạng phần mềm Desktop. Phần Desktop chỉ đảm nhận nhiệm vụ giao diện và tương tác với người dùng, còn dữ liệu và nghiệp vụ chính vẫn được xử lý tập trung tại Backend Java.

### ***1.3.13. TypeScript***

TypeScript là ngôn ngữ lập trình được phát triển dựa trên JavaScript và bổ sung hệ thống kiểu dữ liệu tĩnh. Mã nguồn TypeScript sau quá trình biên dịch được chuyển thành JavaScript để có thể thực thi trong các môi trường hỗ trợ JavaScript.

Điểm khác biệt quan trọng của TypeScript so với JavaScript là khả năng khai báo kiểu cho biến, tham số, giá trị trả về, đối tượng và các cấu trúc dữ liệu khác. Nhờ đó, nhiều lỗi liên quan đến kiểu dữ liệu có thể được phát hiện ngay trong quá trình phát triển thay vì chỉ xuất hiện khi chương trình đang chạy.

TypeScript hỗ trợ các khái niệm như interface, type, generic và enum, giúp mô tả dữ liệu một cách rõ ràng hơn. Đối với một ứng dụng có nhiều dữ liệu nhận từ RESTful API, việc định nghĩa kiểu cho Request và Response giúp các thành phần giao diện sử dụng dữ liệu nhất quán và giảm nguy cơ truy cập sai thuộc tính.

Khi kết hợp với React, TypeScript giúp xác định rõ kiểu của Props, State, dữ liệu biểu mẫu và dữ liệu API. Điều này đặc biệt hữu ích đối với những màn hình có nhiều trường thông tin như sản phẩm, giao dịch, khách hàng hoặc tồn kho. Các component có thể xác định rõ loại dữ liệu được phép nhận, qua đó tăng khả năng kiểm soát mã nguồn.

TypeScript cũng được sử dụng trong phần Electron để tổ chức các thành phần như Main Process, Preload Script và những module hỗ trợ khác. Hệ thống kiểu giúp

việc trao đổi dữ liệu giữa các thành phần này rõ ràng hơn và hỗ trợ tốt quá trình phát hiện lỗi trong khi lập trình.

Trong phần Desktop của hệ thống, TypeScript được sử dụng cùng React và Electron nhằm tăng tính chặt chẽ của mã nguồn. Việc áp dụng kiểu dữ liệu rõ ràng giúp chương trình dễ đọc, dễ bảo trì và phù hợp hơn khi số lượng màn hình cũng như chức năng ngày càng tăng.

## CHƯƠNG 2. THIẾT KẾ VÀ XÂY DỰNG CHƯƠNG TRÌNH

### 2.1. Phân tích yêu cầu bài toán

#### 2.1.1. *Mô tả bài toán quản lý kinh doanh trang sức*

Kinh doanh trang sức là lĩnh vực có đặc thù riêng do hàng hóa thường có giá trị lớn, nhiều thuộc tính cần được quản lý và giá bán có thể thay đổi theo từng thời điểm. Mỗi sản phẩm không chỉ được xác định bởi tên hoặc mã hàng mà còn gắn với các thông tin như loại trang sức, loại vàng, trọng lượng, hàm lượng, đơn giá, tình trạng tồn kho và trạng thái kinh doanh. Vì vậy, quá trình quản lý đòi hỏi dữ liệu phải được tổ chức chặt chẽ, cập nhật chính xác và có khả năng tra cứu nhanh khi phát sinh nghiệp vụ.

Trong hoạt động thực tế, đơn vị kinh doanh cần quản lý toàn bộ vòng đời của sản phẩm từ khi được tiếp nhận vào hệ thống, nhập kho, cập nhật thông tin, điều chỉnh giá cho đến khi sản phẩm được bán cho khách hàng. Mỗi bước đều làm phát sinh dữ liệu và có thể ảnh hưởng đến các thành phần khác. Một sản phẩm sau khi nhập kho phải được ghi nhận chính xác về mã sản phẩm, đặc tính, số lượng và vị trí lưu trữ. Khi phát sinh giao dịch bán hàng, dữ liệu sản phẩm và tồn kho phải được cập nhật tương ứng để phản ánh đúng tình trạng thực tế.

Một trong những nội dung quan trọng của bài toán là quản lý hàng hóa trang sức. Hệ thống cần lưu trữ đầy đủ các thông tin phục vụ nhận diện và phân loại sản phẩm. Các sản phẩm có thể thuộc nhiều nhóm khác nhau như nhẫn, vòng tay, dây chuyền, bông tai hoặc các loại trang sức khác. Bên cạnh loại sản phẩm, thông tin về loại vàng và trọng lượng cũng có ảnh hưởng trực tiếp đến quá trình xác định giá trị của hàng hóa. Việc quản lý các thuộc tính này theo một cấu trúc thống nhất giúp hạn chế nhầm lẫn và hỗ trợ quá trình tìm kiếm, kiểm tra sản phẩm.

Quản lý kho là một phần có liên hệ trực tiếp với quản lý hàng hóa. Hệ thống cần phản ánh được tình trạng hàng hóa hiện có, số lượng hoặc trạng thái của từng sản phẩm và các biến động phát sinh trong quá trình nhập, xuất hoặc bán hàng. Khi một nghiệp vụ làm thay đổi tồn kho được thực hiện, dữ liệu cần được cập nhật đồng bộ để tránh chênh lệch giữa thông tin trên phần mềm và hàng hóa thực tế. Điều này đặc biệt quan

trọng trong lĩnh vực trang sức do giá trị của từng sản phẩm có thể cao và sai lệch nhỏ cũng có thể gây ảnh hưởng đáng kể đến quá trình quản lý.

Bảng giá cũng là một thành phần cần được quản lý riêng trong hệ thống. Giá của sản phẩm trang sức có thể chịu tác động bởi loại vàng, trọng lượng, chi phí gia công và chính sách kinh doanh tại từng thời điểm. Do đó, chương trình cần cho phép cập nhật thông tin giá và sử dụng dữ liệu phù hợp khi thực hiện giao dịch. Việc tổ chức bảng giá tập trung giúp hạn chế tình trạng mỗi nhân viên sử dụng một mức giá khác nhau hoặc phải tra cứu giá từ nhiều nguồn riêng biệt.

Đối với nghiệp vụ bán hàng, hệ thống cần hỗ trợ nhân viên lựa chọn sản phẩm, tra cứu thông tin hàng hóa, xác định khách hàng, áp dụng giá và ghi nhận giao dịch. Một giao dịch có thể bao gồm một hoặc nhiều sản phẩm, vì vậy chương trình phải quản lý được thông tin tổng quát của giao dịch và các chi tiết liên quan đến từng mặt hàng. Sau khi giao dịch được xác nhận, trạng thái hoặc số lượng tồn của sản phẩm cần được điều chỉnh tương ứng. Dữ liệu giao dịch sau đó được lưu trữ để phục vụ tra cứu, đối chiếu và thống kê.

Khách hàng là một đối tượng dữ liệu có liên quan trực tiếp đến quá trình bán hàng. Việc lưu trữ thông tin khách hàng giúp hệ thống liên kết các giao dịch với người mua, hỗ trợ tra cứu lịch sử giao dịch và giảm thời gian nhập lại thông tin đối với khách hàng đã từng sử dụng dịch vụ. Tuy nhiên, thông tin khách hàng cần được quản lý ở mức phù hợp với nhu cầu nghiệp vụ, tránh lưu trữ những dữ liệu không cần thiết.

Bên cạnh hàng hóa và khách hàng, hệ thống còn cần quản lý thông tin nhân viên. Mỗi nhân viên có thể đảm nhiệm những nhiệm vụ khác nhau và không phải tất cả người dùng đều được phép truy cập toàn bộ chức năng. Người quản lý có thể cần theo dõi dữ liệu tổng hợp, trong khi nhân viên bán hàng chủ yếu thao tác với sản phẩm và giao dịch. Quản trị viên lại chịu trách nhiệm đối với tài khoản và quyền truy cập. Vì vậy, bài toán quản lý nhân sự trong hệ thống không chỉ dừng ở việc lưu trữ thông tin nhân viên mà còn liên quan đến việc xác định phạm vi sử dụng chương trình.

Để kiểm soát quyền truy cập, chương trình cần có cơ chế quản lý tài khoản, xác thực và phân quyền. Khi người dùng đăng nhập, hệ thống phải xác định được tài khoản có hợp lệ hay không trước khi cho phép sử dụng các chức năng. Sau khi xác thực thành công, quyền của tài khoản tiếp tục được kiểm tra đối với từng nhóm nghiệp vụ. Việc kiểm soát này giúp hạn chế trường hợp người dùng thực hiện những thao tác không thuộc trách nhiệm của mình, đồng thời góp phần bảo vệ các dữ liệu quan trọng.

Một yêu cầu khác của bài toán là khả năng tìm kiếm và tra cứu dữ liệu. Khi số lượng sản phẩm, nhân viên, khách hàng và giao dịch tăng lên, việc tìm kiếm thủ công trong toàn bộ danh sách sẽ không còn phù hợp. Hệ thống cần hỗ trợ tra cứu theo các tiêu chí thích hợp như mã sản phẩm, tên sản phẩm, loại vàng, thông tin khách hàng hoặc thời gian phát sinh giao dịch. Khả năng tìm kiếm nhanh giúp giảm thời gian thao tác và hỗ trợ người dùng xử lý công việc hiệu quả hơn.

Dữ liệu được tạo ra trong quá trình vận hành còn là cơ sở để thực hiện thống kê và báo cáo. Người quản lý cần có khả năng theo dõi tình hình hàng hóa, số lượng tồn kho, số giao dịch và doanh thu trong từng khoảng thời gian. Các thông tin tổng hợp giúp đánh giá kết quả hoạt động và hỗ trợ kiểm soát quá trình kinh doanh. Vì vậy, dữ liệu ngay từ các nghiệp vụ ban đầu cần được tổ chức chính xác để bảo đảm kết quả thống kê có độ tin cậy.

Xét về luồng nghiệp vụ tổng thể, người dùng trước hết thực hiện đăng nhập vào hệ thống. Sau khi được xác thực và xác định quyền truy cập, người dùng lựa chọn chức năng phù hợp với nhiệm vụ. Trong quá trình vận hành, thông tin sản phẩm được khai báo và quản lý trong kho, bảng giá được cập nhật theo yêu cầu, khách hàng được ghi nhận khi cần thiết và giao dịch được tạo khi phát sinh hoạt động bán hàng. Sau mỗi giao dịch, dữ liệu liên quan được cập nhật, lưu trữ và sử dụng làm cơ sở cho việc tra cứu cũng như lập báo cáo.

Bài toán đặt ra không chỉ yêu cầu chương trình thực hiện được các thao tác thêm, sửa, xóa và tìm kiếm dữ liệu mà còn phải bảo đảm mối liên hệ giữa các nghiệp vụ. Khi một giao dịch được thực hiện, dữ liệu kho phải thay đổi phù hợp; khi một sản phẩm không còn tồn tại hoặc không ở trạng thái được phép giao dịch, hệ thống không được

cho phép bán; khi người dùng không có quyền, yêu cầu phải bị từ chối. Các quy tắc này tạo thành phần logic nghiệp vụ cần được xử lý thống nhất tại Backend thay vì phụ thuộc hoàn toàn vào giao diện.

Từ các yêu cầu nghiệp vụ trên, hệ thống được định hướng xây dựng theo mô hình Client - Server. Ứng dụng desktop chịu trách nhiệm cung cấp giao diện và tiếp nhận thao tác của người dùng, trong khi Backend xử lý nghiệp vụ, kiểm tra quyền và truy xuất dữ liệu. Cơ sở dữ liệu lưu trữ tập trung toàn bộ thông tin liên quan, giúp các chức năng sử dụng chung một nguồn dữ liệu và hạn chế tình trạng thiếu đồng bộ.

Có thể mô tả luồng nghiệp vụ chính của bài toán theo trình tự:



Hình 2.16: Quy trình nghiệp vụ tổng quát của hệ thống

Phạm vi bài toán tập trung vào việc hình thành một quy trình quản lý thống nhất từ dữ liệu đầu vào đến kết quả cuối cùng của hoạt động kinh doanh. Các chức năng được thiết kế có mối liên hệ với nhau thay vì tồn tại như những module độc lập hoàn toàn, qua đó bảo đảm dữ liệu sản phẩm, kho, bảng giá và giao dịch luôn được xử lý trên cùng một hệ thống.

### **2.1.2. Các đối tượng tham gia hệ thống**

Trong quá trình phân tích hệ thống quản lý kinh doanh trang sức, việc xác định các đối tượng tham gia có ý nghĩa quan trọng đối với việc xây dựng chức năng, thiết kế giao diện và tổ chức cơ chế phân quyền. Mỗi đối tượng tham gia vào hệ thống với một vai trò và phạm vi trách nhiệm khác nhau. Từ đó, hệ thống cần cung cấp những chức năng phù hợp, đồng thời kiểm soát các thao tác được phép thực hiện nhằm bảo đảm dữ liệu được sử dụng đúng mục đích và hạn chế các thay đổi không đúng thẩm quyền.

Các đối tượng tham gia trực tiếp vào hệ thống chủ yếu bao gồm quản trị viên hệ thống, người quản lý, nhân viên bán hàng và nhân viên kho. Bên cạnh đó, khách hàng là đối tượng có liên quan đến quá trình kinh doanh nhưng không trực tiếp đăng nhập vào ứng dụng. Thông tin của từng nhóm được sử dụng trong các nghiệp vụ khác nhau và có mối liên hệ với dữ liệu sản phẩm, kho, bảng giá và giao dịch.

Quản trị viên hệ thống là đối tượng có phạm vi quản lý cao nhất đối với các chức năng liên quan đến tài khoản và quyền truy cập. Nhiệm vụ chính của quản trị viên là tạo và quản lý tài khoản người dùng, thiết lập trạng thái hoạt động của tài khoản, quản lý vai trò và phân quyền cho từng nhóm người dùng. Khi có sự thay đổi về nhân sự hoặc trách nhiệm công việc, quản trị viên có thể điều chỉnh quyền để bảo đảm người dùng chỉ được truy cập những chức năng phù hợp với nhiệm vụ được giao.

Quản trị viên còn có trách nhiệm kiểm soát cấu trúc phân quyền của hệ thống. Mỗi tài khoản có thể được liên kết với một hoặc nhiều quyền cụ thể tùy theo cách tổ chức của chương trình. Các quyền này xác định phạm vi thao tác của người dùng đối với những chức năng như xem dữ liệu, thêm mới, cập nhật, thực hiện giao dịch hoặc

truy cập báo cáo. Việc kiểm soát quyền tại mức hệ thống giúp hạn chế tình trạng người dùng có thể thực hiện những thao tác nằm ngoài trách nhiệm của mình.

Đối tượng thứ hai là người quản lý. Đây là nhóm người dùng chịu trách nhiệm theo dõi hoạt động kinh doanh và kiểm soát các dữ liệu nghiệp vụ quan trọng. Người quản lý cần có khả năng theo dõi tình trạng sản phẩm, số lượng tồn kho, bảng giá, giao dịch và các thông tin liên quan đến hoạt động của nhân viên. So với nhân viên thông thường, phạm vi truy cập của người quản lý rộng hơn nhằm phục vụ công tác kiểm tra và đánh giá hoạt động kinh doanh.

Trong nghiệp vụ quản lý hàng hóa, người quản lý có thể kiểm tra danh sách sản phẩm, thông tin loại vàng, trọng lượng, giá trị và trạng thái hiện tại của sản phẩm. Việc theo dõi dữ liệu này giúp người quản lý xác định tình hình hàng hóa và phát hiện những trường hợp có sự chênh lệch hoặc bất thường cần xử lý. Người quản lý cũng có thể thực hiện các nghiệp vụ liên quan đến việc điều chỉnh thông tin sản phẩm trong phạm vi quyền được cấp.

Đối với bảng giá, người quản lý là nhóm có trách nhiệm kiểm soát việc cập nhật và áp dụng giá. Giá của sản phẩm trang sức có thể thay đổi theo từng thời điểm và chịu ảnh hưởng bởi nhiều yếu tố khác nhau. Vì vậy, quyền thay đổi bảng giá cần được giới hạn cho những tài khoản có trách nhiệm phù hợp. Việc phân tách quyền này giúp hạn chế trường hợp nhân viên tự ý thay đổi giá và bảo đảm mức giá được áp dụng thống nhất trong quá trình bán hàng.

Người quản lý còn có nhu cầu theo dõi các báo cáo tổng hợp. Các dữ liệu liên quan đến giao dịch, doanh thu, hàng hóa và tồn kho có thể được tổng hợp theo từng khoảng thời gian để phục vụ việc đánh giá tình hình hoạt động. Thông qua hệ thống, người quản lý có thể tra cứu dữ liệu đã được ghi nhận thay vì phải tổng hợp thủ công từ nhiều nguồn khác nhau.

Nhân viên bán hàng là nhóm người dùng thường xuyên trực tiếp sử dụng ứng dụng trong quá trình phục vụ khách hàng. Phạm vi thao tác của nhân viên bán hàng tập trung vào việc tra cứu hàng hóa, kiểm tra bảng giá, quản lý thông tin khách hàng và

thực hiện giao dịch. Khi khách hàng có nhu cầu mua sản phẩm, nhân viên sử dụng chương trình để tìm kiếm mặt hàng phù hợp, kiểm tra trạng thái sản phẩm và tạo giao dịch dựa trên dữ liệu có sẵn trong hệ thống.

Trong quá trình bán hàng, nhân viên cần xác định chính xác sản phẩm được lựa chọn, thông tin khách hàng và mức giá đang được áp dụng. Sau khi các thông tin được xác nhận, dữ liệu giao dịch được gửi tới Backend để kiểm tra và xử lý. Nếu các điều kiện nghiệp vụ được đáp ứng, giao dịch được lưu vào cơ sở dữ liệu và dữ liệu tồn kho được cập nhật tương ứng.

Nhân viên bán hàng không cần được cấp quyền truy cập tới toàn bộ chức năng quản trị của hệ thống. Những thao tác như phân quyền, chỉnh sửa vai trò hoặc thay đổi các cấu hình quan trọng không thuộc phạm vi công việc của nhóm này. Việc giới hạn chức năng giúp giao diện của nhân viên tập trung vào những nghiệp vụ cần thiết và đồng thời giảm nguy cơ phát sinh các thao tác ngoài thẩm quyền.

Nhân viên kho là đối tượng chịu trách nhiệm theo dõi và quản lý tình trạng hàng hóa trong kho. Công việc của nhóm này liên quan trực tiếp đến việc nhập hàng, kiểm tra tồn kho, xác nhận tình trạng sản phẩm và ghi nhận các biến động của hàng hóa. Vì dữ liệu kho có ảnh hưởng trực tiếp đến khả năng bán hàng và tính chính xác của báo cáo, các thao tác cập nhật cần được thực hiện theo quy trình và được kiểm soát quyền truy cập.

Khi sản phẩm mới được đưa vào kho, nhân viên kho có trách nhiệm kiểm tra và ghi nhận các thông tin cần thiết. Trong quá trình hoạt động, nhân viên có thể tra cứu sản phẩm theo mã, tên hoặc các tiêu chí liên quan để xác định vị trí và trạng thái của hàng hóa. Khi phát sinh nghiệp vụ nhập hoặc xuất, dữ liệu cần được cập nhật kịp thời để số liệu trên hệ thống phản ánh đúng tình trạng thực tế.

Giữa nhân viên bán hàng và nhân viên kho tồn tại mối liên hệ chặt chẽ thông qua dữ liệu hàng hóa. Nhân viên bán hàng cần sử dụng thông tin tồn kho để xác định sản phẩm có thể giao dịch, trong khi dữ liệu giao dịch lại làm thay đổi tình trạng hàng hóa mà nhân viên kho theo dõi. Do đó, các chức năng của hai nhóm người dùng phải

sử dụng chung nguồn dữ liệu và tuân thủ cùng các quy tắc nghiệp vụ để tránh xảy ra sự không thống nhất.

Khách hàng là đối tượng có liên quan đến hệ thống thông qua các giao dịch mua hàng. Khác với những nhóm người dùng nêu trên, khách hàng không trực tiếp đăng nhập hoặc thao tác trên ứng dụng desktop. Thông tin khách hàng được nhân viên ghi nhận và sử dụng trong quá trình tạo giao dịch, tra cứu lịch sử mua hàng hoặc phục vụ các nghiệp vụ chăm sóc khách hàng khi cần thiết.

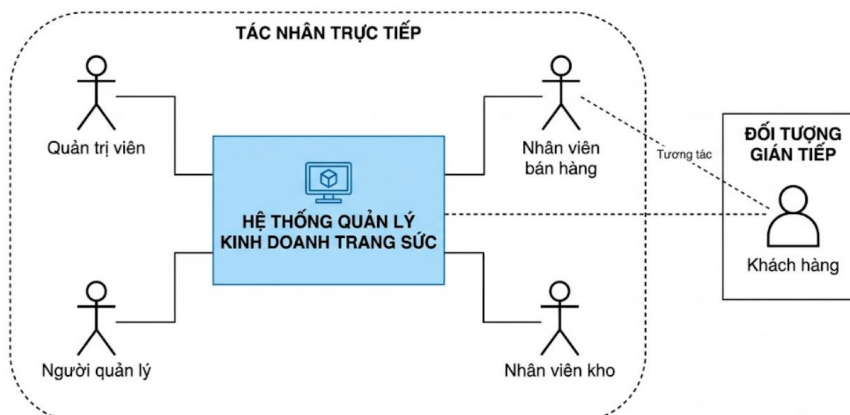
Dữ liệu khách hàng cần được tổ chức phù hợp với yêu cầu sử dụng thực tế. Các thông tin nhận dạng và liên hệ cơ bản có thể được lưu trữ để hỗ trợ nhân viên xác định khách hàng trong lần giao dịch tiếp theo. Khi một giao dịch được tạo, dữ liệu khách hàng có thể được liên kết với giao dịch tương ứng, qua đó hình thành lịch sử mua bán và tạo điều kiện thuận lợi cho việc tra cứu sau này.

Mặc dù mỗi nhóm người dùng có phạm vi trách nhiệm riêng, toàn bộ các đối tượng đều hoạt động trên cùng một hệ thống dữ liệu. Việc cập nhật ở một chức năng có thể ảnh hưởng tới thông tin mà nhóm người dùng khác sử dụng. Chẳng hạn, khi nhân viên kho cập nhật một sản phẩm mới, nhân viên bán hàng có thể tra cứu sản phẩm đó; khi giao dịch bán hàng hoàn thành, số liệu tồn kho thay đổi; khi người quản lý cập nhật bảng giá, mức giá mới sẽ được áp dụng trong các giao dịch tiếp theo.

Để kiểm soát sự tương tác giữa các đối tượng, cơ chế xác thực và phân quyền được áp dụng xuyên suốt quá trình sử dụng. Mỗi người dùng phải đăng nhập bằng tài khoản hợp lệ trước khi truy cập vào hệ thống. Sau khi xác thực thành công, Backend xác định vai trò và quyền của tài khoản để quyết định những chức năng được phép thực hiện. Việc kiểm tra quyền không chỉ được thực hiện tại giao diện mà còn được kiểm soát tại Server nhằm tránh trường hợp người dùng gửi yêu cầu trực tiếp tới các API không thuộc phạm vi cho phép.

Cách phân chia các đối tượng tham gia theo trách nhiệm nghiệp vụ tạo cơ sở cho việc xây dựng Use Case và thiết kế chi tiết các chức năng ở những phần tiếp theo. Đồng thời, đây cũng là nền tảng để xác định cấu trúc Role, Permission và phạm vi dữ

liệu mà từng nhóm người dùng được quyền khai thác trong quá trình vận hành chương trình.



Hình 2.17: Các đối tượng tham gia vào hệ thống

### 2.1.3. Yêu cầu chức năng

Yêu cầu chức năng mô tả những nghiệp vụ và khả năng mà hệ thống cần cung cấp cho người sử dụng trong quá trình quản lý hoạt động kinh doanh trang sức. Các chức năng được xác định dựa trên quy trình vận hành thực tế, đối tượng tham gia hệ thống và mối quan hệ giữa các nhóm dữ liệu như tài khoản, nhân viên, khách hàng, sản phẩm, kho hàng, bảng giá và giao dịch. Mỗi chức năng cần bảo đảm dữ liệu được xử lý chính xác, có khả năng liên kết với những nghiệp vụ liên quan và tuân thủ phạm vi quyền của từng nhóm người dùng.

Hệ thống được xây dựng theo hướng các chức năng không hoạt động hoàn toàn độc lập mà có sự liên kết với nhau. Thông tin sản phẩm được sử dụng trong quản lý kho và giao dịch; bảng giá cung cấp dữ liệu phục vụ quá trình bán hàng; thông tin khách hàng được liên kết với giao dịch; tài khoản và quyền truy cập quyết định những chức năng mà người dùng được phép thực hiện. Vì vậy, việc xác định yêu cầu chức năng cần xem xét đồng thời từng nghiệp vụ riêng và luồng dữ liệu giữa các nghiệp vụ.

#### a. Chức năng đăng nhập và xác thực người dùng

- Hệ thống phải cung cấp chức năng đăng nhập để xác định danh tính người sử dụng trước khi cho phép truy cập vào các chức năng nghiệp vụ. Người

dùng nhập tên đăng nhập và mật khẩu thông qua giao diện Desktop, sau đó thông tin được gửi đến Backend để kiểm tra.

- Backend thực hiện kiểm tra sự tồn tại của tài khoản, trạng thái hoạt động và tính chính xác của mật khẩu. Chỉ những tài khoản hợp lệ và đang được phép hoạt động mới có thể đăng nhập thành công. Khi quá trình xác thực hoàn tất, hệ thống cung cấp thông tin xác thực để người dùng tiếp tục sử dụng các chức năng được bảo vệ.
- Khi thông tin đăng nhập không chính xác, tài khoản không tồn tại hoặc đã bị vô hiệu hóa, hệ thống phải từ chối yêu cầu và trả về thông báo phù hợp. Quá trình xác thực cần được thực hiện tại Backend để tránh phụ thuộc vào việc kiểm tra dữ liệu ở giao diện.

*b. Chức năng quản lý tài khoản*

- Hệ thống cần hỗ trợ quản lý các tài khoản được phép sử dụng ứng dụng. Quản trị viên có thể tạo tài khoản mới, cập nhật thông tin, kiểm tra trạng thái và thay đổi trạng thái hoạt động của tài khoản theo yêu cầu quản lý.
- Mỗi tài khoản được liên kết với thông tin người dùng tương ứng và được sử dụng để xác định quyền khi truy cập hệ thống. Tên đăng nhập cần bảo đảm tính duy nhất nhằm tránh nhầm lẫn trong quá trình xác thực. Mật khẩu không được lưu trực tiếp dưới dạng văn bản thông thường mà cần được xử lý bằng cơ chế mã hóa phù hợp trước khi lưu trữ.
- Hệ thống cũng phải kiểm soát những trường hợp tài khoản bị ngừng sử dụng hoặc xóa theo quy định. Những tài khoản không còn hoạt động không được tiếp tục truy cập các chức năng nghiệp vụ.

*c. Chức năng quản lý vai trò và phân quyền*

- Hệ thống phải có khả năng phân chia quyền truy cập dựa trên vai trò và nhiệm vụ của người dùng. Các nhóm như quản trị viên, người quản lý, nhân viên bán hàng và nhân viên kho có phạm vi công việc khác nhau nên không thể sử dụng toàn bộ chức năng giống nhau.

- Quản trị viên có thể quản lý vai trò, quyền và mối quan hệ giữa vai trò với các quyền cụ thể. Quyền có thể được xác định theo từng chức năng như xem dữ liệu, thêm mới, cập nhật, xóa, thực hiện giao dịch hoặc xem báo cáo.
- Khi người dùng truy cập một chức năng, hệ thống phải kiểm tra quyền trước khi cho phép thực hiện. Việc kiểm tra không chỉ được thực hiện tại giao diện mà còn phải được xử lý tại Backend nhằm ngăn người dùng gửi trực tiếp yêu cầu tới các API không thuộc phạm vi được cấp.

#### *d. Chức năng quản lý nhân viên*

- Hệ thống cần hỗ trợ lưu trữ và quản lý thông tin nhân viên phục vụ hoạt động kinh doanh. Các thông tin cần thiết có thể bao gồm mã nhân viên, họ tên, thông tin liên hệ, vị trí công việc, trạng thái hoạt động và các dữ liệu liên quan.
- Người có quyền quản lý nhân viên có thể thêm mới, cập nhật, tìm kiếm và tra cứu thông tin. Trạng thái của nhân viên cần được quản lý để phân biệt giữa nhân viên đang làm việc và những trường hợp không còn tham gia hoạt động.
- Thông tin nhân viên có thể được liên kết với tài khoản đăng nhập nhằm xác định người thực hiện các nghiệp vụ trên hệ thống. Mối liên kết này cũng tạo cơ sở để theo dõi trách nhiệm đối với những giao dịch hoặc thao tác quan trọng.

#### *e. Chức năng quản lý khách hàng*

- Chương trình cần cho phép quản lý thông tin khách hàng nhằm hỗ trợ quá trình bán hàng và tra cứu lịch sử giao dịch. Nhân viên có thể thêm mới khách hàng khi phát sinh giao dịch, cập nhật những thông tin cần thiết và tìm kiếm khách hàng đã tồn tại trong hệ thống.
- Thông tin khách hàng cần được tổ chức để tránh tạo nhiều bản ghi trùng lặp cho cùng một người khi thực hiện nhiều giao dịch khác nhau. Trong quá trình bán hàng, nhân viên có thể lựa chọn khách hàng đã có hoặc bổ sung khách hàng mới khi cần thiết.

- Các giao dịch được liên kết với khách hàng tương ứng, qua đó hỗ trợ việc tra cứu lịch sử mua hàng và phục vụ công tác quản lý sau này.

*f. Chức năng quản lý loại vàng và loại trang sức*

- Để tổ chức sản phẩm một cách thống nhất, hệ thống cần quản lý các danh mục phục vụ phân loại hàng hóa. Loại vàng được sử dụng để xác định đặc tính của vật liệu, trong khi loại trang sức dùng để phân chia sản phẩm thành các nhóm như nhẫn, vòng, dây chuyền, bông tai hoặc các nhóm khác.
- Người có quyền có thể thêm mới, cập nhật và thay đổi trạng thái của các danh mục này. Các dữ liệu danh mục sau đó được sử dụng khi tạo hoặc chỉnh sửa sản phẩm, giúp hạn chế việc nhập dữ liệu không thống nhất giữa các mặt hàng.
- Việc quản lý danh mục riêng biệt còn hỗ trợ tìm kiếm, lọc dữ liệu và lập báo cáo theo từng nhóm sản phẩm.

*g. Chức năng quản lý sản phẩm trang sức*

- Quản lý sản phẩm là một trong những chức năng trung tâm của chương trình. Hệ thống phải cho phép lưu trữ những thông tin cần thiết của từng sản phẩm như mã sản phẩm, tên sản phẩm, loại trang sức, loại vàng, trọng lượng, giá trị và trạng thái.
- Mã sản phẩm cần có tính duy nhất để phục vụ việc nhận diện và tra cứu. Khi thêm hoặc cập nhật dữ liệu, hệ thống phải kiểm tra các trường bắt buộc và tính hợp lệ của các giá trị như trọng lượng, giá hoặc thông tin phân loại.
- Người dùng có quyền có thể thêm mới, chỉnh sửa, tìm kiếm và xem chi tiết sản phẩm. Chức năng tìm kiếm cần hỗ trợ các tiêu chí phù hợp để người sử dụng có thể nhanh chóng xác định sản phẩm trong trường hợp số lượng hàng hóa lớn.
- Sản phẩm phải được liên kết với dữ liệu kho, bảng giá và giao dịch để bảo đảm quá trình quản lý xuyên suốt từ khi hàng hóa được ghi nhận đến khi được bán.

*h. Chức năng quản lý kho và tồn kho*

- Hệ thống cần quản lý thông tin kho và tình trạng hàng hóa đang được lưu trữ. Người dùng có quyền có thể tạo và cập nhật thông tin kho, đồng thời theo dõi sản phẩm đang tồn tại trong từng kho.
- Dữ liệu tồn kho phải phản ánh chính xác sự thay đổi của hàng hóa khi phát sinh nghiệp vụ nhập, xuất hoặc bán. Mỗi thay đổi liên quan đến số lượng hoặc trạng thái sản phẩm cần được kiểm soát để tránh tình trạng dữ liệu trên hệ thống không khớp với hàng hóa thực tế.
- Khi giao dịch bán hàng được hoàn tất, hệ thống phải cập nhật tồn kho tương ứng. Trường hợp sản phẩm không còn đủ điều kiện giao dịch, chương trình cần ngăn thao tác bán để tránh phát sinh dữ liệu không hợp lệ.
- Ngoài thông tin tồn kho hiện tại, hệ thống có thể lưu lại lịch sử biến động nhằm hỗ trợ việc kiểm tra và đối chiếu khi cần thiết.

*i. Chức năng quản lý bảng giá*

- Giá bán của trang sức có thể thay đổi theo từng giai đoạn nên hệ thống cần cung cấp chức năng quản lý bảng giá. Người dùng được phân quyền có thể tạo mới hoặc cập nhật giá theo loại hàng hóa và thời điểm áp dụng.
- Bảng giá cần có thông tin về thời gian hiệu lực để hệ thống xác định mức giá phù hợp khi thực hiện giao dịch. Việc thay đổi giá cần được quản lý tập trung nhằm hạn chế trường hợp các nhân viên sử dụng những mức giá khác nhau cho cùng một thời điểm.
- Thông tin bảng giá sau khi được cập nhật sẽ được sử dụng trong quá trình bán hàng và các nghiệp vụ có liên quan. Giá đã được áp dụng cho một giao dịch cần được ghi nhận tại thời điểm giao dịch để tránh ảnh hưởng khi bảng giá thay đổi về sau.

*j. Chức năng quản lý giao dịch bán hàng*

- Hệ thống cần hỗ trợ toàn bộ quá trình lập và ghi nhận giao dịch bán hàng. Nhân viên lựa chọn khách hàng, tìm kiếm sản phẩm, kiểm tra tình trạng hàng hóa và xác định giá trước khi tạo giao dịch.
- Một giao dịch có thể bao gồm một hoặc nhiều sản phẩm. Vì vậy, hệ thống cần quản lý cả thông tin chung của giao dịch và thông tin chi tiết của từng sản phẩm tham gia giao dịch. Giá trị thanh toán được tính dựa trên dữ liệu của các mặt hàng và các quy tắc nghiệp vụ đã xác định.
- Trước khi xác nhận giao dịch, Backend phải kiểm tra lại tình trạng sản phẩm, dữ liệu tồn kho và các điều kiện cần thiết. Khi giao dịch được hoàn tất, thông tin phải được lưu vào cơ sở dữ liệu và dữ liệu tồn kho được cập nhật tương ứng.
- Giao dịch đã phát sinh cần có khả năng tra cứu theo mã giao dịch, khách hàng, nhân viên thực hiện hoặc khoảng thời gian để phục vụ quá trình kiểm tra và đối chiếu.

*k. Chức năng tìm kiếm và tra cứu*

- Với số lượng dữ liệu tăng dần trong quá trình sử dụng, hệ thống cần cung cấp khả năng tìm kiếm và lọc dữ liệu tại các module chính. Người dùng có thể tìm kiếm sản phẩm, nhân viên, khách hàng hoặc giao dịch dựa trên những thuộc tính phù hợp với từng loại dữ liệu.
- Kết quả tìm kiếm phải phản ánh đúng điều kiện được nhập và hỗ trợ người dùng nhanh chóng xác định bản ghi cần thiết. Đối với những danh sách có số lượng dữ liệu lớn, hệ thống cần tổ chức việc phân trang hoặc giới hạn dữ liệu phù hợp nhằm tránh tải toàn bộ dữ liệu cùng một lúc.

*l. Chức năng báo cáo và thống kê*

- Dữ liệu được ghi nhận từ các nghiệp vụ cần được tổng hợp để hỗ trợ người quản lý theo dõi hoạt động kinh doanh. Hệ thống cần cung cấp những thông tin thống kê liên quan đến giao dịch, doanh thu, sản phẩm và hàng tồn kho.

- Người quản lý có thể lựa chọn khoảng thời gian phù hợp để xem dữ liệu tổng hợp. Báo cáo phải được xây dựng dựa trên dữ liệu giao dịch đã lưu trữ và bảo đảm sự thống nhất với thông tin trong cơ sở dữ liệu.
- Các thông tin thống kê có thể được trình bày dưới dạng số liệu tổng hợp, bảng dữ liệu hoặc biểu đồ tại giao diện Dashboard. Mục tiêu của chức năng này là hỗ trợ người quản lý nắm bắt nhanh tình trạng hoạt động mà không phải thực hiện tổng hợp thủ công từ nhiều nguồn khác nhau.

*m. Chức năng kiểm soát và xử lý dữ liệu*

- Ngoài các nghiệp vụ trực tiếp, hệ thống phải thực hiện kiểm tra dữ liệu trước khi lưu trữ. Các trường bắt buộc không được phép để trống, các dữ liệu có tính duy nhất phải được kiểm tra trùng lặp và những giá trị số phải nằm trong phạm vi hợp lệ.
- Trong trường hợp yêu cầu không đáp ứng điều kiện nghiệp vụ, hệ thống phải từ chối xử lý và trả về thông báo phù hợp. Cách xử lý lỗi cần được thống nhất giữa các API để ứng dụng Desktop có thể nhận biết và hiển thị thông tin rõ ràng cho người dùng.
- Các thao tác có liên quan đến nhiều bảng dữ liệu, đặc biệt là giao dịch và cập nhật tồn kho, cần bảo đảm tính toàn vẹn trong suốt quá trình xử lý. Nếu một bước quan trọng xảy ra lỗi, dữ liệu không được để lại ở trạng thái chỉ hoàn thành một phần.
- Các yêu cầu chức năng trên tạo thành phạm vi nghiệp vụ chính mà hệ thống cần đáp ứng. Quan hệ giữa các chức năng được tổ chức theo một quy trình liên tục từ xác thực người dùng, quản lý dữ liệu nền, quản lý hàng hóa và kho cho đến quá trình bán hàng và tổng hợp báo cáo. Đây cũng là cơ sở để tiếp tục xây dựng sơ đồ Use Case, thiết kế cơ sở dữ liệu, xác định API và triển khai các module trong những phần tiếp theo của báo cáo.

**2.1.4. Yêu cầu phi chức năng**

Bên cạnh các yêu cầu chức năng, hệ thống cần đáp ứng một số yêu cầu phi chức năng nhằm bảo đảm chương trình có thể vận hành ổn định, an toàn và thuận tiện trong

quá trình sử dụng. Nếu yêu cầu chức năng mô tả hệ thống phải thực hiện những nghiệp vụ nào thì yêu cầu phi chức năng tập trung vào cách hệ thống thực hiện các nghiệp vụ đó, bao gồm hiệu năng, độ tin cậy, bảo mật, khả năng sử dụng, tính toàn vẹn dữ liệu, khả năng bảo trì và khả năng mở rộng.

Về hiệu năng, hệ thống cần bảo đảm thời gian phản hồi phù hợp đối với các thao tác thường xuyên như đăng nhập, tra cứu sản phẩm, tìm kiếm khách hàng, xem tồn kho, tạo giao dịch và lấy dữ liệu báo cáo. Các thao tác không nên gây ra tình trạng chờ đợi kéo dài trong điều kiện sử dụng thông thường. Việc trao đổi dữ liệu giữa ứng dụng Desktop và Backend cần được tổ chức hợp lý, tránh gửi hoặc tải lượng dữ liệu không cần thiết. Đối với những danh sách có số lượng bản ghi lớn, cơ chế phân trang, tìm kiếm theo điều kiện và giới hạn dữ liệu trả về cần được áp dụng để giảm tải cho Server và cơ sở dữ liệu.

Các truy vấn dữ liệu cần được thiết kế phù hợp với cấu trúc cơ sở dữ liệu nhằm hạn chế việc truy xuất dư thừa. Những trường thường xuyên được sử dụng để tìm kiếm hoặc liên kết dữ liệu cần được xem xét xây dựng chỉ mục phù hợp. Ở tầng Backend, việc sử dụng JPA, Hibernate hoặc JDBC phải bảo đảm các truy vấn được kiểm soát và không phát sinh số lượng truy vấn không cần thiết trong một nghiệp vụ.

Về độ ổn định, hệ thống cần duy trì khả năng hoạt động liên tục trong quá trình sử dụng và xử lý đúng các trường hợp lỗi có thể phát sinh. Một lỗi tại một chức năng không nên làm ảnh hưởng đến toàn bộ chương trình. Backend cần có cơ chế xử lý ngoại lệ thống nhất để những lỗi phát sinh được kiểm soát và phản hồi về Client theo cấu trúc rõ ràng thay vì làm cho ứng dụng dừng hoạt động đột ngột.

Ứng dụng Desktop cũng cần xử lý phù hợp khi xảy ra các tình huống như Backend không phản hồi, kết nối mạng bị gián đoạn hoặc yêu cầu xử lý thất bại. Trong những trường hợp này, chương trình cần thông báo trạng thái cho người dùng và tránh để giao diện rơi vào trạng thái không xác định. Các thao tác làm thay đổi dữ liệu chỉ được xem là hoàn thành khi Backend xác nhận kết quả xử lý thành công.

Về bảo mật, người dùng phải được xác thực trước khi truy cập các chức năng yêu cầu quyền sử dụng. Mật khẩu không được lưu dưới dạng văn bản thuần túy mà cần được xử lý bằng thuật toán băm phù hợp. Sau khi đăng nhập thành công, thông tin xác thực được sử dụng để kiểm tra danh tính người dùng trong những yêu cầu tiếp theo.

Các API có liên quan đến dữ liệu nghiệp vụ cần được bảo vệ bằng cơ chế xác thực và phân quyền tại Backend. Việc ẩn một chức năng trên giao diện không được xem là biện pháp bảo mật đầy đủ, vì người dùng vẫn có thể cố gắng gửi yêu cầu trực tiếp tới API. Server phải kiểm tra quyền của tài khoản đối với từng nghiệp vụ trước khi cho phép thao tác được thực hiện.

Token xác thực cần có thời hạn sử dụng và phải được kiểm tra tính hợp lệ trước khi Server tiếp nhận yêu cầu. Khi token không hợp lệ hoặc đã hết hạn, hệ thống cần từ chối truy cập và yêu cầu người dùng thực hiện xác thực lại. Các thông tin nhạy cảm không nên được đưa vào token hoặc trả trực tiếp ra Client nếu không phục vụ cho quá trình sử dụng.

Về tính toàn vẹn dữ liệu, chương trình phải bảo đảm dữ liệu được lưu trữ chính xác và có sự nhất quán giữa các bảng có liên quan. Những trường có yêu cầu bắt buộc phải được kiểm tra trước khi ghi dữ liệu. Các giá trị có tính duy nhất như mã sản phẩm, tên đăng nhập hoặc các mã định danh nghiệp vụ cần được kiểm soát để tránh trùng lặp.

Các mối quan hệ giữa bảng cần được thiết lập thông qua khóa chính, khóa ngoại và các ràng buộc phù hợp. Điều này giúp ngăn việc tạo ra những bản ghi không có dữ liệu liên quan hoặc làm mất liên kết giữa các đối tượng nghiệp vụ. Việc kiểm tra dữ liệu cần được thực hiện đồng thời ở tầng ứng dụng và tầng cơ sở dữ liệu đối với những ràng buộc quan trọng.

Đối với những nghiệp vụ làm thay đổi nhiều nhóm dữ liệu trong cùng một quá trình, hệ thống cần sử dụng transaction để duy trì tính nhất quán. Chẳng hạn, khi một giao dịch bán hàng được hoàn tất, thông tin giao dịch, chi tiết sản phẩm và dữ liệu tồn kho có thể cần được cập nhật đồng thời. Nếu một bước trong quá trình xảy ra lỗi thì những thay đổi liên quan không được để lại ở trạng thái hoàn thành một phần.

Về khả năng sử dụng, giao diện Desktop cần được thiết kế rõ ràng, thống nhất và dễ thao tác. Các chức năng thường xuyên sử dụng phải được bố trí ở vị trí dễ nhận biết, hạn chế việc người dùng phải thực hiện quá nhiều bước để hoàn thành một nghiệp vụ. Tên chức năng, trường nhập liệu và thông báo trên giao diện cần sử dụng cách diễn đạt dễ hiểu và thống nhất giữa các màn hình.

Các biểu mẫu nhập liệu cần thể hiện rõ những trường bắt buộc và kiểm tra dữ liệu trước khi gửi đến Backend. Khi có lỗi, thông báo cần cung cấp đủ thông tin để người sử dụng xác định nội dung cần điều chỉnh. Chương trình cũng cần hạn chế các thao tác có thể gây mất dữ liệu hoặc tạo ra thay đổi ngoài ý muốn.

Giao diện giữa các module cần duy trì cách bố trí và quy tắc tương tác thống nhất. Các thành phần như bảng dữ liệu, hộp tìm kiếm, nút thêm mới, chỉnh sửa và các cửa sổ xác nhận nên có cách sử dụng tương đồng. Điều này giúp giảm thời gian làm quen với chương trình và hạn chế nhầm lẫn khi người dùng chuyển giữa các chức năng.

Về tính tương thích, ứng dụng Desktop được định hướng vận hành trên môi trường Windows. Giao diện cần có khả năng hiển thị ổn định trên các độ phân giải màn hình phổ biến và không bị sai lệch nghiêm trọng khi kích thước cửa sổ thay đổi. Phiên bản Node.js, Electron, JDK, Spring Boot và các thư viện quan trọng cần được quản lý thống nhất trong quá trình phát triển để hạn chế lỗi không tương thích giữa các môi trường.

Phần Backend và Desktop được phát triển độc lập nên việc giao tiếp giữa hai thành phần phải tuân theo một quy ước API thống nhất. Cấu trúc Request, Response, HTTP Status Code và thông tin lỗi cần được xác định rõ để tránh tình trạng Client phải xử lý khác nhau đối với từng module.

Về khả năng bảo trì, mã nguồn cần được tổ chức theo các module và tầng có trách nhiệm rõ ràng. Phía Backend phân chia các thành phần như Controller, Service, Repository, Entity, DTO và Security, trong khi phía Desktop được tổ chức thành các thành phần giao diện, trang chức năng và lớp giao tiếp API. Việc phân chia hợp lý giúp

quá trình tìm kiếm, sửa lỗi và bổ sung chức năng không ảnh hưởng quá lớn đến các phần còn lại.

Các quy tắc nghiệp vụ cần được tập trung tại tầng phù hợp thay vì lặp lại ở nhiều vị trí. Những thành phần được sử dụng chung nên được tái sử dụng thay vì xây dựng lại cho từng chức năng. Tên lớp, phương thức, biến và cấu trúc package cần có quy tắc thống nhất để mã nguồn dễ đọc và thuận lợi cho việc tiếp tục phát triển.

Về khả năng mở rộng, kiến trúc hệ thống cần cho phép bổ sung thêm các module hoặc đối tượng nghiệp vụ mà không phải thay đổi toàn bộ cấu trúc hiện tại. Việc tách ứng dụng Desktop khỏi Backend thông qua RESTful API tạo điều kiện để phân giao diện và phần xử lý nghiệp vụ có thể được nâng cấp tương đối độc lập.

Cấu trúc Backend theo nhiều tầng cũng giúp bổ sung thêm chức năng mới bằng cách mở rộng Entity, Repository, Service và Controller tương ứng. Trong tương lai, hệ thống có thể tiếp tục mở rộng theo hướng quản lý nhiều cửa hàng, bổ sung các báo cáo nâng cao hoặc xây dựng thêm các Client khác sử dụng chung hệ thống API.

Về khả năng kiểm soát lỗi và theo dõi hoạt động, Backend cần ghi nhận các thông tin cần thiết trong quá trình thực thi. Log có thể được sử dụng để theo dõi thời điểm xảy ra lỗi, chức năng đang xử lý và nguyên nhân phát sinh. Nội dung log cần phục vụ mục đích kiểm tra hệ thống nhưng không được ghi trực tiếp các thông tin nhạy cảm như mật khẩu hoặc các dữ liệu xác thực quan trọng.

Đối với các thao tác có ảnh hưởng đáng kể đến dữ liệu, hệ thống có thể lưu các thông tin như thời điểm tạo, thời điểm cập nhật và người thực hiện. Những thông tin này hỗ trợ việc truy vết khi cần đối chiếu dữ liệu và giúp xác định quá trình thay đổi của các bản ghi quan trọng.

Về khả năng sao lưu và phục hồi dữ liệu, cơ sở dữ liệu cần có phương án sao lưu phù hợp để hạn chế nguy cơ mất dữ liệu khi xảy ra lỗi phần cứng, lỗi hệ thống hoặc thao tác ngoài ý muốn. Do dữ liệu sản phẩm, giao dịch và tồn kho có vai trò quan trọng đối với hoạt động kinh doanh, khả năng phục hồi dữ liệu cần được xem xét ngay từ quá trình thiết kế hệ thống.

Các yêu cầu phi chức năng tạo ra những tiêu chí chất lượng mà chương trình cần đáp ứng trong quá trình triển khai. Việc bảo đảm hiệu năng, bảo mật, độ ổn định, tính toàn vẹn, khả năng sử dụng và khả năng bảo trì giúp hệ thống không chỉ hoàn thành đúng chức năng nghiệp vụ mà còn có thể vận hành một cách ổn định và có khả năng tiếp tục phát triển trong những giai đoạn sau.

## **2.2. Phân tích chức năng hệ thống**

### **2.2.1. Chức năng đăng nhập**

Chức năng đăng nhập được xây dựng nhằm xác thực danh tính người dùng trước khi cho phép truy cập vào các chức năng bên trong hệ thống. Đây là bước khởi đầu trong quá trình sử dụng ứng dụng, đồng thời là cơ sở để hệ thống xác định tài khoản, vai trò và phạm vi quyền của người dùng trong các thao tác tiếp theo.

Khi khởi động ứng dụng, người dùng được chuyển tới màn hình đăng nhập. Tại đây, người dùng nhập tên đăng nhập và mật khẩu đã được cấp. Sau khi xác nhận, ứng dụng Desktop gửi thông tin đăng nhập tới Backend thông qua API xác thực. Backend tiếp nhận yêu cầu, kiểm tra dữ liệu đầu vào và thực hiện quá trình xác minh tài khoản.

Trước tiên, hệ thống kiểm tra tên đăng nhập có tồn tại trong cơ sở dữ liệu hay không. Nếu không tìm thấy tài khoản tương ứng, yêu cầu đăng nhập bị từ chối. Trường hợp tài khoản tồn tại, hệ thống tiếp tục kiểm tra trạng thái hoạt động của tài khoản nhằm bảo đảm những tài khoản đã bị vô hiệu hóa hoặc không còn được phép sử dụng không thể truy cập vào chương trình.

Mật khẩu do người dùng nhập không được so sánh trực tiếp với dữ liệu dưới dạng văn bản thuần. Mật khẩu lưu trong cơ sở dữ liệu được xử lý bằng cơ chế băm trước khi lưu trữ. Khi đăng nhập, Backend sử dụng bộ mã hóa mật khẩu để kiểm tra giá trị người dùng nhập với giá trị đã được lưu. Cách xử lý này giúp hạn chế việc để lộ mật khẩu ngay cả trong trường hợp dữ liệu tài khoản bị truy cập trái phép.

Nếu tên đăng nhập, mật khẩu và trạng thái tài khoản đều hợp lệ, Backend xác nhận người dùng đăng nhập thành công. Hệ thống sau đó tạo JWT làm thông tin xác

thực cho phiên sử dụng. Token được trả về ứng dụng Desktop và được sử dụng trong các yêu cầu tiếp theo tới những API yêu cầu xác thực.

Ứng dụng Desktop tiếp nhận token từ Backend và lưu trữ để phục vụ quá trình sử dụng hệ thống. Khi người dùng thực hiện các chức năng như xem sản phẩm, quản lý kho, tạo giao dịch hoặc truy cập báo cáo, token được gửi kèm trong Request. Backend kiểm tra token trước khi chuyển yêu cầu đến tầng xử lý nghiệp vụ tương ứng.

Ngoài việc xác định danh tính, quá trình đăng nhập còn tạo cơ sở để xác định quyền truy cập của người dùng. Sau khi tài khoản được xác thực, hệ thống có thể lấy thông tin vai trò và quyền được gán cho tài khoản. Dựa trên các thông tin này, giao diện chỉ hiển thị những chức năng phù hợp, đồng thời Backend tiếp tục kiểm tra quyền khi người dùng gửi yêu cầu tới các API được bảo vệ.

Trong trường hợp thông tin đăng nhập không hợp lệ, hệ thống không cho phép người dùng truy cập vào màn hình chính. Ứng dụng hiển thị thông báo phù hợp để người dùng kiểm tra lại thông tin đã nhập. Nội dung phản hồi cần đủ để người dùng biết yêu cầu không thành công nhưng không nên cung cấp quá nhiều thông tin về cấu trúc tài khoản hoặc cơ chế xác thực của hệ thống.

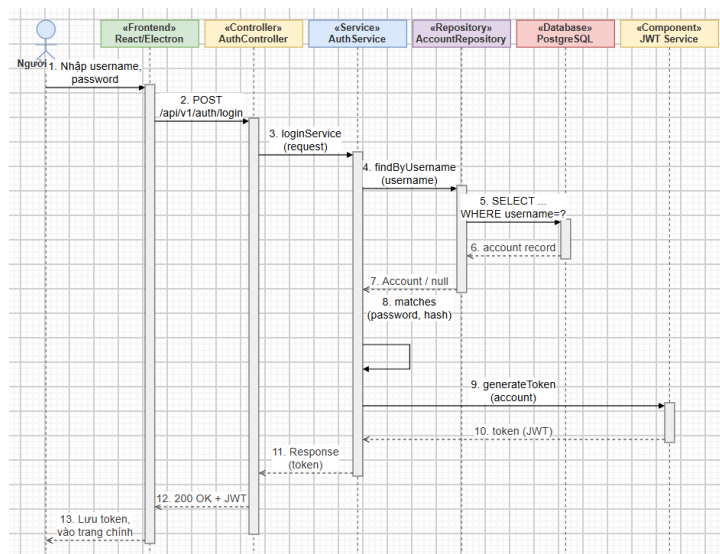
Trường hợp tài khoản đã bị vô hiệu hóa, Backend từ chối yêu cầu đăng nhập ngay cả khi mật khẩu được nhập chính xác. Cơ chế này cho phép quản trị viên kiểm soát khả năng sử dụng chương trình mà không cần xóa hoàn toàn dữ liệu tài khoản khỏi hệ thống.

Quá trình xử lý của chức năng đăng nhập được thực hiện theo luồng chính gồm: người dùng nhập thông tin đăng nhập, Desktop Client gửi Request đến Backend, Backend kiểm tra tài khoản, kiểm tra mật khẩu và trạng thái, tạo JWT khi xác thực thành công, sau đó trả kết quả về Client. Nếu một trong các điều kiện xác thực không được đáp ứng, hệ thống kết thúc quá trình và trả về thông báo lỗi.

Dữ liệu đầu vào của chức năng bao gồm tên đăng nhập và mật khẩu. Dữ liệu đầu ra trong trường hợp thành công bao gồm trạng thái xử lý và token xác thực cần

thiết cho các yêu cầu tiếp theo. Nếu đăng nhập thất bại, hệ thống trả về trạng thái lỗi tương ứng để ứng dụng Desktop hiển thị thông báo cho người dùng.

Chức năng đăng nhập được xử lý chủ yếu tại Backend thay vì chỉ kiểm tra tại phía giao diện. Cách tổ chức này giúp bảo đảm rằng người dùng không thể bỏ qua bước xác thực bằng cách can thiệp trực tiếp vào ứng dụng Desktop. Đồng thời, việc kết hợp giữa Spring Security, cơ chế mã hóa mật khẩu và JWT tạo nền tảng cho quá trình xác thực và kiểm soát truy cập của các chức năng còn lại trong hệ thống.



Hình 2.18: Sơ đồ tuần tự đăng nhập

### 2.2.2. Chức năng quản lý tài khoản

Chức năng quản lý tài khoản được xây dựng nhằm kiểm soát các tài khoản có quyền truy cập và sử dụng hệ thống. Mỗi tài khoản đại diện cho một người dùng cụ thể và được sử dụng trong quá trình đăng nhập, xác thực cũng như ghi nhận các thao tác phát sinh trong quá trình vận hành chương trình. Việc quản lý tài khoản tập trung giúp kiểm soát người dùng đang hoạt động, hạn chế tài khoản không hợp lệ và hỗ trợ duy trì tính an toàn của hệ thống.

Sau khi người dùng có quyền truy cập vào chức năng quản lý tài khoản, hệ thống hiển thị danh sách các tài khoản hiện có cùng những thông tin cần thiết như tên đăng nhập, người sử dụng, trạng thái tài khoản và một số thông tin phục vụ quản lý. Danh

sách cần được tổ chức rõ ràng để người quản trị có thể nhanh chóng kiểm tra tình trạng của từng tài khoản và thực hiện các thao tác phù hợp.

Hệ thống cho phép tạo mới tài khoản khi có người dùng cần được cấp quyền sử dụng chương trình. Khi tạo tài khoản, các thông tin bắt buộc phải được nhập đầy đủ và kiểm tra tính hợp lệ trước khi lưu vào cơ sở dữ liệu. Tên đăng nhập được sử dụng làm thông tin nhận diện trong quá trình xác thực nên phải đảm bảo không trùng với tài khoản đã tồn tại. Trường hợp dữ liệu không đáp ứng yêu cầu, hệ thống từ chối thao tác và trả về thông báo phù hợp.

Mật khẩu của tài khoản được xử lý theo cơ chế bảo mật trước khi được lưu trữ. Hệ thống không lưu mật khẩu dưới dạng văn bản thông thường mà thực hiện băm mật khẩu thông qua cơ chế mã hóa một chiều. Điều này giúp hạn chế khả năng lộ trực tiếp thông tin đăng nhập nếu dữ liệu tài khoản bị truy cập ngoài ý muốn. Việc kiểm tra mật khẩu khi đăng nhập cũng được thực hiện thông qua cơ chế đối chiếu giá trị đã băm thay vì giải mã mật khẩu.

Bên cạnh thao tác tạo mới, hệ thống hỗ trợ cập nhật thông tin tài khoản khi có thay đổi. Các thông tin được phép cập nhật cần được xác định rõ nhằm tránh làm thay đổi những dữ liệu có ảnh hưởng trực tiếp đến tính toàn vẹn của tài khoản. Trước khi ghi nhận thay đổi, Backend thực hiện kiểm tra dữ liệu đầu vào và xác định tài khoản cần cập nhật có tồn tại hay không.

Trạng thái tài khoản là một nội dung quan trọng trong quá trình quản lý. Hệ thống cần phân biệt giữa tài khoản đang hoạt động, tài khoản tạm thời không được phép sử dụng và tài khoản đã được đưa ra khỏi quá trình vận hành. Khi tài khoản ở trạng thái không hoạt động, người dùng không thể tiếp tục đăng nhập dù thông tin tên đăng nhập và mật khẩu chính xác. Cách xử lý này giúp người quản trị có thể ngừng quyền truy cập của một tài khoản mà không nhất thiết phải xóa ngay toàn bộ dữ liệu liên quan.

Đối với các tài khoản đã phát sinh dữ liệu nghiệp vụ, việc xóa trực tiếp khỏi cơ sở dữ liệu có thể làm mất mối liên hệ với các thông tin như giao dịch, lịch sử cập nhật hoặc người thực hiện thao tác. Vì vậy, trong những trường hợp cần ngừng sử dụng tài

khoản, hệ thống có thể thay đổi trạng thái thay vì xóa vật lý bản ghi. Cách tiếp cận này giúp giữ lại thông tin phục vụ việc đối chiếu và bảo đảm các dữ liệu đã phát sinh trước đó vẫn duy trì được mối liên hệ cần thiết.

Hệ thống cũng cần hỗ trợ chức năng tìm kiếm và tra cứu tài khoản. Khi số lượng người dùng tăng lên, việc kiểm tra thủ công toàn bộ danh sách sẽ làm giảm hiệu quả sử dụng. Người quản trị có thể tìm kiếm tài khoản dựa trên các thông tin phù hợp như tên đăng nhập, tên người sử dụng hoặc trạng thái hoạt động. Kết quả tìm kiếm được trả về theo điều kiện đã nhập để hỗ trợ việc quản lý nhanh hơn.

Trong trường hợp số lượng tài khoản lớn, danh sách có thể được tổ chức theo cơ chế phân trang nhằm tránh tải toàn bộ dữ liệu cùng một lúc. Mỗi yêu cầu chỉ lấy một lượng bản ghi phù hợp từ Backend, sau đó người dùng có thể chuyển giữa các trang hoặc kết hợp tìm kiếm để thu hẹp phạm vi dữ liệu. Cách xử lý này góp phần giảm lượng dữ liệu truyền giữa Desktop Client và Server.

Thông tin tài khoản có thể được liên kết với hồ sơ người dùng hoặc thông tin nhân viên tương ứng. Việc tách dữ liệu đăng nhập khỏi thông tin cá nhân giúp hệ thống tổ chức dữ liệu rõ ràng hơn. Tài khoản tập trung vào các thông tin phục vụ xác thực và trạng thái truy cập, trong khi những thông tin như họ tên, email, số điện thoại hoặc dữ liệu nghiệp vụ của nhân viên được quản lý tại các thành phần phù hợp.

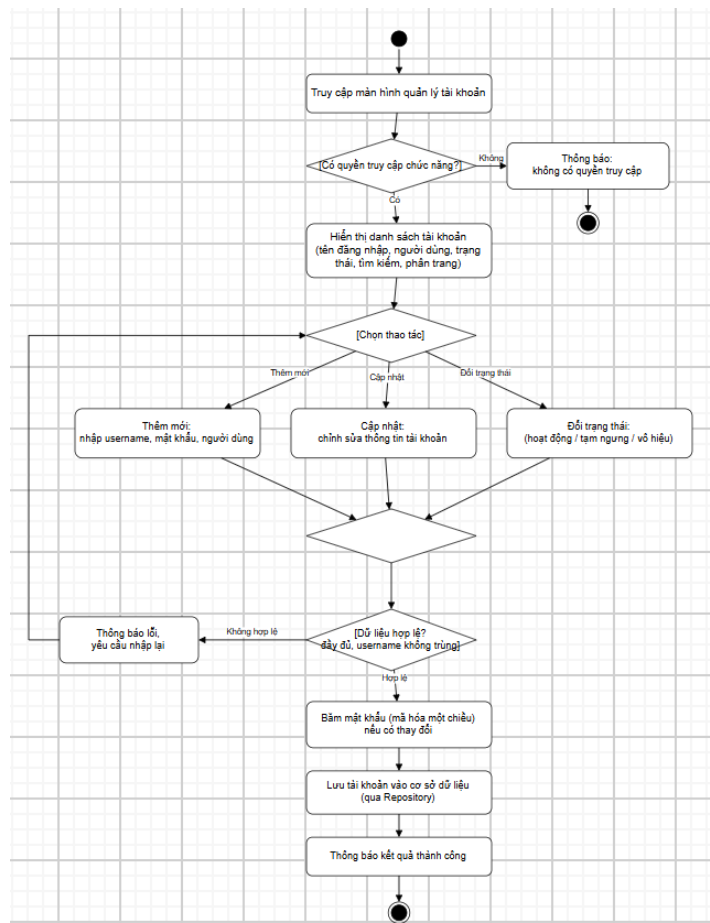
Khi người quản trị thực hiện thao tác tạo, cập nhật hoặc thay đổi trạng thái tài khoản trên ứng dụng Desktop, yêu cầu được gửi tới Backend thông qua RESTful API. Controller tiếp nhận dữ liệu và chuyển tới Service để kiểm tra điều kiện nghiệp vụ. Sau khi dữ liệu hợp lệ, Repository thực hiện thao tác với cơ sở dữ liệu và kết quả được trả về Client để cập nhật giao diện.

Quá trình xử lý tổng quát của chức năng có thể được mô tả theo trình tự: người dùng truy cập màn hình quản lý tài khoản, hệ thống tải danh sách tài khoản, người dùng lựa chọn thao tác cần thực hiện, Backend kiểm tra dữ liệu và điều kiện nghiệp vụ, sau đó lưu thay đổi vào cơ sở dữ liệu và trả kết quả về ứng dụng Desktop. Nếu quá trình

kiểm tra không đạt yêu cầu, dữ liệu không được cập nhật và hệ thống phản hồi nguyên nhân để người dùng xử lý.

Ngoài việc kiểm tra dữ liệu đầu vào, Backend cần xác định người gửi yêu cầu có được phép sử dụng chức năng quản lý tài khoản hay không. Việc kiểm tra này được thực hiện trước khi thao tác thay đổi dữ liệu được chấp nhận. Tuy nhiên, các quy tắc về vai trò, quyền và cách gán quyền cho người dùng được phân tích riêng trong chức năng phân quyền để tránh trùng lặp nội dung giữa các phần.

Chức năng quản lý tài khoản tạo nền tảng cho quá trình xác thực và kiểm soát người sử dụng trong toàn bộ chương trình. Việc quản lý tập trung thông tin đăng nhập, trạng thái và mối liên hệ giữa tài khoản với người dùng giúp hệ thống duy trì dữ liệu nhất quán, kiểm soát quyền truy cập hiệu quả và hỗ trợ theo dõi các hoạt động phát sinh trong quá trình vận hành.



Hình 2.19: Sơ đồ hoạt động quản lý tài khoản

### 2.2.3. Chức năng quản lý nhân viên

Chức năng quản lý nhân viên được xây dựng nhằm lưu trữ, cập nhật và kiểm soát thông tin của các nhân sự tham gia vào quá trình vận hành hệ thống. Nhân viên là một trong những đối tượng quan trọng vì nhiều nghiệp vụ như bán hàng, quản lý kho, theo dõi giao dịch và sử dụng tài khoản đăng nhập đều có thể gắn với từng cá nhân cụ thể. Việc quản lý thông tin nhân viên một cách tập trung giúp hệ thống duy trì dữ liệu nhất quán và hỗ trợ xác định rõ trách nhiệm của người thực hiện nghiệp vụ.

Khi truy cập chức năng quản lý nhân viên, người dùng có quyền được phép xem danh sách nhân viên hiện có trong hệ thống. Danh sách có thể bao gồm các thông tin cần thiết như mã nhân viên, họ tên, ngày sinh, giới tính, số điện thoại, email, địa chỉ, chức vụ, trạng thái làm việc và các dữ liệu liên quan khác tùy theo thiết kế thực tế của chương trình. Các trường thông tin được lựa chọn cần bảo đảm phục vụ đúng nhu cầu quản lý, tránh lưu trữ những dữ liệu không cần thiết.

Hệ thống cho phép thêm mới nhân viên khi đơn vị có nhu cầu bổ sung nhân sự. Trong quá trình tạo mới, các trường bắt buộc phải được kiểm tra trước khi dữ liệu được lưu vào cơ sở dữ liệu. Mã nhân viên cần có tính duy nhất để tránh nhầm lẫn giữa các bản ghi. Các thông tin như số điện thoại hoặc email cũng cần được kiểm tra về định dạng nếu được sử dụng trong hệ thống.

Sau khi dữ liệu đầu vào được gửi từ ứng dụng Desktop, Backend tiếp nhận và thực hiện kiểm tra tính hợp lệ. Nếu phát hiện các trường bị thiếu, sai định dạng hoặc vi phạm ràng buộc dữ liệu, hệ thống từ chối thao tác và trả về thông báo để người dùng chỉnh sửa. Chỉ khi dữ liệu đáp ứng các điều kiện đã xác định thì thông tin nhân viên mới được ghi nhận vào cơ sở dữ liệu.

Bên cạnh chức năng thêm mới, hệ thống hỗ trợ cập nhật thông tin nhân viên khi có thay đổi trong quá trình làm việc. Các thay đổi có thể liên quan đến thông tin liên hệ, vị trí công việc, đơn vị làm việc hoặc trạng thái hoạt động. Việc cập nhật phải bảo đảm không làm mất đi các dữ liệu liên kết đã phát sinh trước đó, đặc biệt đối với những nhân viên đã thực hiện giao dịch hoặc có tài khoản sử dụng hệ thống.

Trạng thái nhân viên cần được quản lý rõ ràng để phản ánh tình trạng làm việc thực tế. Một nhân viên có thể đang hoạt động, tạm ngừng hoặc không còn tham gia vào hệ thống. Trong trường hợp nhân viên nghỉ việc, hệ thống nên thay đổi trạng thái thay vì xóa trực tiếp toàn bộ thông tin nếu bản ghi đó đã có liên kết với các dữ liệu nghiệp vụ trước đó. Cách xử lý này giúp duy trì lịch sử dữ liệu và hỗ trợ việc đối chiếu khi cần thiết.

Chức năng tìm kiếm và tra cứu nhân viên được sử dụng nhằm hỗ trợ người quản lý nhanh chóng xác định thông tin cần thiết trong danh sách lớn. Việc tìm kiếm có thể thực hiện dựa trên mã nhân viên, họ tên, số điện thoại, email, chức vụ hoặc trạng thái. Các điều kiện tìm kiếm cần được lựa chọn phù hợp với nhu cầu sử dụng thực tế để giảm thời gian thao tác.

Đối với danh sách có nhiều bản ghi, hệ thống có thể sử dụng cơ chế phân trang nhằm giảm lượng dữ liệu tải trong một lần. Việc phân trang giúp giao diện phản hồi nhanh hơn và hạn chế việc Backend phải trả về toàn bộ dữ liệu khi không cần thiết. Kết hợp giữa phân trang và tìm kiếm giúp người dùng quản lý thông tin nhân sự hiệu quả hơn.

Thông tin nhân viên có thể được liên kết với tài khoản đăng nhập trong trường hợp nhân viên được phép sử dụng hệ thống. Tuy nhiên, dữ liệu nhân viên và dữ liệu tài khoản nên được quản lý tách biệt về mặt chức năng. Thông tin nhân viên phục vụ quản lý hồ sơ và nghiệp vụ, trong khi tài khoản được sử dụng cho mục đích xác thực và truy cập hệ thống. Cách phân tách này giúp giảm sự phụ thuộc giữa dữ liệu nhân sự và dữ liệu đăng nhập.

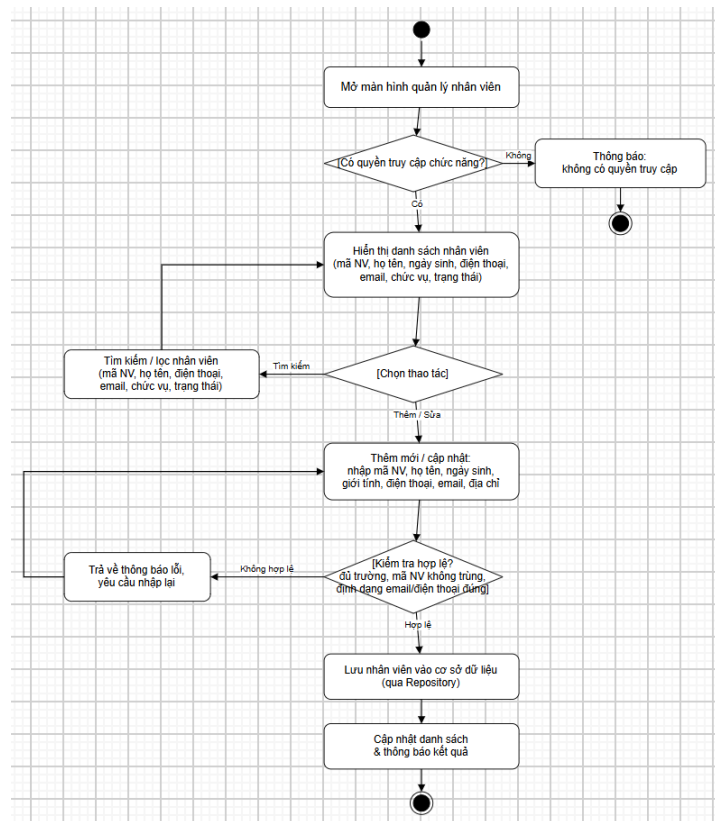
Đối với một nhân viên đã được cấp tài khoản, việc thay đổi trạng thái làm việc có thể ảnh hưởng đến khả năng sử dụng hệ thống. Chẳng hạn, khi nhân viên không còn làm việc, tài khoản tương ứng có thể cần được chuyển sang trạng thái không hoạt động. Tuy nhiên, quy trình xử lý tài khoản được quản lý tại chức năng quản lý tài khoản và cơ chế phân quyền, tránh việc trộn lẫn quá nhiều trách nhiệm vào chức năng quản lý nhân viên.

Chức năng quản lý nhân viên cũng có vai trò hỗ trợ việc ghi nhận người thực hiện các nghiệp vụ trong hệ thống. Khi phát sinh giao dịch, cập nhật dữ liệu hoặc thao tác quản lý kho, thông tin nhân viên liên quan có thể được lưu lại để phục vụ truy vết và đối chiếu sau này. Điều này giúp xác định rõ trách nhiệm đối với những thay đổi quan trọng trong dữ liệu.

Ở mức xử lý hệ thống, ứng dụng Desktop gửi yêu cầu đến Backend thông qua RESTful API. Controller tiếp nhận yêu cầu, Service xử lý các điều kiện nghiệp vụ và Repository thực hiện truy xuất hoặc cập nhật dữ liệu trong PostgreSQL. Kết quả sau đó được trả về Client để cập nhật giao diện tương ứng.

Luồng xử lý tổng quát của chức năng gồm các bước: người dùng truy cập màn hình quản lý nhân viên, hệ thống tải danh sách dữ liệu, người dùng lựa chọn thao tác thêm mới, cập nhật hoặc tìm kiếm, Backend kiểm tra dữ liệu và điều kiện nghiệp vụ, sau đó thực hiện thay đổi trong cơ sở dữ liệu và trả kết quả về ứng dụng. Nếu dữ liệu không hợp lệ, hệ thống không ghi nhận thay đổi và phản hồi lỗi để người dùng xử lý.

Chức năng quản lý nhân viên giúp hình thành một nguồn dữ liệu nhân sự thống nhất trong toàn bộ chương trình. Thông tin nhân viên không chỉ phục vụ việc quản lý hồ sơ mà còn liên kết với các nghiệp vụ khác như tài khoản, giao dịch và quá trình theo dõi hoạt động. Việc tổ chức chức năng này một cách rõ ràng giúp hệ thống dễ dàng kiểm soát nhân sự và hỗ trợ các module khác sử dụng dữ liệu một cách nhất quán.



Hình 2.20: Biểu đồ hoạt động quản lý nhân viên

#### 2.2.4. Chức năng quản lý sản phẩm trang sức

Chức năng quản lý sản phẩm trang sức được xây dựng nhằm quản lý tập trung các thông tin liên quan đến hàng hóa đang được kinh doanh trong hệ thống. Đây là một trong những chức năng quan trọng, bởi dữ liệu sản phẩm được sử dụng làm cơ sở cho nhiều nghiệp vụ khác như quản lý kho, tồn kho, bảng giá, giao dịch mua bán và báo cáo thống kê. Việc tổ chức dữ liệu sản phẩm theo một cấu trúc thống nhất giúp quá trình tra cứu, cập nhật và khai thác thông tin được thực hiện chính xác và thuận tiện hơn.

Mỗi sản phẩm trong hệ thống được xác định bằng một mã riêng nhằm phân biệt với các sản phẩm khác. Mã sản phẩm được sử dụng trong quá trình tìm kiếm, liên kết dữ liệu và xử lý các nghiệp vụ phát sinh. Ngoài mã sản phẩm, hệ thống lưu trữ các thông tin cần thiết như tên sản phẩm, loại trang sức, loại vàng, trọng lượng, mô tả, trạng thái và các thuộc tính liên quan đến đặc điểm của sản phẩm. Việc quản lý đầy đủ các

thông tin này giúp người sử dụng dễ dàng nhận diện và phân loại hàng hóa trong quá trình vận hành.

Khi truy cập vào chức năng quản lý sản phẩm, hệ thống hiển thị danh sách các sản phẩm hiện có. Danh sách cần cung cấp những thông tin chính để người dùng có thể nhanh chóng nhận biết từng mặt hàng mà không phải mở chi tiết từng sản phẩm. Các trường dữ liệu có thể bao gồm mã sản phẩm, tên sản phẩm, loại trang sức, loại vàng, trọng lượng và trạng thái hiện tại. Những thông tin chi tiết hơn được hiển thị khi người dùng thực hiện thao tác xem chi tiết.

Hệ thống cho phép người dùng có quyền tạo mới sản phẩm. Trong quá trình thêm mới, người dùng nhập các thông tin cần thiết thông qua biểu mẫu trên ứng dụng Desktop. Những trường bắt buộc phải được nhập đầy đủ trước khi dữ liệu được gửi tới Backend. Mã sản phẩm phải bảo đảm tính duy nhất, trong khi các giá trị như trọng lượng hoặc các trường số khác cần được kiểm tra để tránh dữ liệu âm, bằng không hoặc không đúng định dạng.

Các thông tin dùng để phân loại sản phẩm cần được lấy từ những danh mục đã được quản lý trong hệ thống. Chẳng hạn, loại vàng và loại trang sức nên được lựa chọn từ dữ liệu có sẵn thay vì để người dùng nhập tự do. Cách xử lý này hạn chế tình trạng cùng một giá trị nhưng được nhập bằng nhiều cách khác nhau, đồng thời giúp quá trình lọc, tìm kiếm và thống kê dữ liệu chính xác hơn.

Sau khi nhận được yêu cầu tạo mới sản phẩm, Backend tiến hành kiểm tra dữ liệu trước khi thực hiện lưu trữ. Các điều kiện cần kiểm tra bao gồm sự đầy đủ của dữ liệu bắt buộc, tính duy nhất của mã sản phẩm, sự tồn tại của các dữ liệu tham chiếu và tính hợp lệ của các thuộc tính. Nếu một trong các điều kiện không được đáp ứng, hệ thống từ chối thao tác và trả về thông tin lỗi phù hợp để người dùng điều chỉnh.

Bên cạnh chức năng thêm mới, hệ thống cho phép cập nhật thông tin sản phẩm khi có thay đổi. Người dùng có thể chỉnh sửa các thuộc tính được phép như tên sản phẩm, loại sản phẩm, loại vàng, trọng lượng, mô tả hoặc trạng thái. Trước khi ghi nhận

thay đổi, Backend tiếp tục kiểm tra dữ liệu nhằm hạn chế trường hợp cập nhật thông tin không hợp lệ hoặc làm mất tính nhất quán của dữ liệu.

Đối với những sản phẩm đã tham gia vào các giao dịch hoặc được liên kết với các dữ liệu nghiệp vụ khác, việc xóa trực tiếp khỏi cơ sở dữ liệu có thể làm ảnh hưởng đến lịch sử hoạt động của hệ thống. Vì vậy, chương trình có thể áp dụng cơ chế thay đổi trạng thái hoặc xóa mềm thay vì loại bỏ hoàn toàn bản ghi. Sản phẩm không còn được kinh doanh có thể được chuyển sang trạng thái ngừng hoạt động nhưng thông tin vẫn được giữ lại để phục vụ việc tra cứu các giao dịch đã phát sinh trước đó.

Chức năng tìm kiếm được xây dựng nhằm hỗ trợ người dùng nhanh chóng xác định sản phẩm cần thiết trong danh sách. Việc tìm kiếm có thể thực hiện theo mã sản phẩm, tên sản phẩm hoặc các thuộc tính phù hợp khác. Ngoài ra, hệ thống có thể hỗ trợ lọc theo loại vàng, loại trang sức hoặc trạng thái sản phẩm để thu hẹp phạm vi dữ liệu cần xem.

Khi số lượng sản phẩm tăng lên, việc tải toàn bộ dữ liệu trong một lần có thể làm giảm hiệu quả xử lý của ứng dụng. Vì vậy, danh sách sản phẩm cần được tổ chức theo cơ chế phân trang. Mỗi lần Client chỉ yêu cầu một lượng dữ liệu phù hợp từ Backend, giúp giảm lượng dữ liệu truyền tải và bảo đảm giao diện phản hồi ổn định hơn.

Chức năng xem chi tiết sản phẩm cho phép người dùng kiểm tra toàn bộ thông tin của một mặt hàng cụ thể. Khi lựa chọn một sản phẩm, ứng dụng gửi yêu cầu đến Backend để lấy dữ liệu tương ứng. Thông tin chi tiết được hiển thị trên giao diện và có thể được sử dụng làm cơ sở cho việc chỉnh sửa hoặc kiểm tra sản phẩm trước khi thực hiện những nghiệp vụ tiếp theo.

Dữ liệu sản phẩm có mối liên hệ trực tiếp với kho và tồn kho nhưng các nội dung này được xử lý tại những chức năng riêng. Quản lý sản phẩm tập trung vào thông tin nhận diện và đặc điểm của hàng hóa, trong khi quản lý kho xác định nơi lưu trữ và quản lý tồn kho phản ánh số lượng hoặc trạng thái thực tế của sản phẩm tại từng thời

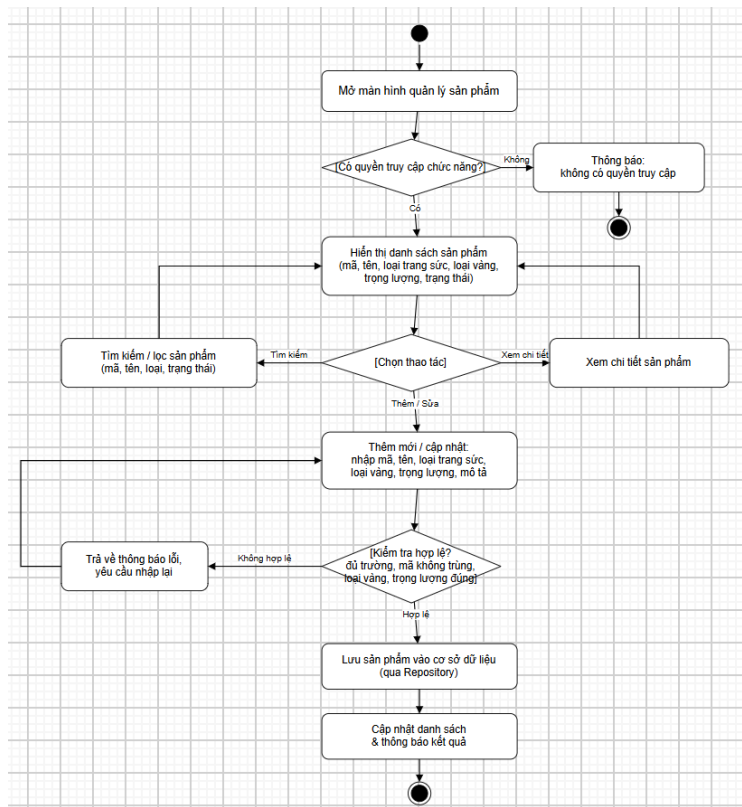
điểm. Việc phân tách này giúp trách nhiệm của từng module rõ ràng hơn và hạn chế sự chồng chéo trong quá trình xử lý.

Tương tự, thông tin về giá bán được quản lý tại chức năng bảng giá thay vì đặt toàn bộ logic giá trong module sản phẩm. Sản phẩm cung cấp thông tin cơ bản của hàng hóa, còn giá được xác định theo các quy tắc và thời điểm áp dụng của bảng giá. Khi phát sinh giao dịch, các thông tin này được liên kết với nhau để tạo ra dữ liệu bán hàng đầy đủ.

Ở phía Backend, các yêu cầu liên quan đến sản phẩm được tiếp nhận thông qua RESTful API. Controller có nhiệm vụ nhận yêu cầu từ ứng dụng Desktop, Service xử lý các điều kiện nghiệp vụ và Repository thực hiện truy xuất hoặc cập nhật dữ liệu trong PostgreSQL. Sau khi quá trình xử lý hoàn tất, kết quả được trả về Client để giao diện cập nhật lại danh sách hoặc hiển thị thông báo tương ứng.

Quá trình xử lý chức năng quản lý sản phẩm có thể bắt đầu từ việc người dùng truy cập màn hình sản phẩm, hệ thống lấy danh sách dữ liệu, sau đó người dùng thực hiện các thao tác như tìm kiếm, xem chi tiết, thêm mới hoặc cập nhật. Mọi thay đổi đều được Backend kiểm tra trước khi được ghi nhận vào cơ sở dữ liệu. Cách xử lý này giúp bảo đảm dữ liệu sản phẩm được duy trì thống nhất và có thể sử dụng ổn định trong các nghiệp vụ liên quan.

Chức năng quản lý sản phẩm trang sức giữ vai trò cung cấp nguồn dữ liệu hàng hóa chung cho toàn bộ hệ thống. Khi thông tin sản phẩm được tổ chức chính xác, các chức năng kho, tồn kho, bảng giá, giao dịch và báo cáo có thể khai thác cùng một nguồn dữ liệu, qua đó giảm sự trùng lặp và nâng cao tính nhất quán trong quá trình vận hành.



Hình 21: Sơ đồ hoạt động quản lý sản phẩm trang sức

### 2.2.5. Chức năng quản lý loại vàng

Chức năng quản lý loại vàng được xây dựng nhằm quản lý tập trung các thông tin dùng để phân loại vật liệu của sản phẩm trang sức. Trong hoạt động kinh doanh trang sức, mỗi sản phẩm có thể được chế tác từ những loại vàng khác nhau và sự khác biệt này ảnh hưởng trực tiếp đến đặc tính của sản phẩm cũng như các nghiệp vụ liên quan. Việc tách loại vàng thành một danh mục riêng giúp dữ liệu được tổ chức thống nhất, hạn chế việc nhập lặp lại hoặc sử dụng nhiều cách biểu diễn khác nhau cho cùng một loại vàng.

Mỗi loại vàng trong hệ thống được xác định bằng một mã riêng và các thông tin mô tả tương ứng. Dữ liệu có thể bao gồm mã loại vàng, tên loại vàng, hàm lượng hoặc độ tinh khiết, đơn vị thể hiện, mô tả và trạng thái sử dụng. Các thuộc tính cụ thể được lựa chọn dựa trên yêu cầu quản lý thực tế, đồng thời phải phù hợp với cấu trúc dữ liệu được thiết kế trong cơ sở dữ liệu.

Mã loại vàng được sử dụng để nhận diện và phân biệt các bản ghi trong quá trình quản lý. Giá trị này cần bảo đảm tính duy nhất nhằm tránh trường hợp nhiều loại vàng có cùng mã nhưng mang thông tin khác nhau. Tên loại vàng cũng cần được tổ chức thống nhất để thuận tiện cho việc hiển thị trên giao diện và sử dụng tại các chức năng khác của hệ thống.

Khi người dùng truy cập màn hình quản lý loại vàng, chương trình hiển thị danh sách các loại vàng đã được khai báo. Danh sách cung cấp những thông tin cơ bản cần thiết để người dùng có thể nhanh chóng kiểm tra và phân biệt từng loại. Với số lượng dữ liệu lớn, hệ thống có thể áp dụng cơ chế phân trang và tìm kiếm để giảm lượng thông tin phải tải trong một lần.

Người dùng có quyền có thể thêm mới một loại vàng khi phát sinh nhu cầu quản lý. Trong quá trình thêm mới, hệ thống yêu cầu nhập đầy đủ các trường bắt buộc và thực hiện kiểm tra dữ liệu trước khi lưu. Các giá trị liên quan đến hàm lượng hoặc độ tinh khiết cần nằm trong phạm vi hợp lệ, trong khi mã hoặc tên có yêu cầu duy nhất phải được kiểm tra để tránh trùng lặp với dữ liệu đã tồn tại.

Sau khi dữ liệu được gửi từ ứng dụng Desktop, Backend tiếp nhận yêu cầu và kiểm tra các điều kiện nghiệp vụ. Nếu dữ liệu thiếu, sai định dạng hoặc vi phạm ràng buộc, hệ thống từ chối thao tác và trả về thông tin lỗi phù hợp. Chỉ những dữ liệu đáp ứng các điều kiện đã xác định mới được lưu vào cơ sở dữ liệu.

Bên cạnh thao tác thêm mới, hệ thống hỗ trợ cập nhật thông tin loại vàng khi có thay đổi. Người dùng có thể điều chỉnh những thuộc tính được phép như tên, mô tả hoặc trạng thái. Những thay đổi có khả năng ảnh hưởng đến dữ liệu đã được sử dụng trong các sản phẩm trước đó cần được kiểm soát nhằm duy trì sự nhất quán của hệ thống.

Đối với một loại vàng đã được liên kết với sản phẩm, việc xóa trực tiếp khỏi cơ sở dữ liệu có thể làm mất mối quan hệ giữa các dữ liệu. Vì vậy, hệ thống nên ưu tiên cơ chế thay đổi trạng thái hoặc xóa mềm đối với những loại vàng không còn được sử

dụng. Bản ghi vẫn được lưu để phục vụ dữ liệu lịch sử nhưng không tiếp tục xuất hiện trong các thao tác tạo mới nếu trạng thái không còn hoạt động.

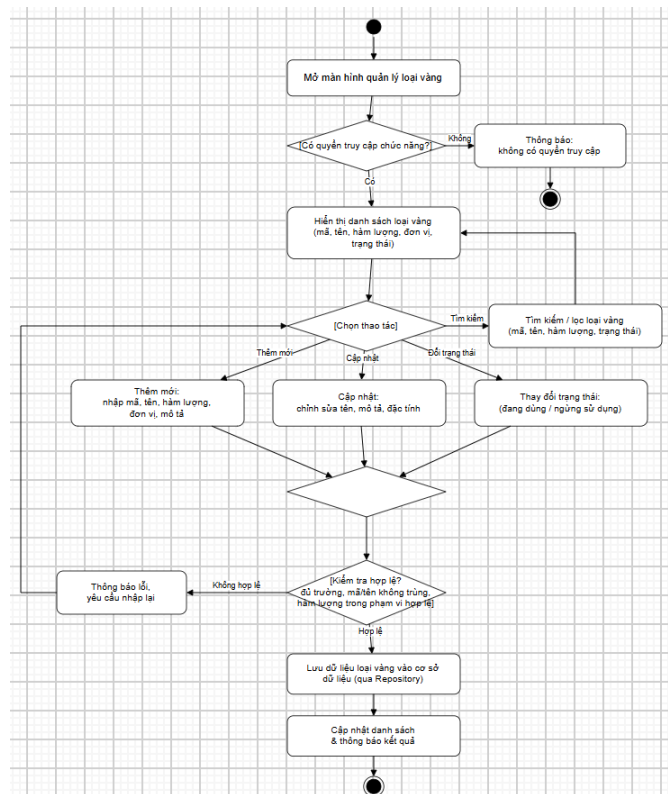
Chức năng tìm kiếm được sử dụng để hỗ trợ việc tra cứu khi danh sách loại vàng tăng lên. Người dùng có thể tìm kiếm theo mã, tên hoặc trạng thái. Việc hỗ trợ tìm kiếm giúp giảm thời gian kiểm tra và tạo thuận lợi khi cần lựa chọn hoặc cập nhật một loại vàng cụ thể.

Dữ liệu loại vàng được sử dụng như một danh mục tham chiếu trong chức năng quản lý sản phẩm trang sức. Khi tạo hoặc chỉnh sửa sản phẩm, người dùng lựa chọn loại vàng từ danh sách đã được thiết lập thay vì nhập trực tiếp bằng văn bản. Điều này giúp các sản phẩm sử dụng chung một nguồn dữ liệu và hạn chế các giá trị không thống nhất.

Chức năng quản lý loại vàng cần được phân biệt với chức năng quản lý bảng giá. Mục này tập trung vào việc xác định và quản lý các loại vàng được sử dụng trong hệ thống, trong khi việc xác định mức giá và thời gian áp dụng được xử lý tại module bảng giá. Việc tách riêng hai chức năng giúp cấu trúc nghiệp vụ rõ ràng và tránh phụ thuộc trực tiếp giữa thông tin phân loại với dữ liệu giá tại từng thời điểm.

Ở phía Backend, yêu cầu quản lý loại vàng được tiếp nhận thông qua RESTful API. Controller thực hiện tiếp nhận dữ liệu, Service chịu trách nhiệm kiểm tra điều kiện nghiệp vụ và Repository thực hiện truy xuất hoặc cập nhật dữ liệu trong PostgreSQL. Kết quả xử lý được trả về ứng dụng Desktop để giao diện cập nhật lại danh sách hoặc hiển thị thông báo tương ứng.

Chức năng quản lý loại vàng tạo ra một nguồn dữ liệu phân loại thống nhất để các chức năng khác có thể sử dụng. Việc kiểm soát mã, tên, đặc tính và trạng thái của từng loại vàng giúp hạn chế dữ liệu trùng lặp, nâng cao tính nhất quán và hỗ trợ quá trình quản lý sản phẩm trang sức trong toàn bộ hệ thống.



Hình 2.22: Biểu đồ hoạt động quản lý loại vàng

### 2.2.6. Chức năng quản lý kho

Chức năng quản lý kho được xây dựng nhằm tổ chức và kiểm soát thông tin về các khu vực lưu trữ hàng hóa trong hệ thống. Trong hoạt động kinh doanh trang sức, kho là nơi tập trung và bảo quản các sản phẩm trước khi được đưa vào giao dịch hoặc phân phối cho các bộ phận liên quan. Việc quản lý kho một cách rõ ràng giúp xác định nơi lưu trữ hàng hóa, hỗ trợ quá trình tra cứu và tạo cơ sở cho việc quản lý tồn kho ở các chức năng tiếp theo.

Mỗi kho trong hệ thống được xác định bằng một mã riêng nhằm phân biệt với các kho khác. Ngoài mã kho, hệ thống có thể lưu trữ các thông tin như tên kho, địa chỉ hoặc vị trí, người phụ trách, mô tả và trạng thái hoạt động. Tùy theo mô hình tổ chức thực tế, một đơn vị có thể có một hoặc nhiều kho khác nhau và mỗi kho có thể phục vụ một phạm vi quản lý riêng.

Mã kho cần bảo đảm tính duy nhất để tránh xảy ra nhầm lẫn trong quá trình liên kết dữ liệu. Tên kho được sử dụng để giúp người dùng dễ nhận biết trên giao diện,

trong khi các thông tin như vị trí hoặc người phụ trách hỗ trợ quá trình quản lý thực tế. Việc tổ chức kho thành một danh mục độc lập giúp các dữ liệu sản phẩm và tồn kho có thể liên kết chính xác tới nơi lưu trữ tương ứng.

Khi người dùng truy cập chức năng quản lý kho, hệ thống hiển thị danh sách các kho hiện có. Danh sách cung cấp những thông tin cơ bản để người dùng nhanh chóng kiểm tra trạng thái và nhận biết từng kho. Đối với hệ thống có nhiều kho, chức năng tìm kiếm và lọc dữ liệu giúp người dùng xác định kho cần quản lý mà không phải kiểm tra toàn bộ danh sách.

Hệ thống cho phép người dùng có quyền tạo mới kho khi đơn vị cần bổ sung địa điểm hoặc khu vực lưu trữ. Khi thực hiện thao tác này, các thông tin bắt buộc phải được nhập đầy đủ và kiểm tra trước khi gửi đến Backend. Những dữ liệu có yêu cầu duy nhất như mã kho phải được kiểm tra trùng lặp, đồng thời các trường thông tin khác cũng cần tuân theo các điều kiện đã được xác định trong thiết kế dữ liệu.

Sau khi ứng dụng Desktop gửi yêu cầu tạo kho, Backend thực hiện kiểm tra thông tin đầu vào và các điều kiện nghiệp vụ. Nếu dữ liệu không hợp lệ hoặc mã kho đã tồn tại, hệ thống từ chối thao tác và trả về thông tin phù hợp. Nếu dữ liệu đáp ứng yêu cầu, kho mới được ghi nhận vào cơ sở dữ liệu và danh sách trên giao diện được cập nhật.

Bên cạnh việc tạo mới, hệ thống hỗ trợ cập nhật thông tin kho trong trường hợp có sự thay đổi về tên, địa điểm, người phụ trách hoặc các thông tin mô tả khác. Trước khi cập nhật, Backend cần xác định kho có tồn tại hay không và kiểm tra tính hợp lệ của dữ liệu mới. Những thay đổi có thể ảnh hưởng đến các dữ liệu đã liên kết cần được kiểm soát nhằm tránh làm mất mối quan hệ giữa kho với các nghiệp vụ đã phát sinh.

Trạng thái của kho được sử dụng để xác định khả năng tiếp tục sử dụng trong các nghiệp vụ của hệ thống. Một kho có thể đang hoạt động hoặc đã ngừng sử dụng. Trong trường hợp kho không còn được sử dụng nhưng đã có dữ liệu liên quan, hệ thống nên thay đổi trạng thái thay vì xóa trực tiếp bản ghi khỏi cơ sở dữ liệu. Cách xử lý này giúp bảo toàn lịch sử dữ liệu và duy trì các mối liên hệ đã được thiết lập trước đó.

Đối với thao tác xóa kho, hệ thống cần kiểm tra các ràng buộc liên quan trước khi cho phép thực hiện. Nếu kho đã được liên kết với sản phẩm, tồn kho hoặc các dữ liệu nghiệp vụ khác, việc xóa vật lý có thể gây mất liên kết và ảnh hưởng đến tính toàn vẹn dữ liệu. Trong trường hợp đó, cơ chế ngừng hoạt động hoặc xóa mềm phù hợp hơn so với việc loại bỏ hoàn toàn kho khỏi hệ thống.

Chức năng tìm kiếm kho giúp người dùng tra cứu nhanh thông tin theo các tiêu chí như mã kho, tên kho, vị trí hoặc trạng thái. Khi dữ liệu tăng lên, việc kết hợp tìm kiếm với phân trang giúp giảm lượng dữ liệu phải tải trong một lần và cải thiện khả năng thao tác trên giao diện Desktop.

Kho có mối quan hệ trực tiếp với hàng hóa nhưng cần phân biệt rõ giữa quản lý kho và quản lý sản phẩm. Chức năng quản lý sản phẩm tập trung vào thông tin nhận diện và đặc điểm của hàng trang sức, trong khi chức năng quản lý kho tập trung vào thông tin về địa điểm hoặc đơn vị lưu trữ. Một sản phẩm có thể được liên kết với dữ liệu kho thông qua các cấu trúc quản lý tồn kho được thiết kế trong hệ thống.

Tương tự, chức năng quản lý kho không trực tiếp đảm nhiệm việc theo dõi số lượng hàng hóa tại từng thời điểm. Những thông tin về số lượng hiện có, biến động nhập xuất hoặc trạng thái tồn được xử lý tại chức năng quản lý tồn kho. Việc tách hai chức năng này giúp tránh việc một module vừa quản lý thông tin kho vừa xử lý toàn bộ biến động hàng hóa, từ đó làm rõ trách nhiệm của từng thành phần.

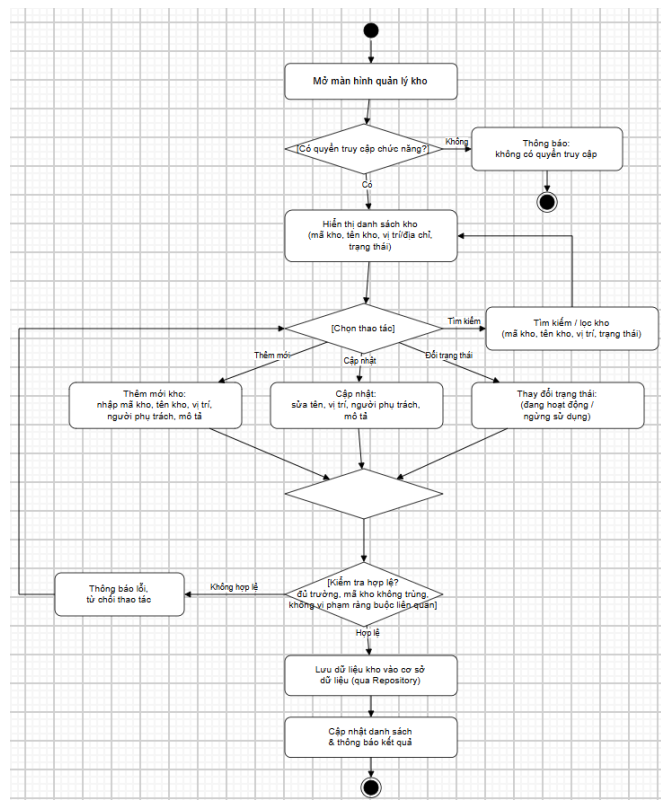
Trong trường hợp hệ thống có nhiều kho, cấu trúc quản lý cần cho phép xác định rõ hàng hóa đang thuộc kho nào. Điều này tạo cơ sở để kiểm tra và tổng hợp dữ liệu theo từng khu vực lưu trữ. Khi một kho ngừng hoạt động, hệ thống cần xử lý các dữ liệu liên quan trước khi thay đổi trạng thái nhằm tránh tình trạng hàng hóa vẫn được ghi nhận tại một kho không còn được sử dụng.

Ở phía Backend, các yêu cầu liên quan đến kho được gửi thông qua RESTful API. Controller tiếp nhận yêu cầu từ ứng dụng Desktop, Service thực hiện kiểm tra điều kiện nghiệp vụ và Repository làm việc với cơ sở dữ liệu PostgreSQL. Sau khi quá

trình xử lý hoàn thành, kết quả được trả về Client để cập nhật giao diện và thông báo trạng thái thao tác cho người dùng.

Luồng xử lý tổng quát của chức năng bắt đầu từ việc người dùng truy cập màn hình quản lý kho, hệ thống tải danh sách kho hiện có, sau đó người dùng có thể tìm kiếm, xem thông tin, thêm mới hoặc cập nhật. Dữ liệu được Backend kiểm tra trước khi thay đổi được ghi nhận vào cơ sở dữ liệu. Đối với những thao tác làm thay đổi trạng thái hoặc loại bỏ kho khỏi quá trình sử dụng, hệ thống cần kiểm tra các mối quan hệ dữ liệu để bảo đảm không ảnh hưởng tới những nghiệp vụ đã tồn tại.

Chức năng quản lý kho tạo ra cấu trúc lưu trữ cần thiết để các module khác có thể xác định chính xác nơi quản lý hàng hóa. Việc quản lý thống nhất mã kho, thông tin vị trí, trạng thái và các dữ liệu liên quan giúp hệ thống duy trì mối quan hệ rõ ràng giữa hàng hóa với nơi lưu trữ, đồng thời tạo nền tảng cho chức năng quản lý tồn kho và các nghiệp vụ phát sinh sau đó.



Hình 2.23: Biểu đồ hoạt động quản lý kho

### **2.2.7. Chức năng quản lý tồn kho**

Chức năng quản lý tồn kho được xây dựng nhằm theo dõi số lượng và trạng thái hàng hóa đang được lưu trữ tại từng kho trong hệ thống. Đây là chức năng có mối liên hệ trực tiếp với sản phẩm, kho và giao dịch mua bán, bởi mọi biến động phát sinh trong quá trình nhập, xuất hoặc bán hàng đều có thể làm thay đổi dữ liệu tồn kho. Việc quản lý tồn kho chính xác giúp hệ thống phản ánh đúng tình trạng hàng hóa thực tế và hỗ trợ người dùng kiểm soát khả năng đáp ứng các nghiệp vụ kinh doanh.

Dữ liệu tồn kho được hình thành dựa trên mối quan hệ giữa sản phẩm và kho. Một sản phẩm có thể được lưu trữ tại một hoặc nhiều kho tùy theo mô hình tổ chức, vì vậy hệ thống cần xác định rõ sản phẩm nào đang thuộc kho nào và số lượng hiện tại là bao nhiêu. Mỗi bản ghi tồn kho có thể bao gồm thông tin về sản phẩm, kho lưu trữ, số lượng hiện có, số lượng khả dụng và thời điểm cập nhật gần nhất.

Khi người dùng truy cập chức năng quản lý tồn kho, hệ thống hiển thị danh sách hàng hóa theo từng kho. Người dùng có thể theo dõi số lượng hiện tại của từng sản phẩm, kiểm tra tình trạng còn hàng hoặc hết hàng và tra cứu thông tin liên quan. Việc hiển thị dữ liệu theo từng kho giúp hạn chế sự nhầm lẫn khi cùng một loại sản phẩm được lưu trữ ở nhiều vị trí khác nhau.

Hệ thống cần hỗ trợ tìm kiếm và lọc dữ liệu tồn kho dựa trên các tiêu chí phù hợp như mã sản phẩm, tên sản phẩm, kho lưu trữ, loại vàng hoặc trạng thái tồn. Khi số lượng dữ liệu lớn, danh sách được tổ chức theo cơ chế phân trang để giảm lượng dữ liệu tải trong một lần và giúp giao diện phản hồi ổn định hơn.

Dữ liệu tồn kho không nên được cập nhật tùy ý mà phải gắn với các nghiệp vụ cụ thể. Khi phát sinh hoạt động nhập hàng, số lượng tồn của sản phẩm tại kho tương ứng được tăng lên. Khi sản phẩm được xuất khỏi kho hoặc được bán, số lượng tồn phải được giảm theo đúng số lượng đã xử lý. Mọi thay đổi cần được Backend kiểm tra trước khi ghi nhận nhằm tránh phát sinh số lượng không hợp lệ.

Trước khi thực hiện thao tác làm giảm tồn kho, hệ thống phải kiểm tra số lượng hiện có. Nếu số lượng yêu cầu lớn hơn số lượng thực tế, nghiệp vụ cần bị từ chối để

tránh trường hợp tồn kho trở thành giá trị âm. Điều kiện này đặc biệt quan trọng đối với quá trình bán hàng, vì một giao dịch chỉ có thể được hoàn tất khi sản phẩm đáp ứng đủ điều kiện về số lượng và trạng thái.

Khi giao dịch bán hàng được xác nhận thành công, dữ liệu tồn kho phải được cập nhật đồng thời với dữ liệu giao dịch. Việc cập nhật này cần được xử lý trong cùng một luồng nghiệp vụ để tránh tình trạng giao dịch đã được lưu nhưng số lượng tồn chưa thay đổi hoặc ngược lại. Trong trường hợp xảy ra lỗi tại một bước quan trọng, hệ thống cần bảo đảm dữ liệu được đưa về trạng thái nhất quán thay vì để lại kết quả chỉ hoàn thành một phần.

Đối với các nghiệp vụ có nhiều thao tác liên quan đến cơ sở dữ liệu, cơ chế transaction được sử dụng để bảo đảm tính toàn vẹn của dữ liệu. Chẳng hạn, khi một giao dịch bao gồm nhiều sản phẩm, hệ thống phải cập nhật tồn kho của từng sản phẩm tương ứng. Nếu một trong các bước cập nhật không thành công, toàn bộ quá trình cần được xử lý phù hợp để tránh chênh lệch giữa dữ liệu giao dịch và dữ liệu tồn kho.

Ngoài số lượng hiện tại, hệ thống có thể lưu lại lịch sử biến động tồn kho. Mỗi lần số lượng thay đổi, thông tin như sản phẩm, kho, loại biến động, số lượng thay đổi, thời gian và người thực hiện có thể được ghi nhận. Dữ liệu lịch sử giúp người quản lý kiểm tra nguyên nhân tăng hoặc giảm tồn kho và hỗ trợ việc đối chiếu khi phát hiện sự chênh lệch.

Lịch sử tồn kho cũng tạo cơ sở để phân biệt các nguyên nhân biến động khác nhau. Việc tăng số lượng có thể đến từ hoạt động nhập hàng hoặc điều chỉnh dữ liệu, trong khi giảm số lượng có thể phát sinh từ giao dịch bán, xuất kho hoặc các nghiệp vụ khác. Việc lưu lại nguyên nhân của từng thay đổi giúp dữ liệu có khả năng truy vết và hạn chế tình trạng không xác định được nguồn gốc của sự chênh lệch.

Trong trường hợp sản phẩm được chuyển giữa các kho, hệ thống cần bảo đảm dữ liệu ở cả kho nguồn và kho nhận được cập nhật thống nhất. Số lượng tại kho nguồn được giảm và số lượng tại kho đích được tăng theo cùng một nghiệp vụ. Trước khi thực

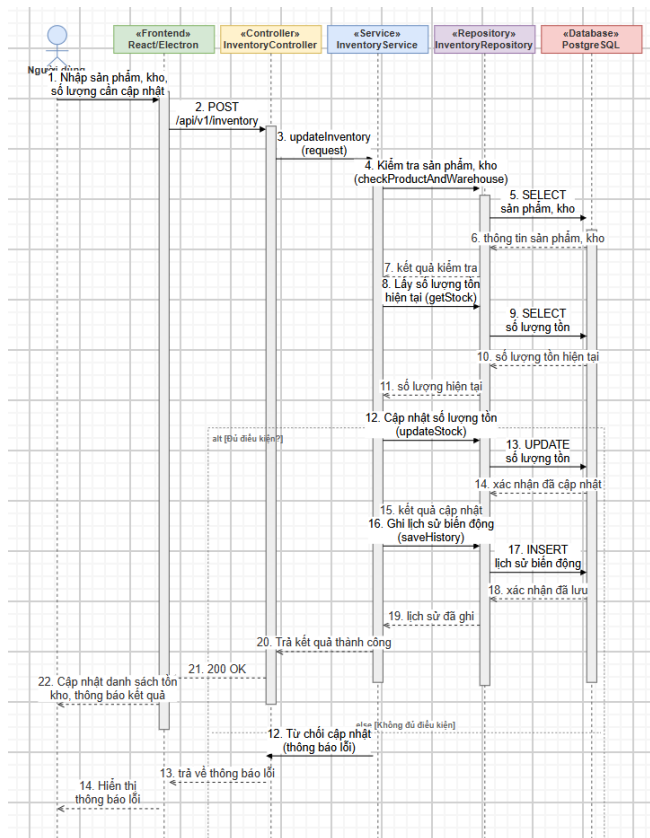
hiện, hệ thống phải kiểm tra kho nguồn có đủ số lượng sản phẩm cần chuyển và kho đích có đang ở trạng thái cho phép tiếp nhận hay không.

Để hạn chế các thay đổi ngoài ý muốn, quyền điều chỉnh tồn kho cần được kiểm soát. Không phải mọi người dùng đều được phép thay đổi trực tiếp số lượng hàng hóa. Những thao tác có ảnh hưởng đến tồn kho chỉ được thực hiện bởi người dùng có quyền phù hợp hoặc thông qua các nghiệp vụ hợp lệ của hệ thống. Cơ chế kiểm tra quyền được thực hiện tại Backend nhằm tránh việc người dùng tác động trực tiếp tới API ngoài phạm vi được cấp.

Ở phía Backend, yêu cầu liên quan đến tồn kho được tiếp nhận thông qua RESTful API. Controller nhận Request từ ứng dụng Desktop và chuyển tới Service để xử lý. Service kiểm tra thông tin sản phẩm, kho, số lượng hiện tại và các điều kiện nghiệp vụ trước khi gọi Repository cập nhật dữ liệu. Sau khi thao tác hoàn tất, kết quả được trả về Client để cập nhật thông tin hiển thị.

Luồng xử lý của chức năng quản lý tồn kho có thể bắt đầu từ việc người dùng lựa chọn kho hoặc sản phẩm cần kiểm tra, hệ thống truy vấn dữ liệu hiện tại và hiển thị kết quả. Khi có nghiệp vụ làm thay đổi số lượng, Backend kiểm tra điều kiện, thực hiện cập nhật và ghi nhận lịch sử biến động nếu cần. Kết quả sau cùng được đồng bộ lại trên giao diện để người dùng theo dõi trạng thái mới nhất của hàng hóa.

Chức năng quản lý tồn kho giữ vai trò duy trì dữ liệu về tình trạng hàng hóa tại từng thời điểm. Dữ liệu chính xác tại chức năng này giúp các nghiệp vụ bán hàng, quản lý kho và báo cáo sử dụng cùng một nguồn thông tin thống nhất, qua đó giảm nguy cơ chênh lệch giữa dữ liệu trên chương trình và hàng hóa thực tế.



Hình 2.24: Biểu đồ tuần tự quản lý tồn kho

### 2.2.8. Chức năng quản lý khách hàng

Chức năng quản lý khách hàng được xây dựng nhằm lưu trữ và quản lý các thông tin liên quan đến người mua hàng trong quá trình kinh doanh trang sức. Khách hàng là đối tượng có mối liên hệ trực tiếp với các giao dịch phát sinh, vì vậy việc tổ chức dữ liệu khách hàng một cách thống nhất giúp hỗ trợ quá trình bán hàng, tra cứu lịch sử giao dịch và hạn chế việc nhập lại thông tin nhiều lần.

Mỗi khách hàng trong hệ thống được xác định bằng một mã riêng hoặc một định danh duy nhất. Bên cạnh mã khách hàng, hệ thống có thể lưu trữ các thông tin cần thiết như họ tên, số điện thoại, email, địa chỉ, ngày sinh và một số dữ liệu liên hệ khác tùy theo phạm vi quản lý. Những thông tin được lưu trữ cần phục vụ trực tiếp cho hoạt động kinh doanh và tránh thu thập các dữ liệu không cần thiết.

Khi người dùng truy cập màn hình quản lý khách hàng, hệ thống hiển thị danh sách các khách hàng đã được ghi nhận trước đó. Danh sách cần thể hiện các thông tin

cơ bản để người dùng có thể nhanh chóng nhận biết và tra cứu từng khách hàng. Những dữ liệu chi tiết hơn có thể được hiển thị khi người dùng thực hiện thao tác xem thông tin cụ thể.

Hệ thống cho phép thêm mới khách hàng trong trường hợp phát sinh người mua chưa có trong cơ sở dữ liệu. Người dùng nhập các thông tin cần thiết thông qua biểu mẫu trên ứng dụng Desktop, sau đó dữ liệu được gửi tới Backend để kiểm tra trước khi lưu trữ. Những trường bắt buộc cần được nhập đầy đủ và các trường có định dạng riêng như số điện thoại hoặc email cần được kiểm tra tính hợp lệ.

Một yêu cầu quan trọng của chức năng này là hạn chế tạo trùng dữ liệu khách hàng. Trong thực tế, một khách hàng có thể thực hiện nhiều giao dịch khác nhau nên nếu mỗi lần mua hàng đều tạo một bản ghi mới sẽ làm dữ liệu bị phân tán và khó quản lý. Vì vậy, hệ thống cần kiểm tra một số thông tin có khả năng dùng để nhận diện như số điện thoại, email hoặc mã khách hàng trước khi tạo mới.

Trong trường hợp thông tin khách hàng đã tồn tại, nhân viên có thể lựa chọn bản ghi hiện có để sử dụng cho giao dịch thay vì tạo lại. Cách xử lý này giúp duy trì một hồ sơ khách hàng thống nhất và tạo điều kiện thuận lợi cho việc tra cứu lịch sử mua hàng sau này.

Bên cạnh chức năng thêm mới, hệ thống hỗ trợ cập nhật thông tin khách hàng khi có thay đổi. Người dùng có quyền có thể chỉnh sửa các dữ liệu như họ tên, số điện thoại, email hoặc địa chỉ. Trước khi lưu thay đổi, Backend kiểm tra tính hợp lệ của dữ liệu và bảo đảm những thông tin có yêu cầu duy nhất không bị trùng với khách hàng khác.

Việc cập nhật thông tin khách hàng không được làm ảnh hưởng đến các giao dịch đã phát sinh trước đó. Các giao dịch được liên kết với khách hàng thông qua định danh trong cơ sở dữ liệu, do đó khi thông tin liên hệ thay đổi, mối quan hệ giữa khách hàng và lịch sử giao dịch vẫn được duy trì.

Hệ thống cần cung cấp chức năng tìm kiếm và tra cứu khách hàng để hỗ trợ nhân viên trong quá trình bán hàng. Các tiêu chí tìm kiếm có thể bao gồm mã khách hàng,

họ tên, số điện thoại hoặc email. Việc tìm kiếm nhanh đặc biệt cần thiết khi khách hàng đã từng mua hàng và nhân viên cần lấy lại thông tin mà không phải nhập lại từ đầu.

Khi số lượng khách hàng tăng lên, danh sách cần được tổ chức theo cơ chế phân trang. Việc chỉ tải một phần dữ liệu trong mỗi lần yêu cầu giúp giảm lượng dữ liệu trao đổi giữa ứng dụng Desktop và Backend, đồng thời cải thiện tốc độ hiển thị danh sách. Người dùng có thể kết hợp phân trang với chức năng tìm kiếm để xác định nhanh bản ghi cần sử dụng.

Khách hàng được liên kết với dữ liệu giao dịch để hỗ trợ theo dõi lịch sử mua bán. Khi một giao dịch được tạo, hệ thống ghi nhận khách hàng tương ứng nếu có. Nhờ đó, người dùng có thể kiểm tra các giao dịch đã phát sinh đối với một khách hàng cụ thể và theo dõi thông tin liên quan khi cần thiết.

Tuy nhiên, chức năng quản lý khách hàng không trực tiếp xử lý toàn bộ nghiệp vụ bán hàng. Mục này tập trung vào việc lưu trữ, cập nhật và tra cứu dữ liệu khách hàng, trong khi các bước lựa chọn sản phẩm, tính giá trị giao dịch và ghi nhận giao dịch được xử lý tại chức năng quản lý giao dịch mua bán. Sự phân tách này giúp mỗi module có phạm vi trách nhiệm rõ ràng hơn.

Đối với những khách hàng đã có giao dịch trong hệ thống, việc xóa trực tiếp bản ghi cần được hạn chế để tránh làm mất mối liên hệ với dữ liệu lịch sử. Trong trường hợp không còn nhu cầu sử dụng thông tin khách hàng, hệ thống có thể áp dụng cơ chế thay đổi trạng thái hoặc xóa mềm thay vì loại bỏ hoàn toàn dữ liệu khỏi cơ sở dữ liệu.

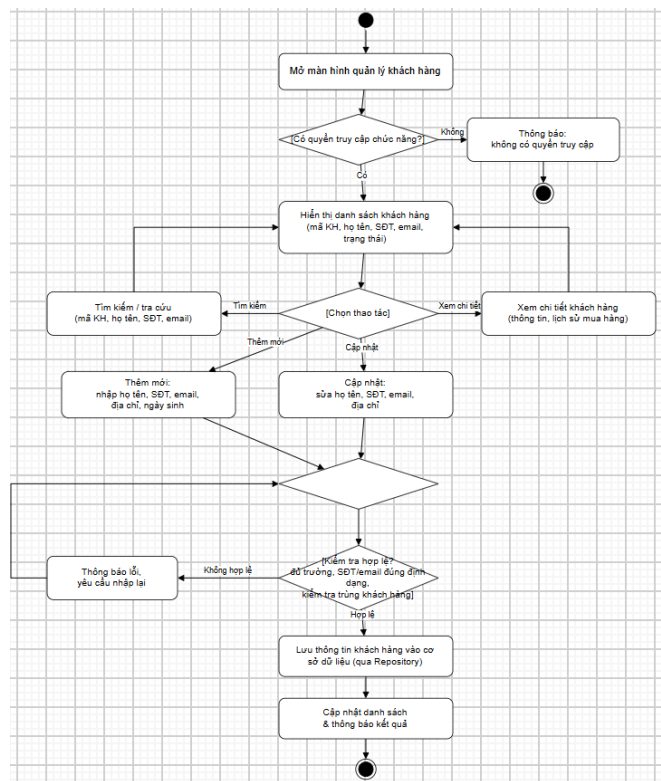
Việc quản lý trạng thái khách hàng cũng có thể được sử dụng để xác định bản ghi còn được phép sử dụng trong các nghiệp vụ mới hay không. Một khách hàng đã ngừng hoạt động vẫn có thể được giữ lại để phục vụ tra cứu các giao dịch cũ, nhưng không nhất thiết xuất hiện trong danh sách lựa chọn khi lập giao dịch mới.

Ở phía Backend, các yêu cầu liên quan đến khách hàng được tiếp nhận thông qua RESTful API. Controller tiếp nhận yêu cầu từ ứng dụng Desktop, Service thực hiện các bước kiểm tra như xác minh dữ liệu đầu vào, kiểm tra trùng lặp và xác định điều

kiện nghiệp vụ, sau đó Repository thực hiện truy xuất hoặc cập nhật dữ liệu trong PostgreSQL.

Luồng xử lý tổng quát bắt đầu khi người dùng truy cập màn hình quản lý khách hàng. Hệ thống tải danh sách dữ liệu, sau đó người dùng có thể tìm kiếm, xem chi tiết, thêm mới hoặc cập nhật thông tin. Dữ liệu được kiểm tra tại Backend trước khi thay đổi được ghi nhận. Sau khi xử lý thành công, kết quả được trả về ứng dụng Desktop để giao diện cập nhật lại thông tin tương ứng.

Chức năng quản lý khách hàng giúp hình thành một nguồn dữ liệu khách hàng tập trung và có khả năng liên kết với các giao dịch trong hệ thống. Việc hạn chế dữ liệu trùng lặp, hỗ trợ tra cứu nhanh và duy trì mối quan hệ với lịch sử mua bán giúp quá trình quản lý thông tin khách hàng được thực hiện nhất quán và thuận lợi hơn.



Hình 2.25: Biểu đồ hoạt động chức năng quản lý khách hàng

### 2.2.9. Chức năng quản lý bảng giá

Chức năng quản lý bảng giá được xây dựng nhằm kiểm soát và cập nhật các mức giá được sử dụng trong quá trình kinh doanh trang sức. Đối với mặt hàng trang

sức, giá trị sản phẩm có thể thay đổi theo từng thời điểm do phụ thuộc vào loại vàng, trọng lượng, chi phí gia công và chính sách kinh doanh. Vì vậy, việc quản lý giá cần được tổ chức thành một chức năng riêng thay vì gắn cố định hoàn toàn vào dữ liệu sản phẩm.

Mỗi bảng giá trong hệ thống cần được xác định rõ về thời điểm áp dụng, phạm vi sử dụng và trạng thái hiệu lực. Việc lưu trữ thông tin theo từng bảng giá giúp hệ thống có thể phân biệt giữa mức giá hiện tại và các mức giá đã từng được áp dụng trước đó. Điều này có ý nghĩa quan trọng đối với việc tra cứu lịch sử và bảo đảm các giao dịch cũ không bị thay đổi khi giá mới được cập nhật.

Khi người dùng truy cập màn hình quản lý bảng giá, hệ thống hiển thị danh sách các bảng giá đã được tạo. Thông tin hiển thị có thể bao gồm mã bảng giá, tên bảng giá, thời gian bắt đầu áp dụng, thời gian kết thúc nếu có, trạng thái và các thông tin mô tả liên quan. Danh sách cần giúp người sử dụng nhanh chóng xác định bảng giá nào đang được áp dụng và bảng giá nào đã hết hiệu lực.

Hệ thống cho phép người dùng có quyền tạo mới bảng giá khi cần thay đổi mức giá áp dụng trong hoạt động kinh doanh. Khi tạo mới, người dùng nhập các thông tin chung của bảng giá và thiết lập các mức giá tương ứng với từng loại hàng hóa hoặc loại vàng theo thiết kế nghiệp vụ. Các dữ liệu phải được kiểm tra trước khi lưu nhằm hạn chế trường hợp thiếu thông tin hoặc thiết lập giá không hợp lệ.

Thời gian hiệu lực là một yếu tố quan trọng trong chức năng này. Hệ thống cần xác định rõ thời điểm một bảng giá bắt đầu được sử dụng và khi nào bảng giá không còn hiệu lực. Nếu tồn tại nhiều bảng giá có khoảng thời gian chồng lấn nhau, Backend cần áp dụng quy tắc nghiệp vụ phù hợp để tránh trường hợp không xác định được mức giá nào cần sử dụng tại một thời điểm cụ thể.

Bên cạnh việc tạo mới, người dùng có thể cập nhật các thông tin được phép thay đổi trước khi bảng giá được áp dụng hoặc trong những trường hợp nghiệp vụ cho phép. Những bảng giá đã được sử dụng trong các giao dịch cần được kiểm soát chặt chẽ khi

chỉnh sửa, bởi việc thay đổi dữ liệu lịch sử có thể làm sai lệch thông tin của các giao dịch trước đó.

Đối với bảng giá không còn được sử dụng, hệ thống có thể chuyển trạng thái sang hết hiệu lực hoặc ngừng hoạt động thay vì xóa trực tiếp. Việc giữ lại dữ liệu cũ giúp bảo toàn lịch sử áp dụng giá và hỗ trợ quá trình kiểm tra khi cần đối chiếu một giao dịch đã phát sinh tại thời điểm trước đó.

Chức năng tìm kiếm và lọc bảng giá giúp người dùng tra cứu nhanh theo mã, tên, trạng thái hoặc khoảng thời gian áp dụng. Với số lượng bảng giá tăng lên theo thời gian, việc lọc theo trạng thái hiện hành hoặc lịch sử sẽ giúp quá trình quản lý thuận tiện hơn và tránh nhầm lẫn giữa các phiên bản giá khác nhau.

Bảng giá có mối liên hệ với sản phẩm và loại vàng nhưng không thay thế các chức năng quản lý những đối tượng này. Dữ liệu sản phẩm cung cấp thông tin nhận diện và đặc điểm của hàng hóa, dữ liệu loại vàng cung cấp thông tin phân loại vật liệu, còn bảng giá tập trung vào mức giá được áp dụng trong từng giai đoạn. Việc tách biệt giúp cấu trúc dữ liệu rõ ràng và hạn chế việc phải sửa trực tiếp giá trong từng sản phẩm khi có biến động.

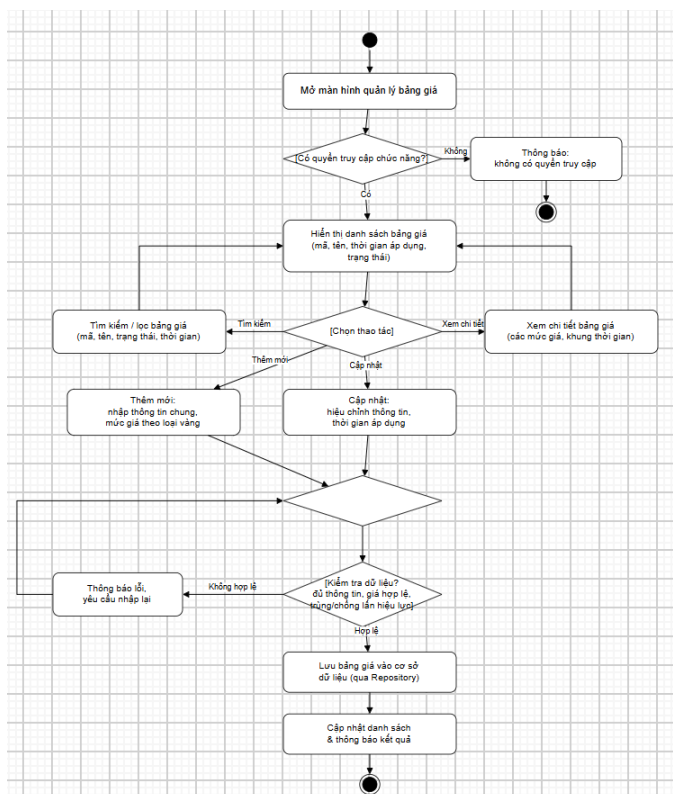
Khi phát sinh giao dịch, hệ thống cần xác định mức giá đang có hiệu lực tại thời điểm thực hiện. Mức giá đó được sử dụng làm cơ sở để tính toán giá trị giao dịch. Tuy nhiên, giá thực tế đã áp dụng cho giao dịch cần được lưu cùng dữ liệu giao dịch nhằm bảo đảm lịch sử không bị thay đổi khi bảng giá được cập nhật trong tương lai.

Ở phía Backend, các yêu cầu liên quan đến bảng giá được tiếp nhận thông qua RESTful API. Controller nhận dữ liệu từ ứng dụng Desktop, Service thực hiện kiểm tra thời gian hiệu lực, dữ liệu giá và các điều kiện nghiệp vụ, sau đó Repository truy xuất hoặc cập nhật dữ liệu trong PostgreSQL. Kết quả xử lý được trả về Client để hiển thị trạng thái tương ứng.

Luồng xử lý tổng quát bắt đầu từ việc người dùng truy cập màn hình quản lý bảng giá, hệ thống tải danh sách dữ liệu, sau đó người dùng có thể tìm kiếm, xem chi tiết, tạo mới hoặc cập nhật bảng giá. Backend kiểm tra dữ liệu, xác định tính hợp lệ của

thời gian áp dụng và trạng thái trước khi ghi nhận thay đổi. Sau khi xử lý hoàn tất, danh sách bảng giá được cập nhật lại trên giao diện.

Chức năng quản lý bảng giá giúp tách riêng dữ liệu giá khỏi thông tin cơ bản của sản phẩm, từ đó hỗ trợ việc thay đổi giá theo từng thời điểm mà không làm mất dữ liệu lịch sử. Cách tổ chức này tạo điều kiện cho hệ thống xác định đúng mức giá khi phát sinh giao dịch, đồng thời duy trì khả năng tra cứu và đối chiếu các mức giá đã từng được sử dụng.



Hình 2.26: Biểu đồ hoạt động chức năng bảng giá

### 2.2.10. Chức năng quản lý giao dịch mua bán

Chức năng quản lý giao dịch mua bán được xây dựng nhằm ghi nhận, xử lý và theo dõi các nghiệp vụ phát sinh trong quá trình kinh doanh trang sức. Đây là một trong những chức năng trung tâm của hệ thống vì có sự liên kết trực tiếp với sản phẩm, khách hàng, bảng giá, tồn kho và nhân viên thực hiện giao dịch. Mỗi giao dịch sau khi hoàn thành không chỉ tạo ra dữ liệu mua bán mà còn có thể làm thay đổi trạng thái hàng hóa

và số lượng tồn kho, do đó quá trình xử lý cần bảo đảm tính chính xác và nhất quán giữa các thành phần liên quan.

Hệ thống cho phép phân biệt các loại giao dịch theo nghiệp vụ thực tế, chẳng hạn giao dịch bán sản phẩm cho khách hàng và giao dịch mua hàng hoặc thu mua trang sức khi phạm vi kinh doanh có yêu cầu. Việc xác định loại giao dịch ngay từ thời điểm tạo giúp hệ thống lựa chọn đúng quy trình xử lý, xác định chiều biến động của hàng hóa và tổ chức dữ liệu phục vụ quá trình tra cứu sau này.

Mỗi giao dịch được xác định bằng một mã riêng nhằm bảo đảm khả năng nhận diện và tra cứu. Thông tin chung của giao dịch có thể bao gồm mã giao dịch, loại giao dịch, khách hàng, nhân viên thực hiện, thời gian phát sinh, tổng giá trị, trạng thái và các thông tin cần thiết khác. Bên cạnh dữ liệu chung, hệ thống cần quản lý phần chi tiết giao dịch để ghi nhận từng sản phẩm hoặc mặt hàng tham gia trong cùng một giao dịch.

Một giao dịch có thể chứa một hoặc nhiều sản phẩm. Đối với từng dòng chi tiết, hệ thống cần ghi nhận sản phẩm, số lượng, đơn giá áp dụng và giá trị tương ứng. Việc tách dữ liệu giao dịch thành phần thông tin chung và phần chi tiết giúp tránh lặp lại các dữ liệu như khách hàng, thời gian hoặc nhân viên đối với từng sản phẩm, đồng thời phù hợp với mô hình dữ liệu quan hệ.

Khi người dùng truy cập màn hình giao dịch, hệ thống cung cấp các chức năng cần thiết để tạo giao dịch mới, tra cứu các giao dịch đã phát sinh và xem thông tin chi tiết. Danh sách giao dịch có thể hiển thị các thông tin chính như mã giao dịch, loại giao dịch, khách hàng, nhân viên thực hiện, thời gian, tổng giá trị và trạng thái để người dùng nhanh chóng xác định bản ghi cần kiểm tra.

Đối với nghiệp vụ bán hàng, nhân viên trước tiên lựa chọn hoặc tìm kiếm khách hàng nếu giao dịch cần gắn với thông tin người mua. Sau đó, các sản phẩm được lựa chọn từ dữ liệu hiện có trong hệ thống. Trước khi một sản phẩm được đưa vào giao dịch, Backend cần kiểm tra sản phẩm có tồn tại, có đang ở trạng thái cho phép giao dịch và có đủ số lượng khả dụng hay không.

Mức giá sử dụng trong giao dịch được xác định dựa trên dữ liệu bảng giá hoặc quy tắc giá đang có hiệu lực tại thời điểm thực hiện. Hệ thống cần lấy đúng mức giá áp dụng thay vì phụ thuộc hoàn toàn vào giá trị do người dùng nhập trực tiếp. Cách xử lý này giúp giảm sai sót và bảo đảm việc tính toán được thực hiện thống nhất giữa các giao dịch.

Khi một mức giá đã được sử dụng cho giao dịch, giá trị đó cần được lưu lại trong phần chi tiết giao dịch. Đây là yêu cầu quan trọng vì bảng giá có thể thay đổi trong tương lai. Nếu giao dịch chỉ tham chiếu tới mức giá hiện tại mà không lưu giá thực tế tại thời điểm phát sinh, các giao dịch cũ có thể bị thay đổi giá trị khi bảng giá được cập nhật.

Trong quá trình lập giao dịch, hệ thống thực hiện tính toán giá trị từng sản phẩm và tổng giá trị của toàn bộ giao dịch. Việc tính toán nên được kiểm tra tại Backend để bảo đảm kết quả không phụ thuộc hoàn toàn vào dữ liệu được tính ở giao diện. Client có thể thực hiện tính toán tạm thời để hiển thị cho người dùng, nhưng Server vẫn cần xác minh lại các giá trị quan trọng trước khi ghi nhận giao dịch.

Trước khi giao dịch được xác nhận, hệ thống cần kiểm tra lại toàn bộ dữ liệu tại thời điểm xử lý. Việc kiểm tra bao gồm sự tồn tại của khách hàng nếu có, trạng thái của sản phẩm, số lượng tồn, giá đang áp dụng và quyền của người thực hiện. Bước kiểm tra cuối cùng này giúp hạn chế trường hợp dữ liệu đã thay đổi trong khoảng thời gian từ khi người dùng mở màn hình đến khi xác nhận giao dịch.

Sau khi các điều kiện được đáp ứng, Backend tiến hành tạo bản ghi giao dịch và các bản ghi chi tiết tương ứng. Đồng thời, dữ liệu tồn kho được cập nhật theo loại giao dịch. Đối với giao dịch bán, số lượng hàng hóa tại kho được giảm. Đối với giao dịch nhập hoặc mua hàng, số lượng tồn có thể được tăng sau khi nghiệp vụ được xác nhận và hàng hóa được ghi nhận vào kho.

Việc tạo giao dịch và cập nhật tồn kho cần được thực hiện trong cùng một transaction ở tầng Backend. Đây là yêu cầu cần thiết để bảo đảm tính nhất quán dữ liệu. Nếu giao dịch đã được lưu nhưng thao tác cập nhật kho thất bại, dữ liệu sẽ không còn

phản ánh đúng tình trạng thực tế. Ngược lại, nếu tồn kho đã thay đổi nhưng giao dịch chưa được ghi nhận, hệ thống cũng không thể xác định nguyên nhân biến động.

Khi một bước quan trọng trong quá trình xử lý xảy ra lỗi, transaction cần được rollback để các thay đổi chưa hoàn thành được khôi phục. Cơ chế này giúp giao dịch hoặc được xử lý đầy đủ hoặc không được ghi nhận, hạn chế tình trạng dữ liệu chỉ hoàn thành một phần.

Đối với nghiệp vụ bán hàng, hệ thống không cho phép số lượng tồn kho trở thành giá trị âm. Nếu một sản phẩm chỉ còn một số lượng nhất định nhưng người dùng yêu cầu bán nhiều hơn số lượng khả dụng, Backend phải từ chối giao dịch và phản hồi thông tin phù hợp. Việc kiểm tra này được thực hiện ngay trước khi cập nhật nhằm hạn chế sai lệch trong trường hợp có nhiều thao tác cùng sử dụng dữ liệu hàng hóa.

Sau khi giao dịch hoàn tất, trạng thái của giao dịch được cập nhật để phản ánh kết quả xử lý. Các trạng thái có thể được thiết kế phù hợp với quy trình thực tế như đang tạo, đã hoàn thành hoặc đã hủy. Việc sử dụng trạng thái giúp hệ thống phân biệt những giao dịch mới được khởi tạo với những giao dịch đã tạo ảnh hưởng chính thức tới dữ liệu kinh doanh.

Đối với giao dịch đã hoàn thành, việc chỉnh sửa trực tiếp các thông tin quan trọng cần được hạn chế. Nếu cho phép thay đổi tùy ý một giao dịch cũ, dữ liệu tồn kho, doanh thu và báo cáo có thể mất tính nhất quán. Khi cần xử lý giao dịch sai hoặc bị hủy, hệ thống nên thực hiện theo một quy trình nghiệp vụ riêng, trong đó các dữ liệu liên quan được điều chỉnh tương ứng và có khả năng truy vết.

Chức năng tìm kiếm giao dịch giúp người dùng tra cứu dữ liệu dựa trên các tiêu chí như mã giao dịch, loại giao dịch, khách hàng, nhân viên thực hiện, trạng thái hoặc khoảng thời gian. Khi số lượng giao dịch tăng lên, việc kết hợp tìm kiếm với phân trang giúp giảm thời gian tải dữ liệu và hỗ trợ người quản lý kiểm tra lịch sử hoạt động thuận tiện hơn.

Chức năng xem chi tiết giao dịch cho phép người dùng kiểm tra đầy đủ các thông tin đã được ghi nhận, bao gồm dữ liệu chung và danh sách sản phẩm thuộc giao

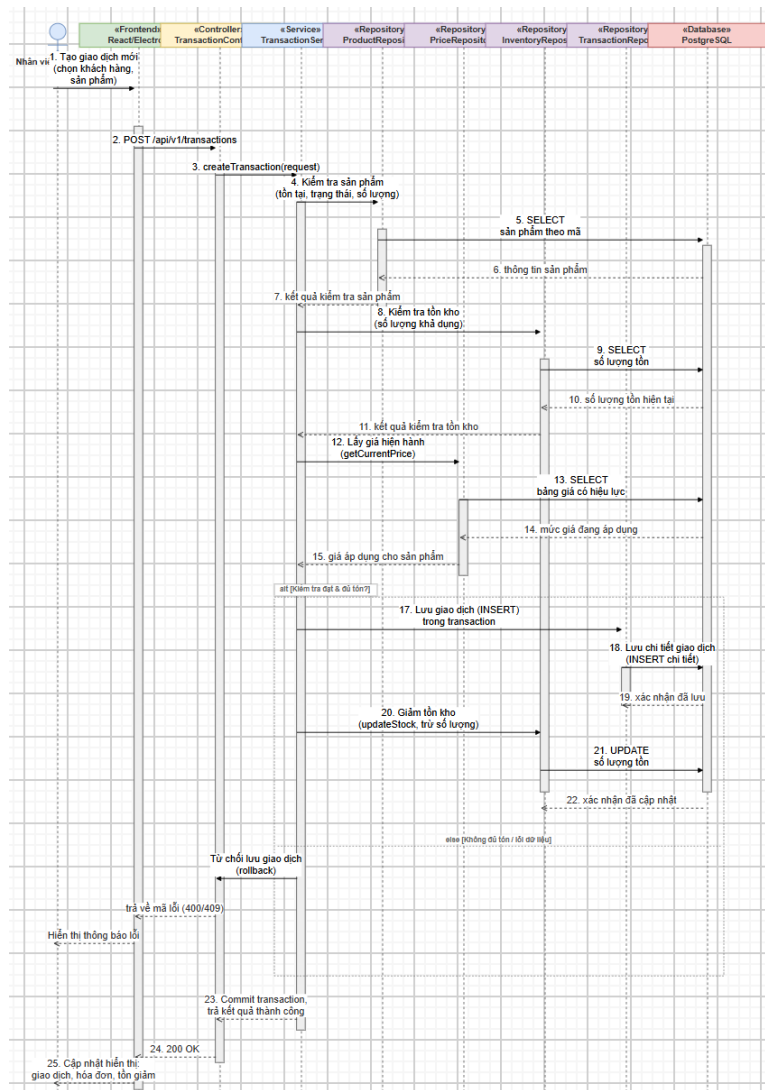
dịch. Việc giữ lại thông tin tại thời điểm phát sinh giúp quá trình đối chiếu không bị ảnh hưởng bởi những thay đổi sau này của sản phẩm, bảng giá hoặc thông tin liên quan.

Thông tin người thực hiện giao dịch cũng cần được lưu để hỗ trợ việc kiểm tra và truy vết. Dữ liệu này giúp xác định nhân viên nào đã tạo hoặc xác nhận giao dịch tại một thời điểm cụ thể. Tuy nhiên, việc phân tích chi tiết quyền của từng nhóm người dùng không được xử lý tại chức năng này mà được trình bày riêng trong phần quản lý phân quyền.

Ở phía Backend, quá trình xử lý giao dịch đi qua nhiều thành phần khác nhau. Controller tiếp nhận yêu cầu từ ứng dụng Desktop và chuyển đến Service. Service kiểm tra khách hàng, sản phẩm, giá và dữ liệu tồn kho, đồng thời thực hiện các quy tắc nghiệp vụ cần thiết. Các Repository tương ứng được sử dụng để truy xuất và cập nhật dữ liệu trong PostgreSQL. Toàn bộ các thay đổi quan trọng được quản lý trong phạm vi transaction trước khi kết quả cuối cùng được trả về Client.

Luồng xử lý chính của một giao dịch bắt đầu khi nhân viên tạo giao dịch mới, lựa chọn khách hàng và sản phẩm, hệ thống lấy các thông tin cần thiết và tính toán giá trị. Khi người dùng xác nhận, Backend tiến hành kiểm tra lại dữ liệu, tạo giao dịch, lưu chi tiết giao dịch, cập nhật tồn kho và ghi nhận kết quả. Sau khi toàn bộ quá trình thành công, ứng dụng Desktop nhận phản hồi và hiển thị trạng thái hoàn thành cho người dùng.

Chức năng quản lý giao dịch mua bán tạo ra mối liên kết giữa nhiều thành phần nghiệp vụ của hệ thống. Sản phẩm cung cấp thông tin hàng hóa, bảng giá cung cấp cơ sở xác định giá, khách hàng cung cấp thông tin người mua, tồn kho phản ánh tình trạng hàng hóa và dữ liệu giao dịch là nguồn thông tin quan trọng cho quá trình thống kê. Việc xử lý các thành phần này trong một quy trình thống nhất giúp dữ liệu kinh doanh được duy trì chính xác và có khả năng đối chiếu trong suốt quá trình vận hành.



Hình 2.27: Biểu đồ tuần tự giao dịch mua bán

### 2.2.11. Chức năng báo cáo thống kê

Chức năng báo cáo thống kê được xây dựng nhằm tổng hợp các dữ liệu phát sinh trong quá trình vận hành hệ thống và chuyển chúng thành thông tin có ý nghĩa phục vụ công tác theo dõi, đánh giá và quản lý. Khác với các chức năng nghiệp vụ trực tiếp như quản lý sản phẩm, kho hoặc giao dịch, chức năng báo cáo không làm phát sinh dữ liệu chính mà chủ yếu khai thác dữ liệu đã được lưu trữ để cung cấp cái nhìn tổng quan về tình hình hoạt động.

Dữ liệu phục vụ báo cáo được lấy từ nhiều nguồn khác nhau trong hệ thống, bao gồm giao dịch, sản phẩm, tồn kho, khách hàng, nhân viên và các thông tin liên quan.

Việc tổng hợp cân bảo đảm sử dụng cùng một nguồn dữ liệu đã được ghi nhận trong quá trình xử lý nghiệp vụ, từ đó hạn chế sự chênh lệch giữa số liệu báo cáo và dữ liệu thực tế.

Một nội dung quan trọng của chức năng này là báo cáo doanh thu. Hệ thống cần có khả năng tổng hợp giá trị các giao dịch đã hoàn thành trong một khoảng thời gian xác định. Người dùng có thể lựa chọn khoảng thời gian cần xem như theo ngày, tháng hoặc một khoảng tùy chọn, sau đó Backend thực hiện truy vấn và tổng hợp dữ liệu tương ứng.

Dữ liệu doanh thu cần được tính dựa trên những giao dịch hợp lệ và đã hoàn tất. Các giao dịch chưa hoàn thành, bị hủy hoặc không còn hiệu lực không nên được đưa vào tổng doanh thu nếu quy tắc nghiệp vụ không cho phép. Cách xử lý này giúp số liệu phản ánh đúng hoạt động kinh doanh thực tế.

Bên cạnh doanh thu, hệ thống có thể cung cấp thống kê về số lượng giao dịch phát sinh trong từng khoảng thời gian. Thông tin này giúp người quản lý theo dõi mức độ hoạt động và có thêm cơ sở để đánh giá biến động trong quá trình kinh doanh. Số lượng giao dịch có thể được trình bày kết hợp với tổng doanh thu để tạo ra cái nhìn đầy đủ hơn thay vì chỉ dựa trên một chỉ số duy nhất.

Báo cáo về sản phẩm cũng là một phần cần thiết trong hệ thống. Dữ liệu giao dịch có thể được tổng hợp để xác định số lượng sản phẩm đã bán, nhóm sản phẩm thường xuyên xuất hiện trong giao dịch hoặc tình trạng tiêu thụ của từng mặt hàng. Những thông tin này hỗ trợ người quản lý theo dõi khả năng luân chuyển hàng hóa và xác định những sản phẩm cần được chú ý.

Đối với dữ liệu tồn kho, chức năng báo cáo có thể cung cấp thông tin tổng hợp về số lượng hàng hóa hiện đang được lưu trữ tại từng kho. Người dùng có thể xem tình trạng tồn theo sản phẩm, theo kho hoặc theo nhóm hàng hóa. Việc tổng hợp này giúp quá trình kiểm tra hàng tồn trở nên thuận tiện hơn so với việc tra cứu riêng lẻ từng bản ghi.

Trong trường hợp hệ thống quản lý nhiều kho, dữ liệu có thể được phân nhóm theo từng kho để hỗ trợ so sánh tình trạng hàng hóa giữa các khu vực lưu trữ. Tuy nhiên, báo cáo chỉ thực hiện việc tổng hợp và trình bày dữ liệu; các thao tác điều chỉnh số lượng vẫn thuộc phạm vi của chức năng quản lý tồn kho.

Dữ liệu khách hàng cũng có thể được khai thác trong quá trình thống kê. Dựa trên lịch sử giao dịch, hệ thống có thể xác định số lượng khách hàng đã phát sinh mua bán trong từng giai đoạn hoặc tổng hợp số giao dịch liên quan đến từng khách hàng. Những thông tin này hỗ trợ việc theo dõi hoạt động kinh doanh nhưng không thay thế cho chức năng quản lý hồ sơ khách hàng.

Để bảo đảm báo cáo phù hợp với nhu cầu sử dụng, hệ thống cần hỗ trợ các điều kiện lọc dữ liệu. Người dùng có thể lựa chọn khoảng thời gian, trạng thái giao dịch, kho hoặc các tiêu chí phù hợp trước khi thực hiện thống kê. Backend tiếp nhận các điều kiện này và xây dựng truy vấn tương ứng để chỉ lấy những dữ liệu cần thiết.

Khi thực hiện truy vấn báo cáo, hệ thống cần chú ý đến hiệu năng do các phép tổng hợp thường phải xử lý số lượng bản ghi lớn hơn so với thao tác tra cứu thông thường. Các truy vấn cần được thiết kế phù hợp, hạn chế tải toàn bộ dữ liệu lên Backend rồi mới tính toán nếu cơ sở dữ liệu có thể thực hiện trực tiếp các phép tổng hợp như đếm, tính tổng hoặc nhóm dữ liệu.

Kết quả thống kê được trả về ứng dụng Desktop dưới dạng dữ liệu có cấu trúc. Tùy theo loại báo cáo, giao diện có thể trình bày dưới dạng các chỉ số tổng quan, bảng dữ liệu hoặc biểu đồ. Việc lựa chọn cách hiển thị cần phù hợp với loại thông tin; các số liệu tổng hợp có thể được đặt trên Dashboard, trong khi dữ liệu chi tiết hơn được trình bày dưới dạng bảng để người dùng dễ tra cứu.

Biểu đồ có thể được sử dụng để thể hiện xu hướng theo thời gian hoặc so sánh giữa các nhóm dữ liệu. Chẳng hạn, doanh thu theo từng tháng có thể được trình bày bằng biểu đồ đường hoặc cột, trong khi tỷ lệ phân bố giữa các nhóm sản phẩm có thể được thể hiện bằng dạng biểu đồ phù hợp. Phần biểu đồ chỉ đóng vai trò trực quan hóa dữ liệu đã được Backend tổng hợp và không thay đổi bản chất của dữ liệu nguồn.

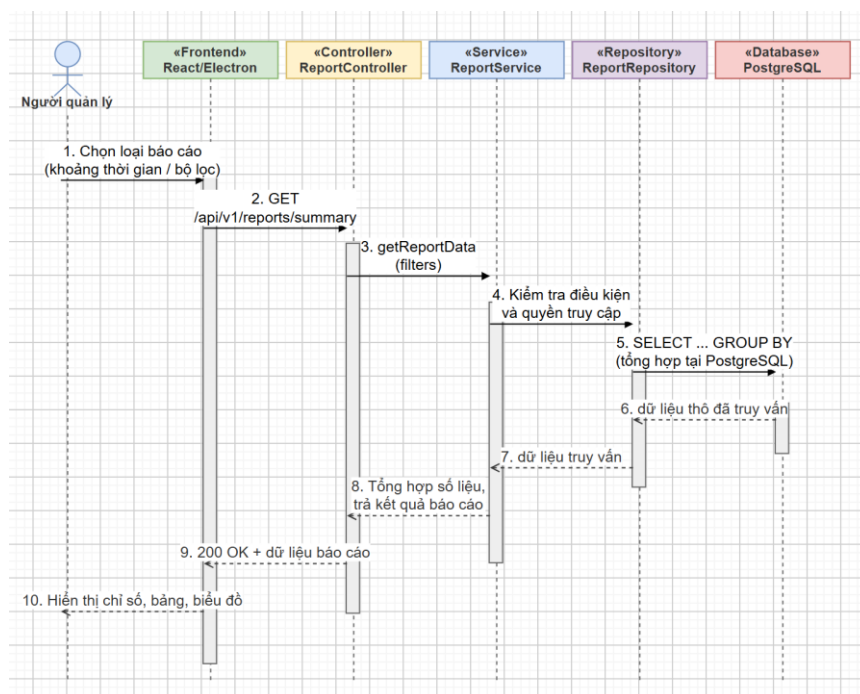
Một yêu cầu quan trọng của chức năng báo cáo là tính nhất quán với dữ liệu nghiệp vụ. Khi giao dịch được tạo, hủy hoặc thay đổi trạng thái theo quy trình hợp lệ, báo cáo cần phản ánh đúng thay đổi đó. Vì vậy, số liệu thống kê không nên được nhập hoặc chỉnh sửa thủ công mà phải được sinh ra từ dữ liệu đã tồn tại trong hệ thống.

Quyền truy cập báo cáo cũng cần được kiểm soát vì một số thông tin như doanh thu hoặc dữ liệu tổng hợp có thể chỉ phù hợp với người quản lý. Backend cần kiểm tra quyền của tài khoản trước khi trả dữ liệu báo cáo. Tuy nhiên, quy tắc cấp quyền cụ thể được trình bày tại chức năng phân quyền để tránh lặp lại nội dung giữa các mục.

Ở phía Backend, khi người dùng yêu cầu xem báo cáo, ứng dụng Desktop gửi các điều kiện thống kê tới RESTful API. Controller tiếp nhận yêu cầu và chuyển tới Service. Service xác định phạm vi dữ liệu, thực hiện các bước tổng hợp cần thiết và gọi Repository để truy vấn PostgreSQL. Kết quả sau đó được chuyển thành dữ liệu phù hợp trước khi trả về Client.

Luồng xử lý tổng quát bắt đầu từ việc người dùng mở màn hình báo cáo, lựa chọn loại báo cáo và điều kiện cần thống kê. Backend kiểm tra yêu cầu, truy vấn dữ liệu, thực hiện tính toán và tổng hợp, sau đó trả kết quả về ứng dụng Desktop. Giao diện tiếp nhận dữ liệu và trình bày dưới dạng số liệu, bảng hoặc biểu đồ tương ứng.

Chức năng báo cáo thống kê giúp chuyển đổi dữ liệu nghiệp vụ thành thông tin phục vụ công tác quản lý. Khi dữ liệu giao dịch, sản phẩm và tồn kho được ghi nhận chính xác, hệ thống có thể cung cấp các số liệu tổng hợp nhất quán, hỗ trợ người quản lý theo dõi tình hình hoạt động và đánh giá kết quả kinh doanh dựa trên dữ liệu đã được lưu trữ.



Hình 2.28: Biểu đồ tuần tự báo cáo thống kê

### 2.2.12. Chức năng phân quyền

Chức năng phân quyền được xây dựng nhằm kiểm soát phạm vi chức năng mà mỗi người dùng được phép sử dụng sau khi đăng nhập vào hệ thống. Trong quá trình vận hành, các nhóm người dùng có nhiệm vụ và trách nhiệm khác nhau nên không thể sử dụng toàn bộ chức năng với cùng một mức quyền. Việc tổ chức phân quyền giúp giới hạn khả năng truy cập, giảm nguy cơ thao tác ngoài phạm vi công việc và bảo vệ các dữ liệu quan trọng của hệ thống.

Cơ chế phân quyền được xây dựng dựa trên mối quan hệ giữa tài khoản, vai trò và quyền. Tài khoản đại diện cho người sử dụng hệ thống; vai trò được dùng để tập hợp một nhóm quyền có cùng phạm vi nghiệp vụ; quyền xác định cụ thể chức năng hoặc hành động mà người dùng được phép thực hiện. Thay vì gán trực tiếp từng quyền cho mọi tài khoản, hệ thống có thể gán vai trò cho người dùng và liên kết các quyền cần thiết với vai trò đó. Cách tổ chức này giúp việc quản lý trở nên rõ ràng hơn khi số lượng người dùng tăng lên.

Các vai trò được xác định dựa trên trách nhiệm thực tế trong quá trình vận hành. Quản trị viên có phạm vi thao tác rộng hơn đối với các chức năng quản lý hệ thống;

người quản lý cần truy cập các chức năng phục vụ công tác theo dõi và kiểm soát nghiệp vụ; nhân viên bán hàng tập trung vào các nghiệp vụ liên quan đến khách hàng và giao dịch; nhân viên kho chủ yếu làm việc với dữ liệu kho và tồn kho. Việc xác định vai trò theo trách nhiệm giúp hạn chế tình trạng cấp quyền vượt quá nhu cầu sử dụng thực tế.

Mỗi quyền trong hệ thống được xác định bằng một mã riêng để Backend có thể kiểm tra trong quá trình xử lý yêu cầu. Mã quyền cần có ý nghĩa rõ ràng và không trùng lặp. Quyền có thể được tổ chức theo từng nhóm nghiệp vụ như tài khoản, nhân viên, sản phẩm, kho, bảng giá, giao dịch hoặc báo cáo. Trong từng nhóm, quyền tiếp tục được phân chia theo hành động cụ thể như xem, tạo mới, cập nhật hoặc thực hiện một nghiệp vụ đặc biệt.

Ngoài việc xác định quyền theo chức năng, hệ thống có thể phân loại quyền theo mức độ hiển thị và thao tác trên giao diện. Một quyền có thể đại diện cho nhóm chức năng, mục menu hoặc hành động cụ thể trên một màn hình. Cách tổ chức này tạo điều kiện để ứng dụng Desktop xác định thành phần nào cần hiển thị cho người dùng sau khi đăng nhập. Người dùng không có quyền đối với một chức năng có thể không nhìn thấy menu tương ứng hoặc không được phép sử dụng các thao tác liên quan.

Tuy nhiên, việc ẩn menu hoặc nút trên giao diện chỉ có ý nghĩa hỗ trợ trải nghiệm sử dụng, không phải là cơ chế bảo mật duy nhất. Mọi yêu cầu quan trọng vẫn cần được kiểm tra tại Backend. Nếu một người dùng không có quyền nhưng cố tình gửi trực tiếp Request tới API, Server phải từ chối yêu cầu trước khi nghiệp vụ được thực hiện. Việc kiểm tra đồng thời tại Client và Server giúp cơ chế phân quyền vừa trực quan trên giao diện vừa bảo đảm an toàn ở tầng xử lý.

Sau khi người dùng đăng nhập thành công, Backend xác định tài khoản và các thông tin quyền liên quan. Khi người dùng truy cập một chức năng hoặc gửi yêu cầu tới API, hệ thống kiểm tra quyền cần thiết trước khi chuyển yêu cầu tới phần xử lý nghiệp vụ. Nếu người dùng có quyền phù hợp, yêu cầu tiếp tục được xử lý. Ngược lại, hệ thống trả về trạng thái từ chối truy cập và không thực hiện thay đổi dữ liệu.

Hệ thống cần cho phép quản trị viên quản lý danh sách vai trò và quyền theo phạm vi được cho phép. Khi tạo một vai trò mới, quản trị viên có thể lựa chọn các quyền cần thiết để hình thành tập quyền của vai trò đó. Khi yêu cầu nghiệp vụ thay đổi, quyền của vai trò có thể được điều chỉnh mà không cần sửa trực tiếp từng tài khoản đang sử dụng vai trò này.

Mối quan hệ giữa vai trò và quyền thường có tính nhiều – nhiều. Một vai trò có thể sở hữu nhiều quyền khác nhau và một quyền cũng có thể được sử dụng bởi nhiều vai trò. Vì vậy, hệ thống cần có cấu trúc dữ liệu trung gian để lưu mối quan hệ giữa hai đối tượng này. Cách thiết kế đó giúp việc bổ sung hoặc loại bỏ quyền khỏi vai trò được thực hiện linh hoạt mà không phải thay đổi cấu trúc của các bảng dữ liệu chính.

Trong trường hợp hệ thống có nhiều phạm vi quản lý, quyền còn có thể được xác định theo phạm vi áp dụng. Một quyền có thể được sử dụng ở phạm vi toàn hệ thống, trong phạm vi doanh nghiệp hoặc trong một đơn vị cụ thể tùy theo mô hình triển khai. Việc kết hợp giữa quyền thao tác và phạm vi dữ liệu giúp kiểm soát không chỉ người dùng được phép làm gì mà còn được thực hiện trên nhóm dữ liệu nào.

Khi một người dùng thay đổi vị trí công việc, hệ thống có thể thay đổi vai trò tương ứng thay vì phải thiết lập lại từng quyền từ đầu. Tương tự, khi một vai trò được cập nhật tập quyền, các tài khoản sử dụng vai trò đó có thể được áp dụng cấu hình quyền mới theo cơ chế của hệ thống. Điều này giúp giảm khối lượng quản trị và bảo đảm các tài khoản có cùng trách nhiệm sử dụng một tập quyền thống nhất.

Đối với những thao tác có ảnh hưởng lớn đến dữ liệu như quản lý tài khoản, cập nhật phân quyền, thay đổi bảng giá hoặc truy cập báo cáo quan trọng, hệ thống cần kiểm tra quyền chặt chẽ trước khi xử lý. Việc chỉ xác định người dùng đã đăng nhập là chưa đủ, bởi Authentication và Authorization giải quyết hai vấn đề khác nhau. Đăng nhập xác định người dùng là ai, còn phân quyền xác định người dùng đó được phép thực hiện những hành động nào.

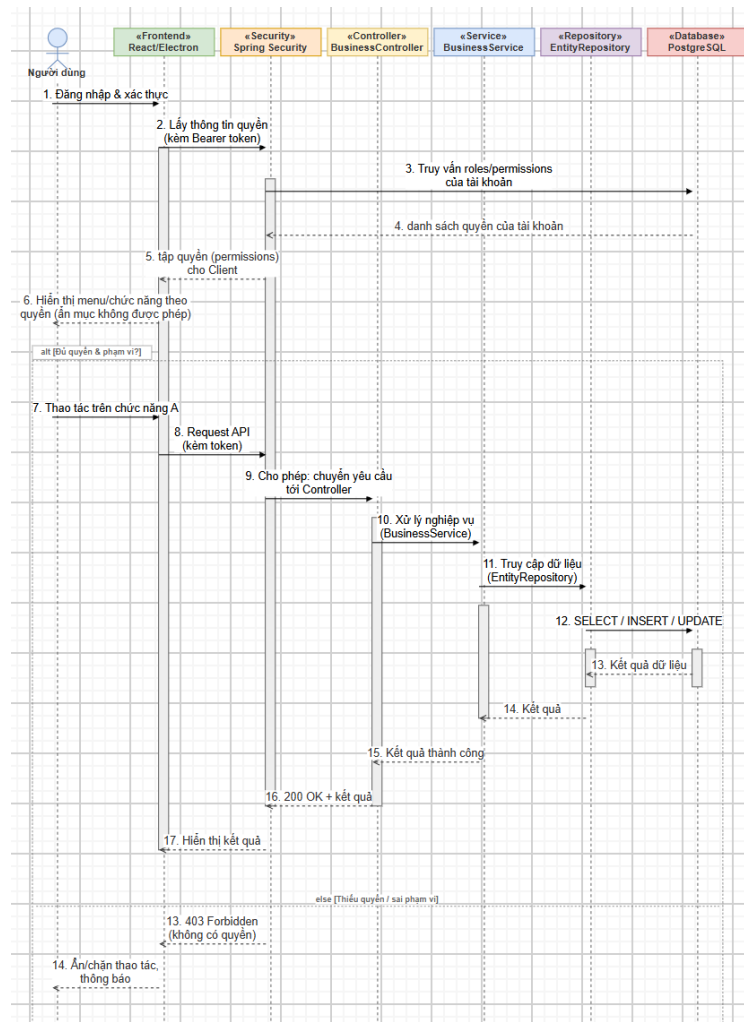
Quá trình xử lý của chức năng phân quyền được thực hiện chủ yếu tại Backend thông qua Spring Security và các thông tin Role, Permission được lưu trong cơ sở dữ

liệu. Spring Security hỗ trợ xác định trạng thái xác thực của yêu cầu, trong khi các quy tắc phân quyền của ứng dụng được sử dụng để quyết định API hoặc phương thức nghiệp vụ nào được phép thực hiện. Cách tổ chức này giúp cơ chế kiểm soát truy cập được đặt gần với tầng xử lý nghiệp vụ và không phụ thuộc hoàn toàn vào ứng dụng Desktop.

Ở phía giao diện, thông tin quyền được sử dụng để điều chỉnh các thành phần mà người dùng có thể nhìn thấy hoặc thao tác. Thanh menu có thể chỉ hiển thị các module được phép truy cập; trong từng màn hình, các nút thêm, sửa hoặc các hành động đặc biệt cũng có thể được kiểm soát theo quyền. Điều này giúp giao diện phù hợp hơn với từng nhóm người dùng và hạn chế các thao tác không cần thiết.

Luồng xử lý tổng quát bắt đầu từ việc người dùng đăng nhập và được xác thực. Hệ thống xác định vai trò và tập quyền của tài khoản, sau đó ứng dụng Desktop hiển thị các chức năng tương ứng. Khi người dùng thực hiện một thao tác, yêu cầu được gửi tới Backend cùng thông tin xác thực. Backend kiểm tra tài khoản, quyền cần thiết và phạm vi truy cập trước khi quyết định cho phép hoặc từ chối yêu cầu.

Chức năng phân quyền đóng vai trò liên kết giữa cơ chế xác thực và các module nghiệp vụ của hệ thống. Việc tổ chức quyền theo vai trò, chức năng và phạm vi giúp kiểm soát hoạt động của từng nhóm người dùng mà không làm thay đổi logic chính của các module sản phẩm, kho, giao dịch hay báo cáo. Qua đó, hệ thống có thể duy trì một cơ chế truy cập thống nhất trong toàn bộ quá trình vận hành.



Hình 2.29: Biểu đồ tuần tự chức năng phân quyền

## 2.3. Kiến trúc hệ thống

### 2.3.1. Kiến trúc tổng thể hệ thống

Hệ thống được xây dựng theo mô hình Client - Server, trong đó ứng dụng Desktop đóng vai trò là Client, Backend Java Spring Boot đảm nhiệm xử lý nghiệp vụ và PostgreSQL được sử dụng làm hệ quản trị cơ sở dữ liệu. Các thành phần được tách biệt về trách nhiệm nhưng phối hợp với nhau thông qua RESTful API, tạo thành một kiến trúc thống nhất từ giao diện người dùng đến tầng lưu trữ dữ liệu.

Phía Client được phát triển bằng React kết hợp Electron. React đảm nhiệm việc xây dựng giao diện, quản lý các thành phần hiển thị và xử lý tương tác của người dùng. Electron cung cấp môi trường để giao diện React có thể hoạt động dưới dạng ứng dụng Desktop trên hệ điều hành. Client chịu trách nhiệm tiếp nhận thao tác từ người dùng,

thực hiện một số kiểm tra dữ liệu cơ bản và gửi yêu cầu tới Backend khi cần truy xuất hoặc thay đổi dữ liệu.

Backend được xây dựng bằng Java Spring Boot và là thành phần trung tâm trong quá trình xử lý nghiệp vụ. Các yêu cầu từ Desktop Client được tiếp nhận thông qua RESTful API, sau đó đi qua các tầng Controller, Service và Repository. Mỗi tầng đảm nhiệm một nhiệm vụ riêng nhằm hạn chế việc logic nghiệp vụ, xử lý HTTP và thao tác dữ liệu bị trộn lẫn trong cùng một thành phần.

Controller đóng vai trò tiếp nhận Request và trả Response. Service chịu trách nhiệm xử lý các quy tắc nghiệp vụ của hệ thống, trong khi Repository thực hiện truy xuất và cập nhật dữ liệu. JPA, Hibernate và JDBC được sử dụng tại tầng truy cập dữ liệu tùy theo yêu cầu xử lý. Cấu trúc này giúp Backend có khả năng tổ chức mã nguồn rõ ràng và thuận lợi hơn cho việc bảo trì.

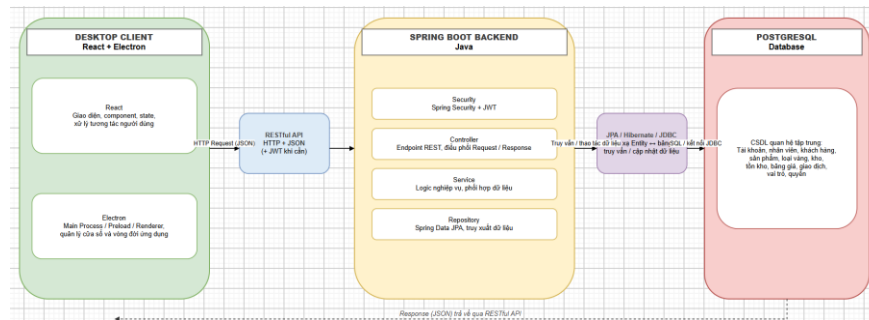
PostgreSQL là thành phần lưu trữ dữ liệu tập trung của hệ thống. Các thông tin liên quan đến tài khoản, nhân viên, khách hàng, sản phẩm, loại vàng, kho, tồn kho, bảng giá, giao dịch, vai trò và quyền được tổ chức thành các bảng có quan hệ với nhau. Những ràng buộc như khóa chính, khóa ngoại và các constraint được sử dụng để duy trì tính toàn vẹn của dữ liệu.

Các thành phần bảo mật được tích hợp tại Backend nhằm kiểm soát quá trình truy cập. Spring Security hỗ trợ xác thực người dùng và kiểm tra quyền, trong khi JWT được sử dụng để mang thông tin xác thực trong các yêu cầu từ Client. Nhờ đó, những API cần bảo vệ chỉ được xử lý khi yêu cầu đến từ người dùng hợp lệ và có đủ quyền cần thiết.

Luồng tổng quát của hệ thống bắt đầu từ thao tác của người dùng trên ứng dụng Desktop. Client tạo Request và gửi tới Backend. Backend xác thực yêu cầu, xử lý nghiệp vụ và làm việc với cơ sở dữ liệu nếu cần thiết. Kết quả sau đó được đóng gói thành Response và gửi trở lại Client để hiển thị cho người dùng.

Việc phân tách thành Desktop Client, Backend và Database giúp mỗi thành phần tập trung vào một nhóm nhiệm vụ cụ thể. Cấu trúc này hạn chế sự phụ thuộc trực tiếp

giữa giao diện và cơ sở dữ liệu, đồng thời tạo điều kiện để các thành phần có thể được phát triển, kiểm thử và mở rộng tương đối độc lập.



Hình 2.30: Kiến trúc tổng thể hệ thống

### 2.3.2. Kiến trúc Desktop Client

Desktop Client là thành phần trực tiếp cung cấp giao diện và tiếp nhận các thao tác của người dùng. Phần này được xây dựng bằng React kết hợp Electron nhằm tận dụng khả năng phát triển giao diện theo mô hình component của React đồng thời triển khai chương trình dưới dạng ứng dụng Desktop.

React chịu trách nhiệm chính trong việc xây dựng giao diện. Các màn hình của hệ thống được chia thành nhiều thành phần như trang đăng nhập, Dashboard, quản lý sản phẩm, nhân viên, khách hàng, kho, tồn kho, bảng giá, giao dịch và báo cáo. Mỗi màn hình tiếp tục được chia thành các component nhỏ hơn như bảng dữ liệu, biểu mẫu nhập liệu, hộp thoại và thanh điều hướng.

Cách tổ chức theo component giúp hạn chế việc lặp lại mã nguồn. Những thành phần xuất hiện tại nhiều màn hình có thể được xây dựng một lần và sử dụng lại. Điều này giúp giao diện duy trì sự thống nhất và thuận lợi hơn khi cần thay đổi cách hiển thị hoặc hành vi của một thành phần dùng chung.

Electron cung cấp môi trường thực thi cho ứng dụng Desktop. Main Process chịu trách nhiệm quản lý vòng đời ứng dụng, tạo cửa sổ chương trình và xử lý các chức năng có liên quan trực tiếp đến môi trường Desktop. Renderer Process đảm nhiệm việc hiển thị giao diện React và xử lý tương tác của người dùng.

Preload Script được sử dụng làm lớp trung gian trong những trường hợp giao diện cần truy cập các chức năng của Electron hoặc môi trường hệ điều hành. Việc sử

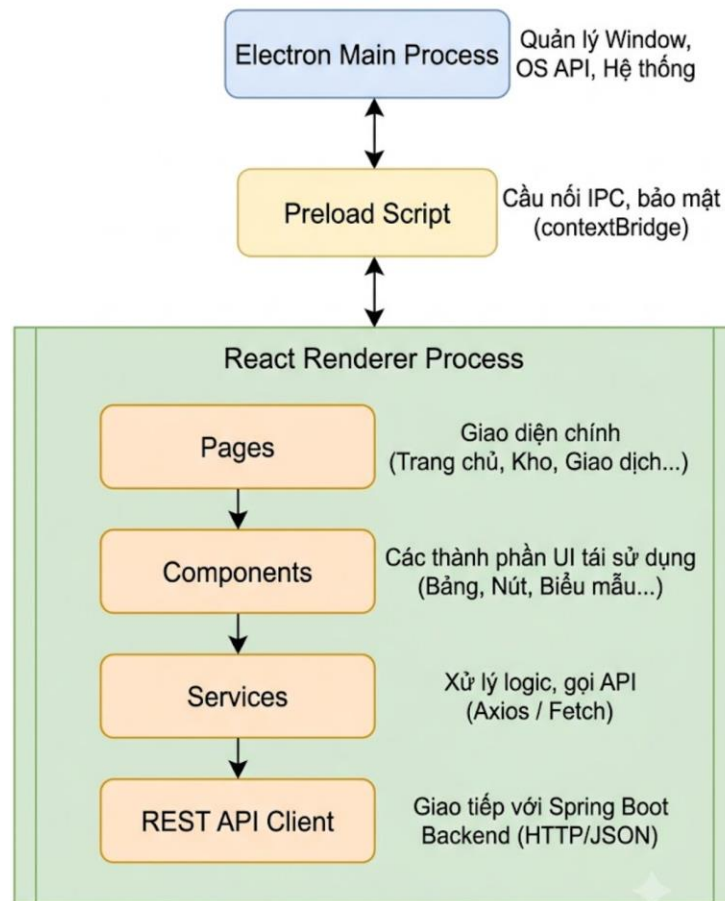
dùng Preload giúp giới hạn những API được phép cung cấp cho Renderer Process, từ đó tránh để giao diện truy cập trực tiếp vào toàn bộ môi trường Node.js.

Phần Desktop không truy cập trực tiếp PostgreSQL. Mọi thao tác cần dữ liệu đều được thực hiện thông qua các Service phía Client. Các Service này có nhiệm vụ xây dựng Request, gửi tới RESTful API và xử lý Response trả về từ Spring Boot. Việc tách phần gọi API khỏi các component giao diện giúp mã nguồn dễ tổ chức hơn.

Sau khi người dùng đăng nhập, thông tin xác thực được Client lưu trữ để sử dụng trong các Request tiếp theo. Các yêu cầu tới API được bảo vệ sẽ gửi JWT trong Header. Khi Server trả về lỗi liên quan đến xác thực hoặc quyền, ứng dụng xử lý kết quả và hiển thị trạng thái phù hợp cho người dùng.

Desktop Client cũng thực hiện các kiểm tra cơ bản trước khi gửi dữ liệu. Các biểu mẫu có thể kiểm tra trường bắt buộc, kiểu dữ liệu hoặc định dạng nhằm hạn chế những Request không hợp lệ. Tuy nhiên, các kiểm tra này không thay thế Validation tại Backend vì dữ liệu gửi từ Client vẫn phải được Server xác minh lại.

Kiến trúc Desktop được tổ chức theo hướng tách giao diện, xử lý trạng thái và giao tiếp API. Cách phân chia này giúp giảm việc đưa quá nhiều trách nhiệm vào một component và tạo thuận lợi cho quá trình mở rộng số lượng màn hình trong tương lai.



Hình 2.31: Kiến trúc Desktop Client

### 2.3.3. Kiến trúc Backend Spring Boot

Backend Spring Boot đảm nhiệm phần lớn các xử lý nghiệp vụ của hệ thống và đóng vai trò trung gian giữa ứng dụng Desktop với cơ sở dữ liệu. Kiến trúc Backend được tổ chức theo mô hình nhiều tầng nhằm phân chia rõ trách nhiệm của từng thành phần và hạn chế sự phụ thuộc trực tiếp giữa các lớp.

Tầng Controller là điểm tiếp nhận các yêu cầu HTTP từ Client. Controller khai báo các endpoint của RESTful API, xác định phương thức HTTP, tiếp nhận dữ liệu đầu vào và chuyển yêu cầu đến tầng Service. Controller không đảm nhiệm trực tiếp các nghiệp vụ phức tạp mà chủ yếu đóng vai trò điều phối Request và Response.

Tầng Service là nơi xử lý các quy tắc nghiệp vụ chính. Các thao tác như kiểm tra dữ liệu, xác minh điều kiện, tính toán, cập nhật trạng thái và phối hợp nhiều

Repository được thực hiện tại đây. Việc tập trung logic nghiệp vụ trong Service giúp các Controller giữ được cấu trúc đơn giản và tránh lặp lại quy tắc xử lý.

Tầng Repository chịu trách nhiệm làm việc với dữ liệu. Spring Data JPA được sử dụng cho phần lớn các thao tác truy xuất thông thường, trong khi JDBC hoặc Native Query có thể được sử dụng với những trường hợp cần truy vấn trực tiếp hoặc tối ưu theo yêu cầu riêng. Repository không đảm nhiệm các quy tắc nghiệp vụ mà chỉ tập trung vào thao tác dữ liệu.

Các Entity được sử dụng để biểu diễn cấu trúc dữ liệu dưới dạng đối tượng Java. Hibernate đảm nhiệm việc ánh xạ giữa Entity và các bảng trong PostgreSQL. Quan hệ giữa các Entity được xây dựng tương ứng với quan hệ giữa các bảng trong cơ sở dữ liệu.

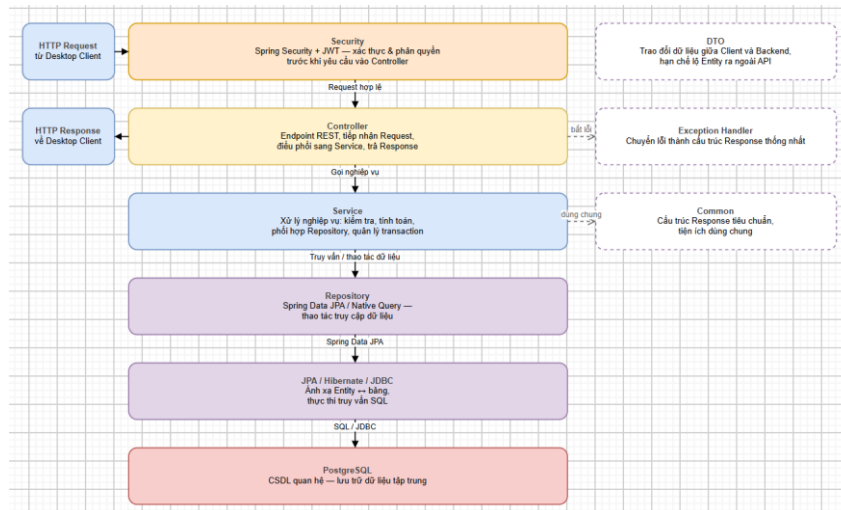
DTO được sử dụng để truyền dữ liệu giữa Client và Backend. Việc sử dụng DTO giúp hạn chế việc đưa trực tiếp Entity ra ngoài API, đồng thời cho phép hệ thống kiểm soát chính xác những trường nào được nhận và trả về trong mỗi nghiệp vụ.

Phản bảo mật được tổ chức riêng với các thành phần liên quan đến Spring Security và JWT. Trước khi một yêu cầu được chuyển tới chức năng nghiệp vụ, hệ thống kiểm tra trạng thái xác thực và quyền truy cập. Các API công khai và API cần bảo vệ được cấu hình riêng dựa trên yêu cầu của hệ thống.

Backend cũng sử dụng các thành phần xử lý Exception và chuẩn hóa Response. Khi một lỗi phát sinh, hệ thống chuyển đổi lỗi thành cấu trúc phản hồi thống nhất thay vì để mỗi Controller tự xử lý theo một cách khác nhau. Cách tổ chức này giúp Desktop Client dễ dàng xử lý kết quả trả về.

Đối với những nghiệp vụ có nhiều thao tác dữ liệu liên quan, transaction được áp dụng tại tầng Service để bảo đảm tính nhất quán. Nếu một bước trong quá trình xử lý thất bại, các thay đổi liên quan có thể được rollback nhằm tránh để dữ liệu ở trạng thái không hoàn chỉnh.

Cấu trúc nhiều tầng của Spring Boot giúp mỗi thành phần đảm nhận một trách nhiệm rõ ràng. Điều này tạo thuận lợi cho việc kiểm thử, sửa lỗi và mở rộng Backend khi hệ thống bổ sung thêm các module nghiệp vụ.



Hình 2.32: Sơ đồ kiến trúc spring boot

#### 2.3.4. Kiến trúc cơ sở dữ liệu

Cơ sở dữ liệu được thiết kế theo mô hình quan hệ và sử dụng PostgreSQL làm hệ quản trị chính. Dữ liệu được tổ chức thành nhiều bảng có mối liên hệ với nhau, trong đó mỗi bảng đại diện cho một nhóm thông tin hoặc đối tượng nghiệp vụ cụ thể của hệ thống.

Nhóm dữ liệu xác thực và phân quyền bao gồm các bảng liên quan đến tài khoản, hồ sơ người dùng, vai trò, quyền và mối quan hệ giữa vai trò với quyền. Các bảng này tạo cơ sở cho quá trình đăng nhập, xác định danh tính và kiểm soát phạm vi chức năng mà người dùng được phép sử dụng.

Nhóm dữ liệu nhân sự lưu trữ các thông tin cần thiết về nhân viên và mối liên hệ với tài khoản sử dụng hệ thống. Việc tách dữ liệu nhân viên khỏi tài khoản giúp phân biệt rõ thông tin phục vụ nghiệp vụ nhân sự với thông tin dùng cho xác thực.

Nhóm dữ liệu hàng hóa bao gồm sản phẩm trang sức, loại vàng và các dữ liệu phân loại liên quan. Sản phẩm được liên kết với những danh mục cần thiết thông qua khóa ngoại, giúp hạn chế dữ liệu nhập tự do và duy trì cách phân loại thống nhất.

Nhóm dữ liệu kho bao gồm thông tin kho và dữ liệu tồn kho. Quan hệ giữa sản phẩm và kho được sử dụng để xác định số lượng hoặc trạng thái hàng hóa tại từng vị trí lưu trữ. Việc tách bảng kho và tồn kho giúp phân biệt thông tin của nơi lưu trữ với số liệu hàng hóa thực tế.

Nhóm dữ liệu bảng giá lưu lại các mức giá và khoảng thời gian áp dụng. Việc lưu lịch sử giá cho phép hệ thống xác định mức giá phù hợp tại từng thời điểm và tránh làm thay đổi các giao dịch đã phát sinh khi bảng giá mới được cập nhật.

Nhóm dữ liệu giao dịch được tổ chức thành phần thông tin chung và phần chi tiết. Bảng giao dịch lưu các thông tin như mã giao dịch, khách hàng, nhân viên, thời gian, trạng thái và tổng giá trị; bảng chi tiết giao dịch lưu các sản phẩm cụ thể thuộc từng giao dịch.

Các bảng được liên kết bằng khóa chính và khóa ngoại. Những trường cần bảo đảm duy nhất được thiết lập ràng buộc UNIQUE, các trường bắt buộc sử dụng NOT NULL và những dữ liệu có quy tắc đặc biệt có thể sử dụng CHECK hoặc các constraint thích hợp.

Index được áp dụng đối với các trường thường xuyên được sử dụng trong tìm kiếm, liên kết hoặc sắp xếp dữ liệu. Việc xây dựng index cần cân bằng giữa tốc độ truy vấn và chi phí cập nhật dữ liệu, tránh tạo quá nhiều index không cần thiết.

Cơ sở dữ liệu đóng vai trò là nguồn dữ liệu tập trung của toàn bộ hệ thống. Backend là thành phần duy nhất trực tiếp truy cập và thay đổi dữ liệu, trong khi ứng dụng Desktop chỉ sử dụng thông tin thông qua RESTful API. Cách tổ chức này giúp việc kiểm soát dữ liệu được tập trung và giảm nguy cơ Client thao tác trực tiếp với cơ sở dữ liệu.

#### **2.3.5. *Luồng giao tiếp giữa React Electron và Spring Boot***

Ứng dụng React Electron và Backend Spring Boot được phát triển thành hai thành phần độc lập và giao tiếp với nhau thông qua RESTful API. Client không cần biết chi tiết cách Backend tổ chức nghiệp vụ hoặc lưu trữ dữ liệu; ngược lại, Backend không phụ thuộc trực tiếp vào cách giao diện được trình bày.

Khi người dùng thực hiện một thao tác trên giao diện, React xử lý sự kiện và gọi Service tương ứng. Service phía Client tạo một HTTP Request với phương thức, URL, Header và dữ liệu cần thiết, sau đó gửi Request đến API do Spring Boot cung cấp.

Với các yêu cầu chỉ lấy dữ liệu, Client thường sử dụng GET. Khi tạo mới dữ liệu, POST được sử dụng. Các thao tác cập nhật có thể sử dụng PUT hoặc PATCH, trong khi DELETE được sử dụng đối với các nghiệp vụ xóa khi hệ thống cho phép.

Dữ liệu trao đổi chủ yếu được biểu diễn bằng JSON. Khi Client gửi dữ liệu tới Backend, Spring Boot chuyển dữ liệu JSON thành đối tượng Java tương ứng. Sau khi xử lý, kết quả Java tiếp tục được chuyển thành JSON trước khi gửi trở lại Client.

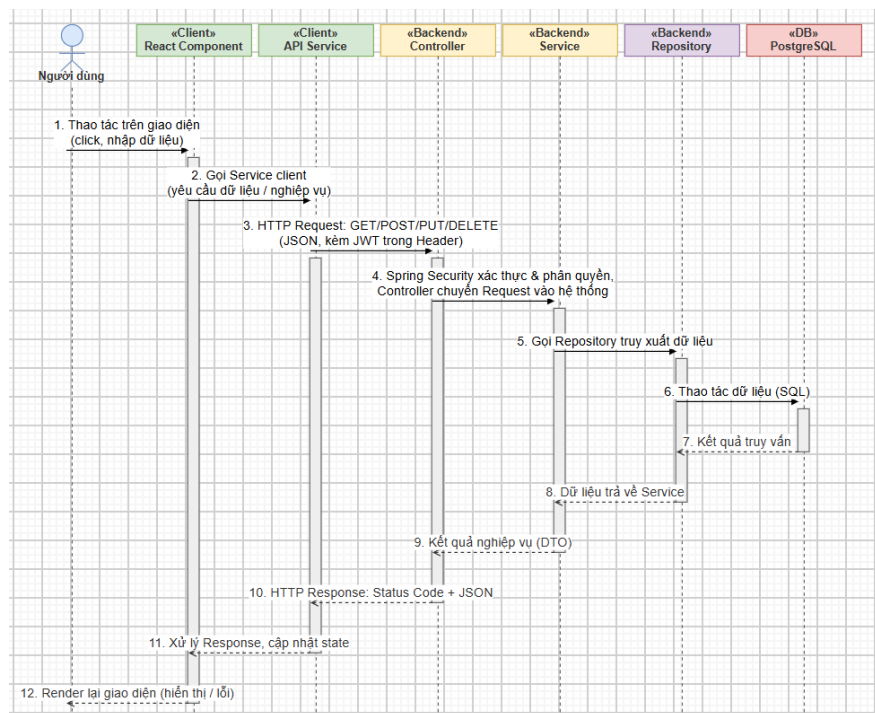
Đối với các API cần xác thực, Client gửi JWT trong Header của Request. Backend kiểm tra token trước khi thực hiện nghiệp vụ. Nếu token không hợp lệ, hết hạn hoặc người dùng không có quyền phù hợp, yêu cầu bị từ chối và Client nhận về HTTP Status Code tương ứng.

Sau khi Backend xử lý thành công, Response được gửi trở lại ứng dụng Desktop. Client tiếp nhận dữ liệu, cập nhật state và render lại những component có liên quan. Nếu Backend trả về lỗi, Client xử lý Response và hiển thị thông báo phù hợp cho người dùng.

Luồng giao tiếp này giúp React Electron không cần kết nối trực tiếp tới PostgreSQL. Toàn bộ dữ liệu đều đi qua Backend, nhờ đó các quy tắc nghiệp vụ và kiểm tra quyền được thực hiện tập trung tại Server.

Quá trình giao tiếp giữa hai thành phần có thể được thực hiện nhiều lần trong cùng một màn hình. Một trang quản lý sản phẩm có thể gọi API lấy danh sách, API lấy chi tiết, API thêm mới hoặc API cập nhật. Tuy nhiên, các API đều tuân theo cùng một quy tắc giao tiếp Request - Response.

Việc tách biệt Client và Server thông qua RESTful API tạo điều kiện để hai phần có thể được phát triển độc lập nhưng vẫn duy trì khả năng tích hợp thông qua một giao diện giao tiếp thống nhất.



Hình 2.33: Sơ đồ luồng cách client nói chuyện với server

### 2.3.6. *Luồng xử lý Request - Response*

Mô hình Request - Response là cơ chế chính được sử dụng để trao đổi dữ liệu giữa Desktop Client và Backend. Mỗi thao tác cần dữ liệu hoặc cần thực hiện nghiệp vụ sẽ bắt đầu bằng một Request từ Client và kết thúc bằng một Response từ Server.

Khi người dùng thực hiện thao tác trên giao diện, React thu thập dữ liệu cần thiết và chuyển cho lớp giao tiếp API. Request được tạo với URL tương ứng với tài nguyên cần xử lý, HTTP Method phù hợp và các Header cần thiết. Nếu nghiệp vụ yêu cầu gửi dữ liệu, nội dung được đưa vào Request Body dưới dạng JSON.

Request sau khi tới Spring Boot trước tiên đi qua các thành phần bảo mật nếu endpoint được bảo vệ. Hệ thống kiểm tra token, xác định người dùng và đánh giá quyền truy cập. Nếu không đáp ứng điều kiện, Request bị dừng và Server trả về Response lỗi mà không chuyển tiếp vào phần nghiệp vụ.

Khi yêu cầu được phép tiếp tục, Controller tiếp nhận Request, chuyển đổi dữ liệu và thực hiện Validation ở mức cần thiết. Nếu dữ liệu không hợp lệ, hệ thống trả về lỗi cho Client. Nếu dữ liệu hợp lệ, Controller gọi Service tương ứng.

Service xử lý các quy tắc nghiệp vụ và có thể gọi một hoặc nhiều Repository. Trong quá trình xử lý, Service có thể kiểm tra sự tồn tại của dữ liệu, điều kiện trạng thái, quyền nghiệp vụ hoặc thực hiện các phép tính cần thiết.

Repository thực hiện truy vấn hoặc cập nhật dữ liệu trong PostgreSQL. Kết quả sau đó được trả về Service. Service hoàn thiện kết quả nghiệp vụ và chuyển dữ liệu về Controller để xây dựng Response.

Response gồm HTTP Status Code và dữ liệu trả về. Trường hợp thành công, hệ thống có thể trả dữ liệu mà Client cần sử dụng. Trường hợp thất bại, Response chứa thông tin lỗi theo cấu trúc thống nhất để Client có thể nhận biết nguyên nhân.

Các mã trạng thái HTTP được sử dụng để phản ánh kết quả của Request. 200 được sử dụng cho yêu cầu thành công, 201 cho trường hợp tạo mới tài nguyên, 400 đối với dữ liệu yêu cầu không hợp lệ, 401 khi chưa được xác thực, 403 khi không có quyền và 404 khi tài nguyên không tồn tại. Những lỗi không mong muốn tại Server được phản ánh bằng mã trạng thái phù hợp thuộc nhóm 5xx.

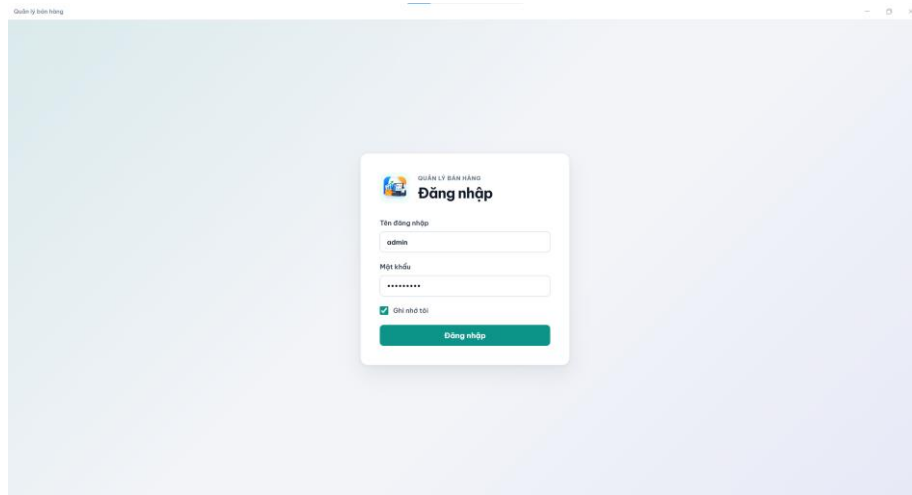
Khi Client nhận Response, ứng dụng kiểm tra trạng thái và nội dung trả về. Nếu yêu cầu thành công, dữ liệu được đưa vào state hoặc hiển thị trực tiếp trên giao diện. Nếu xảy ra lỗi, ứng dụng lựa chọn cách xử lý phù hợp như hiển thị thông báo, yêu cầu người dùng sửa dữ liệu hoặc chuyển về màn hình đăng nhập trong trường hợp thông tin xác thực không còn hợp lệ.

Toàn bộ quá trình Request - Response được thực hiện theo nguyên tắc Client không trực tiếp can thiệp vào xử lý nghiệp vụ nội bộ của Backend. Client chỉ gửi dữ liệu đầu vào và nhận kết quả, trong khi Backend chịu trách nhiệm kiểm tra và quyết định kết quả cuối cùng.

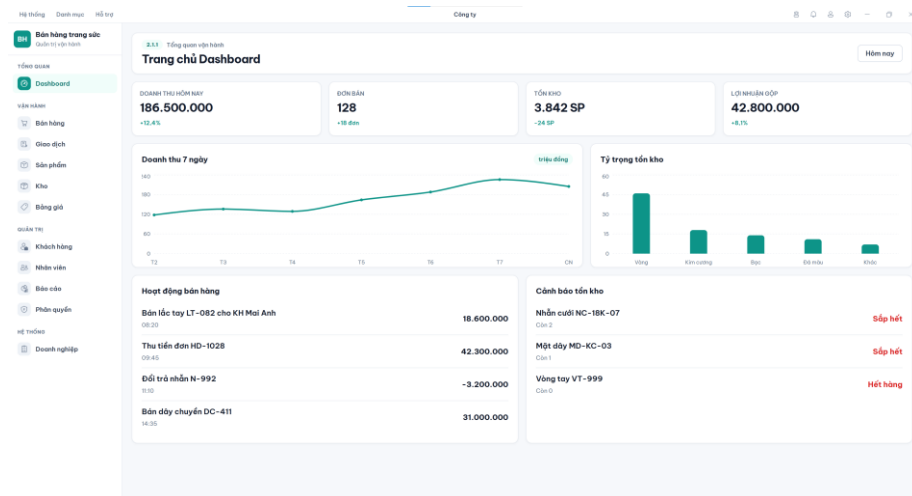
Cơ chế này tạo nên một luồng xử lý thống nhất trong toàn hệ thống, từ thao tác của người dùng đến truy xuất cơ sở dữ liệu và phản hồi kết quả. Việc duy trì cùng một cách xử lý Request - Response giữa các module giúp Desktop và Backend dễ tích hợp hơn, đồng thời hạn chế sự khác biệt trong cách các chức năng trao đổi dữ liệu.

## 2.4. Thiết kế giao diện

### 2.4.1. Thiết kế giao diện đăng nhập



Hình 2.34: Thiết kế giao diện đăng nhập



Hình 2.35: Thiết kế giao diện dashboard



Hệ thống

Quản trị

Hỗ trợ

Bản hàng trong sức

Quản trị vận hành

Tổng quan

Dashboard

Vận hành

Bán hàng

Quản trị

Sản phẩm

Kho

Bảng giá

Quản trị

Khách hàng

**Nhân viên**

Báo cáo

Phân quyền

Hệ thống

Quản trị

Công ty

2.3.4

Quản lý nhân viên

Tìm nhanh trong bảng

Thêm mới

Tổng nhân viên

42

Số nhân viên

39

Số nhân viên

3

Số nhân viên

1,8 tỷ

2 dòng

| STT | Mã NV | Họ tên         | Chức vụ     | Cửa hàng  | CA         | Doanh số    | Trạng thái |
|-----|-------|----------------|-------------|-----------|------------|-------------|------------|
| 1   | NV001 | Nguyễn Mai Anh | Thu ngân    | CN Hà Nội | Sáng       | 86.000.000  | Đang làm   |
| 2   | NV002 | Trần Quốc Huy  | Tư vấn      | CN Hà Nội | Chiều      | 112.000.000 | Đang làm   |
| 3   | NV003 | Lê Minh Trang  | Quản lý kho | CN HCM    | Hành chính | 0           | Đang làm   |

Thêm mới

Sửa

Xóa

Bỏ qua

Thông tin

NV001

Mã NV

NV001

Họ tên

Nguyễn Mai Anh

Chức vụ

Thu ngân

Cửa hàng

CN Hà Nội

CA

Sáng

Doanh số

86.000.000

Trạng thái

Đang làm

Hình 2.38: Thiết kế giao diện quản lý nhân viên

Hệ thống

Quản trị

Hỗ trợ

Bản hàng trong sức

Quản trị vận hành

Tổng quan

Dashboard

Vận hành

Bán hàng

Quản trị

Sản phẩm

Kho

Bảng giá

Quản trị

Khách hàng

**Nhân viên**

Báo cáo

Phân quyền

Hệ thống

Quản trị

Công ty

2.3.5

Quản lý khách hàng

Tìm nhanh trong bảng

Thêm mới

Tổng khách

1280

VIP

82

VIP

16

Mức lợi

38%

2 dòng

| STT | Mã KH | Tên khách hàng | Điện thoại | Hạng   | Tổng mua    | Cộng nợ    | Lần mua cuối |
|-----|-------|----------------|------------|--------|-------------|------------|--------------|
| 1   | KH001 | Phạm Thu Hà    | 0901234567 | VIP    | 420.000.000 | 0          | 07/10/2026   |
| 2   | KH002 | Đỗ Hoàng Nam   | 0912345678 | Gold   | 168.000.000 | 12.000.000 | 06/10/2026   |
| 3   | KH003 | Nguyễn Lan Chi | 0988123123 | Silver | 52.000.000  | 0          | 02/10/2026   |

Thêm mới

Sửa

Xóa

Bỏ qua

Thông tin

KH001

Mã KH

KH001

Tên khách hàng

Phạm Thu Hà

Điện thoại

0901234567

Hạng

VIP

Tổng mua

420.000.000

Cộng nợ

0

Lần mua cuối

07/10/2026

Hình 2.39: Thiết kế giao diện quản lý khách hàng



Hệ thống

Bán hàng trong sắc

Quản trị vận hành

Tổng quan

Dashboard

Vận hành

Bán hàng

Giao dịch

Sản phẩm

Kho

Bảng giá

Quản trị

Khách hàng

Nhân viên

Báo cáo

Phân quyền

Hệ thống

Doanh nghiệp

Công ty

23.8

Giao dịch

Thủ chi, công nợ, hoàn tiền và đổi xuất thành tiền.

Tìm nhanh trong bảng

Thêm mới

DAO DAO

246

THU

228 triệu

CHI

126 triệu

CHỈ SỐ HIỆU

4

3 dòng

Đánh sách

| STT | Mã số | Loại          | Số tương | Số tiền     | Phương thức  | Người tạo  | Trạng thái |
|-----|-------|---------------|----------|-------------|--------------|------------|------------|
| 1   | GD001 | Thu bán hàng  | HD1029   | 12.600.000  | Tiền mặt     | Mai Anh    | Đã ghi sổ  |
| 2   | GD002 | Thu công nợ   | KHO02    | 8.000.000   | Chuyển khoản | Quốc Huy   | Đã ghi sổ  |
| 3   | GD003 | Chi nhập hàng | PH008    | 126.000.000 | Chuyển khoản | Minh Trang | Chờ duyệt  |

Quản trị

GD001

Sửa

Xóa

Bỏ qua

Mã số

GD001

Loại

Thu bán hàng

Đã tương

HD1029

Số tiền

12.600.000

Phương thức

Tiền mặt

Người tạo

Mai Anh

Trạng thái

Đã ghi sổ

Hình 42: Thiết kế giao diện giao dịch

Hệ thống

Bán hàng trong sắc

Quản trị vận hành

Tổng quan

Dashboard

Vận hành

Bán hàng

Giao dịch

Sản phẩm

Kho

Bảng giá

Quản trị

Khách hàng

Nhân viên

Báo cáo

Phân quyền

Hệ thống

Doanh nghiệp

Công ty

23.9

Tổng hợp số liệu

Báo cáo

Xuất Excel

Báo cáo tồn

Báo cáo nhập xuất

Báo cáo bán hàng

Báo cáo mua hàng

Vấn tồn

3842

nhập tồn kho

58

xuất tồn kho

74

còn kho

23

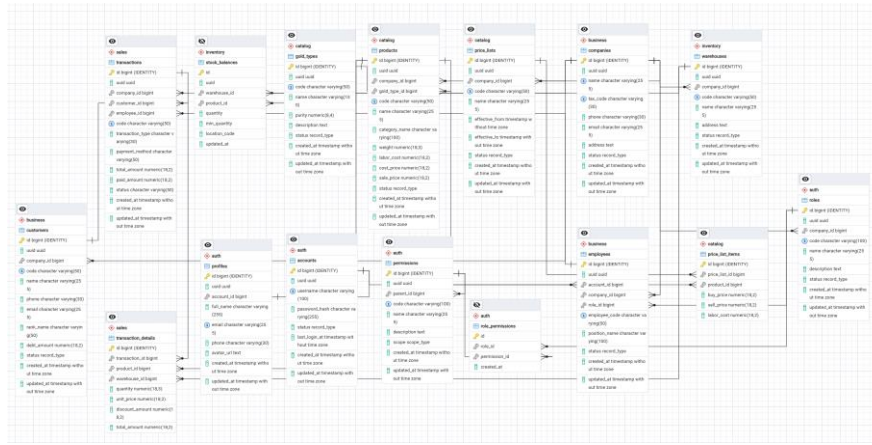
| Mã kho | Sản phẩm          | Tồn đầu | nhập | xuất | Tồn cuối | VITH   | CẢNH BÁO |
|--------|-------------------|---------|------|------|----------|--------|----------|
| KHO-01 | Nhãn hoa B&K      | 12      | 5    | 8    | 9        | TU A1  | Ổn       |
| KHO-02 | Một dây kim cương | 4       | 0    | 3    | 1        | TU VIP | Sắp hết  |
| KHO-03 | Vòng tay vàng     | 7       | 2    | 9    | 0        | TU B2  | Hết      |

Hình 2.43: Thiết kế giao diện báo cáo

133

## 2.5. Thiết kế cơ sở dữ liệu

### 2.5.1. Sơ đồ thực thể liên kết ERD



Hình 2.44: Sơ đồ thực thể ERD

Sơ đồ thực thể liên kết ERD được sử dụng để mô tả cấu trúc dữ liệu và mối quan hệ giữa các thực thể trong hệ thống. Cơ sở dữ liệu được tổ chức thành các nhóm chức năng chính gồm xác thực và phân quyền, quản lý doanh nghiệp, quản lý danh mục sản phẩm, quản lý kho và quản lý giao dịch. Việc phân chia dữ liệu theo từng nhóm giúp cấu trúc cơ sở dữ liệu rõ ràng hơn, đồng thời thuận tiện cho quá trình phát triển và bảo trì hệ thống.

Nhóm xác thực và phân quyền gồm các bảng accounts, profiles, roles, permissions và role\_permissions. Bảng accounts lưu thông tin phục vụ đăng nhập và xác thực người dùng. Mỗi tài khoản được liên kết với một hồ sơ thông qua bảng profiles. Bảng roles lưu thông tin vai trò của người dùng, trong khi bảng permissions lưu các quyền chức năng của hệ thống. Quan hệ nhiều – nhiều giữa vai trò và quyền được triển khai thông qua bảng trung gian role\_permissions. Bảng permissions còn hỗ trợ quan hệ cha – con thông qua trường parent\_id, giúp tổ chức quyền theo cấu trúc phân cấp.

Nhóm quản lý doanh nghiệp gồm các bảng companies, employees và customers. Bảng companies lưu thông tin doanh nghiệp và đóng vai trò là một trong những thực thể trung tâm của hệ thống. Một doanh nghiệp có thể có nhiều nhân viên và nhiều khách hàng. Bảng employees liên kết nhân viên với tài khoản, doanh nghiệp và vai trò tương

ứng, trong khi bảng customers lưu thông tin khách hàng phục vụ cho các giao dịch phát sinh trong quá trình kinh doanh.

Nhóm danh mục sản phẩm gồm các bảng gold\_types, products, price\_lists và price\_list\_items. Bảng gold\_types lưu thông tin các loại vàng, bao gồm mã, tên, độ tinh khiết, mô tả và trạng thái sử dụng. Bảng products lưu thông tin sản phẩm trang sức như mã sản phẩm, tên sản phẩm, loại vàng, nhóm sản phẩm, trọng lượng, tiền công, giá vốn và giá bán. Mỗi sản phẩm có thể liên kết với một loại vàng cụ thể để phục vụ quá trình phân loại và tra cứu.

Thông tin bảng giá được quản lý thông qua hai bảng price\_lists và price\_list\_items. Bảng price\_lists lưu thông tin chung của từng bảng giá như mã, tên, thời gian bắt đầu áp dụng, thời gian kết thúc và trạng thái. Bảng price\_list\_items lưu giá mua, giá bán và tiền công của từng sản phẩm trong mỗi bảng giá. Cách tổ chức này giúp hệ thống lưu được nhiều bảng giá khác nhau theo từng thời điểm mà không làm thay đổi trực tiếp thông tin cơ bản của sản phẩm.

Nhóm quản lý kho gồm các bảng warehouses và stock\_balances. Bảng warehouses lưu thông tin về các kho thuộc từng doanh nghiệp. Bảng stock\_balances được sử dụng để liên kết giữa kho và sản phẩm, đồng thời lưu số lượng tồn, số lượng tối thiểu và vị trí lưu trữ của sản phẩm trong kho. Mỗi cặp sản phẩm và kho chỉ có một bản ghi tồn kho tương ứng nhằm tránh trùng lặp dữ liệu.

Nhóm giao dịch gồm các bảng transactions và transaction\_details. Bảng transactions lưu thông tin chung của giao dịch như doanh nghiệp, khách hàng, nhân viên thực hiện, mã giao dịch, loại giao dịch, phương thức thanh toán, tổng tiền, số tiền đã thanh toán và trạng thái. Một giao dịch có thể bao gồm nhiều sản phẩm khác nhau nên bảng transaction\_details được sử dụng để lưu thông tin chi tiết của từng mặt hàng. Bảng transaction\_details chứa các thông tin như sản phẩm, kho, số lượng, đơn giá, số tiền giảm giá và thành tiền. Mỗi giao dịch có thể có nhiều bản ghi chi tiết, trong khi một sản phẩm cũng có thể xuất hiện trong nhiều giao dịch khác nhau. Cách thiết kế này

giúp tách thông tin chung của giao dịch khỏi dữ liệu chi tiết, hạn chế việc lặp lại thông tin và phù hợp với mô hình cơ sở dữ liệu quan hệ.

Các mối quan hệ giữa các bảng được thiết lập thông qua khóa chính và khóa ngoại. Một doanh nghiệp có thể liên kết với nhiều vai trò, nhân viên, khách hàng, sản phẩm, kho, bảng giá và giao dịch. Một vai trò có thể có nhiều quyền và một quyền có thể được sử dụng bởi nhiều vai trò thông qua bảng `role_permissions`. Một loại vàng có thể được sử dụng bởi nhiều sản phẩm. Một kho có thể chứa nhiều sản phẩm và một sản phẩm cũng có thể tồn tại trong nhiều kho thông qua bảng `stock_balances`.

Bên cạnh đó, một khách hàng có thể phát sinh nhiều giao dịch, một nhân viên có thể thực hiện nhiều giao dịch và mỗi giao dịch có thể chứa nhiều sản phẩm. Các mối quan hệ này tạo thành cấu trúc dữ liệu liên kết chặt chẽ, giúp hệ thống bảo đảm tính nhất quán trong quá trình lưu trữ và xử lý thông tin.

Sơ đồ ERD là cơ sở quan trọng để triển khai cấu trúc bảng trong PostgreSQL, thiết lập các ràng buộc dữ liệu và xây dựng các Entity trong Java. Thông qua JPA và Hibernate, các mối quan hệ trong cơ sở dữ liệu có thể được ánh xạ thành các quan hệ giữa các đối tượng trong chương trình, tạo điều kiện thuận lợi cho quá trình truy xuất và xử lý dữ liệu.

### 2.5.2. Thiết kế bảng tài khoản

Bảng 2.1: Bảng dữ liệu tài khoản

| STT | Tên trường | Kiểu dữ liệu | Ràng buộc                     | Mô tả                                           |
|-----|------------|--------------|-------------------------------|-------------------------------------------------|
| 1   | id         | BIGINT       | PRIMARY KEY,<br>IDENTITY      | Khóa chính<br>của tài<br>khoản, tự<br>động tăng |
| 2   | uuid       | UUID         | NOT NULL, DEFAULT<br>uuidv7() | Mã định<br>danh<br>UUID,                        |

|   |               |              |                                        |                                             |
|---|---------------|--------------|----------------------------------------|---------------------------------------------|
|   |               |              |                                        | được<br>PostgreSQL<br>tự động<br>sinh       |
| 3 | username      | VARCHAR(100) | NOT NULL, UNIQUE                       | Tên đăng<br>nhập của<br>người dùng          |
| 4 | password_hash | VARCHAR(255) | NOT NULL                               | Mật khẩu<br>sau khi<br>được bấm             |
| 5 | status        | record_type  | NOT NULL, DEFAULT<br>active            | Trạng thái<br>hoạt động<br>của tài<br>khoản |
| 6 | last_login_at | TIMESTAMP    | NULL                                   | Thời điểm<br>đăng nhập<br>gần nhất          |
| 7 | created_at    | TIMESTAMP    | NOT NULL, DEFAULT<br>CURRENT_TIMESTAMP | Thời điểm<br>tạo tài<br>khoản               |
| 8 | updated_at    | TIMESTAMP    | NOT NULL, DEFAULT<br>CURRENT_TIMESTAMP | Thời điểm<br>cập nhật<br>gần nhất           |

Bảng 2.2: Cấu trúc bảng hồ sơ người dùng

| STT | Tên trường | Kiểu dữ liệu | Ràng buộc                                                                                                       | Mô tả                                       |
|-----|------------|--------------|-----------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| 1   | id         | BIGINT       | PRIMARY KEY;<br>IDENTITY                                                                                        | Khóa chính nội bộ, tự động tăng.            |
| 2   | uuid       | UUID         | NOT NULL                                                                                                        | Mã định danh UUID, được PostgreSQL tự sinh. |
| 3   | account_id | BIGINT       | NOT NULL;<br>UNIQUE<br>(account_id); FK →<br>auth.accounts(id);<br>ON DELETE<br>CASCADE ON<br>UPDATE<br>CASCADE | Khóa ngoại xác định tài khoản.              |
| 4   | full_name  | VARCHAR(255) | NOT NULL                                                                                                        | Họ và tên người dùng.                       |
| 5   | email      | VARCHAR(255) | UNIQUE (email)                                                                                                  | Địa chỉ email.                              |
| 6   | phone      | VARCHAR(30)  |                                                                                                                 | Số điện thoại.                              |

| STT | Tên trường | Kiểu dữ liệu | Ràng buộc | Mô tả                                |
|-----|------------|--------------|-----------|--------------------------------------|
| 7   | avatar_url | TEXT         |           | Đường dẫn ảnh đại diện.              |
| 8   | created_at | TIMESTAMP    | NOT NULL  | Thời điểm tạo bản ghi.               |
| 9   | updated_at | TIMESTAMP    | NOT NULL  | Thời điểm cập nhật bản ghi gần nhất. |

Bảng 2.3: Cấu trúc bảng doanh nghiệp

| STT | Tên trường | Kiểu dữ liệu | Ràng buộc                  | Mô tả                                       |
|-----|------------|--------------|----------------------------|---------------------------------------------|
| 1   | id         | BIGINT       | PRIMARY KEY;<br>IDENTITY   | Khóa chính nội bộ, tự động tăng.            |
| 2   | uuid       | UUID         | NOT NULL                   | Mã định danh UUID, được PostgreSQL tự sinh. |
| 3   | name       | VARCHAR(255) | NOT NULL;<br>UNIQUE (name) | Tên hiển thị.                               |

| STT | Tên trường | Kiểu dữ liệu       | Ràng buộc                      | Mô tả                                |
|-----|------------|--------------------|--------------------------------|--------------------------------------|
| 4   | tax_code   | VARCHAR(30)        | NOT NULL;<br>UNIQUE (tax_code) | Mã số thuế của doanh nghiệp.         |
| 5   | phone      | VARCHAR(30)        |                                | Số điện thoại.                       |
| 6   | email      | VARCHAR(255)       |                                | Địa chỉ email.                       |
| 7   | address    | TEXT               |                                | Địa chỉ doanh nghiệp.                |
| 8   | status     | public.record_type | NOT NULL                       | Trạng thái của bản ghi.              |
| 9   | created_at | TIMESTAMP          | NOT NULL                       | Thời điểm tạo bản ghi.               |
| 10  | updated_at | TIMESTAMP          | NOT NULL                       | Thời điểm cập nhật bản ghi gần nhất. |

Bảng 2.4: Cấu trúc bảng vai trò

| STT | Tên trường  | Kiểu dữ liệu | Ràng buộc                                                                                                     | Mô tả                                                                |
|-----|-------------|--------------|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| 1   | id          | BIGINT       | PRIMARY KEY;<br>IDENTITY                                                                                      | Khóa chính nội bộ, tự động tăng.                                     |
| 2   | uuid        | UUID         | NOT NULL                                                                                                      | Mã định danh UUID, được PostgreSQL tự sinh.                          |
| 3   | company_id  | BIGINT       | UNIQUE<br>(company_id, code);<br>FK →<br>business.companies(id); ON DELETE<br>CASCADE ON<br>UPDATE<br>CASCADE | Doanh nghiệp sở hữu vai trò; có thể để trống với vai trò dùng chung. |
| 4   | code        | VARCHAR(100) | NOT NULL;<br>UNIQUE<br>(company_id, code)                                                                     | Mã vai trò.                                                          |
| 5   | name        | VARCHAR(255) | NOT NULL                                                                                                      | Tên vai trò.                                                         |
| 6   | description | TEXT         |                                                                                                               | Mô tả bổ sung.                                                       |

| STT | Tên trường | Kiểu dữ liệu       | Ràng buộc | Mô tả                                |
|-----|------------|--------------------|-----------|--------------------------------------|
| 7   | status     | public.record_type | NOT NULL  | Trạng thái của bản ghi.              |
| 8   | created_at | TIMESTAMP          | NOT NULL  | Thời điểm tạo bản ghi.               |
| 9   | updated_at | TIMESTAMP          | NOT NULL  | Thời điểm cập nhật bản ghi gần nhất. |

Bảng 2.5: Cấu trúc bảng quyền

| STT | Tên trường | Kiểu dữ liệu | Ràng buộc                | Mô tả                                       |
|-----|------------|--------------|--------------------------|---------------------------------------------|
| 1   | id         | BIGINT       | PRIMARY KEY;<br>IDENTITY | Khóa chính nội bộ, tự động tăng.            |
| 2   | uuid       | UUID         | NOT NULL                 | Mã định danh UUID, được PostgreSQL tự sinh. |

| STT | Tên trường  | Kiểu dữ liệu      | Ràng buộc                                                                     | Mô tả                                        |
|-----|-------------|-------------------|-------------------------------------------------------------------------------|----------------------------------------------|
| 3   | parent_id   | BIGINT            | FK →<br>auth.permissions(id);<br>ON DELETE<br>CASCADE ON<br>UPDATE<br>CASCADE | Quyền cha dùng để tổ chức cấu trúc phân cấp. |
| 4   | code        | VARCHAR<br>(100)  | NOT NULL;<br>UNIQUE (code)                                                    | Mã quyền.                                    |
| 5   | name        | VARCHAR<br>(255)  | NOT NULL                                                                      | Tên quyền.                                   |
| 6   | description | TEXT              |                                                                               | Mô tả bổ sung.                               |
| 7   | scope       | public.scope_type | NOT NULL                                                                      | Phạm vi áp dụng của quyền.                   |
| 8   | created_at  | TIMESTAMP         | NOT NULL                                                                      | Thời điểm tạo bản ghi.                       |
| 9   | updated_at  | TIMESTAMP         | NOT NULL                                                                      | Thời điểm cập nhật bản ghi gần nhất.         |

Bảng 2.6: Cấu trúc bảng liên kết vai trò - quyền

| STT | Tên trường    | Kiểu dữ liệu | Ràng buộc                                                                                                             | Mô tả                            |
|-----|---------------|--------------|-----------------------------------------------------------------------------------------------------------------------|----------------------------------|
| 1   | id            | BIGINT       | PRIMARY KEY;<br>IDENTITY                                                                                              | Khóa chính nội bộ, tự động tăng. |
| 2   | role_id       | BIGINT       | NOT NULL;<br>UNIQUE (role_id, permission_id); FK<br>→ auth.roles(id);<br>ON DELETE CASCADE ON<br>UPDATE CASCADE       | Vai trò được gán quyền.          |
| 3   | permission_id | BIGINT       | NOT NULL;<br>UNIQUE (role_id, permission_id); FK<br>→ auth.permissions(id);<br>ON DELETE CASCADE ON<br>UPDATE CASCADE | Quyền được gán cho vai trò.      |
| 4   | created_at    | TIMESTAMP    | NOT NULL                                                                                                              | Thời điểm tạo bản ghi.           |

Bảng 2.7: Cấu trúc bảng nhân viên

| STT | Tên trường | Kiểu dữ liệu | Ràng buộc                                                                            | Mô tả                                       |
|-----|------------|--------------|--------------------------------------------------------------------------------------|---------------------------------------------|
| 1   | id         | BIGINT       | PRIMARY KEY;<br>IDENTITY                                                             | Khóa chính nội bộ, tự động tăng.            |
| 2   | uuid       | UUID         | NOT NULL                                                                             | Mã định danh UUID, được PostgreSQL tự sinh. |
| 3   | account_id | BIGINT       | NOT NULL; FK →<br>auth.accounts(id);<br>ON DELETE<br>CASCADE ON<br>UPDATE<br>CASCADE | Khóa ngoại xác định tài khoản.              |

| STT | Tên trường    | Kiểu dữ liệu     | Ràng buộc                                                                                                                                  | Mô tả                                |
|-----|---------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| 4   | company_id    | BIGINT           | NOT NULL;<br>UNIQUE<br>(company_id,<br>employee_code);<br>FK →<br>business.companies<br>(id); ON DELETE<br>CASCADE ON<br>UPDATE<br>CASCADE | Khóa ngoại xác định<br>doanh nghiệp. |
| 5   | role_id       | BIGINT           | FK →<br>auth.roles(id); ON<br>DELETE SET<br>NULL ON<br>UPDATE<br>CASCADE                                                                   | Khóa ngoại xác định<br>vai trò.      |
| 6   | employee_code | VARCHAR<br>(50)  | NOT NULL;<br>UNIQUE<br>(company_id,<br>employee_code)                                                                                      | Mã nhân viên trong<br>doanh nghiệp.  |
| 7   | position_name | VARCHAR<br>(100) |                                                                                                                                            | Tên vị trí/chức vụ<br>của nhân viên. |

| STT | Tên trường | Kiểu dữ liệu           | Ràng buộc | Mô tả                                |
|-----|------------|------------------------|-----------|--------------------------------------|
| 8   | status     | public.recor<br>d_type | NOT NULL  | Trạng thái của bản ghi.              |
| 9   | created_at | TIMESTA<br>MP          | NOT NULL  | Thời điểm tạo bản ghi.               |
| 10  | updated_at | TIMESTA<br>MP          | NOT NULL  | Thời điểm cập nhật bản ghi gần nhất. |

Bảng 2.8: Cấu trúc bảng khách hàng

| STT | Tên trường | Kiểu dữ liệu | Ràng buộc                | Mô tả                                       |
|-----|------------|--------------|--------------------------|---------------------------------------------|
| 1   | id         | BIGINT       | PRIMARY KEY;<br>IDENTITY | Khóa chính nội bộ, tự động tăng.            |
| 2   | uuid       | UUID         | NOT NULL                 | Mã định danh UUID, được PostgreSQL tự sinh. |

| STT | Tên trường | Kiểu dữ liệu     | Ràng buộc                                                                                                                  | Mô tả                             |
|-----|------------|------------------|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| 3   | company_id | BIGINT           | NOT NULL;<br>UNIQUE<br>(company_id, code);<br>FK →<br>business.companies(id); ON DELETE<br>CASCADE ON<br>UPDATE<br>CASCADE | Khóa ngoại xác định doanh nghiệp. |
| 4   | code       | VARCHAR<br>(50)  | NOT NULL;<br>UNIQUE<br>(company_id, code)                                                                                  | Mã khách hàng trong doanh nghiệp. |
| 5   | name       | VARCHAR<br>(255) | NOT NULL                                                                                                                   | Tên hiển thị.                     |
| 6   | phone      | VARCHAR<br>(30)  |                                                                                                                            | Số điện thoại.                    |
| 7   | email      | VARCHAR<br>(255) |                                                                                                                            | Địa chỉ email.                    |
| 8   | rank_name  | VARCHAR<br>(50)  |                                                                                                                            | Hạng khách hàng.                  |

| STT | Tên trường  | Kiểu dữ liệu       | Ràng buộc | Mô tả                                |
|-----|-------------|--------------------|-----------|--------------------------------------|
| 9   | debt_amount | NUMERIC<br>(18, 2) | NOT NULL  | Số tiền công nợ hiện tại.            |
| 10  | status      | public.record_type | NOT NULL  | Trạng thái của bản ghi.              |
| 11  | created_at  | TIMESTAMP          | NOT NULL  | Thời điểm tạo bản ghi.               |
| 12  | updated_at  | TIMESTAMP          | NOT NULL  | Thời điểm cập nhật bản ghi gần nhất. |

Bảng 2.9: Cấu trúc bảng loại vàng

| STT | Tên trường | Kiểu dữ liệu | Ràng buộc                | Mô tả                                       |
|-----|------------|--------------|--------------------------|---------------------------------------------|
| 1   | id         | BIGINT       | PRIMARY KEY;<br>IDENTITY | Khóa chính nội bộ, tự động tăng.            |
| 2   | uuid       | UUID         | NOT NULL                 | Mã định danh UUID, được PostgreSQL tự sinh. |

| STT | Tên trường  | Kiểu dữ liệu       | Ràng buộc                  | Mô tả                                  |
|-----|-------------|--------------------|----------------------------|----------------------------------------|
| 3   | code        | VARCHAR(50)        | NOT NULL;<br>UNIQUE (code) | Mã loại vàng.                          |
| 4   | name        | VARCHAR(100)       | NOT NULL                   | Tên loại vàng.                         |
| 5   | purity      | NUMERIC(8,4)       |                            | Độ tinh khiết/hàm lượng của loại vàng. |
| 6   | description | TEXT               |                            | Mô tả bổ sung.                         |
| 7   | status      | public.record_type | NOT NULL                   | Trạng thái của bản ghi.                |
| 8   | created_at  | TIMESTAMP          | NOT NULL                   | Thời điểm tạo bản ghi.                 |
| 9   | updated_at  | TIMESTAMP          | NOT NULL                   | Thời điểm cập nhật bản ghi gần nhất.   |

Bảng 2.10: Cấu trúc bảng sản phẩm trang sức

| STT | Tên trường | Kiểu dữ liệu | Ràng buộc                                                                                                   | Mô tả                                       |
|-----|------------|--------------|-------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| 1   | id         | BIGINT       | PRIMARY KEY;<br>IDENTITY                                                                                    | Khóa chính nội bộ, tự động tăng.            |
| 2   | uuid       | UUID         | NOT NULL                                                                                                    | Mã định danh UUID, được PostgreSQL tự sinh. |
| 3   | company_id | BIGINT       | NOT NULL;<br>UNIQUE<br>(company_id, code); FK → business.companies(id); ON DELETE CASCADE ON UPDATE CASCADE | Khóa ngoại xác định doanh nghiệp.           |

| STT | Tên trường    | Kiểu dữ liệu   | Ràng buộc                                                         | Mô tả                           |
|-----|---------------|----------------|-------------------------------------------------------------------|---------------------------------|
| 4   | gold_type_id  | BIGINT         | FK → catalog.gold_types(id); ON DELETE SET NULL ON UPDATE CASCADE | Loại vàng của sản phẩm.         |
| 5   | code          | VARCHAR(50)    | NOT NULL;<br>UNIQUE (company_id, code)                            | Mã sản phẩm trong doanh nghiệp. |
| 6   | name          | VARCHAR(255)   | NOT NULL                                                          | Tên sản phẩm trang sức.         |
| 7   | category_name | VARCHAR(100)   | NOT NULL                                                          | Tên nhóm/loại sản phẩm.         |
| 8   | weight        | NUMERIC(18, 3) | NOT NULL                                                          | Trọng lượng sản phẩm.           |
| 9   | labor_cost    | NUMERIC(18, 2) | NOT NULL                                                          | Tiền công chế tác.              |

| STT | Tên trường | Kiểu dữ liệu       | Ràng buộc | Mô tả                                |
|-----|------------|--------------------|-----------|--------------------------------------|
| 10  | cost_price | NUMERIC(18, 2)     | NOT NULL  | Giá vốn của sản phẩm.                |
| 11  | sale_price | NUMERIC(18, 2)     | NOT NULL  | Giá bán cơ bản của sản phẩm.         |
| 12  | status     | public.record_type | NOT NULL  | Trạng thái của bản ghi.              |
| 13  | created_at | TIMESTAMP          | NOT NULL  | Thời điểm tạo bản ghi.               |
| 14  | updated_at | TIMESTAMP          | NOT NULL  | Thời điểm cập nhật bản ghi gần nhất. |

Bảng 2.11: Cấu trúc bảng kho

| STT | Tên trường | Kiểu dữ liệu | Ràng buộc                | Mô tả                            |
|-----|------------|--------------|--------------------------|----------------------------------|
| 1   | id         | BIGINT       | PRIMARY KEY;<br>IDENTITY | Khóa chính nội bộ, tự động tăng. |

| STT | Tên trường | Kiểu dữ liệu  | Ràng buộc                                                                                                   | Mô tả                                       |
|-----|------------|---------------|-------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| 2   | uuid       | UUID          | NOT NULL                                                                                                    | Mã định danh UUID, được PostgreSQL tự sinh. |
| 3   | company_id | BIGINT        | NOT NULL;<br>UNIQUE<br>(company_id, code); FK → business.companies(id); ON DELETE CASCADE ON UPDATE CASCADE | Khóa ngoại xác định doanh nghiệp.           |
| 4   | code       | VARCHAR (50)  | NOT NULL;<br>UNIQUE<br>(company_id, code)                                                                   | Mã kho trong doanh nghiệp.                  |
| 5   | name       | VARCHAR (255) | NOT NULL                                                                                                    | Tên kho.                                    |
| 6   | address    | TEXT          |                                                                                                             | Địa chỉ hoặc vị trí kho.                    |

| STT | Tên trường | Kiểu dữ liệu       | Ràng buộc | Mô tả                                |
|-----|------------|--------------------|-----------|--------------------------------------|
| 7   | status     | public.record_type | NOT NULL  | Trạng thái của bản ghi.              |
| 8   | created_at | TIMESTAMP          | NOT NULL  | Thời điểm tạo bản ghi.               |
| 9   | updated_at | TIMESTAMP          | NOT NULL  | Thời điểm cập nhật bản ghi gần nhất. |

Bảng 2.12: Cấu trúc bảng tồn kho

| STT | Tên trường | Kiểu dữ liệu | Ràng buộc                | Mô tả                                       |
|-----|------------|--------------|--------------------------|---------------------------------------------|
| 1   | id         | BIGINT       | PRIMARY KEY;<br>IDENTITY | Khóa chính nội bộ, tự động tăng.            |
| 2   | uuid       | UUID         | NOT NULL                 | Mã định danh UUID, được PostgreSQL tự sinh. |

| STT | Tên trường   | Kiểu dữ liệu | Ràng buộc                                                                                                                      | Mô tả                         |
|-----|--------------|--------------|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| 3   | warehouse_id | BIGINT       | NOT NULL;<br>UNIQUE<br>(warehouse_id, product_id);<br>FK → inventory.warehouses(id);<br>ON DELETE CASCADE<br>ON UPDATE CASCADE | Khóa ngoại xác định kho.      |
| 4   | product_id   | BIGINT       | NOT NULL;<br>UNIQUE<br>(warehouse_id, product_id);<br>FK → catalog.products(id);<br>ON DELETE CASCADE<br>ON UPDATE CASCADE     | Khóa ngoại xác định sản phẩm. |

| STT | Tên trường    | Kiểu dữ liệu   | Ràng buộc                             | Mô tả                                |
|-----|---------------|----------------|---------------------------------------|--------------------------------------|
| 5   | quantity      | NUMERIC(18, 3) | NOT NULL;<br>CHECK<br>(quantity >= 0) | Số lượng tồn hiện tại.               |
| 6   | min_quantity  | NUMERIC(18, 3) | NOT NULL                              | Mức tồn kho tối thiểu để cảnh báo.   |
| 7   | location_code | VARCHAR(100)   |                                       | Mã vị trí lưu trữ trong kho.         |
| 8   | updated_at    | TIMESTAMP      | NOT NULL                              | Thời điểm cập nhật bản ghi gần nhất. |

Bảng 2.13: Cấu trúc bảng bảng giá

| STT | Tên trường | Kiểu dữ liệu | Ràng buộc                | Mô tả                                       |
|-----|------------|--------------|--------------------------|---------------------------------------------|
| 1   | id         | BIGINT       | PRIMARY KEY;<br>IDENTITY | Khóa chính nội bộ, tự động tăng.            |
| 2   | uuid       | UUID         | NOT NULL                 | Mã định danh UUID, được PostgreSQL tự sinh. |

| STT | Tên trường     | Kiểu dữ liệu       | Ràng buộc                                                                                                    | Mô tả                             |
|-----|----------------|--------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------|
| 3   | company_id     | BIGINT             | NOT NULL;<br>UNIQUE<br>(company_id, code); FK → business.companies (id); ON DELETE CASCADE ON UPDATE CASCADE | Khóa ngoại xác định doanh nghiệp. |
| 4   | code           | VARCHAR<br>(50)    | NOT NULL;<br>UNIQUE<br>(company_id, code)                                                                    | Mã bảng giá.                      |
| 5   | name           | VARCHAR<br>(255)   | NOT NULL                                                                                                     | Tên bảng giá.                     |
| 6   | effective_from | TIMESTAMP          | NOT NULL                                                                                                     | Thời điểm bắt đầu hiệu lực.       |
| 7   | effective_to   | TIMESTAMP          |                                                                                                              | Thời điểm kết thúc hiệu lực.      |
| 8   | status         | public.record_type | NOT NULL                                                                                                     | Trạng thái của bản ghi.           |

| STT | Tên trường | Kiểu dữ liệu | Ràng buộc | Mô tả                                |
|-----|------------|--------------|-----------|--------------------------------------|
| 9   | created_at | TIMESTAMP    | NOT NULL  | Thời điểm tạo bản ghi.               |
| 10  | updated_at | TIMESTAMP    | NOT NULL  | Thời điểm cập nhật bản ghi gần nhất. |

Bảng 2.14: Cấu trúc bảng chi tiết bảng giá

| STT | Tên trường    | Kiểu dữ liệu | Ràng buộc                                                                                        | Mô tả                            |
|-----|---------------|--------------|--------------------------------------------------------------------------------------------------|----------------------------------|
| 1   | id            | BIGINT       | PRIMARY KEY; IDENTITY                                                                            | Khóa chính nội bộ, tự động tăng. |
| 2   | price_list_id | BIGINT       | NOT NULL; UNIQUE (price_list_id, product_id); FK → catalog.price_lists(id); ON DELETE CASCADE ON | Bảng giá chứa dòng giá này.      |

| STT | Tên trường | Kiểu dữ liệu   | Ràng buộc                                                                                                                                   | Mô tả                                |
|-----|------------|----------------|---------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
|     |            |                | UPDATE<br>CASCADE                                                                                                                           |                                      |
| 3   | product_id | BIGINT         | NOT NULL;<br>UNIQUE<br>(price_list_id,<br>product_id); FK<br>→<br>catalog.products(<br>id); ON<br>DELETE<br>CASCADE ON<br>UPDATE<br>CASCADE | Khóa ngoại xác định<br>sản phẩm.     |
| 4   | buy_price  | NUMERIC(18, 2) | NOT NULL                                                                                                                                    | Giá mua áp dụng.                     |
| 5   | sell_price | NUMERIC(18, 2) | NOT NULL                                                                                                                                    | Giá bán áp dụng.                     |
| 6   | labor_cost | NUMERIC(18, 2) | NOT NULL                                                                                                                                    | Tiền công áp dụng<br>trong bảng giá. |

Bảng 2.15: Cấu trúc bảng giao dịch

| STT | Tên trường  | Kiểu dữ liệu | Ràng buộc                                                                                             | Mô tả                                       |
|-----|-------------|--------------|-------------------------------------------------------------------------------------------------------|---------------------------------------------|
| 1   | id          | BIGINT       | PRIMARY KEY; IDENTITY                                                                                 | Khóa chính nội bộ, tự động tăng.            |
| 2   | uuid        | UUID         | NOT NULL                                                                                              | Mã định danh UUID, được PostgreSQL tự sinh. |
| 3   | company_id  | BIGINT       | NOT NULL; UNIQUE (company_id, code); FK → business.companies(id); ON DELETE CASCADE ON UPDATE CASCADE | Khóa ngoại xác định doanh nghiệp.           |
| 4   | customer_id | BIGINT       | FK → business.customers(id); ON DELETE SET NULL ON                                                    | Khóa ngoại xác định khách hàng.             |

| STT | Tên trường       | Kiểu dữ liệu       | Ràng buộc                                                                        | Mô tả                             |
|-----|------------------|--------------------|----------------------------------------------------------------------------------|-----------------------------------|
|     |                  |                    | UPDATE<br>CASCADE                                                                |                                   |
| 5   | employee_id      | BIGINT             | FK →<br>business.employees(id); ON<br>DELETE SET<br>NULL ON<br>UPDATE<br>CASCADE | Khóa ngoại xác<br>định nhân viên. |
| 6   | code             | VARCHAR(<br>50)    | NOT NULL;<br>UNIQUE<br>(company_id,<br>code)                                     | Mã giao dịch.                     |
| 7   | transaction_type | VARCHAR(<br>30)    | NOT NULL                                                                         | Loại giao dịch mua<br>hoặc bán.   |
| 8   | payment_method   | VARCHAR(<br>50)    |                                                                                  | Phương thức thanh<br>toán.        |
| 9   | total_amount     | NUMERIC(<br>18, 2) | NOT NULL;<br>CHECK<br>(total_amount<br>≥ 0 AND                                   | Tổng giá trị giao<br>dịch.        |

| STT | Tên trường  | Kiểu dữ liệu   | Ràng buộc                                                         | Mô tả                                |
|-----|-------------|----------------|-------------------------------------------------------------------|--------------------------------------|
|     |             |                | paid_amount >= 0)                                                 |                                      |
| 10  | paid_amount | NUMERIC(18, 2) | NOT NULL;<br>CHECK<br>(total_amount >= 0 AND<br>paid_amount >= 0) | Số tiền đã thanh toán.               |
| 11  | status      | VARCHAR(50)    | NOT NULL                                                          | Trạng thái của bản ghi.              |
| 12  | created_at  | TIMESTAMP      | NOT NULL                                                          | Thời điểm tạo bản ghi.               |
| 13  | updated_at  | TIMESTAMP      | NOT NULL                                                          | Thời điểm cập nhật bản ghi gần nhất. |

Bảng 2.16: Bảng chứa dữ liệu mua bán

| STT | Tên trường     | Kiểu dữ liệu | Ràng buộc                                                                  | Mô tả                            |
|-----|----------------|--------------|----------------------------------------------------------------------------|----------------------------------|
| 1   | id             | BIGINT       | PRIMARY KEY; IDENTITY                                                      | Khóa chính nội bộ, tự động tăng. |
| 2   | transaction_id | BIGINT       | NOT NULL; FK → sales.transactions(id); ON DELETE CASCADE ON UPDATE CASCADE | Giao dịch chứa dòng chi tiết.    |
| 3   | product_id     | BIGINT       | NOT NULL; FK → catalog.products(id); ON DELETE RESTRICT ON UPDATE CASCADE  | Khóa ngoại xác định sản phẩm.    |
| 4   | warehouse_id   | BIGINT       | FK → inventory.warehouses(id); ON                                          | Khóa ngoại xác định kho.         |

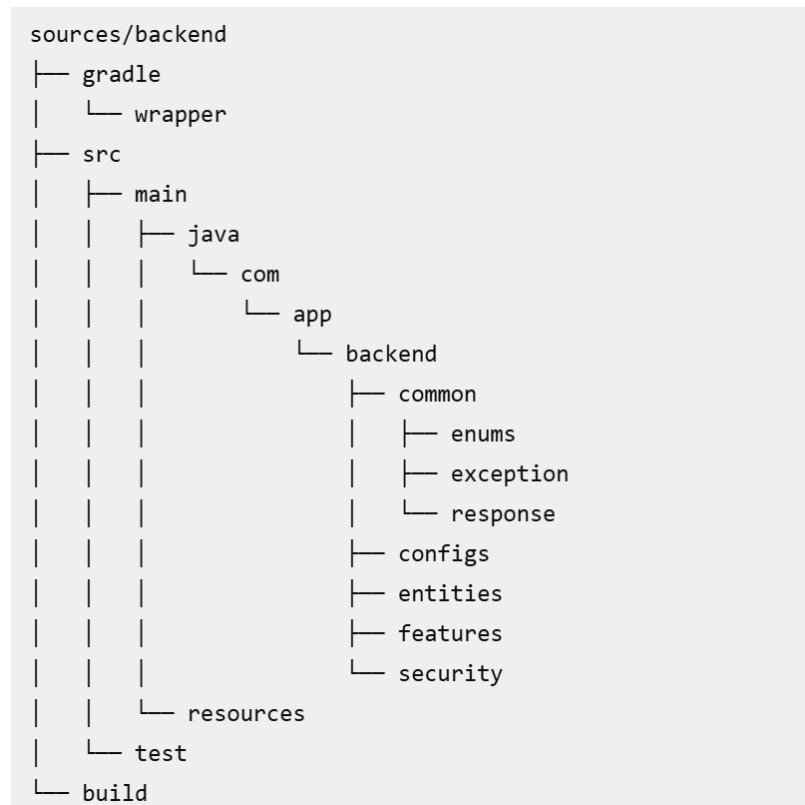
| STT | Tên trường      | Kiểu dữ liệu      | Ràng buộc                                                        | Mô tả                              |
|-----|-----------------|-------------------|------------------------------------------------------------------|------------------------------------|
|     |                 |                   | DELETE SET<br>NULL ON<br>UPDATE<br>CASCADE                       |                                    |
| 5   | quantity        | NUMERIC(18,<br>3) | NOT NULL;<br>CHECK<br>(quantity > 0)                             | Số lượng sản phẩm trong giao dịch. |
| 6   | unit_price      | NUMERIC(18,<br>2) | NOT NULL;<br>CHECK<br>(unit_price >= 0 AND<br>total_amount >= 0) | Đơn giá tại thời điểm giao dịch.   |
| 7   | discount_amount | NUMERIC(18,<br>2) | NOT NULL                                                         | Số tiền giảm giá.                  |
| 8   | total_amount    | NUMERIC(18,<br>2) | NOT NULL;<br>CHECK<br>(unit_price >= 0 AND<br>total_amount >= 0) | Thành tiền của dòng giao dịch.     |

## 2.6. Tổ chức dự án và mô tả mã nguồn

### 2.6.1. Cấu trúc Backend Spring Boot

Phần Backend của hệ thống được xây dựng bằng Java Spring Boot và sử dụng Gradle để quản lý thư viện, biên dịch và thực hiện quá trình build ứng dụng. Mã nguồn được tổ chức thành các thư mục có trách nhiệm riêng nhằm giúp chương trình dễ quản lý, hạn chế sự phụ thuộc giữa các thành phần và thuận tiện khi mở rộng thêm chức năng.

Cấu trúc tổng thể của Backend được tổ chức như sau:



Hình 2.45: Cấu trúc thư mục backend

Thư mục gradle/wrapper chứa các tệp cần thiết của Gradle Wrapper. Thành phần này giúp dự án sử dụng đúng phiên bản Gradle đã được cấu hình mà không yêu cầu mỗi máy phát triển phải cài đặt Gradle riêng. Nhờ đó, quá trình build dự án có thể được thực hiện nhất quán giữa các môi trường khác nhau.

Thư mục `src/main/java` chứa toàn bộ mã nguồn Java chính của Backend. Package gốc của chương trình là `com.app.backend`, bên trong được chia thành các nhóm chức năng riêng.

Thư mục `common` chứa các thành phần được sử dụng chung trong nhiều module. Việc tách các thành phần dùng chung ra khỏi từng chức năng giúp hạn chế việc lặp mã nguồn và tạo ra cách xử lý thống nhất trong toàn hệ thống.

Thư mục `common/enums` chứa các kiểu liệt kê được sử dụng trong chương trình. Các enum được dùng để biểu diễn những tập giá trị xác định trước như trạng thái bản ghi, phạm vi quyền hoặc các loại dữ liệu nghiệp vụ. Việc sử dụng enum giúp mã nguồn rõ nghĩa hơn và hạn chế việc sử dụng các chuỗi giá trị không thống nhất.

Thư mục `common/exception` chịu trách nhiệm quản lý các lớp ngoại lệ và cơ chế xử lý lỗi chung. Các lỗi phát sinh trong quá trình xử lý nghiệp vụ có thể được chuyển đổi thành cấu trúc phản hồi phù hợp trước khi gửi về ứng dụng Desktop. Cách tổ chức này giúp các Controller không phải tự xử lý lỗi theo nhiều cách khác nhau.

Thư mục `common/response` chứa các lớp phục vụ chuẩn hóa dữ liệu Response của REST API. Kết quả thành công và thất bại được tổ chức theo cùng một cấu trúc, giúp Desktop Client có thể xử lý phản hồi từ các API một cách thống nhất.

Thư mục `configs` chứa các lớp cấu hình chung của Spring Boot. Những thiết lập liên quan đến ứng dụng, CORS, Bean hoặc các thành phần dùng chung có thể được đặt tại đây. Việc tập trung các lớp cấu hình giúp tách phần thiết lập hệ thống khỏi mã nguồn xử lý nghiệp vụ.

Thư mục `entities` chứa các Entity đại diện cho dữ liệu được lưu trữ trong PostgreSQL. Mỗi Entity được ánh xạ tới bảng tương ứng trong cơ sở dữ liệu thông qua JPA và Hibernate. Các Entity cũng thể hiện những mối quan hệ giữa các bảng như một – một, một – nhiều hoặc nhiều – một.

Thư mục `features` là khu vực chứa các module nghiệp vụ chính của hệ thống. Các chức năng như xác thực, tài khoản, nhân viên, khách hàng, sản phẩm, kho, bảng giá, giao dịch và báo cáo có thể được tổ chức thành từng module riêng trong thư mục

này. Việc tổ chức theo chức năng giúp các lớp liên quan đến cùng một nghiệp vụ được nhóm lại với nhau, thuận tiện khi tìm kiếm và bảo trì mã nguồn.

Trong từng module nghiệp vụ, mã nguồn có thể tiếp tục được phân chia theo các thành phần như Controller, Service, Repository và DTO. Controller tiếp nhận yêu cầu từ Client, Service xử lý nghiệp vụ, Repository đảm nhiệm truy xuất dữ liệu và DTO được sử dụng để trao đổi dữ liệu giữa Client với Backend.

Thư mục security chứa các thành phần liên quan đến bảo mật của hệ thống. Đây là nơi tổ chức cơ chế xác thực, xử lý JWT, cấu hình Spring Security và kiểm tra quyền truy cập. Các yêu cầu gửi tới API cần bảo vệ được xử lý qua các thành phần bảo mật trước khi được chuyển tới phần nghiệp vụ.

Thư mục src/main/resources chứa các tài nguyên và tệp cấu hình của ứng dụng Spring Boot. Những thông tin như cấu hình kết nối PostgreSQL, cổng chạy Server, thông số JWT và các thiết lập khác của ứng dụng được quản lý tại khu vực này.

Thư mục src/test được sử dụng để lưu các mã nguồn phục vụ quá trình kiểm thử Backend. Các thành phần nghiệp vụ, Repository hoặc API có thể được kiểm tra độc lập nhằm phát hiện lỗi trước khi tích hợp hoàn chỉnh với ứng dụng Desktop.

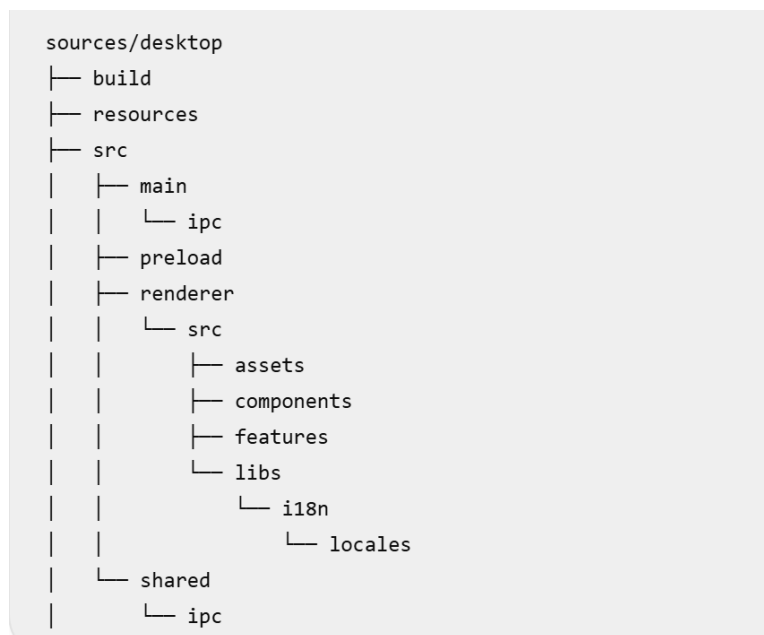
Thư mục build được Gradle tạo ra trong quá trình biên dịch và đóng gói chương trình. Thư mục này chứa các tệp được sinh tự động như class đã biên dịch, kết quả kiểm thử và tệp đóng gói. Đây không phải là nơi lưu mã nguồn chính và có thể được tạo lại thông qua quá trình build.

Cách tổ chức Backend trên giúp phân chia rõ các thành phần dùng chung, cấu hình, dữ liệu, nghiệp vụ và bảo mật. Cấu trúc này phù hợp với kiến trúc nhiều tầng của Spring Boot, đồng thời tạo điều kiện để từng module có thể được phát triển và bảo trì tương đối độc lập.

### **2.6.2. Thư mục desktop Electron**

Phần Desktop của hệ thống được xây dựng bằng Electron kết hợp React và TypeScript. Mã nguồn được tổ chức theo hướng tách riêng Main Process, Preload Process, Renderer Process và các thành phần dùng chung. Cách tổ chức này giúp phân

định rõ trách nhiệm giữa phần xử lý của Electron, giao diện người dùng và cơ chế giao tiếp nội bộ của ứng dụng.



Hình 2.46: Cấu trúc thư mục desktop

Thư mục build chứa các thành phần được tạo ra trong quá trình build và đóng gói ứng dụng Desktop. Đây không phải là khu vực lưu trữ mã nguồn chính mà chủ yếu phục vụ quá trình tạo phiên bản ứng dụng có thể chạy hoặc phân phối.

Thư mục resources chứa các tài nguyên phục vụ ứng dụng Electron như biểu tượng chương trình, hình ảnh hoặc các tệp cần thiết khi đóng gói và triển khai ứng dụng.

Thư mục src/main chứa mã nguồn thuộc Main Process của Electron. Main Process chịu trách nhiệm quản lý vòng đời ứng dụng, cửa sổ chương trình và các chức năng cần tương tác trực tiếp với môi trường Desktop. Bên trong, thư mục ipc được sử dụng để tổ chức các xử lý giao tiếp IPC giữa Main Process và các thành phần khác của ứng dụng.

Thư mục src/preload chứa mã nguồn Preload Script. Thành phần này đóng vai trò trung gian giữa Main Process và Renderer Process, cung cấp những chức năng cần

thiết cho giao diện thông qua một lớp giao tiếp được kiểm soát thay vì để Renderer truy cập trực tiếp vào toàn bộ môi trường Node.js.

Thư mục `src/renderer` chứa phần giao diện của ứng dụng. Trong đó, `renderer/src` là khu vực chính chứa mã nguồn React và các thành phần phục vụ việc xây dựng giao diện người dùng.

Thư mục `assets` chứa các tài nguyên được sử dụng trong giao diện như hình ảnh, biểu tượng và các tệp hỗ trợ hiển thị.

Thư mục `components` chứa các thành phần giao diện có khả năng tái sử dụng tại nhiều màn hình khác nhau. Việc tách các component dùng chung giúp giảm sự lặp lại của mã nguồn và duy trì cách trình bày thống nhất trong toàn bộ ứng dụng.

Thư mục `features` được sử dụng để tổ chức mã nguồn theo từng nhóm chức năng nghiệp vụ. Các chức năng như đăng nhập, sản phẩm, kho, khách hàng, giao dịch hoặc báo cáo có thể được phân chia thành những module riêng tại đây. Cách tổ chức theo feature giúp những thành phần liên quan đến cùng một nghiệp vụ được đặt gần nhau, thuận tiện cho quá trình phát triển và bảo trì.

Thư mục `libs` chứa các thư viện hoặc thành phần hỗ trợ được sử dụng trong phần giao diện. Trong đó, `lib` phục vụ cơ chế đa ngôn ngữ của ứng dụng và thư mục `locales` chứa các tài nguyên ngôn ngữ tương ứng.

Thư mục `src/shared` chứa những thành phần được sử dụng chung giữa các phần của ứng dụng Electron. Thư mục `shared/ipc` có vai trò tập trung các định nghĩa liên quan đến giao tiếp IPC, giúp Main Process, Preload và Renderer sử dụng cùng một quy ước khi trao đổi dữ liệu.

Việc tách cấu trúc Desktop thành Main Process, Preload, Renderer và Shared giúp mã nguồn có tính tổ chức rõ ràng. Main Process tập trung vào chức năng của môi trường Desktop, Renderer đảm nhiệm giao diện người dùng, Preload kiểm soát cầu nối giữa hai phía và Shared cung cấp các thành phần dùng chung. Cấu trúc này phù hợp với mô hình hoạt động của Electron và hỗ trợ tốt cho việc mở rộng ứng dụng trong quá trình phát triển.

## CHƯƠNG 3. PHÁT TRIỂN HỆ THỐNG

### 3.1. Phát triển tầng dữ liệu

#### 3.1.1. Kết nối PostgreSQL

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/GoldChuan2026
    username: postgres
    password: admin
    hikari:
      pool-name: data-gold-chuan2026
      maximum-pool-size: 20
      minimum-idle: 10
```

Hình 3.47: Kết nối sql

Hệ thống sử dụng PostgreSQL làm hệ quản trị cơ sở dữ liệu chính cho Backend Spring Boot. Việc kết nối cơ sở dữ liệu được cấu hình trong tệp application.yml thông qua nhóm thuộc tính spring.datasource. Backend sử dụng JDBC Driver của PostgreSQL để thiết lập kết nối và thực hiện trao đổi dữ liệu với cơ sở dữ liệu.

Trong cấu hình hiện tại, Backend kết nối đến PostgreSQL tại địa chỉ localhost, cổng 5432 và sử dụng cơ sở dữ liệu GoldChuan2026. Tài khoản postgres được sử dụng để xác thực kết nối trong môi trường phát triển.

Chuỗi kết nối được cấu hình theo dạng:

**jdbc:postgresql://localhost:5432/GoldChuan2026**

Bên cạnh thông tin kết nối cơ bản, hệ thống sử dụng HikariCP để quản lý Connection Pool. HikariCP được Spring Boot sử dụng nhằm duy trì một tập hợp các kết nối cơ sở dữ liệu có thể tái sử dụng, hạn chế việc phải tạo mới kết nối cho mỗi yêu cầu truy vấn.

Connection Pool của hệ thống được đặt tên data-gold-chuan2026. Giá trị maximum-pool-size được thiết lập là 20, cho phép tối đa 20 kết nối được quản lý trong

Pool. Thuộc tính `minimum-idle` có giá trị 10, giúp duy trì tối thiểu 10 kết nối ở trạng thái sẵn sàng. Cấu hình này giúp Backend có thể xử lý nhiều yêu cầu truy xuất dữ liệu mà không phải liên tục khởi tạo và đóng kết nối PostgreSQL.

Sau khi Spring Boot khởi động, `DataSource` được tạo dựa trên các thông tin đã khai báo. Các thành phần Spring Data JPA, Hibernate và JDBC trong hệ thống sử dụng `DataSource` này để thực hiện các thao tác truy vấn, thêm mới, cập nhật và xử lý dữ liệu trong PostgreSQL.

Việc tập trung thông tin kết nối tại tệp cấu hình giúp mã nguồn nghiệp vụ không phụ thuộc trực tiếp vào thông tin của cơ sở dữ liệu. Khi thay đổi máy chủ PostgreSQL, cổng kết nối hoặc tên cơ sở dữ liệu, chỉ cần điều chỉnh cấu hình mà không phải sửa đổi các lớp Service, Repository hoặc Entity.

### 3.1.2. *Xây dựng Entity*

Trong hệ thống, các Entity được xây dựng dựa trên JPA và được Hibernate sử dụng để ánh xạ giữa các đối tượng Java với các bảng trong cơ sở dữ liệu PostgreSQL. Mỗi Entity đại diện cho một bảng dữ liệu và chứa các thuộc tính tương ứng với các cột của bảng đó.

Các lớp Entity được khai báo bằng annotation `@Entity` để JPA nhận biết đây là đối tượng cần quản lý. Annotation `@Table` được sử dụng để xác định bảng tương ứng trong cơ sở dữ liệu, trong khi `@Id` được dùng để đánh dấu khóa chính của Entity. Các trường dữ liệu có thể được ánh xạ bằng `@Column` nhằm xác định tên cột, tính bắt buộc, độ dài hoặc các thuộc tính liên quan.

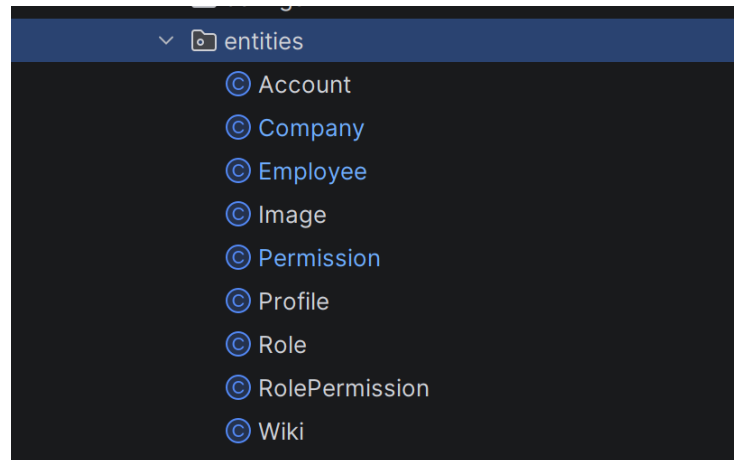
Trong dự án, các Entity được xây dựng tương ứng với các bảng chính như Account, Profile, Company, Role, Permission, Employee, Customer, GoldType, Product, Warehouse, StockBalance, PriceList, Transaction và TransactionDetail. Việc ánh xạ này giúp chương trình thao tác với dữ liệu dưới dạng đối tượng Java thay vì phải xử lý trực tiếp từng dòng dữ liệu trong PostgreSQL.

JPA chỉ định nghĩa các quy tắc và annotation phục vụ ánh xạ đối tượng – quan hệ, còn Hibernate là implementation được sử dụng để thực hiện việc ánh xạ và tương

tác với cơ sở dữ liệu. Khi chương trình thực hiện thao tác thêm, sửa, xóa hoặc truy vấn Entity, Hibernate sẽ chuyển các thao tác đó thành câu lệnh SQL tương ứng.

Đối với các trường định danh UUID, giá trị không được sinh trực tiếp trong Java mà được PostgreSQL tự động tạo thông qua giá trị mặc định `uuidv7()`. Vì vậy, Entity chỉ thực hiện ánh xạ trường UUID với cột trong cơ sở dữ liệu và nhận lại giá trị sau khi bản ghi được tạo.

Việc xây dựng Entity bằng JPA giúp giảm lượng mã nguồn thao tác dữ liệu thủ công, đồng thời tạo sự thống nhất giữa mô hình đối tượng trong Java và mô hình dữ liệu trong PostgreSQL. Đây cũng là cơ sở để xây dựng Repository bằng Spring Data JPA ở các bước tiếp theo.



Hình 3.48: Entities được tạo bởi JPA

### 3.1.3. *Xây dựng Repository với Spring Data JPA*

Tầng Repository được xây dựng bằng Spring Data JPA nhằm thực hiện các thao tác truy xuất và quản lý dữ liệu trong PostgreSQL. Repository đóng vai trò trung gian giữa tầng nghiệp vụ và tầng dữ liệu, giúp Service làm việc với các Entity mà không cần trực tiếp viết các câu lệnh SQL cho những thao tác cơ bản.

Trong hệ thống, mỗi Repository được xây dựng tương ứng với một Entity và kế thừa `JpaRepository`. `JpaRepository` cung cấp sẵn nhiều phương thức thao tác dữ liệu như thêm mới, cập nhật, xóa, tìm kiếm theo khóa chính và lấy danh sách dữ liệu.

Ví dụ, `AuthRepository` được xây dựng để thao tác với Entity `Account`:

```

package com.app.backend.features.auth.repositories;

> import ...

2 usages
@Repository
public interface AuthRepository extends JpaRepository<Account, Long> {
    /**
     * Find account by username
     * @param username The username by user
     * @return Account
     */
    1 usage
    Optional<Account> findByUsername(String username);
}

```

Hình 3.49: Tìm kiếm tài khoản bằng JPA

Trong đó, Account là Entity mà Repository quản lý, còn Long là kiểu dữ liệu của khóa chính id trong bảng tài khoản.

Ngoài các phương thức được cung cấp sẵn, Spring Data JPA cho phép xây dựng truy vấn thông qua tên phương thức. Trong AuthRepository, phương thức:

### **Optional<Account> findByUsername(String username)**

được sử dụng để tìm tài khoản theo username. Dựa vào quy tắc đặt tên findByUsername, Spring Data JPA tự động tạo truy vấn tương ứng mà không cần lập trình viên viết câu lệnh SQL thủ công.

Kiểu Optional<Account> được sử dụng để biểu diễn trường hợp có hoặc không tìm thấy tài khoản. Cách xử lý này giúp hạn chế việc trả về giá trị null và thuận tiện hơn khi kiểm tra sự tồn tại của dữ liệu tại tầng Service.

Annotation @Repository được sử dụng để xác định interface này thuộc tầng truy cập dữ liệu của ứng dụng Spring. Các Repository sau đó được Spring quản lý và có thể được inject vào Service thông qua Dependency Injection.

Ngoài AuthRepository, các chức năng khác của hệ thống cũng có thể xây dựng Repository tương ứng với từng Entity như ProductRepository, CustomerRepository, WarehouseRepository, TransactionRepository và các Repository phục vụ những module nghiệp vụ khác.

Việc sử dụng Spring Data JPA giúp giảm đáng kể số lượng mã nguồn cần viết cho tầng truy cập dữ liệu. Những thao tác CRUD thông thường được cung cấp sẵn, trong khi các truy vấn đơn giản có thể được xây dựng dựa trên tên phương thức. Đối với những truy vấn phức tạp hoặc yêu cầu tối ưu riêng, hệ thống có thể sử dụng JPQL, Native Query hoặc JDBC tùy theo yêu cầu xử lý.

#### 3.1.4. *Xử lý Transaction*

Trong hệ thống, Transaction được sử dụng để bảo đảm tính toàn vẹn của dữ liệu khi một nghiệp vụ cần thực hiện đồng thời nhiều thao tác với cơ sở dữ liệu. Các thao tác thuộc cùng một nghiệp vụ phải được thực hiện thành công toàn bộ hoặc được hoàn tác nếu xảy ra lỗi trong quá trình xử lý.

```
@Transactional
public CompanyDto.Response createCompany(CompanyDto.Request request, JwtClaim claim) {
    Long accountId = currentAccountId(claim);
    CompanyView company = companyRepository.create(
        accountId,
        request.name().trim(),
        blankToNull(request.address()),
        blankToNull(request.phone()),
        blankToNull(request.email()),
        request.taxCode().trim(),
        statusValue(request.status())
    );
    return CompanyDto.Response.from(company);
}
```

Hình 3.50: Sử dụng transaction vào sql

Spring Boot hỗ trợ quản lý Transaction thông qua Spring Transaction Management. Trong dự án, Transaction được xử lý chủ yếu tại tầng Service bằng annotation `@Transactional`. Khi một phương thức Service được đánh dấu `@Transactional`, Spring sẽ mở một Transaction trước khi phương thức bắt đầu thực hiện và kết thúc Transaction sau khi toàn bộ quá trình xử lý hoàn thành.

Nếu các thao tác trong phương thức thực hiện thành công, Transaction sẽ được commit và dữ liệu được lưu chính thức vào PostgreSQL. Ngược lại, khi xảy ra lỗi trong quá trình xử lý, Transaction có thể được rollback để đưa dữ liệu trở về trạng thái trước khi nghiệp vụ bắt đầu.

Cơ chế này đặc biệt quan trọng đối với nghiệp vụ giao dịch mua bán. Khi một giao dịch được tạo, hệ thống có thể phải thực hiện nhiều thao tác liên tiếp như tạo thông tin giao dịch, thêm các chi tiết giao dịch, kiểm tra tồn kho, cập nhật số lượng tồn và tính toán tổng tiền. Các thao tác này cần được thực hiện trong cùng một Transaction để tránh tình trạng chỉ một phần dữ liệu được cập nhật.

Ví dụ, khi thực hiện giao dịch bán sản phẩm, hệ thống trước tiên kiểm tra số lượng tồn kho. Nếu số lượng đáp ứng yêu cầu, thông tin giao dịch và các chi tiết giao dịch được lưu, sau đó số lượng tồn của sản phẩm được cập nhật. Nếu xảy ra lỗi khi cập nhật tồn kho hoặc lưu chi tiết giao dịch, toàn bộ quá trình cần được rollback để tránh trường hợp giao dịch đã được tạo nhưng tồn kho chưa được cập nhật.

Transaction cũng được sử dụng trong những nghiệp vụ có liên quan đến nhiều bảng dữ liệu. Việc đặt Transaction tại tầng Service giúp tập trung ranh giới nghiệp vụ tại một vị trí rõ ràng, thay vì để từng Repository tự quản lý các thao tác độc lập.

Trong Spring, @Transactional có thể được áp dụng trực tiếp lên phương thức xử lý nghiệp vụ. Các Repository được gọi bên trong phương thức sẽ cùng tham gia vào Transaction đang hoạt động. Điều này giúp việc phối hợp nhiều thao tác thêm, sửa hoặc xóa dữ liệu trở nên đơn giản hơn.

Đối với các lỗi Runtime Exception phát sinh trong quá trình xử lý, Spring mặc định thực hiện rollback Transaction. Với những trường hợp ngoại lệ khác, hệ thống có thể cấu hình thêm điều kiện rollback phù hợp với yêu cầu nghiệp vụ.

Việc áp dụng Transaction giúp hệ thống hạn chế tình trạng dữ liệu không đồng bộ giữa giao dịch, chi tiết giao dịch và tồn kho. Đây là cơ chế quan trọng để bảo đảm tính nhất quán và độ tin cậy của dữ liệu trong quá trình vận hành.

## **3.2. Phát triển tầng nghiệp vụ**

### **3.2.1. Xây dựng Service Layer**

Service Layer được đặt giữa Controller và Repository. Controller chỉ nhận request, kiểm tra định dạng cơ bản và gọi service; service mới là nơi quyết định dữ liệu có hợp lệ về mặt nghiệp vụ hay không.

Trong backend Spring Boot, các lớp service như AuthService, AccountService, CompanyService và MockupService được đánh dấu bằng @Service. Cách tổ chức này giúp Spring quản lý vòng đời đối tượng, hỗ trợ dependency injection và cho phép đặt transaction ở đúng nơi xử lý nghiệp vụ.

Các phương thức service được thiết kế theo từng hành động nghiệp vụ. Ví dụ, loginService xử lý đăng nhập, getMeService lấy thông tin tài khoản hiện tại, createCompany tạo doanh nghiệp mới, updateCompany cập nhật doanh nghiệp và module trong MockupService cung cấp dữ liệu cho các phân hệ quản lý trên giao diện.

```
2 usage
@Service
@RequiredArgsConstructor
public class CompanyService {
    private final CompanyRepository companyRepository;
1 usage
```

Hình 3.51: Service Layer trong spring boot

### 3.2.2. *Xử lý nghiệp vụ tài khoản*

Nghiệp vụ tài khoản gồm đăng nhập, lấy thông tin người dùng hiện tại và kiểm tra trạng thái nhân viên trước khi cho phép sử dụng hệ thống. Khi người dùng gửi tên đăng nhập và mật khẩu, AuthService tìm tài khoản theo username. Nếu không có tài khoản phù hợp, hệ thống trả lỗi đăng nhập không hợp lệ.

Sau khi tìm được tài khoản, mật khẩu người dùng nhập vào được so khớp với mật khẩu đã mã hóa. Hệ thống không so sánh mật khẩu dạng rõ, nhờ vậy dữ liệu đăng nhập được bảo vệ tốt hơn khi lưu trong cơ sở dữ liệu.

Tài khoản hợp lệ vẫn phải gắn với nhân viên đang hoạt động. Nếu nhân viên bị khóa hoặc không còn hoạt động, hệ thống không cấp quyền truy cập. Bước này phù hợp với môi trường cửa hàng, nơi tài khoản có thể tồn tại nhưng nhân viên đã nghỉ việc hoặc tạm dừng sử dụng hệ thống.

```

public LoginDto.Response loginService(LoginDto.Request request) {
    var account = authRepository.findByUsername(request.username())
        .orElseThrow(() -> new AppException(ErrorCode.AUTH_INVALID_CREDENTIALS));
    if (!passwordEncoder.matches(request.password(), account.getPassword())) {
        throw new AppException(ErrorCode.AUTH_INVALID_CREDENTIALS);
    }

    var employee = employeeRepository.findById(account.getId())
        .orElseThrow(() -> new AppException(ErrorCode.AUTH_EMPLOYEE_INACTIVE));
    if (employee.getStatus() != RecordType.active) {
        throw new AppException(ErrorCode.AUTH_EMPLOYEE_INACTIVE);
    }

    String token = jwtService.generateToken(account);

    return new LoginDto.Response(token);
}

```

Hình 3.52: Đăng nhập tại tầng service

### 3.2.3. *Xử lý nghiệp vụ sản phẩm*

Nghiệp vụ sản phẩm quản lý thông tin trang sức gồm mã sản phẩm, tên sản phẩm, nhóm hàng, tuổi vàng, trọng lượng, giá vốn, giá bán và trạng thái kinh doanh. Các dữ liệu này là cơ sở cho bán hàng, tồn kho, bảng giá và báo cáo doanh thu.

Khi thêm mới sản phẩm, hệ thống cần kiểm tra mã sản phẩm không bị trùng trong phạm vi doanh nghiệp, tên sản phẩm không được bỏ trống, giá bán và trọng lượng phải là giá trị hợp lệ. Khi cập nhật sản phẩm, hệ thống giữ lại mã định danh để bảo đảm các giao dịch cũ vẫn liên kết đúng sản phẩm.

Khi xóa hoặc ngừng bán sản phẩm, cách xử lý phù hợp là chuyển trạng thái thay vì xóa vật lý ngay lập tức. Điều này giúp báo cáo và lịch sử giao dịch không bị mất liên kết.

### 3.2.4. *Xử lý nghiệp vụ kho*

Nghiệp vụ kho theo dõi nơi lưu trữ sản phẩm, vị trí tủ kệ, số lượng nhập, số lượng xuất và trạng thái cảnh báo. Mỗi kho thuộc một doanh nghiệp hoặc chi nhánh, vì vậy mọi truy vấn kho cần gắn với phạm vi công ty của người dùng đang đăng nhập.

Khi tạo kho, hệ thống cần kiểm tra mã kho và tên kho. Khi cập nhật, thông tin địa chỉ, trạng thái và mô tả kho được điều chỉnh nhưng các dữ liệu phát sinh như phiếu nhập xuất vẫn phải giữ nguyên quan hệ với kho.

Trên giao diện, màn hình quản lý kho hiển thị tồn đầu, nhập, xuất, tồn cuối và cảnh báo để người quản lý nhận biết nhanh những sản phẩm sắp hết hoặc đã hết hàng.

### **3.2.5. *Xử lý nghiệp vụ tồn kho***

Tồn kho được tính từ số lượng đầu kỳ cộng nhập trong kỳ và trừ xuất trong kỳ. Với sản phẩm trang sức, tồn kho không chỉ là số lượng mà còn gắn với giá trị hàng hóa lớn, vị trí lưu trữ và trạng thái cảnh báo.

Khi phát sinh giao dịch bán hàng, hệ thống phải kiểm tra số lượng tồn trước khi cho phép hoàn tất hóa đơn. Nếu số lượng yêu cầu lớn hơn tồn thực tế, service cần trả lỗi nghiệp vụ để giao diện thông báo cho người dùng.

Các thay đổi tồn kho nên được xử lý trong transaction. Nếu một giao dịch gồm nhiều sản phẩm mà một dòng bị lỗi, toàn bộ giao dịch không được ghi một phần, tránh tình trạng hóa đơn đã tạo nhưng tồn kho không khớp.

### **3.2.6. *Xử lý nghiệp vụ bảng giá***

Bảng giá lưu giá mua, giá bán, công chế tác và thời điểm hiệu lực. Vì giá vàng và trang sức có thể thay đổi thường xuyên, bảng giá cần hỗ trợ nhiều phiên bản theo ngày áp dụng.

Khi tạo bảng giá mới, hệ thống kiểm tra khoảng hiệu lực để tránh hai bảng giá cùng loại áp dụng chồng lấn. Khi lập hóa đơn, service cần lấy bảng giá còn hiệu lực tại thời điểm giao dịch để tính đơn giá.

Bảng giá không nên ghi đè dữ liệu giao dịch đã phát sinh. Giá tại thời điểm bán cần được lưu vào chi tiết giao dịch để báo cáo doanh thu sau này không bị thay đổi khi cập nhật giá mới.

### **3.2.7. *Validation dữ liệu***

Validation được thực hiện ở nhiều mức. Tầng DTO kiểm tra dữ liệu bắt buộc, định dạng và kiểu dữ liệu; tầng service kiểm tra ràng buộc nghiệp vụ như trạng thái tài khoản, mã dữ liệu trùng, tồn kho không đủ hoặc bảng giá hết hiệu lực.

Các request quan trọng dùng @Valid để Spring tự động kích hoạt kiểm tra dữ liệu trước khi vào service. Nếu dữ liệu sai, GlobalExceptionHandler chuyển lỗi validate thành response lỗi thống nhất.

Validation giúp giảm lỗi nhập liệu, bảo vệ dữ liệu trong database và tạo phản hồi rõ ràng cho người dùng. Đây là phần bắt buộc vì hệ thống bán hàng có nhiều dữ liệu nhạy cảm như giá, công nợ và thanh toán.

### 3.2.8. Xử lý Exception

Exception được chuẩn hóa bằng ApplicationException, ErrorCode và GlobalExceptionHandler. Khi service phát hiện lỗi nghiệp vụ, service chủ động ném ApplicationException thay vì trả null hoặc chuỗi lỗi rời rạc.

GlobalExceptionHandler tiếp nhận lỗi và chuyển thành response có cấu trúc gồm type, title, status, detail và instance. Cách này giúp frontend chỉ cần xử lý một kiểu lỗi thống nhất cho toàn bộ API.

Các lỗi database hoặc lỗi ngoài dự kiến được ghi log nội bộ nhưng không trả trực tiếp câu lệnh SQL hoặc stack trace cho người dùng. Điều này vừa bảo mật, vừa giúp thông báo lỗi trên giao diện gọn và dễ hiểu hơn.

*Bảng 3.17: Bảng nghiệp vụ Exception*

| Nhóm nghiệp vụ | Dữ liệu chính                            | Kết quả xử lý                              |
|----------------|------------------------------------------|--------------------------------------------|
| Tài khoản      | username, password, employee status      | Xác thực đăng nhập và sinh token           |
| Sản phẩm       | mã sản phẩm, tuổi vàng, trọng lượng, giá | Quản lý danh mục sản phẩm phục vụ bán hàng |

|                       |                                             |                                            |
|-----------------------|---------------------------------------------|--------------------------------------------|
| <b>Kho và tồn kho</b> | kho, sản phẩm, số lượng, vị trí             | Theo dõi nhập xuất tồn và cảnh báo sắp hết |
| <b>Bảng giá</b>       | giá mua, giá bán, công, hiệu lực            | Áp dụng giá đúng thời điểm giao dịch       |
| <b>Giao dịch</b>      | khách hàng, nhân viên, sản phẩm, thanh toán | Lập hóa đơn và cập nhật tồn kho            |
| <b>Báo cáo</b>        | khoảng thời gian, nhóm dữ liệu, chỉ số      | Tổng hợp số liệu quản trị                  |

### 3.3. Phát triển tầng API và bảo mật

#### 3.3.1. Xây dựng Controller

Controller được xây dựng theo từng nhóm chức năng, ví dụ AuthController xử lý đăng nhập, AccountController lấy thông tin tài khoản hiện tại, CompanyController quản lý công ty và MockupController cung cấp dữ liệu cho các phân hệ giao diện.

Controller không chứa nhiều logic nghiệp vụ. Nhiệm vụ chính của controller là định nghĩa endpoint, nhận tham số từ request, gọi service tương ứng và trả về ResponseEntity với ApiResponse chuẩn hóa.

#### 3.3.2. Chuẩn hóa Response API

Response thành công dùng ApiResponse.Success với ba trường chính là success, msg và data. success cho biết request xử lý thành công, msg chứa thông báo, data chứa dữ liệu trả về.

Việc chuẩn hóa response giúp frontend không phải viết nhiều nhánh xử lý khác nhau. Khi nhận response thành công, client chỉ cần đọc data để hiển thị lên màn hình.

### **3.3.3. Xây dựng chức năng đăng nhập**

Chức năng đăng nhập bắt đầu từ POST /api/v1/auth/login. Người dùng gửi username và password, AuthController chuyển dữ liệu sang AuthService để kiểm tra.

AuthService tìm tài khoản theo username, kiểm tra mật khẩu, kiểm tra nhân viên liên kết với tài khoản và trạng thái hoạt động. Nếu mọi điều kiện hợp lệ, hệ thống sinh JWT và trả về token cho client.

### **3.3.4. Mã hóa mật khẩu bằng Bcrypt**

SpringSecurity khai báo BCryptPasswordEncoder để xử lý mật khẩu. Khi đăng nhập, passwordEncoder.matches so sánh mật khẩu người dùng nhập với mật khẩu đã băm đang lưu trong database.

Bcrypt phù hợp cho mật khẩu vì mỗi giá trị băm có salt và chi phí tính toán riêng. Điều này làm giảm rủi ro lộ mật khẩu gốc nếu dữ liệu tài khoản bị truy cập trái phép.

### **3.3.5. Sinh JWT**

Sau khi đăng nhập hợp lệ, JwtService tạo JWT dựa trên thông tin tài khoản. Token này đại diện cho phiên làm việc của người dùng trên ứng dụng desktop.

JWT giúp backend hoạt động theo kiểu stateless. Máy chủ không cần lưu session truyền thống; mỗi request tự mang thông tin xác thực qua token.

### **3.3.6. Xác thực JWT**

JwtAuthenticationFilter chạy trước UsernamePasswordAuthenticationFilter. Filter đọc header Authorization, kiểm tra tiền tố Bearer, tách token và gọi JwtService để phân tích token.

Nếu token hợp lệ, filter tạo UsernamePasswordAuthenticationToken và đưa vào SecurityContextHolder. Từ thời điểm đó, controller hoặc service có thể biết request đang thuộc về tài khoản nào.

### 3.3.7. *Cấu hình Spring Security*

Cấu hình bảo mật tắt CSRF vì API hoạt động theo mô hình token, bật CORS để desktop client có thể gọi backend, và đặt sessionCreationPolicy là STATELESS.

Một số đường dẫn được cho phép truy cập công khai như /api/v1/auth/login, /api/v1/health/ping và Swagger. Các request còn lại yêu cầu xác thực.

## 3.4. **Phát triển ứng dụng Desktop**

### 3.4.1. *Khởi tạo React Electron*

Dự án desktop được đặt trong thư mục sources/desktop và sử dụng Electron Vite. Cấu hình package.json, electron.vite.config.ts và các file tsconfig tách riêng môi trường main, preload và renderer.

React được dùng để xây dựng các màn hình nghiệp vụ. Cách tổ chức theo feature giúp mỗi phân hệ có page, action và kiểu dữ liệu riêng, thuận tiện khi mở rộng.

### 3.4.2. *Xây dựng Electron Main Process*

Main process quản lý vòng đời ứng dụng Electron, tạo cửa sổ BrowserWindow và xử lý các sự kiện hệ điều hành. Các file src/main/app.ts, src/main/window.ts và src/main/index.ts đảm nhiệm phần này.

Main process cũng đăng ký các kênh IPC phục vụ điều khiển cửa sổ như thu nhỏ, phóng to, đóng ứng dụng. Việc tách IPC riêng giúp mã nguồn dễ bảo trì hơn.

### 3.4.3. *Xây dựng Preload Script*

Preload script là cầu nối an toàn giữa main process và renderer process. File src/preload/index.ts định nghĩa các API được phép truy cập từ giao diện.

Cách làm này giúp renderer không truy cập trực tiếp Node.js hoặc Electron API nhạy cảm. Ứng dụng vẫn có thể dùng chức năng desktop nhưng giữ được ranh giới bảo mật giữa giao diện và hệ thống.

#### **3.4.4. *Xây dựng Renderer Process***

Renderer process là phần hiển thị giao diện React. File main.tsx khởi tạo ứng dụng, còn App.tsx quản lý trạng thái đăng nhập, profile hiện tại và trang đang mở.

Renderer chịu trách nhiệm gọi API, hiển thị dữ liệu, nhận thao tác người dùng và phản hồi lỗi. Các màn hình được chia thành workspace, auth, overview và system để phù hợp với nhóm chức năng.

#### **3.4.5. *Xây dựng giao diện React***

Giao diện React gồm màn hình đăng nhập, dashboard, quản lý sản phẩm, quản lý kho, nhân viên, khách hàng, bảng giá, bán hàng, giao dịch, báo cáo và phân quyền. Các thành phần dùng chung như TitleBar, WindowControls, ToastProvider và AppNavbar giúp giao diện thống nhất.

Các màn hình quản lý dùng bố cục bảng dữ liệu kết hợp khu vực thao tác. Người dùng có thể tìm kiếm, chọn dòng, thêm mới, cập nhật hoặc xóa dữ liệu theo cùng một quy tắc.

#### **3.4.6. *Xây dựng hệ thống Router***

Ứng dụng desktop hiện điều hướng theo state activePage trong App.tsx. Khi người dùng chọn chức năng trên thanh điều hướng, activePage thay đổi và AuthPage hoặc WorkspaceLayout hiển thị màn hình tương ứng.

Cách điều hướng này gọn cho ứng dụng desktop vì không phụ thuộc URL trình duyệt. Đồng thời, hệ thống vẫn có thể mở rộng sang React Router nếu số lượng màn hình tăng hoặc cần deep link nội bộ.

#### **3.4.7. *Xây dựng Service gọi API***

Các hàm gọi API được đặt trong thư mục actions. auth.api.ts xử lý đăng nhập và lấy thông tin tài khoản; workspace.api.ts xử lý dashboard, dữ liệu phân hệ, thao tác thêm sửa xóa và báo cáo.

Hàm request trong `libs/index.ts` chuẩn hóa base URL, header Content-Type, Accept-Language, Locale và cách đọc lỗi. Nhờ vậy mỗi màn hình không phải tự viết lại logic gọi fetch.

## KẾT LUẬN

### 1. Kết quả thu được

Sau quá trình phân tích, thiết kế và triển khai, đề tài đã xây dựng được một ứng dụng Desktop phục vụ quản lý hoạt động kinh doanh trang sức với các chức năng nghiệp vụ cơ bản. Hệ thống được tổ chức theo mô hình Client - Server, trong đó ứng dụng Desktop đảm nhiệm phân giao diện và tương tác với người dùng, còn Backend chịu trách nhiệm xử lý nghiệp vụ, xác thực, phân quyền và làm việc với cơ sở dữ liệu.

Phần Backend được phát triển bằng Java Spring Boot, cung cấp các RESTful API phục vụ quá trình trao đổi dữ liệu với ứng dụng Desktop. Cấu trúc Backend được tổ chức theo các tầng Controller, Service và Repository nhằm phân tách rõ trách nhiệm của từng thành phần. Dữ liệu được lưu trữ trên PostgreSQL và được truy xuất thông qua Spring Data JPA, Hibernate và JDBC tùy theo yêu cầu xử lý.

Hệ thống đã triển khai cơ chế đăng nhập và kiểm soát truy cập người dùng. Spring Security được sử dụng để hỗ trợ xác thực và phân quyền, kết hợp với JWT để duy trì thông tin xác thực trong quá trình Client gửi yêu cầu tới Backend. Các tài khoản được quản lý theo vai trò và quyền nhằm giới hạn phạm vi chức năng phù hợp với từng nhóm người dùng.

Phần Desktop Client được xây dựng bằng React, TypeScript và Electron. Giao diện được chia thành các màn hình và component riêng biệt, hỗ trợ người dùng thực hiện các nghiệp vụ trên hệ thống. Ứng dụng Desktop đã kết nối được với Backend thông qua RESTful API và có khả năng tiếp nhận, hiển thị cũng như cập nhật dữ liệu theo kết quả xử lý từ Server.

Các chức năng chính đã được xây dựng bao gồm quản lý tài khoản, nhân viên, khách hàng, sản phẩm trang sức, loại vàng, kho, tồn kho, bảng giá, giao dịch mua bán, báo cáo thống kê và phân quyền. Các dữ liệu có mối liên hệ với nhau được tổ chức trong cùng một hệ thống, tạo điều kiện cho việc quản lý và tra cứu thông tin thuận tiện hơn.

Thông qua quá trình thực hiện đề tài, các kiến thức về Java, Spring Boot, JPA, JDBC, PostgreSQL, RESTful API, bảo mật ứng dụng và phát triển Desktop đã được vận dụng vào một bài toán cụ thể. Kết quả đạt được cho thấy hệ thống đã đáp ứng được các yêu cầu cơ bản đặt ra và có thể tiếp tục được mở rộng trong những giai đoạn phát triển sau.

## **2. Hạn chế của đề tài**

Mặc dù đã hoàn thành các chức năng chính, hệ thống vẫn còn một số hạn chế do phạm vi và thời gian thực hiện của đề tài. Trước hết, chương trình hiện tập trung vào các nghiệp vụ quản lý cốt lõi và chưa được phát triển đầy đủ cho mô hình có nhiều chi nhánh hoạt động đồng thời. Việc đồng bộ dữ liệu giữa nhiều điểm sử dụng theo thời gian thực chưa được triển khai hoàn chỉnh.

Hệ thống hiện chưa tích hợp trực tiếp với các thiết bị phần cứng thường được sử dụng trong hoạt động kinh doanh như máy quét mã vạch, máy in hóa đơn hoặc cân điện tử. Vì vậy, một số thao tác vẫn cần được thực hiện thủ công thông qua giao diện phần mềm.

Các chức năng báo cáo và thống kê mới tập trung vào những thông tin cơ bản phục vụ quá trình theo dõi hoạt động. Hệ thống chưa hỗ trợ nhiều dạng phân tích nâng cao, so sánh theo nhiều tiêu chí hoặc các công cụ hỗ trợ dự báo dựa trên dữ liệu lịch sử.

Phản triển khai hiện tại chủ yếu phục vụ quá trình xây dựng và kiểm thử trong môi trường phát triển, chưa được tối ưu đầy đủ cho mô hình triển khai Cloud hoặc môi trường có số lượng lớn người dùng và dữ liệu. Hiệu năng của hệ thống khi xử lý khối lượng giao dịch lớn vẫn cần được kiểm tra và tối ưu thêm.

Ngoài ra, một số chức năng liên quan đến sao lưu dữ liệu, giám sát hệ thống, quản lý phiên bản triển khai và phục hồi khi xảy ra sự cố chưa được xây dựng ở mức hoàn chỉnh. Đây là những nội dung cần được bổ sung nếu hệ thống được đưa vào sử dụng trong môi trường thực tế.

## **3. Hướng phát triển**

Trong tương lai, hệ thống có thể được mở rộng theo hướng hỗ trợ nhiều chi nhánh, nhiều kho và nhiều điểm bán trên cùng một nền tảng. Dữ liệu giữa các đơn vị được quản lý tập trung, đồng thời phân quyền theo từng phạm vi cụ thể nhằm bảo đảm mỗi người dùng chỉ được truy cập dữ liệu phù hợp với nhiệm vụ và đơn vị làm việc của mình.

Hệ thống có thể phát triển thêm cơ chế đồng bộ dữ liệu theo thời gian thực giữa các điểm sử dụng. Khi một giao dịch phát sinh tại một cửa hàng hoặc kho, dữ liệu tồn kho, doanh

thu và trạng thái hàng hóa có thể được cập nhật ngay trên toàn hệ thống. Điều này giúp giảm độ trễ thông tin và hỗ trợ người quản lý theo dõi hoạt động kinh doanh chính xác hơn.

Một hướng phát triển khác là tích hợp các thiết bị phần cứng phục vụ trực tiếp cho nghiệp vụ. Máy quét mã vạch có thể được sử dụng để nhận diện sản phẩm nhanh chóng trong quá trình nhập kho và bán hàng. Máy in hóa đơn có thể hỗ trợ tự động xuất chứng từ sau khi giao dịch hoàn thành, trong khi cân điện tử có thể được kết nối để lấy dữ liệu trọng lượng trực tiếp, giảm thao tác nhập thủ công.

Hệ thống cũng có thể bổ sung chức năng quản lý tem sản phẩm, mã vạch hoặc mã QR. Mỗi sản phẩm có thể được gắn một mã định danh riêng để phục vụ việc kiểm kê, tra cứu và theo dõi lịch sử. Việc sử dụng mã định danh giúp rút ngắn thời gian thao tác và hạn chế sai sót khi có số lượng hàng hóa lớn.

Phân báo cáo và Dashboard có thể được mở rộng theo hướng trực quan và chuyên sâu hơn. Hệ thống có thể bổ sung các báo cáo về doanh thu theo ngày, tháng, quý hoặc năm; so sánh doanh thu giữa các khoảng thời gian; thống kê sản phẩm bán chạy; hàng tồn lâu; giá trị tồn kho; hiệu suất nhân viên và mức độ hoạt động của từng kho. Các biểu đồ và chỉ số tổng hợp giúp người quản lý nắm bắt tình hình nhanh hơn.

Hệ thống có thể phát triển thêm cơ chế cảnh báo tự động. Các cảnh báo có thể bao gồm hàng sắp hết, sản phẩm tồn kho trong thời gian dài, bảng giá sắp hết hiệu lực, giao dịch có giá trị lớn hoặc các trường hợp dữ liệu có dấu hiệu bất thường. Những cảnh báo này có thể được hiển thị trực tiếp trên Dashboard hoặc gửi tới người quản lý.

Một hướng mở rộng quan trọng là xây dựng chức năng quản lý lịch sử thay đổi dữ liệu. Các thao tác như cập nhật bảng giá, điều chỉnh tồn kho, thay đổi thông tin sản phẩm hoặc cập nhật quyền người dùng có thể được ghi nhận đầy đủ về thời gian, tài khoản thực hiện và nội dung thay đổi. Cơ chế này hỗ trợ truy vết và kiểm tra khi xảy ra sai lệch dữ liệu.

Hệ thống có thể bổ sung cơ chế phê duyệt đối với các nghiệp vụ quan trọng. Những thao tác như điều chỉnh tồn kho, cập nhật giá, hủy giao dịch hoặc thay đổi quyền có thể yêu cầu người có thẩm quyền xác nhận trước khi chính thức được áp dụng. Điều này giúp giảm nguy cơ thao tác sai hoặc thay đổi dữ liệu ngoài quy trình.

Về bảo mật, hệ thống có thể tiếp tục nâng cấp cơ chế xác thực bằng xác thực hai lớp, quản lý thiết bị đăng nhập, giới hạn số lần đăng nhập sai và ghi nhận lịch sử truy cập. Các token xác thực cũng có thể được quản lý chặt chẽ hơn nhằm giảm nguy cơ sử dụng trái phép.

Về dữ liệu, có thể xây dựng cơ chế sao lưu tự động theo định kỳ và phục hồi dữ liệu khi xảy ra sự cố. Các bản sao lưu có thể được lưu tại nhiều vị trí khác nhau hoặc trên hạ tầng Cloud để giảm nguy cơ mất dữ liệu do lỗi phần cứng hoặc thao tác ngoài ý muốn.

Hệ thống cũng có thể được triển khai trên hạ tầng Cloud và đóng gói bằng các công nghệ như container để thuận tiện hơn trong quá trình triển khai, cập nhật và mở rộng. Việc sử dụng môi trường triển khai tập trung giúp giảm công sức cài đặt Backend tại từng máy và hỗ trợ quản lý phiên bản ứng dụng hiệu quả hơn.

Phần Desktop có thể phát triển thêm cơ chế tự động cập nhật phiên bản. Khi có phiên bản mới, ứng dụng có thể kiểm tra và thông báo cho người dùng, sau đó thực hiện cập nhật mà không cần cài đặt lại thủ công toàn bộ chương trình.

Ngoài ứng dụng Desktop, hệ thống có thể phát triển thêm phiên bản Web hoặc Mobile sử dụng chung Backend RESTful API. Người quản lý có thể theo dõi báo cáo và các chỉ số quan trọng từ thiết bị di động, trong khi các nghiệp vụ trực tiếp tại cửa hàng vẫn được thực hiện thông qua ứng dụng Desktop.

Trong tương lai xa hơn, dữ liệu lịch sử có thể được khai thác để xây dựng các chức năng phân tích và dự báo. Hệ thống có thể hỗ trợ dự đoán nhu cầu hàng hóa, xu hướng doanh thu hoặc nhóm sản phẩm có khả năng tiêu thụ tốt dựa trên dữ liệu đã thu thập. Đây có thể là nền tảng để ứng dụng các kỹ thuật phân tích dữ liệu hoặc trí tuệ nhân tạo vào hoạt động quản lý kinh doanh.

Ngoài ra, có thể bổ sung chức năng xuất báo cáo ra các định dạng như Excel hoặc PDF để phục vụ lưu trữ, in ấn và chia sẻ dữ liệu. Việc hỗ trợ xuất báo cáo giúp người quản lý dễ dàng sử dụng dữ liệu của hệ thống trong các hoạt động tổng hợp và báo cáo bên ngoài chương trình.

Các hướng phát triển trên không chỉ mở rộng số lượng chức năng mà còn hướng tới việc nâng cao độ ổn định, khả năng tự động hóa, tính bảo mật và khả năng triển khai thực tế

của hệ thống. Đây là cơ sở để tiếp tục hoàn thiện chương trình thành một giải pháp quản lý kinh doanh trang sức có phạm vi sử dụng rộng hơn trong tương lai.

## PHỤ LỤC

Toàn bộ mã nguồn của hệ thống được lưu trữ và quản lý trên GitHub. Repository bao gồm mã nguồn Backend Java Spring Boot, ứng dụng Desktop React Electron, các tệp cấu hình Gradle, cấu hình cơ sở dữ liệu, script SQL và các tài liệu kỹ thuật liên quan đến quá trình phát triển hệ thống.

Địa chỉ repository:

[https://github.com/hungminhdev205/phan\\_mem\\_qlbh](https://github.com/hungminhdev205/phan_mem_qlbh)

Mã nguồn được tổ chức theo từng thành phần nhằm phân tách rõ phần xử lý Backend, giao diện Desktop và các tài nguyên hỗ trợ. Việc sử dụng GitHub giúp quản lý phiên bản, theo dõi lịch sử thay đổi và hỗ trợ lưu trữ tập trung toàn bộ nội dung của dự án.

## TÀI LIỆU THAM KHẢO

- [1] Trần Bá Hùng, Bài giảng học phần Công nghệ Java, Khoa Công nghệ Thông tin, Trường Đại học Công nghệ Đông Á, 2026.
- [2] Oracle Corporation, JDK 21 Documentation, Oracle, tài liệu chính thức về nền tảng Java và Java Development Kit.
- [3] Oracle Corporation, Java SE 21 – JDBC API (java.sql), Oracle, tài liệu chính thức về Java Database Connectivity.
- [4] Spring Team, Spring Framework Documentation, Spring, tài liệu chính thức về Spring Framework, IoC, Dependency Injection, truy cập dữ liệu và phát triển ứng dụng web.
- [5] Spring Team, Spring Boot Reference Documentation, Spring, tài liệu chính thức về xây dựng và cấu hình ứng dụng Spring Boot.
- [6] Oliver Gierke, Thomas Darimont, Christoph Strobl, Mark Paluch, Jay Bryant, Greg Turnquist, Spring Data JPA Reference Documentation, Spring.
- [7] Hibernate Team, Hibernate ORM Documentation, Hibernate, tài liệu chính thức về ánh xạ đối tượng – quan hệ và triển khai Jakarta Persistence.
- [8] The PostgreSQL Global Development Group, PostgreSQL Documentation, PostgreSQL Global Development Group.
- [9] Spring Team, Spring Security Reference Documentation, Spring, tài liệu chính thức về xác thực, phân quyền và bảo mật ứng dụng Spring.
- [10] M. Jones, J. Bradley, N. Sakimura, RFC 7519: JSON Web Token (JWT), Internet Engineering Task Force, 2015.
- [11] Roy Thomas Fielding, Architectural Styles and the Design of Network-based Software Architectures, Doctoral Dissertation, University of California, Irvine, 2000.
- [12] React Team, React Documentation – React Reference, tài liệu chính thức của React.
- [13] Electron Team, Electron Documentation, tài liệu chính thức về phát triển ứng dụng desktop bằng Electron.

[14] Microsoft, The TypeScript Handbook, tài liệu chính thức về ngôn ngữ TypeScript.

[15] Gradle, Inc., Gradle User Manual, tài liệu chính thức về hệ thống build và quản lý dependency Gradle.